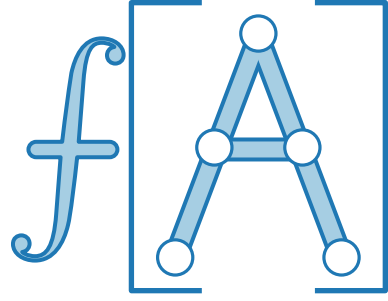


Datenstrukturen

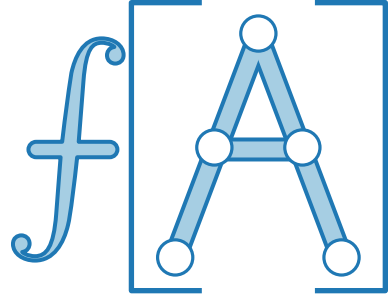
Eine **Datenstruktur** ist ein Konzept, um Daten zu



Datenstrukturen

Eine **Datenstruktur** ist ein Konzept, um Daten zu

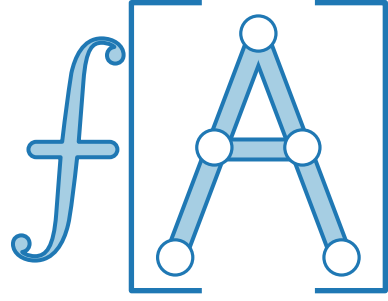
- **speichern,**



Datenstrukturen

Eine **Datenstruktur** ist ein Konzept, um Daten zu

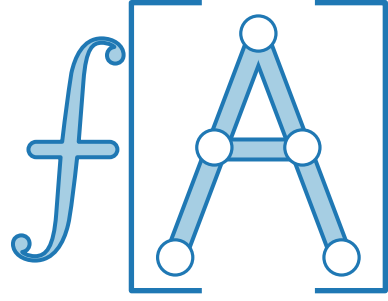
- **speichern,**
- **organisieren** und



Datenstrukturen

Eine **Datenstruktur** ist ein Konzept, um Daten zu

- **speichern,**
- **organisieren** und
- **verwalten.**

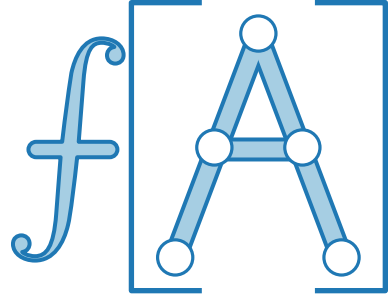


Datenstrukturen

Eine **Datenstruktur** ist ein Konzept, um Daten zu

- **speichern,**
- **organisieren** und
- **verwalten.**

Sie besteht aus



Datenstrukturen



Eine **Datenstruktur** ist ein Konzept, um Daten zu

- **speichern,**
- **organisieren** und
- **verwalten.**

Sie besteht aus

- **Datenwerten,**

Datenstrukturen



Eine **Datenstruktur** ist ein Konzept, um Daten zu

- **speichern,**
- **organisieren** und
- **verwalten.**

Sie besteht aus

- **Datenwerten,**
- ihren **Relationen** und

Datenstrukturen



Eine **Datenstruktur** ist ein Konzept, um Daten zu

- **speichern,**
- **organisieren** und
- **verwalten.**

Sie besteht aus

- **Datenwerten,**
- ihren **Relationen** und
- **Operationen**, die auf die Daten angewendet werden können.

Datenstrukturen



Eine **Datenstruktur** ist ein Konzept, um Daten zu

- **speichern,**
- **organisieren** und
- **verwalten.**

Sie besteht aus

- **Datenwerten,**
- ihren **Relationen** und
- **Operationen**, die auf die Daten angewendet werden können.

Bemerkungen.

- Wir betrachten Datenstrukturen aus der Sicht des Designers / Implementierers (und nicht unbedingt aus Sicht des Nutzers)

Datenstrukturen



Eine **Datenstruktur** ist ein Konzept, um Daten zu

- **speichern,**
- **organisieren** und
- **verwalten.**

Sie besteht aus

- **Datenwerten,**
- ihren **Relationen** und
- **Operationen**, die auf die Daten angewendet werden können.

Bemerkungen.

- Wir betrachten Datenstrukturen aus der Sicht des Designers / Implementierers (und nicht unbedingt aus Sicht des Nutzers)
- Eine Datenstruktur zu definieren und eine zu implementieren sind zwei sehr unterschiedliche Aufgaben.

Datenstrukturen



Eine **Datenstruktur** ist ein Konzept, um Daten zu

- **speichern,**
- **organisieren** und
- **verwalten.**

- Wie schauen die Daten aus?



Sie besteht aus

- **Datenwerten,**
- ihren **Relationen** und
- **Operationen**, die auf die Daten angewendet werden können.

Bemerkungen.

- Wir betrachten Datenstrukturen aus der Sicht des Designers / Implementierers (und nicht unbedingt aus Sicht des Nutzers)
- Eine Datenstruktur zu definieren und eine zu implementieren sind zwei sehr unterschiedliche Aufgaben.

Datenstrukturen



Eine **Datenstruktur** ist ein Konzept, um Daten zu

- **speichern,**
- **organisieren** und
- **verwalten.**

- Wie schauen die Daten aus?
- Wie viel Platz brauchen wir?



Sie besteht aus

- **Datenwerten,**
- ihren **Relationen** und
- **Operationen**, die auf die Daten angewendet werden können.

Bemerkungen.

- Wir betrachten Datenstrukturen aus der Sicht des Designers / Implementierers (und nicht unbedingt aus Sicht des Nutzers)
- Eine Datenstruktur zu definieren und eine zu implementieren sind zwei sehr unterschiedliche Aufgaben.

Datenstrukturen



Eine **Datenstruktur** ist ein Konzept, um Daten zu

- **speichern,**
- **organisieren** und
- **verwalten.**



- Wie schauen die Daten aus?
- Wie viel Platz brauchen wir?
- Dynamisch or statisch?

Sie besteht aus

- **Datenwerten,**
- ihren **Relationen** und
- **Operationen**, die auf die Daten angewendet werden können.

Bemerkungen.

- Wir betrachten Datenstrukturen aus der Sicht des Designers / Implementierers (und nicht unbedingt aus Sicht des Nutzers)
- Eine Datenstruktur zu definieren und eine zu implementieren sind zwei sehr unterschiedliche Aufgaben.

Datenstrukturen



Eine **Datenstruktur** ist ein Konzept, um Daten zu

- **speichern,**
- **organisieren** und
- **verwalten.**

Sie besteht aus

- **Datenwerten,**
- ihren **Relationen** und
- **Operationen**, die auf die Daten angewendet werden können.



- Wie schauen die Daten aus?
- Wie viel Platz brauchen wir?
- Dynamisch or statisch?
- Welche Operationen brauchen wir?

Bemerkungen.

- Wir betrachten Datenstrukturen aus der Sicht des Designers / Implementierers (und nicht unbedingt aus Sicht des Nutzers)
- Eine Datenstruktur zu definieren und eine zu implementieren sind zwei sehr unterschiedliche Aufgaben.

Datenstrukturen



Eine **Datenstruktur** ist ein Konzept, um Daten zu

- **speichern,**
- **organisieren** und
- **verwalten.**

Sie besteht aus

- **Datenwerten,**
- ihren **Relationen** und
- **Operationen**, die auf die Daten angewendet werden können.



- Wie schauen die Daten aus?
- Wie viel Platz brauchen wir?
- Dynamisch or statisch?
- Welche Operationen brauchen wir?
- Wie schnell sollen sie sein?

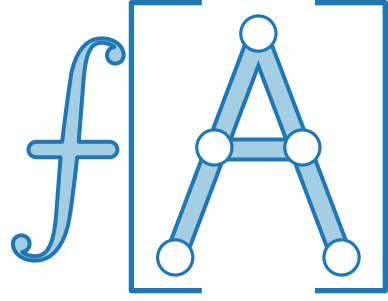
Bemerkungen.

- Wir betrachten Datenstrukturen aus der Sicht des Designers / Implementierers (und nicht unbedingt aus Sicht des Nutzers)
- Eine Datenstruktur zu definieren und eine zu implementieren sind zwei sehr unterschiedliche Aufgaben.

Das „Strg+F“ Problem

Problem. STRINGMATCHING

Geg. Strings T (Text) und M (Muster) über einem Alphabet Σ

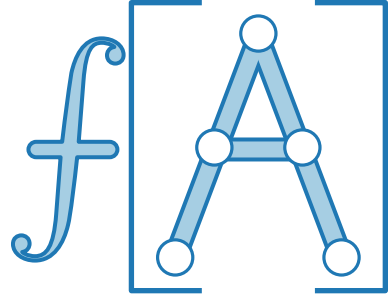


Das „Strg+F“ Problem

Problem. STRINGMATCHING

Geg. Strings T (Text) und M (Muster) über einem Alphabet Σ

Ges. Alle Vorkommen von M in T .



Das „Strg+F“ Problem



Problem. STRINGMATCHING

Geg. Strings T (Text) und M (Muster) über einem Alphabet Σ

Ges. Alle Vorkommen von M in T .

Beispiel.

$$\Sigma = \{a,b,c\}$$

Das „Strg+F“ Problem



Problem. STRINGMATCHING

Geg. Strings T (Text) und M (Muster) über einem Alphabet Σ

Ges. Alle Vorkommen von M in T .

Beispiel.

$$\Sigma = \{a,b,c\}$$

$$M = cbc$$

Das „Strg+F“ Problem



Problem. STRINGMATCHING

Geg. Strings T (Text) und M (Muster) über einem Alphabet Σ

Ges. Alle Vorkommen von M in T .

Beispiel.

$\Sigma = \{a,b,c\}$

$M = cbc$

$T = \text{ c b c c a b c b c b c a c b }$

Das „Strg+F“ Problem



Problem. STRINGMATCHING

Geg. Strings T (Text) und M (Muster) über einem Alphabet Σ

Ges. Alle Vorkommen von M in T .

Beispiel.

$\Sigma = \{a,b,c\}$

$M = cbc$

$T =$ $\begin{matrix} c & b & c & c & a & b & c & b & c & b & c & a & c & b \\ (1) & (2) & (3) & (4) & (5) & (6) & (7) & (8) & (9) & (10) & (11) & (12) & (13) & (14) \end{matrix}$

Das „Strg+F“ Problem



Problem. STRINGMATCHING

Geg. Strings T (Text) und M (Muster) über einem Alphabet Σ

Ges. Alle Vorkommen von M in T .

Beispiel.

$\Sigma = \{a,b,c\}$

$M = cbc$

$T =$ $\begin{matrix} c & b & c & c & a & b & c & b & c & b & c & a & c & b \\ (1) & (2) & (3) & (4) & (5) & (6) & (7) & (8) & (9) & (10) & (11) & (12) & (13) & (14) \end{matrix}$

An welchen Positionen in T kommt M vor?

Das „Strg+F“ Problem



Problem. STRINGMATCHING

Geg. Strings T (Text) und M (Muster) über einem Alphabet Σ

Ges. Alle Vorkommen von M in T .

Beispiel.

$\Sigma = \{a,b,c\}$

$M = cbc$

$T =$

c	b	c
(1)	(2)	(3)
c	b	c

 c a b c b c b c a c b
(4) (5) (6) (7) (8) (9) (10) (11) (12) (13) (14)

An welchen Positionen in T kommt M vor?

↑
(1)

Das „Strg+F“ Problem



Problem. STRINGMATCHING

Geg. Strings T (Text) und M (Muster) über einem Alphabet Σ

Ges. Alle Vorkommen von M in T .

Beispiel.

$$\Sigma = \{a, b, c\}$$

$$M = cbc$$

$$T = \begin{array}{cccccccccccccccc} c & b & c & c & a & b & c & b & c & b & c & a & c & b \\ (1) & (2) & (3) & (4) & (5) & (6) & (7) & (8) & (9) & (10) & (11) & (12) & (13) & (14) \\ c & b & c & & & & c & b & c & & & & & \end{array}$$

An welchen Positionen in T kommt M vor?

↑
(1)

↑
(7)

Das „Strg+F“ Problem



Problem. STRINGMATCHING

Geg. Strings T (Text) und M (Muster) über einem Alphabet Σ

Ges. Alle Vorkommen von M in T .

Beispiel.

$\Sigma = \{a,b,c\}$

$M = cbc$

$T =$

c	b	c	c	a	b	c	b	c	a	c	b
(1)	(2)	(3)	(4)	(5)	(6)	(7)	(8)	(9)	(10)	(11)	(12)
c	b	c				c	b	c			
									c	b	c

Arrows point to positions (1), (7), and (9) below the text.

An welchen Positionen in T kommt M vor?

Das „Strg+F“ Problem



Problem. STRINGMATCHING

Geg. Strings T (Text) und M (Muster) über einem Alphabet Σ

Ges. Alle Vorkommen von M in T .

Beispiel.

$\Sigma = \{a,b,c\}$

$M = cbc$

$T =$

c	b	c	c	a	b	c	b	c	a	c	b
(1)	(2)	(3)	(4)	(5)	(6)	(7)	(8)	(9)	(10)	(11)	(12)
c	b	c				c	b	c			
									c	b	c

Arrows point to positions (1), (7), and (9) below the text.

An welchen Positionen in T kommt M vor?

Anwendungen.

Das „Strg+F“ Problem



Problem. STRINGMATCHING

Geg. Strings T (Text) und M (Muster) über einem Alphabet Σ

Ges. Alle Vorkommen von M in T .

Beispiel.

$\Sigma = \{a,b,c\}$

$M = cbc$

$T =$

c	b	c	c	a	b	c	b	c	a	c	b
(1)	(2)	(3)	(4)	(5)	(6)	(7)	(8)	(9)	(10)	(11)	(12)
c	b	c				c	b	c			
									c	b	c

Arrows point to positions (1), (7), and (9) below the text.

An welchen Positionen in T kommt M vor?

Anwendungen.

- Ein Dokument / E-Book durchsuchen

Das „Strg+F“ Problem



Problem. STRINGMATCHING

Geg. Strings T (Text) und M (Muster) über einem Alphabet Σ

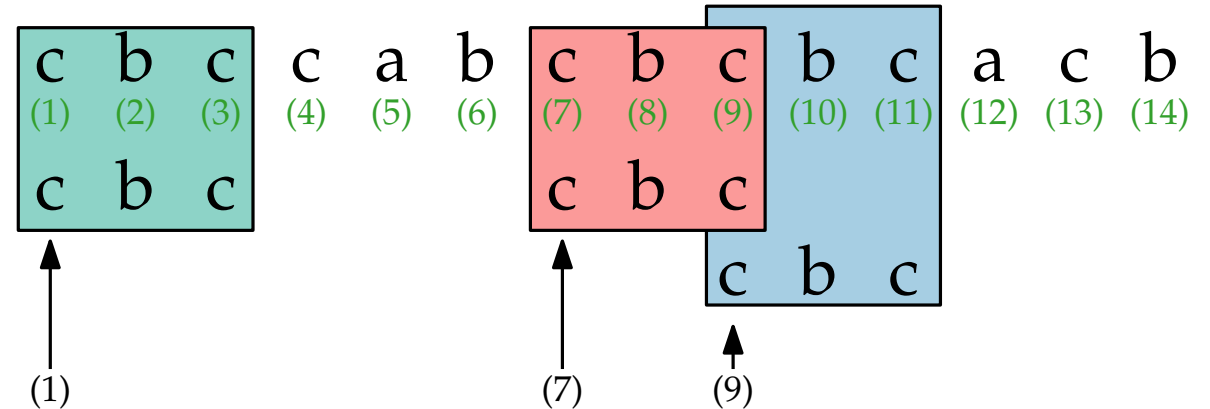
Ges. Alle Vorkommen von M in T .

Beispiel.

$\Sigma = \{a,b,c\}$

$M = cbc$

$T =$



An welchen Positionen in T kommt M vor?

Anwendungen.

- Ein Dokument / E-Book durchsuchen
- Ein Muster in einer DNA-Sequenz suchen

Das „Strg+F“ Problem



Problem. STRINGMATCHING

Geg. Strings T (Text) und M (Muster) über einem Alphabet Σ

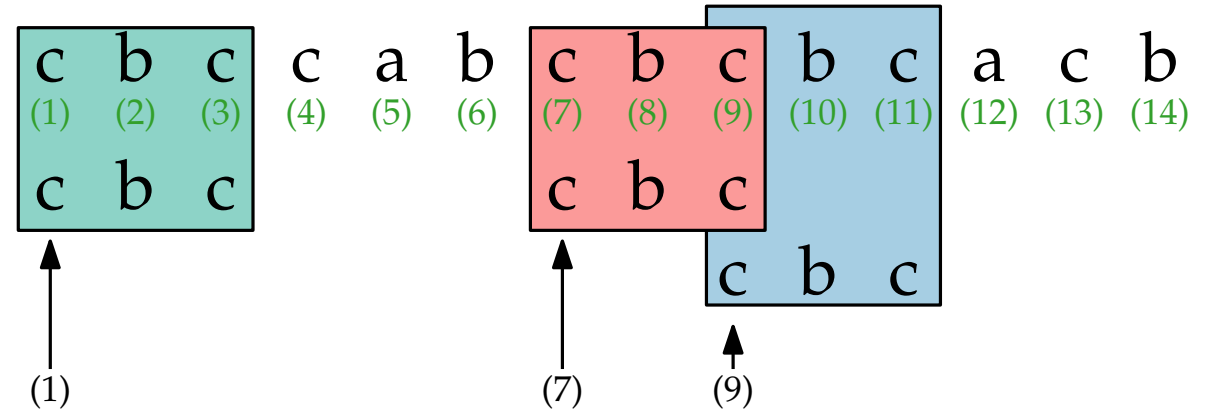
Ges. Alle Vorkommen von M in T .

Beispiel.

$$\Sigma = \{a,b,c\}$$

$$M = cbc$$

$$T =$$



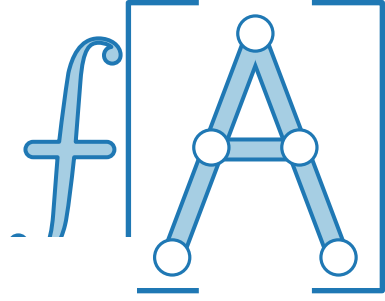
An welchen Positionen in T kommt M vor?

Anwendungen.

- Ein Dokument / E-Book durchsuchen
- Ein Muster in einer DNA-Sequenz suchen
- Suchmaschinen: Erkennen, ob eine Webseite relevant zur Suchanfrage ist

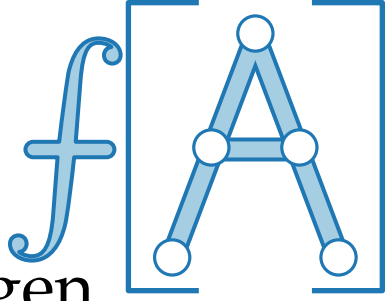
Notation

Wir gehen davon aus, dass T und M als Arrays vorliegen



$T =$ **c** **b** **c** **c** **a** **b** **c** **b** **c** **b** **c** **a** **c** **b**
 (1) (2) (3) (4) (5) (6) (7) (8) (9) (10) (11) (12) (13) (14)

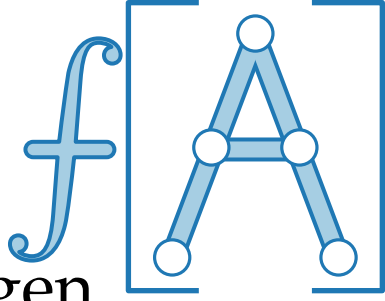
Notation



Wir gehen davon aus, dass T und M als Arrays vorliegen, mit $n = |T|$ Einträgen
 $T[1], T[2], \dots, T[n]$

$T =$ **c** **b** **c** **c** **a** **b** **c** **b** **c** **b** **c** **a** **c** **b**
 (1) (2) (3) (4) (5) (6) (7) (8) (9) (10) (11) (12) (13) (14)

Notation

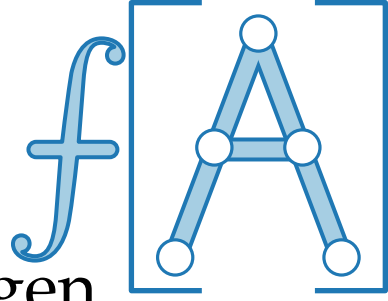


Wir gehen davon aus, dass T und M als Arrays vorliegen, mit $n = |T|$ Einträgen
 $T[1], T[2], \dots, T[n]$

$T =$

		$T[3]$											
c	b	c	c	a	b	c	b	c	b	c	a	c	b
(1)	(2)	(3)	(4)	(5)	(6)	(7)	(8)	(9)	(10)	(11)	(12)	(13)	(14)

Notation

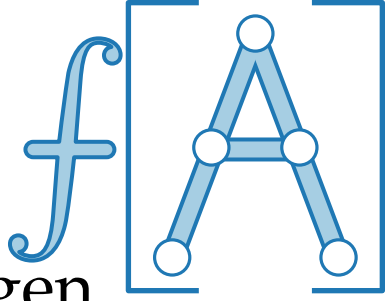


Wir gehen davon aus, dass T und M als Arrays vorliegen, mit $n = |T|$ Einträgen $T[1], T[2], \dots, T[n]$ und $m = |M|$ Einträgen $M[1], M[2], \dots, M[m]$.

$T =$

		$T[3]$											
c	b	c	c	a	b	c	b	c	b	c	a	c	b
(1)	(2)	(3)	(4)	(5)	(6)	(7)	(8)	(9)	(10)	(11)	(12)	(13)	(14)

Notation



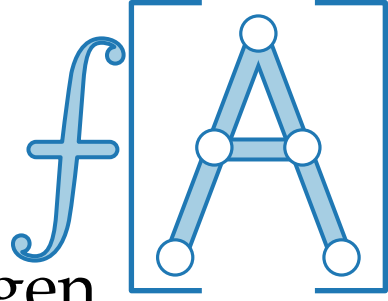
Wir gehen davon aus, dass T und M als Arrays vorliegen, mit $n = |T|$ Einträgen $T[1], T[2], \dots, T[n]$ und $m = |M|$ Einträgen $M[1], M[2], \dots, M[m]$.

$T =$

		$T[3]$											
c	b	c	c	a	b	c	b	c	b	c	a	c	b
(1)	(2)	(3)	(4)	(5)	(6)	(7)	(8)	(9)	(10)	(11)	(12)	(13)	(14)

$T[i, j]$ mit $1 \leq i \leq j \leq n$ bezeichnet den Teilstring von T bestehend aus $T[i], T[i+1], \dots, T[j]$.

Notation



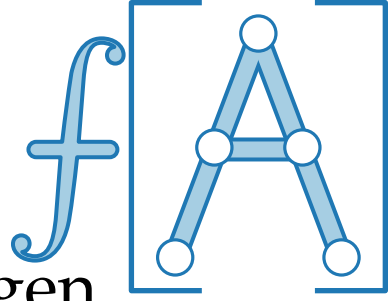
Wir gehen davon aus, dass T und M als Arrays vorliegen, mit $n = |T|$ Einträgen $T[1], T[2], \dots, T[n]$ und $m = |M|$ Einträgen $M[1], M[2], \dots, M[m]$.

$T =$

		$T[3]$			$T[6, 11]$								
c	b	c	c	a	b	c	b	c	b	c	a	c	b
(1)	(2)	(3)	(4)	(5)	(6)	(7)	(8)	(9)	(10)	(11)	(12)	(13)	(14)

$T[i, j]$ mit $1 \leq i \leq j \leq n$ bezeichnet den Teilstring von T bestehend aus $T[i], T[i+1], \dots, T[j]$.

Notation



Wir gehen davon aus, dass T und M als Arrays vorliegen, mit $n = |T|$ Einträgen $T[1], T[2], \dots, T[n]$ und $m = |M|$ Einträgen $M[1], M[2], \dots, M[m]$.

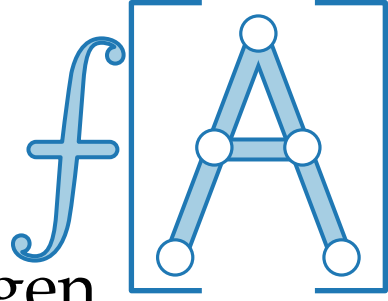
$T =$ $T[3]$ $T[6, 11]$
c b c c a b c b c b c a c b
(1) (2) (3) (4) (5) (6) (7) (8) (9) (10) (11) (12) (13) (14)

$T[i, j]$ mit $1 \leq i \leq j \leq n$ bezeichnet den Teilstring von T bestehend aus $T[i], T[i+1], \dots, T[j]$.

Jeder Teilstring $T[i, j]$ heißt **Infix** von T .

$T =$ c b c c a b c b c b c a c b
 (1) (2) (3) (4) (5) (6) (7) (8) (9) (10) (11) (12) (13) (14)

Notation



Wir gehen davon aus, dass T und M als Arrays vorliegen, mit $n = |T|$ Einträgen $T[1], T[2], \dots, T[n]$ und $m = |M|$ Einträgen $M[1], M[2], \dots, M[m]$.

$T =$ $T[3]$ $T[6, 11]$

c	b	c	c	a	b	c	b	c	b	c	a	c	b
(1)	(2)	(3)	(4)	(5)	(6)	(7)	(8)	(9)	(10)	(11)	(12)	(13)	(14)

$T[i, j]$ mit $1 \leq i \leq j \leq n$ bezeichnet den Teilstring von T bestehend aus $T[i], T[i+1], \dots, T[j]$.

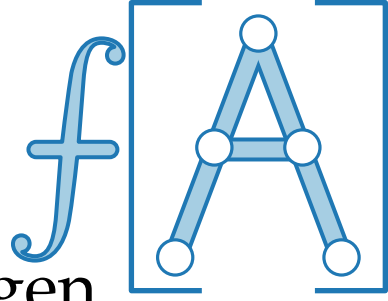
Jeder Teilstring $T[i, j]$ heißt **Infix** von T .

$T =$ $T[1, 3]$

c	b	c	c	a	b	c	b	c	b	c	a	c	b
(1)	(2)	(3)	(4)	(5)	(6)	(7)	(8)	(9)	(10)	(11)	(12)	(13)	(14)

Infix

Notation



Wir gehen davon aus, dass T und M als Arrays vorliegen, mit $n = |T|$ Einträgen $T[1], T[2], \dots, T[n]$ und $m = |M|$ Einträgen $M[1], M[2], \dots, M[m]$.

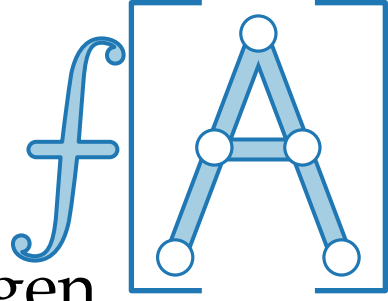
$T =$ $T[3]$ $T[6, 11]$
c b c c a b c b c b c a c b
(1) (2) (3) (4) (5) (6) (7) (8) (9) (10) (11) (12) (13) (14)

$T[i, j]$ mit $1 \leq i \leq j \leq n$ bezeichnet den Teilstring von T bestehend aus $T[i], T[i+1], \dots, T[j]$.

Jeder Teilstring $T[i, j]$ heißt **Infix** von T .

$T =$ c b c c a b c b c b c a c b
 (1) (2) (3) (4) (5) (6) (7) (8) (9) (10) (11) (12) (13) (14)
 Infix Infix

Notation



Wir gehen davon aus, dass T und M als Arrays vorliegen, mit $n = |T|$ Einträgen $T[1], T[2], \dots, T[n]$ und $m = |M|$ Einträgen $M[1], M[2], \dots, M[m]$.

$T =$ $T[3]$ $T[6, 11]$

c	b	c	c	a	b	c	b	c	b	c	a	c	b
(1)	(2)	(3)	(4)	(5)	(6)	(7)	(8)	(9)	(10)	(11)	(12)	(13)	(14)

$T[i, j]$ mit $1 \leq i \leq j \leq n$ bezeichnet den Teilstring von T bestehend aus $T[i], T[i+1], \dots, T[j]$.

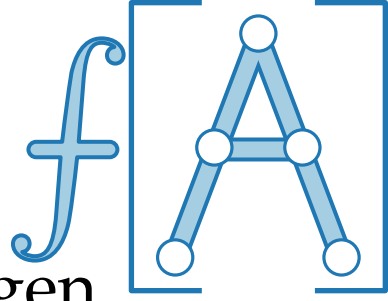
Jeder Teilstring $T[i, j]$ heißt **Infix** von T .

$T =$

c	b	c	c	a	b	c	b	c	a	c	b		
(1)	(2)	(3)	(4)	(5)	(6)	(7)	(8)	(9)	(10)	(11)	(12)	(13)	(14)

Infix Infix Infix

Notation



Wir gehen davon aus, dass T und M als Arrays vorliegen, mit $n = |T|$ Einträgen $T[1], T[2], \dots, T[n]$ und $m = |M|$ Einträgen $M[1], M[2], \dots, M[m]$.

$T =$ $\begin{array}{cccccccccccccc} & & & T[3] & & & T[6, 11] & & & & & & & \\ c & b & c & c & a & b & c & b & c & b & c & a & c & b \\ (1) & (2) & (3) & (4) & (5) & (6) & (7) & (8) & (9) & (10) & (11) & (12) & (13) & (14) \end{array}$

$T[i, j]$ mit $1 \leq i \leq j \leq n$ bezeichnet den Teilstring von T bestehend aus $T[i], T[i+1], \dots, T[j]$.

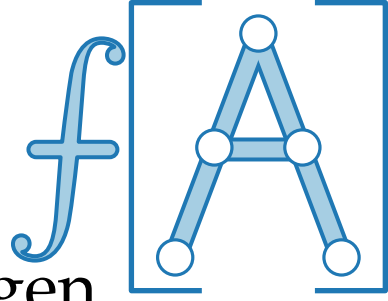
Jeder Teilstring $T[i, j]$ heißt **Infix** von T .

$T[1, j]$ heißt auch **Präfix** von T .

$T =$ $\begin{array}{cccccccccccccc} c & b & c & c & a & b & c & b & c & a & c & b \\ (1) & (2) & (3) & (4) & (5) & (6) & (7) & (8) & (9) & (10) & (11) & (12) & (13) & (14) \end{array}$

Infix Infix Infix

Notation



Wir gehen davon aus, dass T und M als Arrays vorliegen, mit $n = |T|$ Einträgen $T[1], T[2], \dots, T[n]$ und $m = |M|$ Einträgen $M[1], M[2], \dots, M[m]$.

$T =$ $\begin{array}{cccccccccccccc} & & & T[3] & & & T[6, 11] & & & & & & & \\ c & b & \boxed{c} & c & a & \boxed{b} & \boxed{c} & \boxed{b} & \boxed{c} & \boxed{b} & \boxed{c} & a & c & b \\ (1) & (2) & (3) & (4) & (5) & (6) & (7) & (8) & (9) & (10) & (11) & (12) & (13) & (14) \end{array}$

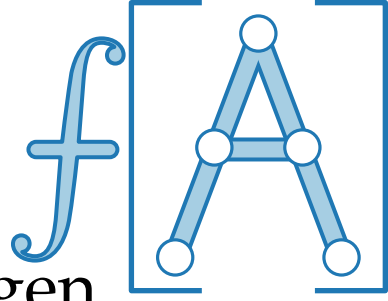
$T[i, j]$ mit $1 \leq i \leq j \leq n$ bezeichnet den Teilstring von T bestehend aus $T[i], T[i+1], \dots, T[j]$.

Jeder Teilstring $T[i, j]$ heißt **Infix** von T .

$T[1, j]$ heißt auch **Präfix** von T .

$T =$ $\begin{array}{cccccccccccccc} & & \text{Präfix} & & & & & & & & & & & \\ \boxed{c} & \boxed{b} & \boxed{c} & c & a & \boxed{b} & \boxed{c} & \boxed{b} & c & b & \boxed{c} & \boxed{a} & \boxed{c} & \boxed{b} \\ (1) & (2) & (3) & (4) & (5) & (6) & (7) & (8) & (9) & (10) & (11) & (12) & (13) & (14) \\ & & \text{Infix} & & & \text{Infix} & & & & & \text{Infix} & & & \end{array}$

Notation



Wir gehen davon aus, dass T und M als Arrays vorliegen, mit $n = |T|$ Einträgen $T[1], T[2], \dots, T[n]$ und $m = |M|$ Einträgen $M[1], M[2], \dots, M[m]$.

$T =$ $\begin{array}{cccccccccccccc} & & & T[3] & & & T[6,11] & & & & & & & \\ c & b & c & c & a & b & c & b & c & b & c & a & c & b \\ (1) & (2) & (3) & (4) & (5) & (6) & (7) & (8) & (9) & (10) & (11) & (12) & (13) & (14) \end{array}$

$T[i, j]$ mit $1 \leq i \leq j \leq n$ bezeichnet den Teilstring von T bestehend aus $T[i], T[i+1], \dots, T[j]$.

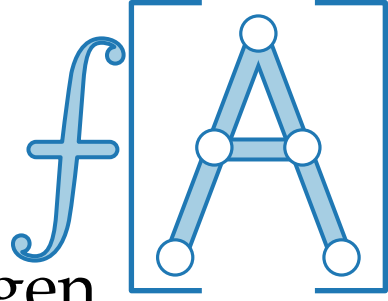
Jeder Teilstring $T[i, j]$ heißt **Infix** von T .

$T[1, j]$ heißt auch **Präfix** von T .

$T[i, n]$ heißt auch **Suffix** von T .

$T =$ $\begin{array}{cccccccccccccc} & & \text{Präfix} & & & & & & & & & & & \\ c & b & c & c & a & b & c & b & c & b & c & a & c & b \\ (1) & (2) & (3) & (4) & (5) & (6) & (7) & (8) & (9) & (10) & (11) & (12) & (13) & (14) \end{array}$
 $\begin{array}{ccccccc} & \text{Infix} & & \text{Infix} & & \text{Infix} & \\ & & & & & & \end{array}$

Notation



Wir gehen davon aus, dass T und M als Arrays vorliegen, mit $n = |T|$ Einträgen $T[1], T[2], \dots, T[n]$ und $m = |M|$ Einträgen $M[1], M[2], \dots, M[m]$.

$T =$

		$T[3]$			$T[6, 11]$								
c	b	c	c	a	b	c	b	c	a	c	b		
(1)	(2)	(3)	(4)	(5)	(6)	(7)	(8)	(9)	(10)	(11)	(12)	(13)	(14)

$T[i, j]$ mit $1 \leq i \leq j \leq n$ bezeichnet den Teilstring von T bestehend aus $T[i], T[i+1], \dots, T[j]$.

Jeder Teilstring $T[i, j]$ heißt **Infix** von T .

$T[1, j]$ heißt auch **Präfix** von T .

$T[i, n]$ heißt auch **Suffix** von T .

$T =$

		Präfix							Suffix				
c	b	c	c	a	b	c	b	c	a	c	b		
(1)	(2)	(3)	(4)	(5)	(6)	(7)	(8)	(9)	(10)	(11)	(12)	(13)	(14)
		Infix				Infix				Infix			

Algorithmische Komplexität



Vorkommen von (Präfixen von) M könnten überlappen.

➡ Ein einziger links-nach-rechts Durchlauf von T reicht nicht aus, um alle Vorkommen von M zu finden!

$T =$

c	b	c	c	a	b	c	b	c	b	c	a	c	b
(1)	(2)	(3)	(4)	(5)	(6)	(7)	(8)	(9)	(10)	(11)	(12)	(13)	(14)

Algorithmische Komplexität



Vorkommen von (Präfixen von) M könnten überlappen.

➡ Ein einziger links-nach-rechts Durchlauf von T reicht nicht aus, um alle Vorkommen von M zu finden!

$T =$ $\underset{(1)}{c}$ $\underset{(2)}{b}$ $\underset{(3)}{c}$ $\underset{(4)}{c}$ $\underset{(5)}{a}$ $\underset{(6)}{b}$ $\underset{(7)}{c}$ $\underset{(8)}{b}$ $\underset{(9)}{c}$ $\underset{(10)}{b}$ $\underset{(11)}{c}$ $\underset{(12)}{a}$ $\underset{(13)}{c}$ $\underset{(14)}{b}$

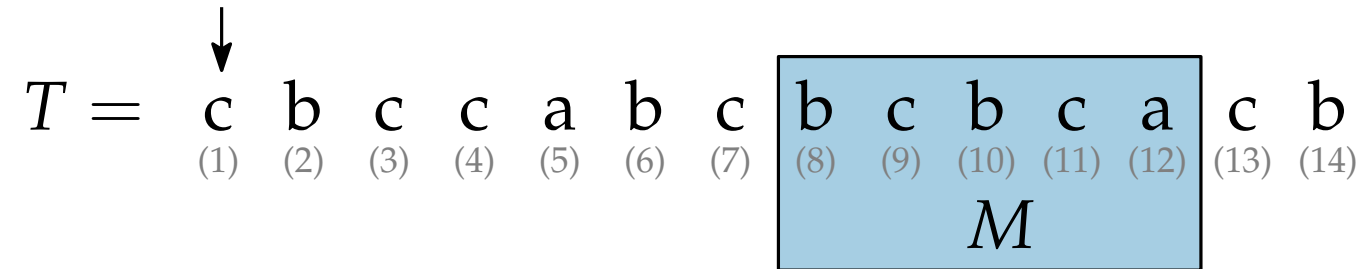
M

Algorithmische Komplexität



Vorkommen von (Präfixen von) M könnten überlappen.

➡ Ein einziger links-nach-rechts Durchlauf von T reicht nicht aus, um alle Vorkommen von M zu finden!

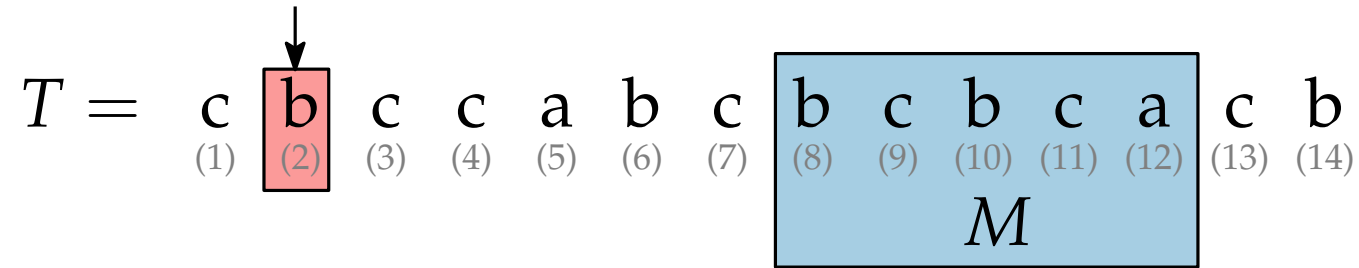


Algorithmische Komplexität

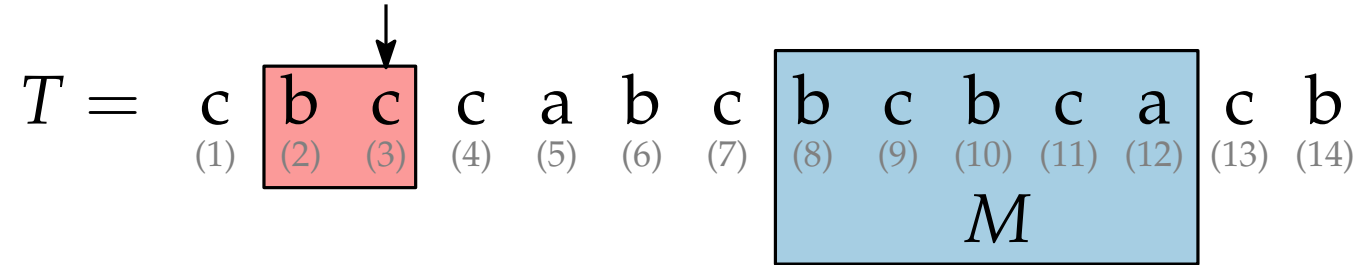


Vorkommen von (Präfixen von) M könnten überlappen.

→ Ein einziger links-nach-rechts Durchlauf von T reicht nicht aus, um alle Vorkommen von M zu finden!



➡ Ein einziger links-nach-rechts Durchlauf von T reicht nicht aus, um alle Vorkommen von M zu finden!

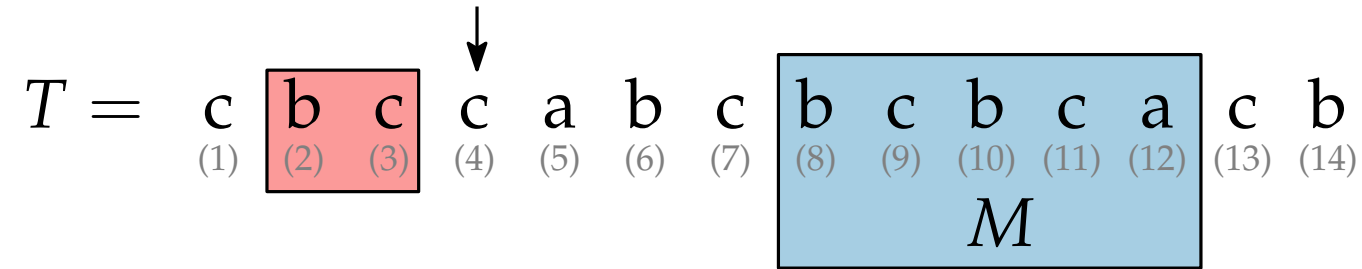


Algorithmische Komplexität



Vorkommen von (Präfixen von) M könnten überlappen.

➡ Ein einziger links-nach-rechts Durchlauf von T reicht nicht aus, um alle Vorkommen von M zu finden!

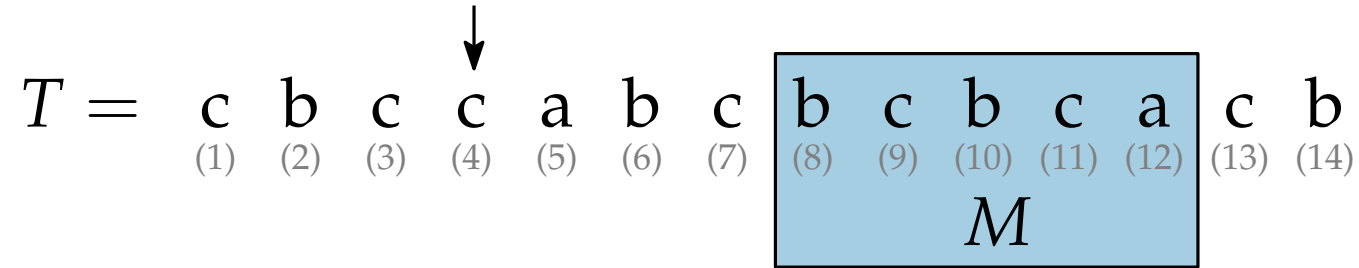


Algorithmische Komplexität



Vorkommen von (Präfixen von) M könnten überlappen.

➡ Ein einziger links-nach-rechts Durchlauf von T reicht nicht aus, um alle Vorkommen von M zu finden!

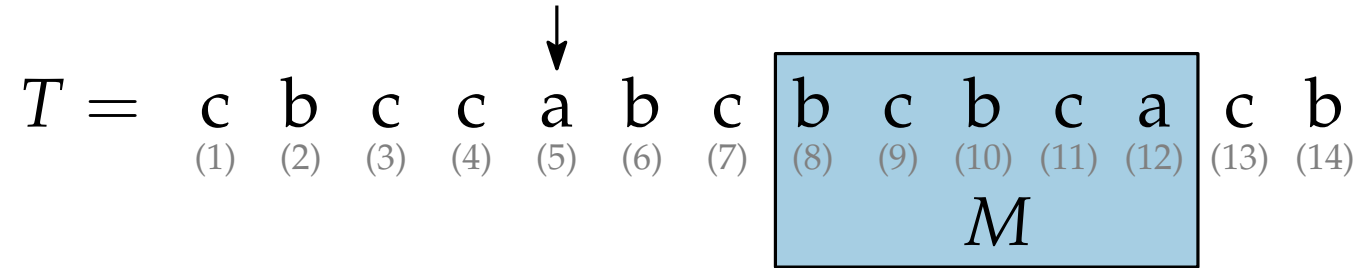


Algorithmische Komplexität



Vorkommen von (Präfixen von) M könnten überlappen.

➡ Ein einziger links-nach-rechts Durchlauf von T reicht nicht aus, um alle Vorkommen von M zu finden!

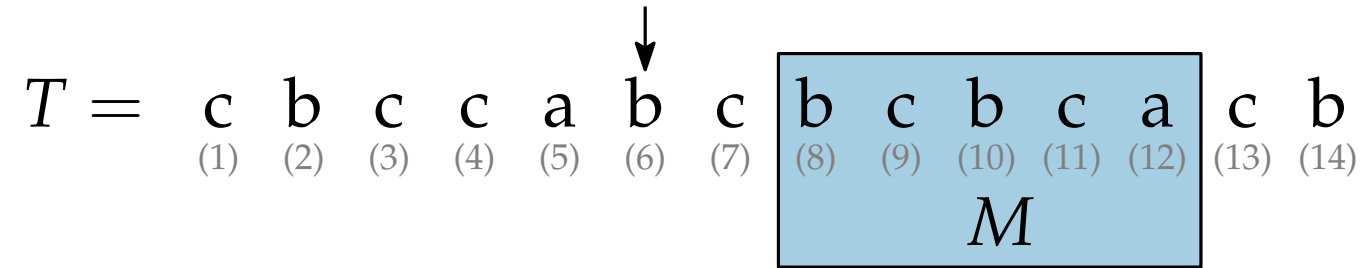


Algorithmische Komplexität



Vorkommen von (Präfixen von) M könnten überlappen.

➡ Ein einziger links-nach-rechts Durchlauf von T reicht nicht aus, um alle Vorkommen von M zu finden!

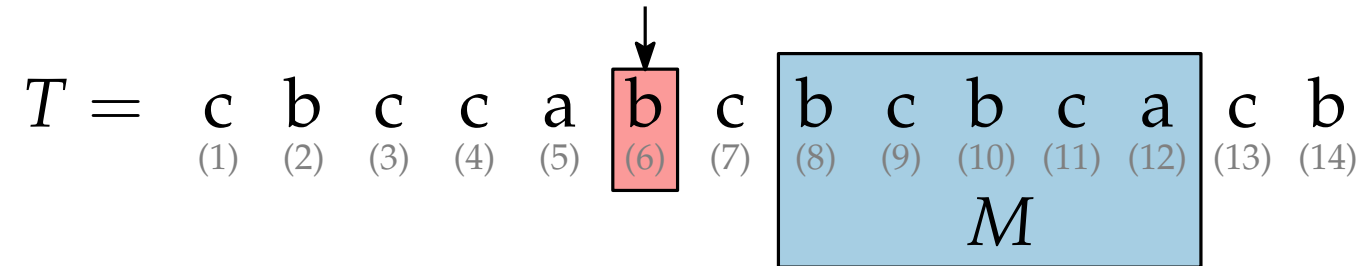


Algorithmische Komplexität



Vorkommen von (Präfixen von) M könnten überlappen.

➔ Ein einziger links-nach-rechts Durchlauf von T reicht nicht aus, um alle Vorkommen von M zu finden!

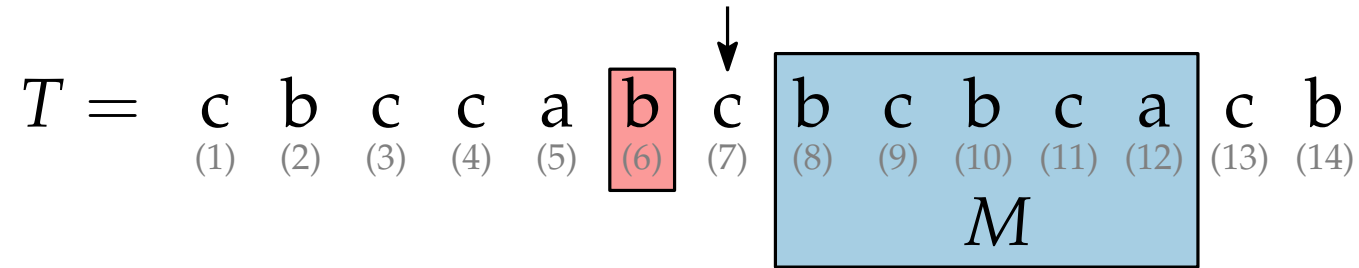


Algorithmische Komplexität



Vorkommen von (Präfixen von) M könnten überlappen.

➡ Ein einziger links-nach-rechts Durchlauf von T reicht nicht aus, um alle Vorkommen von M zu finden!

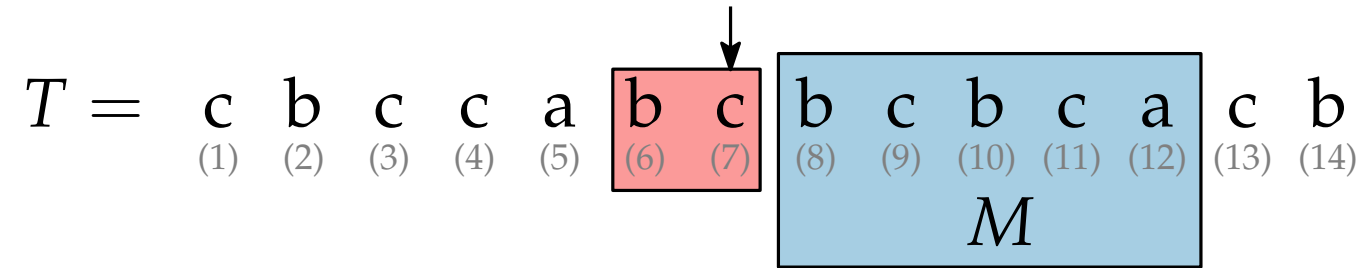


Algorithmische Komplexität



Vorkommen von (Präfixen von) M könnten überlappen.

➡ Ein einziger links-nach-rechts Durchlauf von T reicht nicht aus, um alle Vorkommen von M zu finden!

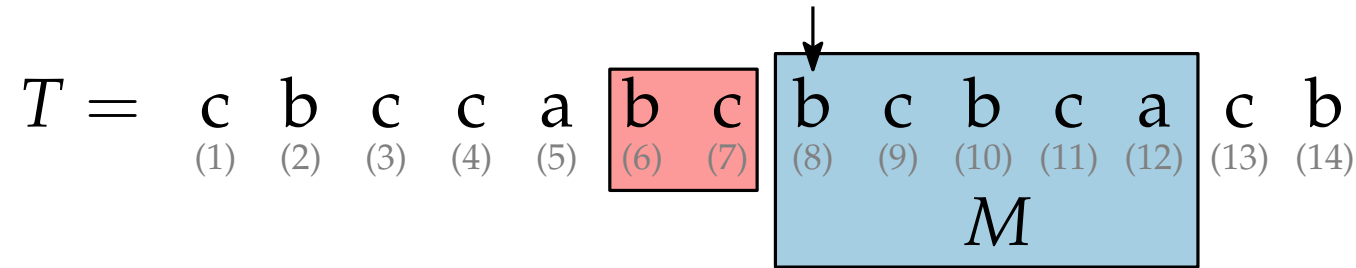


Algorithmische Komplexität



Vorkommen von (Präfixen von) M könnten überlappen.

➡ Ein einziger links-nach-rechts Durchlauf von T reicht nicht aus, um alle Vorkommen von M zu finden!

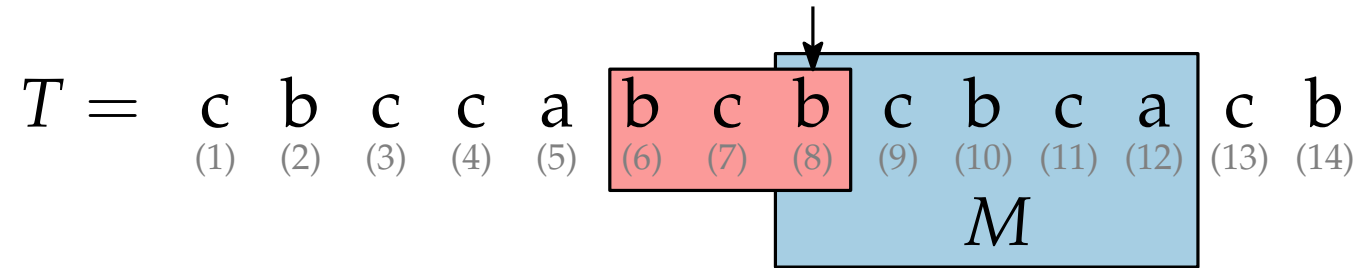


Algorithmische Komplexität



Vorkommen von (Präfixen von) M könnten überlappen.

➡ Ein einziger links-nach-rechts Durchlauf von T reicht nicht aus, um alle Vorkommen von M zu finden!

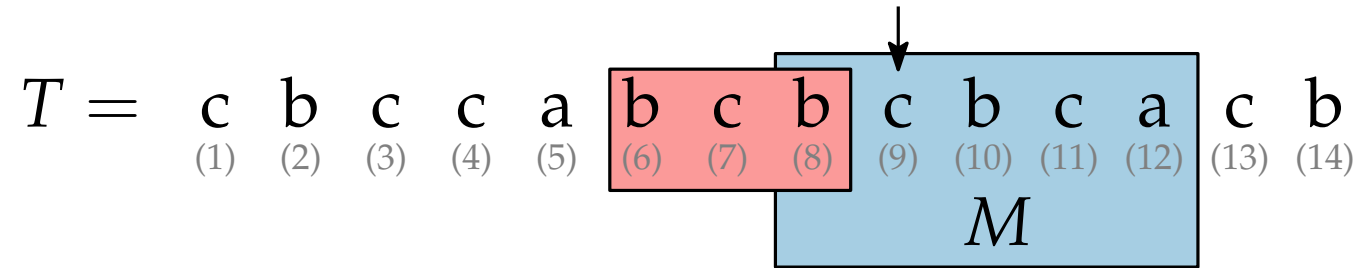


Algorithmische Komplexität



Vorkommen von (Präfixen von) M könnten überlappen.

➡ Ein einziger links-nach-rechts Durchlauf von T reicht nicht aus, um alle Vorkommen von M zu finden!

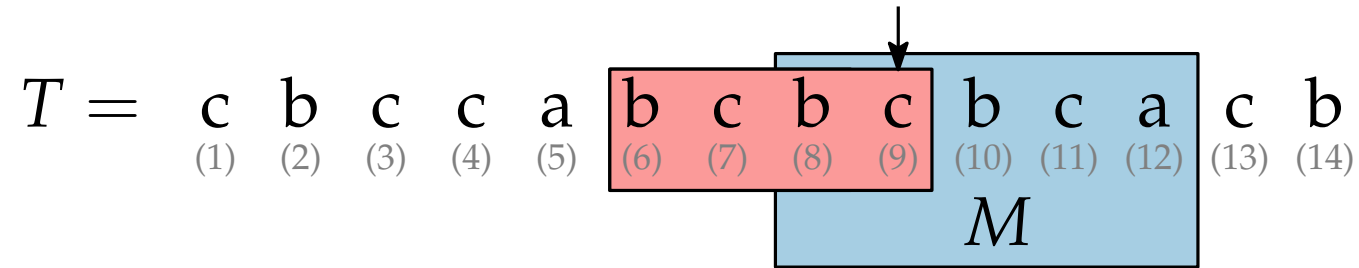


Algorithmische Komplexität



Vorkommen von (Präfixen von) M könnten überlappen.

➔ Ein einziger links-nach-rechts Durchlauf von T reicht nicht aus, um alle Vorkommen von M zu finden!

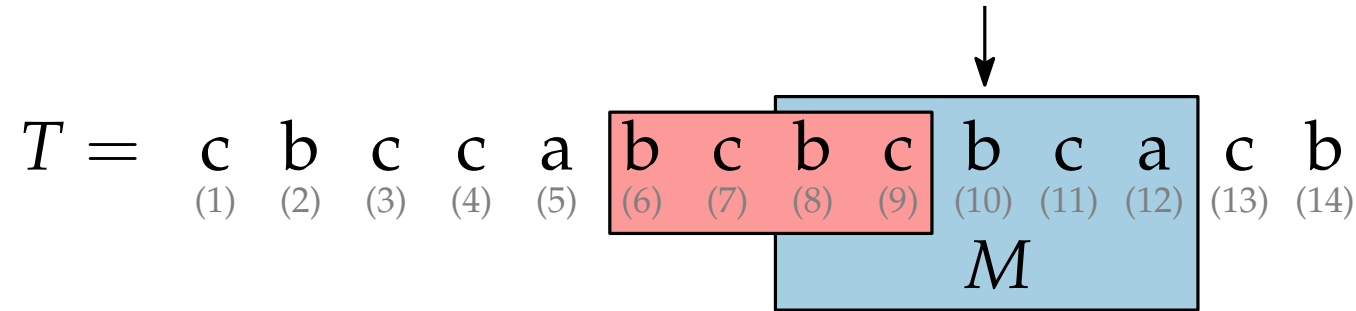


Algorithmische Komplexität



Vorkommen von (Präfixen von) M könnten überlappen.

➔ Ein einziger links-nach-rechts Durchlauf von T reicht nicht aus, um alle Vorkommen von M zu finden!

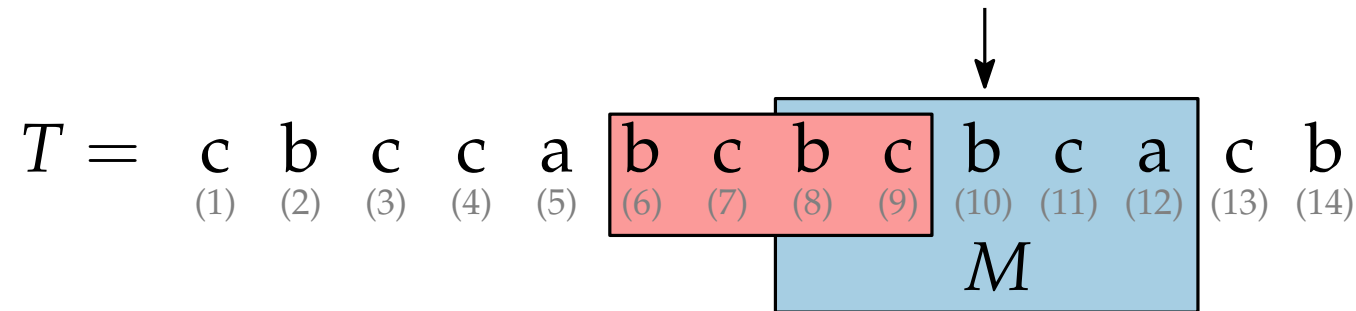


Algorithmische Komplexität



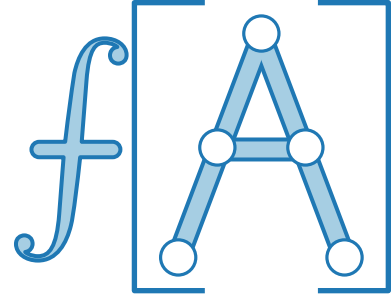
Vorkommen von (Präfixen von) M könnten überlappen.

➔ Ein einziger links-nach-rechts Durchlauf von T reicht nicht aus, um alle Vorkommen von M zu finden!



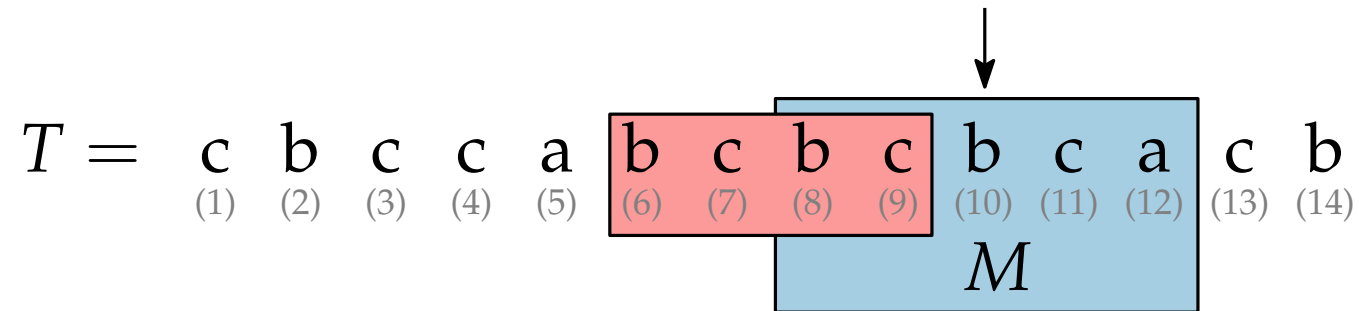
Beobachtung. STRINGMATCHING kann in $\mathcal{O}(nm)$ Zeit gelöst werden.

Algorithmische Komplexität



Vorkommen von (Präfixen von) M könnten überlappen.

→ Ein einziger links-nach-rechts Durchlauf von T reicht nicht aus, um alle Vorkommen von M zu finden!



Beobachtung. STRINGMATCHING kann in $\mathcal{O}(nm)$ Zeit gelöst werden.

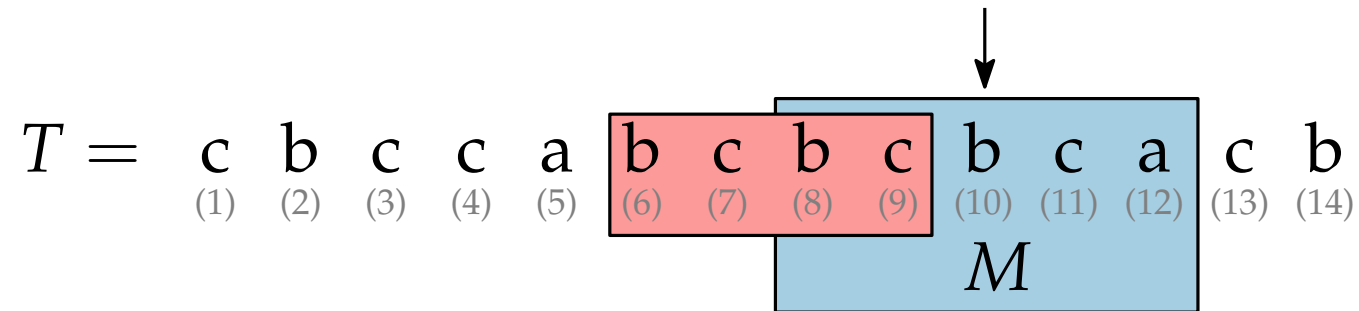
Satz. STRINGMATCHING kann in $\mathcal{O}(n + m)$ Zeit gelöst werden, und diese Zeitschranke ist optimal. [Knuth, Morris, Pratt '77]

Algorithmische Komplexität



Vorkommen von (Präfixen von) M könnten überlappen.

➔ Ein einziger links-nach-rechts Durchlauf von T reicht nicht aus, um alle Vorkommen von M zu finden!

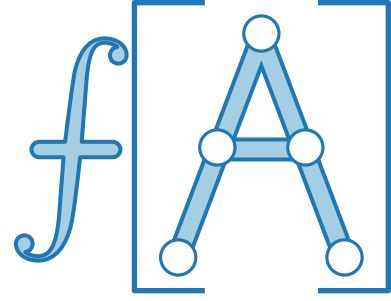


Beobachtung. STRINGMATCHING kann in $\mathcal{O}(nm)$ Zeit gelöst werden.

Satz. STRINGMATCHING kann in $\mathcal{O}(n + m)$ Zeit gelöst werden, und diese Zeitschranke ist optimal. [Knuth, Morris, Pratt '77]

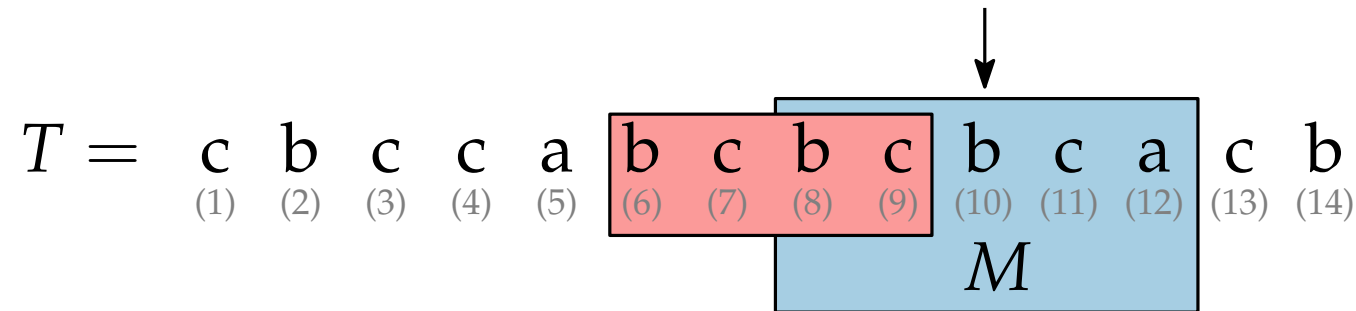
Häufig werden mehrere Anfragen $M_1, M_2, M_3, \dots$ auf dem gleichen Text T ausgeführt.

Algorithmische Komplexität



Vorkommen von (Präfixen von) M könnten überlappen.

→ Ein einziger links-nach-rechts Durchlauf von T reicht nicht aus, um alle Vorkommen von M zu finden!



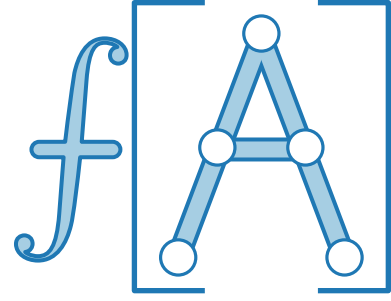
Beobachtung. STRINGMATCHING kann in $\mathcal{O}(nm)$ Zeit gelöst werden.

Satz. STRINGMATCHING kann in $\mathcal{O}(n + m)$ Zeit gelöst werden, und diese Zeitschranke ist optimal. [Knuth, Morris, Pratt '77]

Häufig werden mehrere Anfragen $M_1, M_2, M_3, \dots$ auf dem gleichen Text T ausgeführt.

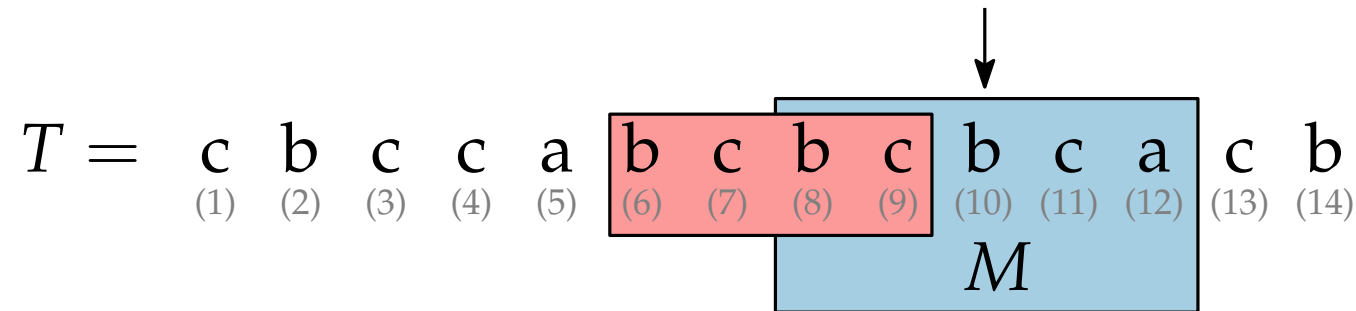
Ziel. Entwickeln einer Datenstruktur für T , so dass jede Anfrage M_i in Zeit *unabhängig von* n beantwortet werden kann.

Algorithmische Komplexität



Vorkommen von (Präfixen von) M könnten überlappen.

→ Ein einziger links-nach-rechts Durchlauf von T reicht nicht aus, um alle Vorkommen von M zu finden!



Beobachtung. STRINGMATCHING kann in $\mathcal{O}(nm)$ Zeit gelöst werden.

Satz. STRINGMATCHING kann in $\mathcal{O}(n + m)$ Zeit gelöst werden, und diese Zeitschranke ist optimal. [Knuth, Morris, Pratt '77]

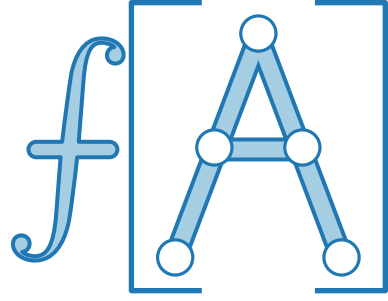
Häufig werden mehrere Anfragen $M_1, M_2, M_3, \dots$ auf dem gleichen Text T ausgeführt.

Ziel. Entwickeln einer Datenstruktur für T , so dass jede Anfrage M_i in Zeit *unabhängig von* n beantwortet werden kann.

Wir betrachten zwei solcher Datenstrukturen: **Suffixbäume** und **Suffixarrays**.

Suffixbäume – Definitionen

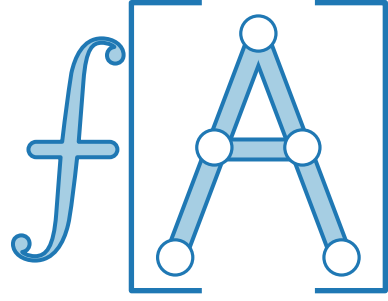
Idee. Repräsentiere T als Suchbaum.



Suffixbäume – Definitionen

$T = a b c a b a b c a$

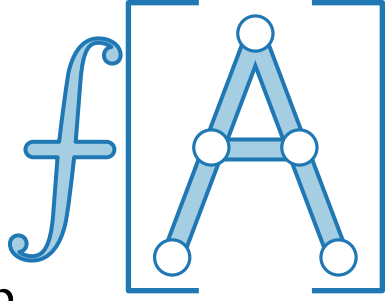
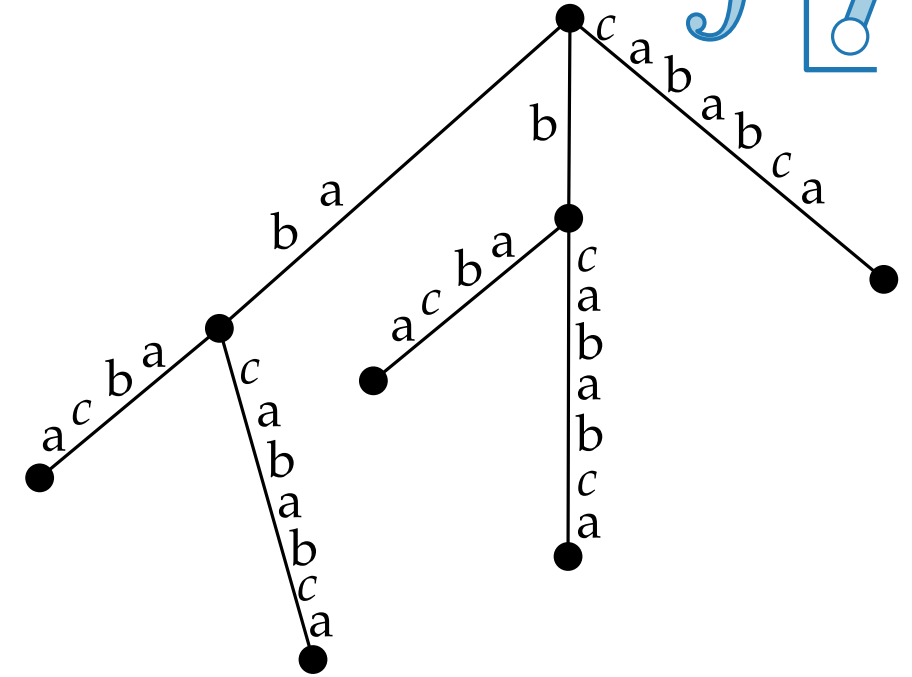
Idee. Repräsentiere T als Suchbaum.



Suffixbäume – Definitionen

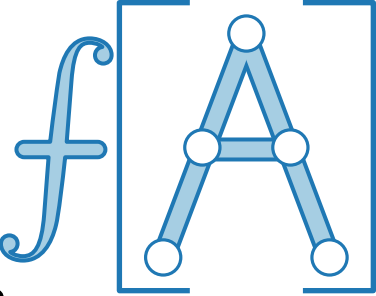
$T = a b c a b a b c a$

Idee. Repräsentiere T als Suchbaum.

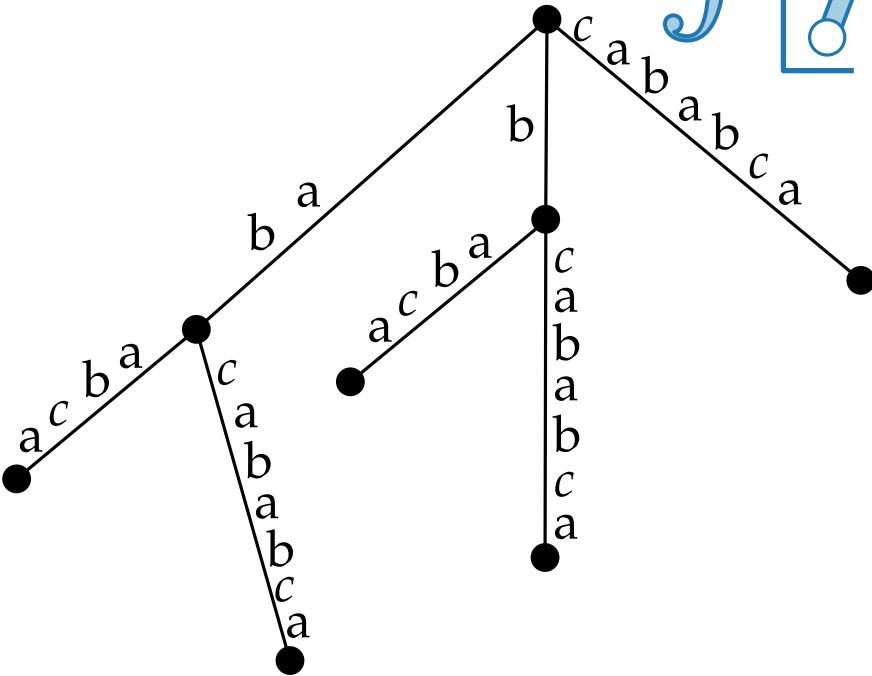


Suffixbäume – Definitionen

$T = a\,b\,c\,a\,b\,a\,b\,c\,a$ $M = \boxed{b\,a\,b}$

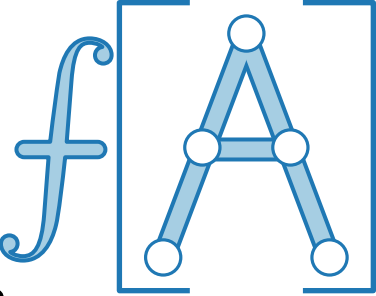


Idee. Repräsentiere T als Suchbaum.

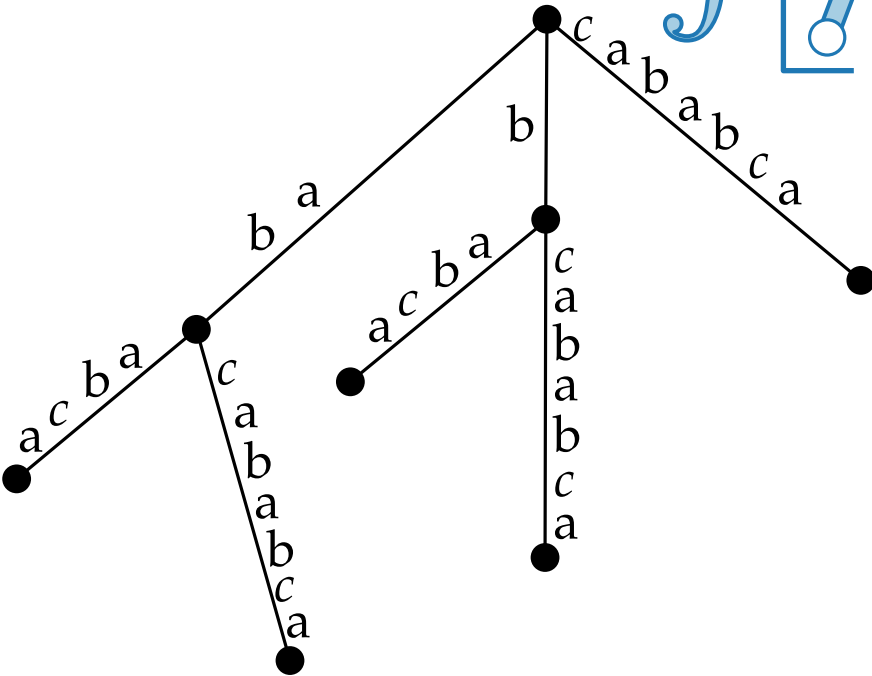


Suffixbäume – Definitionen

$T = a\,b\,c\,a\,\boxed{b\,a\,b}\,c\,a$ $M = \boxed{b\,a\,b}$

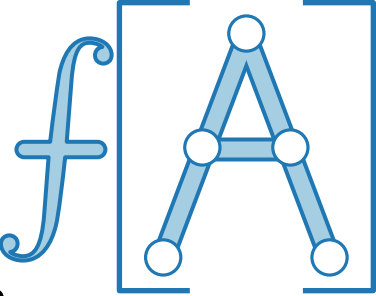


Idee. Repräsentiere T als Suchbaum.

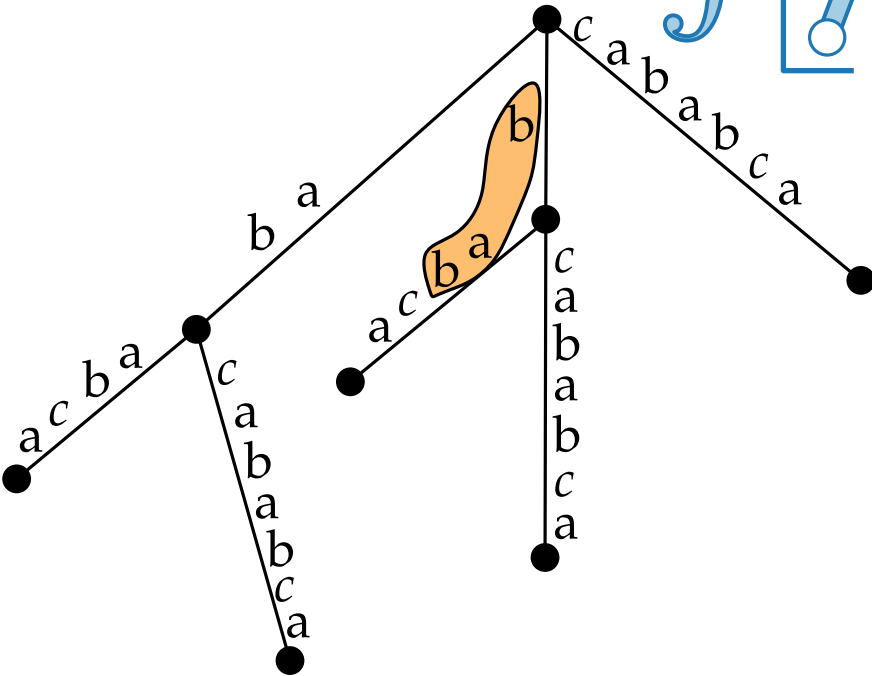


Suffixbäume – Definitionen

$T = a\,b\,c\,a\,\boxed{b\,a\,b}\,c\,a$ $M = \boxed{b\,a\,b}$



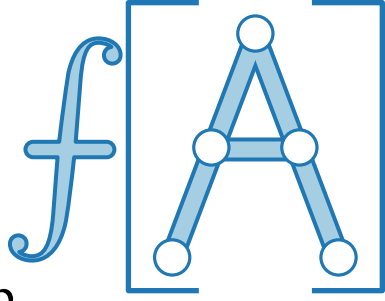
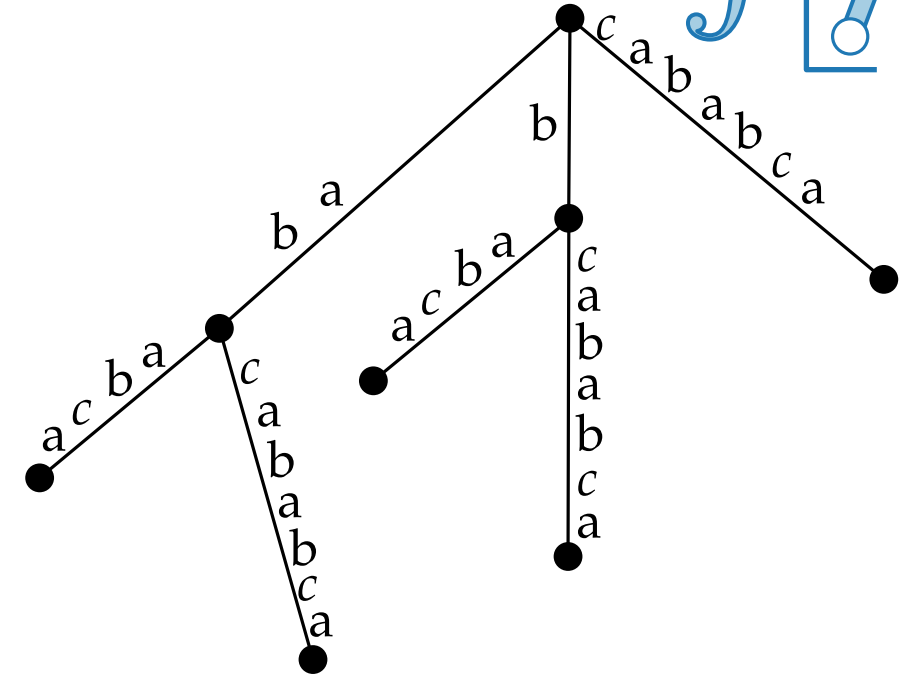
Idee. Repräsentiere T als Suchbaum.



Suffixbäume – Definitionen

$T = a b c a b a b c a$

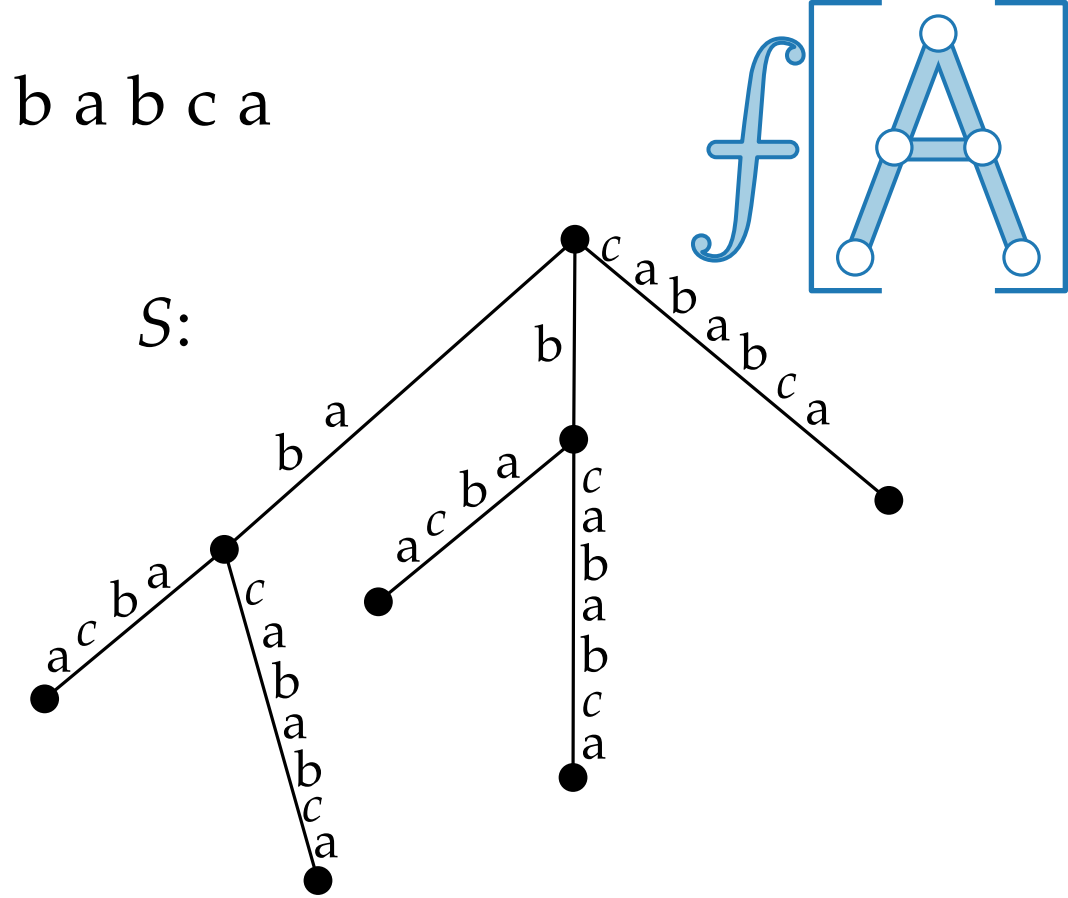
Idee. Repräsentiere T als Suchbaum.



Suffixbäume – Definitionen $T = a b c a b a b c a$

Idee. Repräsentiere T als Suchbaum.

Ein **Σ -Baum** ist ein Wurzelbaum $S = (V, E)$ mit Kantenbeschriftungen aus Σ , so dass für jeden $v \in V$

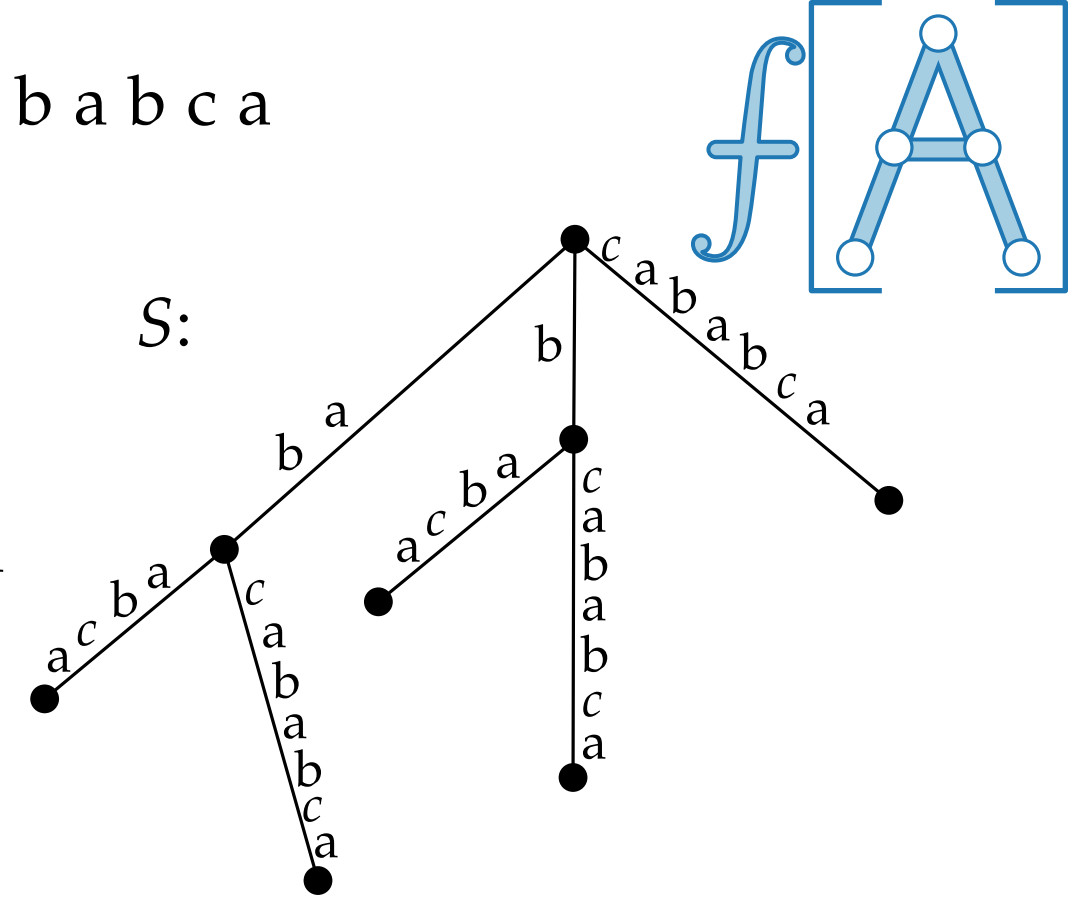


Suffixbäume – Definitionen $T = a b c a b a b c a$

Idee. Repräsentiere T als Suchbaum.

Ein **Σ -Baum** ist ein Wurzelbaum $S = (V, E)$ mit Kantenbeschriftungen aus Σ , so dass für jeden $v \in V$

- Die Beschriftungen der Kanten zu den Kindern von v starten mit unterschiedlichen Elementen von Σ ;



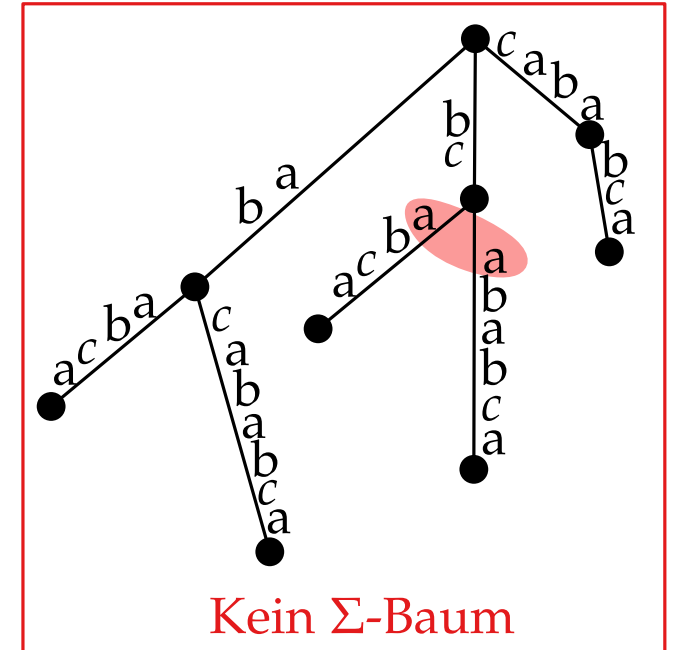
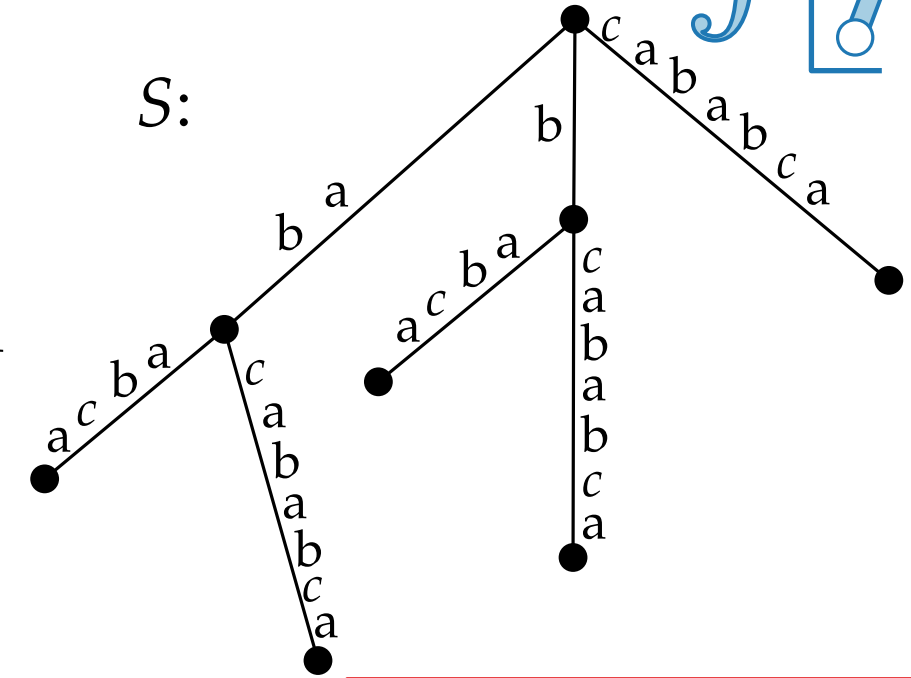
Suffixbäume – Definitionen

$T = a b c a b a b c a$

Idee. Repräsentiere T als Suchbaum.

Ein **Σ -Baum** ist ein Wurzelbaum $S = (V, E)$ mit Kantenbeschriftungen aus Σ , so dass für jeden $v \in V$

- Die Beschriftungen der Kanten zu den Kindern von v starten mit unterschiedlichen Elementen von Σ ;
- wenn v nicht die Wurzel ist, dann ist v ein Blatt oder hat mindestens 2 Kinder



Kein Σ -Baum

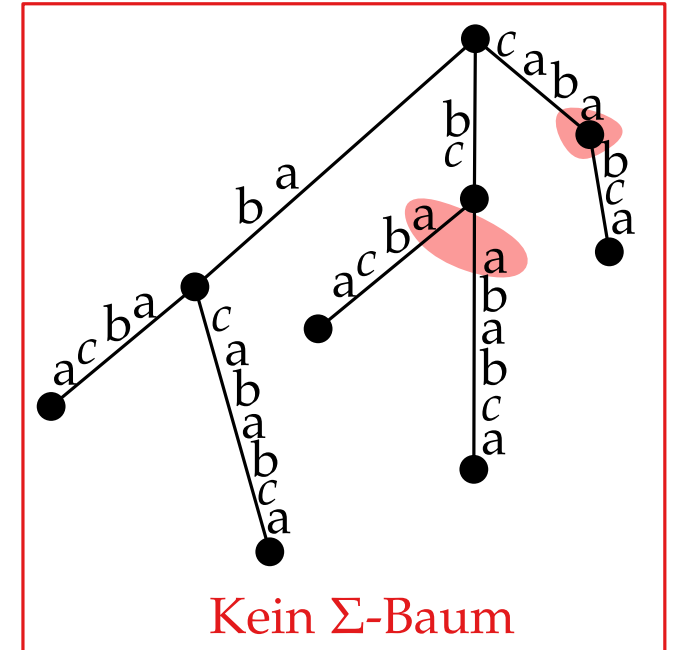
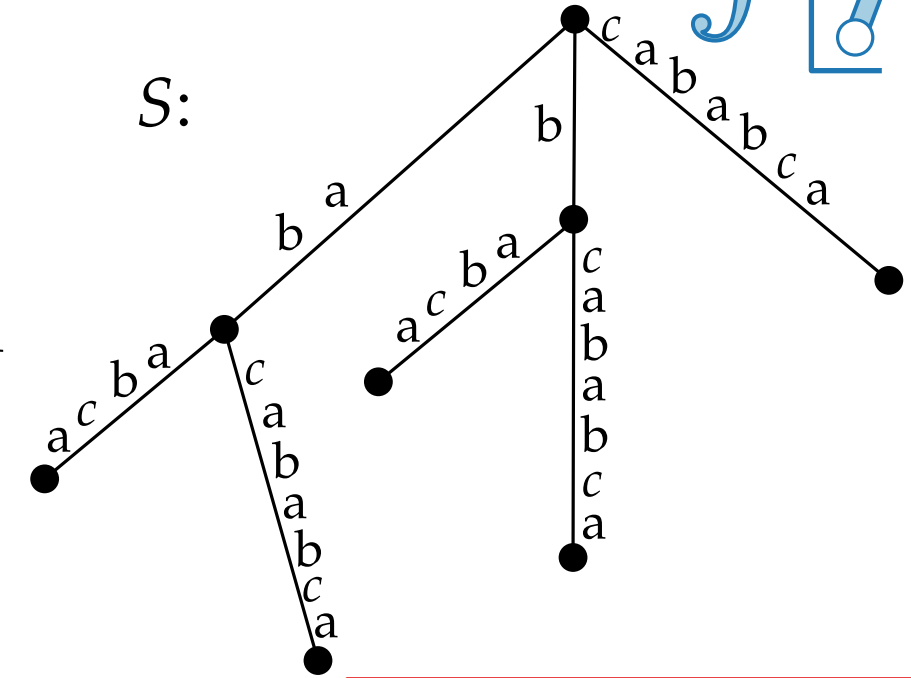
Suffixbäume – Definitionen

$T = a b c a b a b c a$

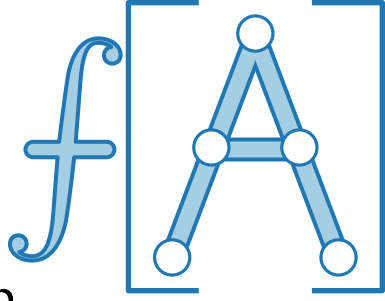
Idee. Repräsentiere T als Suchbaum.

Ein **Σ -Baum** ist ein Wurzelbaum $S = (V, E)$ mit Kantenbeschriftungen aus Σ , so dass für jeden $v \in V$

- Die Beschriftungen der Kanten zu den Kindern von v starten mit unterschiedlichen Elementen von Σ ;
- wenn v nicht die Wurzel ist, dann ist v ein Blatt oder hat mindestens 2 Kinder



Kein Σ -Baum



Suffixbäume – Definitionen

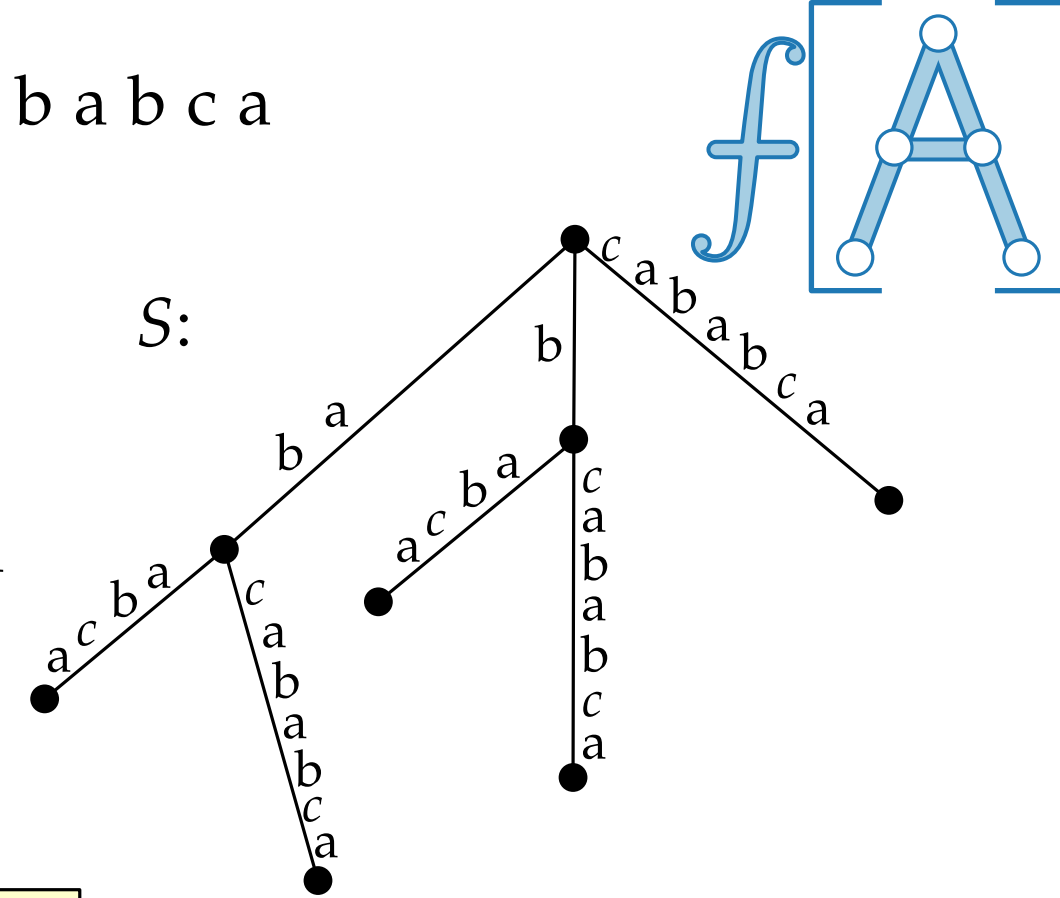
$T = a b c a b a b c a$

Idee. Repräsentiere T als Suchbaum.

Ein **Σ -Baum** ist ein Wurzelbaum $S = (V, E)$ mit Kantenbeschriftungen aus Σ , so dass für jeden $v \in V$

- Die Beschriftungen der Kanten zu den Kindern von v starten mit unterschiedlichen Elementen von Σ ;
- wenn v nicht die Wurzel ist, dann ist v ein Blatt oder hat mindestens 2 Kinder

Def. ■ $\bar{v}$ = Verkettung der Beschriftungen auf dem Pfad von der Wurzel zu v ;



Suffixbäume – Definitionen

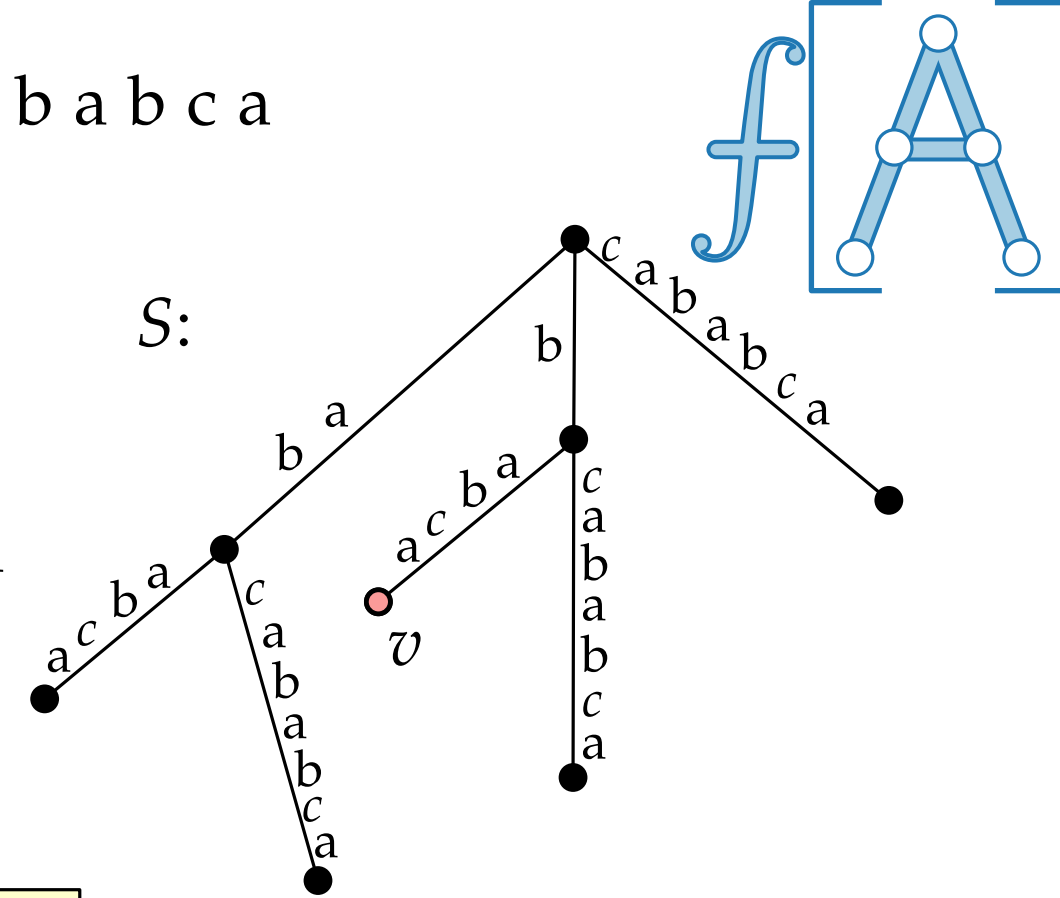
$T = a b c a b a b c a$

Idee. Repräsentiere T als Suchbaum.

Ein **Σ -Baum** ist ein Wurzelbaum $S = (V, E)$ mit Kantenbeschriftungen aus Σ , so dass für jeden $v \in V$

- Die Beschriftungen der Kanten zu den Kindern von v starten mit unterschiedlichen Elementen von Σ ;
- wenn v nicht die Wurzel ist, dann ist v ein Blatt oder hat mindestens 2 Kinder

Def. ■ $\bar{v}$ = Verkettung der Beschriftungen auf dem Pfad von der Wurzel zu v ;



Suffixbäume – Definitionen

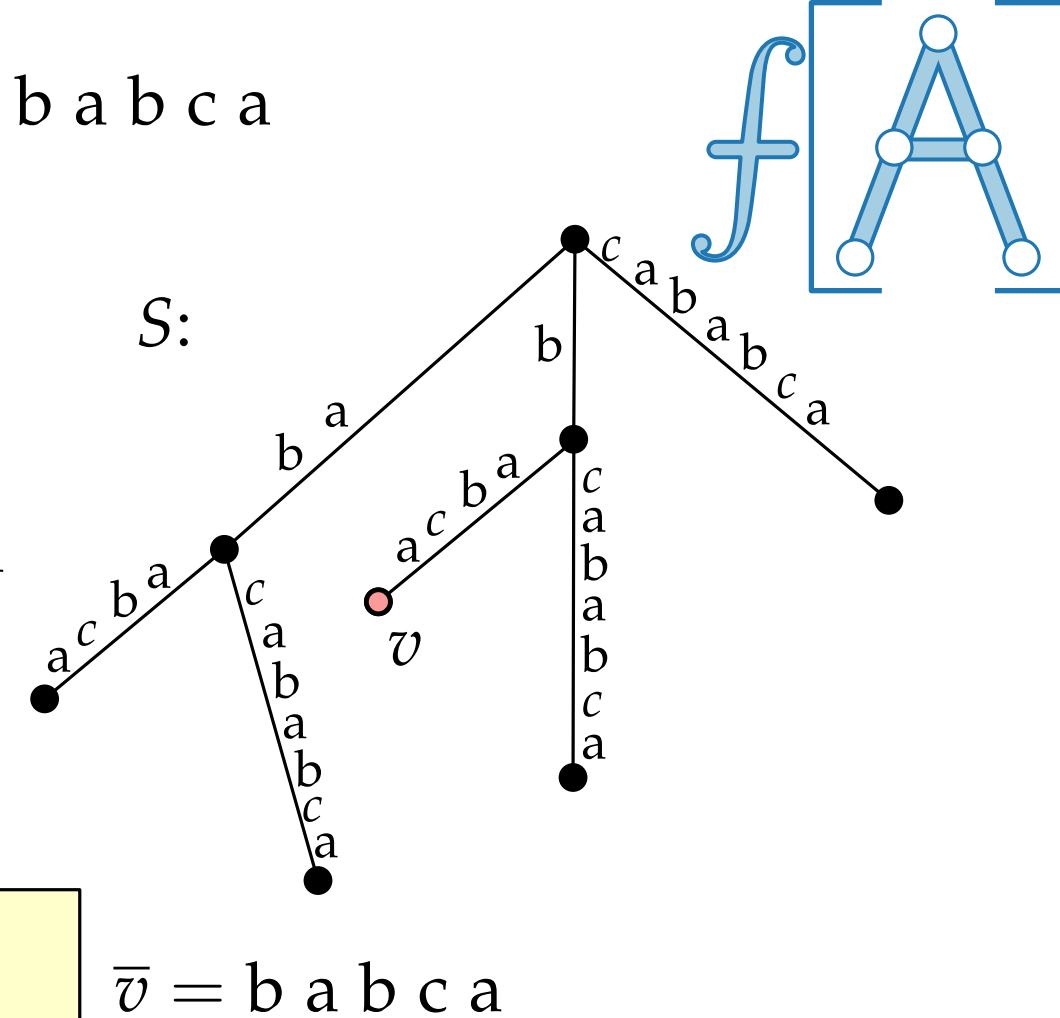
$T = a b c a b a b c a$

Idee. Repräsentiere T als Suchbaum.

Ein **Σ -Baum** ist ein Wurzelbaum $S = (V, E)$ mit Kantenbeschriftungen aus Σ , so dass für jeden $v \in V$

- Die Beschriftungen der Kanten zu den Kindern von v starten mit unterschiedlichen Elementen von Σ ;
- wenn v nicht die Wurzel ist, dann ist v ein Blatt oder hat mindestens 2 Kinder

Def. ■ $\bar{v}$ = Verkettung der Beschriftungen auf dem Pfad von der Wurzel zu v ;



Suffixbäume – Definitionen

$T = a b c a b a b c a$

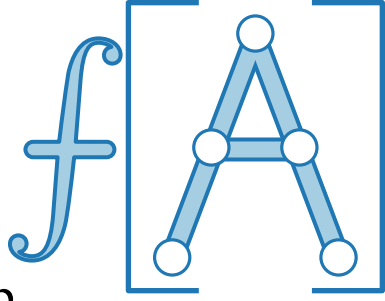
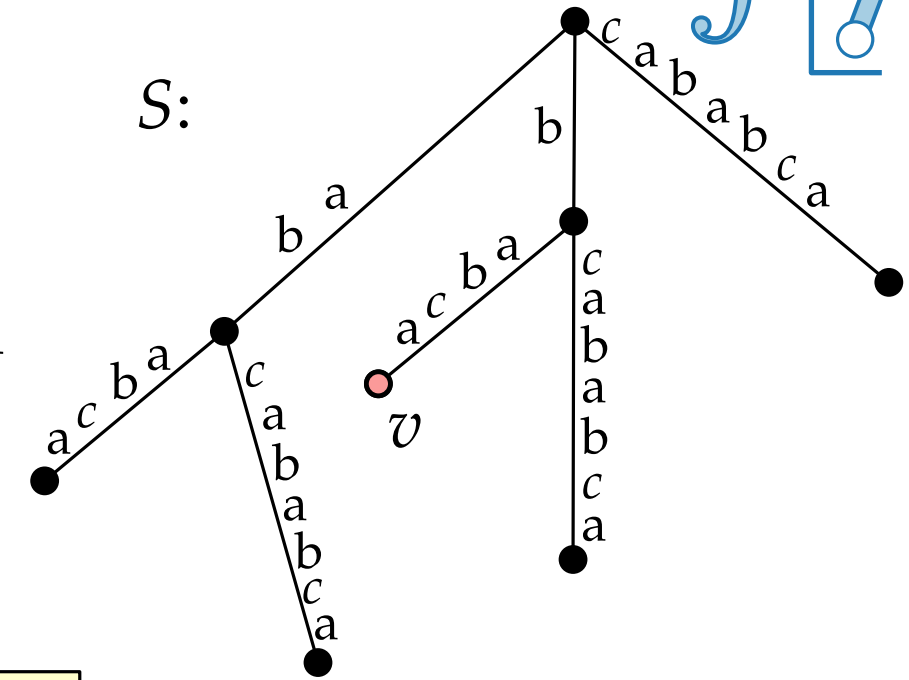
Idee. Repräsentiere T als Suchbaum.

Ein **Σ -Baum** ist ein Wurzelbaum $S = (V, E)$ mit Kantenbeschriftungen aus Σ , so dass für jeden $v \in V$

- Die Beschriftungen der Kanten zu den Kindern von v starten mit unterschiedlichen Elementen von Σ ;
- wenn v nicht die Wurzel ist, dann ist v ein Blatt oder hat mindestens 2 Kinder

Def.

- $\bar{v}$ = Verkettung der Beschriftungen auf dem Pfad von der Wurzel zu v ;
- $d(v) = |\bar{v}|$ ist die **Stringtiefe** von v ;



Suffixbäume – Definitionen

$T = a b c a b a b c a$

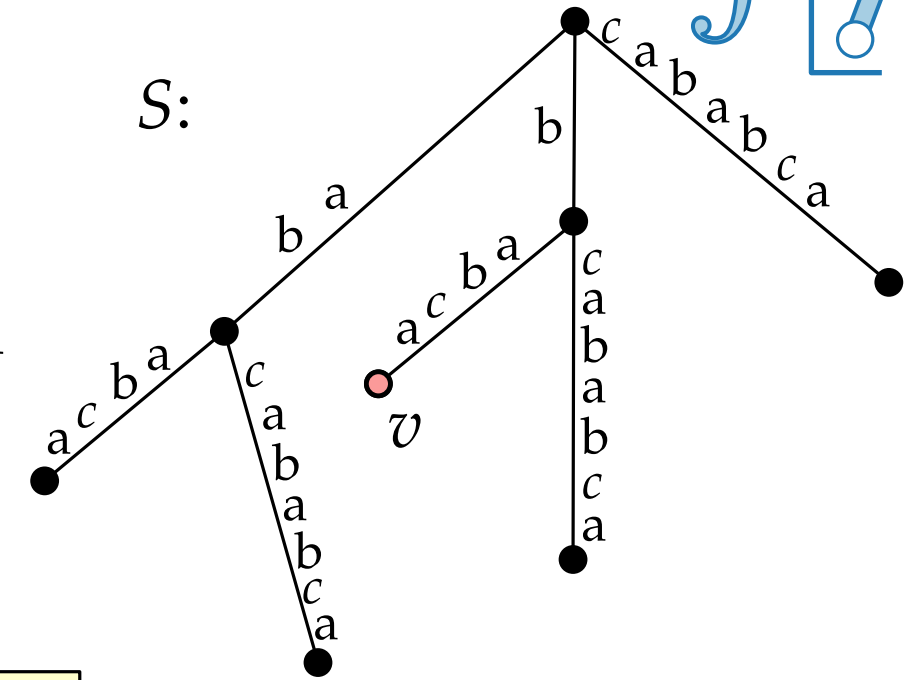
Idee. Repräsentiere T als Suchbaum.

Ein **Σ -Baum** ist ein Wurzelbaum $S = (V, E)$ mit Kantenbeschriftungen aus Σ , so dass für jeden $v \in V$

- Die Beschriftungen der Kanten zu den Kindern von v starten mit unterschiedlichen Elementen von Σ ;
- wenn v nicht die Wurzel ist, dann ist v ein Blatt oder hat mindestens 2 Kinder

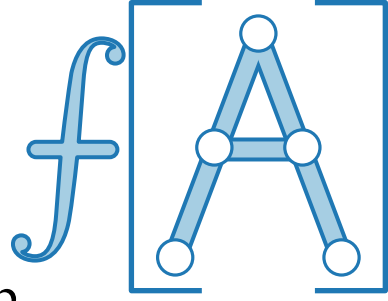
Def. ■ $\bar{v}$ = Verkettung der Beschriftungen auf dem Pfad von der Wurzel zu v ;

- $d(v) = |\bar{v}|$ ist die **Stringtiefe** von v ;



$\bar{v} = b a b c a$

$d(v) = |\bar{v}| = 5$



Suffixbäume – Definitionen $T = a b c a b a b c a$

$$T = a \, b \, c \, a \, b \, a \, b \, c \, a$$

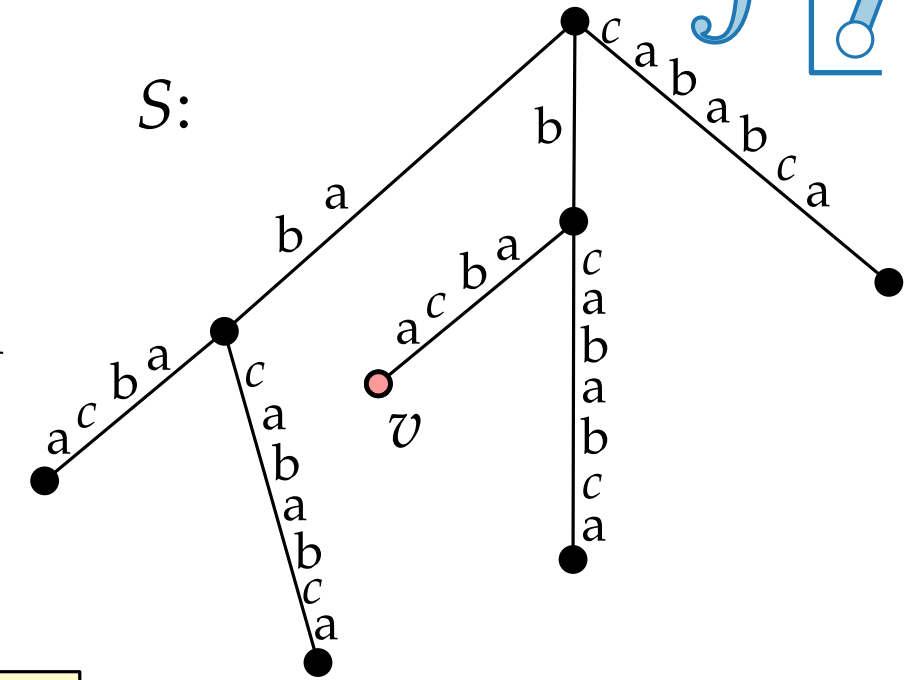
Idee. Repräsentiere T als Suchbaum.

Ein **Σ -Baum** ist ein Wurzelbaum $S = (V, E)$ mit Kantenbeschriftungen aus Σ , so dass für jeden $v \in V$

- Die Beschriftungen der Kanten zu den Kindern von v starten mit unterschiedlichen Elementen von Σ ;
- wenn v nicht die Wurzel ist, dann ist v ein Blatt oder hat mindestens 2 Kinder

Def. ■ $\bar{v}$ = Verkettung der Beschriftungen auf dem Pfad von der Wurzel zu v ;

- $d(v) = |\bar{v}|$ ist die **Stringtiefe** von v ;
- S **enthält** einen String α , wenn es ein $v \in V$ und einen String β gibt, so dass $\bar{v} = \alpha\beta$;



$$\overline{v} = \mathbf{b} \mathbf{a} \mathbf{b} \mathbf{c} \mathbf{a}$$

$$d(v) = |\overline{v}| = 5$$

Suffixbäume – Definitionen

$T = a b c a b a b c a$

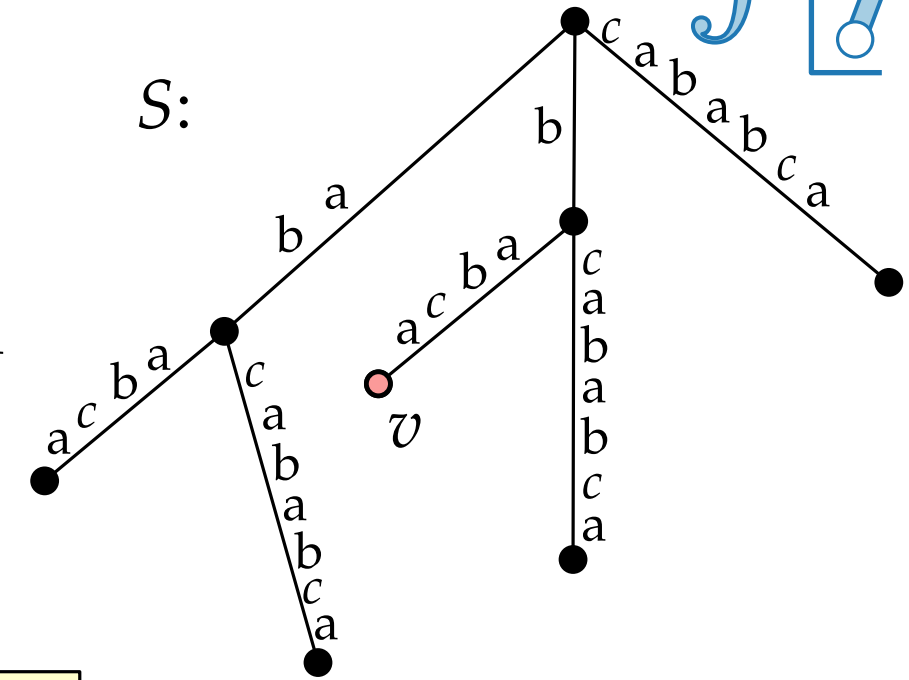
Idee. Repräsentiere T als Suchbaum.

Ein **Σ -Baum** ist ein Wurzelbaum $S = (V, E)$ mit Kantenbeschriftungen aus Σ , so dass für jeden $v \in V$

- Die Beschriftungen der Kanten zu den Kindern von v starten mit unterschiedlichen Elementen von Σ ;
- wenn v nicht die Wurzel ist, dann ist v ein Blatt oder hat mindestens 2 Kinder

Def.

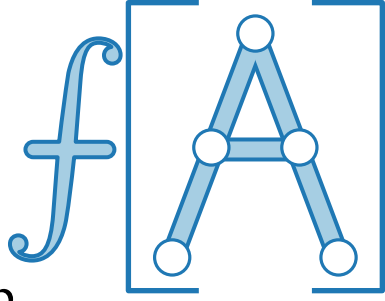
- $\bar{v}$ = Verkettung der Beschriftungen auf dem Pfad von der Wurzel zu v ;
- $d(v) = |\bar{v}|$ ist die **Stringtiefe** von v ;
- S **enthält** einen String α , wenn es ein $v \in V$ und einen String β gibt, so dass $\bar{v} = \alpha\beta$;



$\bar{v} = b a b c a$

$d(v) = |\bar{v}| = 5$

S enthält $\alpha =$ $b a b$



The diagram shows a tree structure S with nodes and edges labeled with letters a , b , and c . A specific path is highlighted in orange and labeled α . A red node is labeled v . A blue icon of a magnifying glass is in the top right corner.

$$\overline{v} = \mathbf{b} \, \mathbf{a} \, \mathbf{b} \, \mathbf{c} \, \mathbf{a}$$
$$d(v) = |\overline{v}| = 5$$

S enthält $a = b a b$

Suffixbäume – Definitionen $T = a b c a b a b c a$

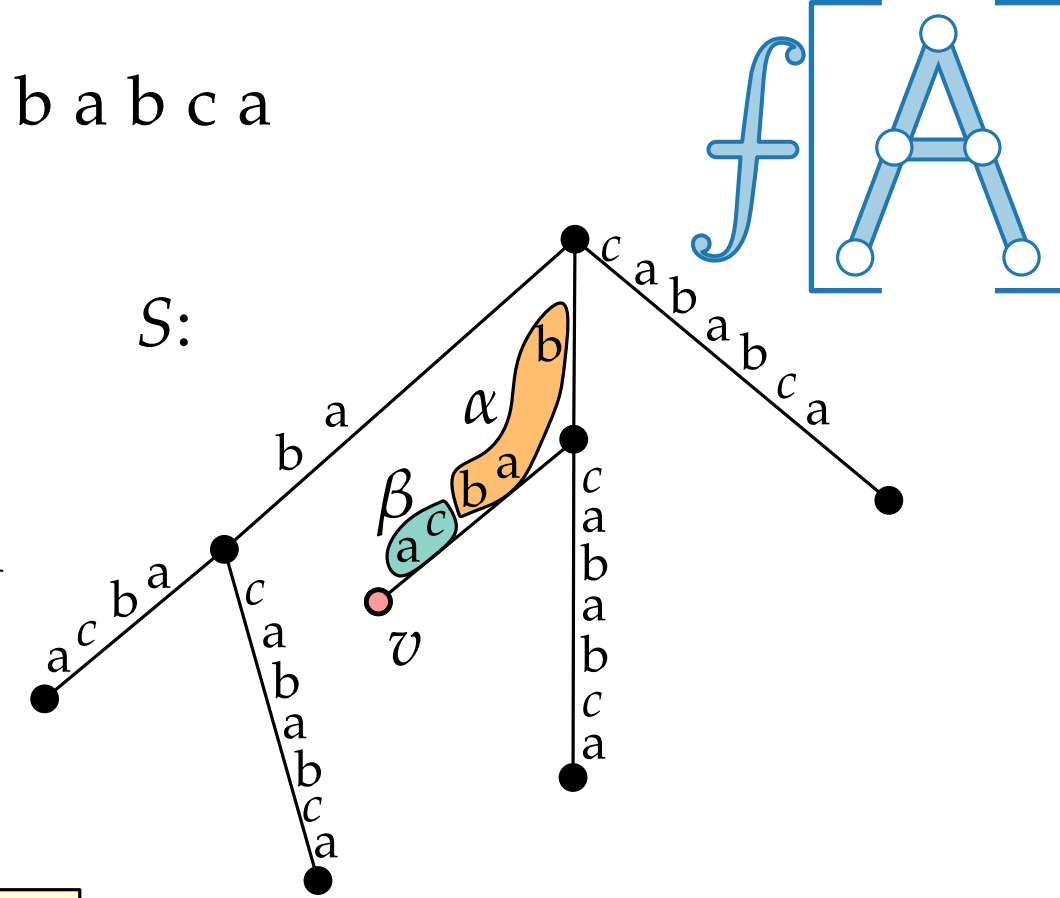
Idee. Repräsentiere T als Suchbaum.

Ein **Σ -Baum** ist ein Wurzelbaum $S = (V, E)$ mit Kantenbeschriftungen aus Σ , so dass für jeden $v \in V$

- Die Beschriftungen der Kanten zu den Kindern von v starten mit unterschiedlichen Elementen von Σ ;
- wenn v nicht die Wurzel ist, dann ist v ein Blatt oder hat mindestens 2 Kinder

Def. ■ $\bar{v}$ = Verkettung der Beschriftungen auf dem Pfad von der Wurzel zu v ;

- $d(v) = |\bar{v}|$ ist die **Stringtiefe** von v ;
- S **enthält** einen String α , wenn es ein $v \in V$ und einen String β gibt, so dass $\bar{v} = \alpha\beta$;



$$\overline{v} = \mathbf{b} \, \mathbf{a} \, \mathbf{b} \, \mathbf{c} \, \mathbf{a}$$

$$d(v) = |\overline{v}| = 5$$

S enthält $\alpha = \mathbf{b\ a\ b}$, weil
 $\bar{v} = \alpha\beta$ mit $\beta = \mathbf{c\ a}$.

Suffixbäume – Definitionen $T = a b c a b a b c a$

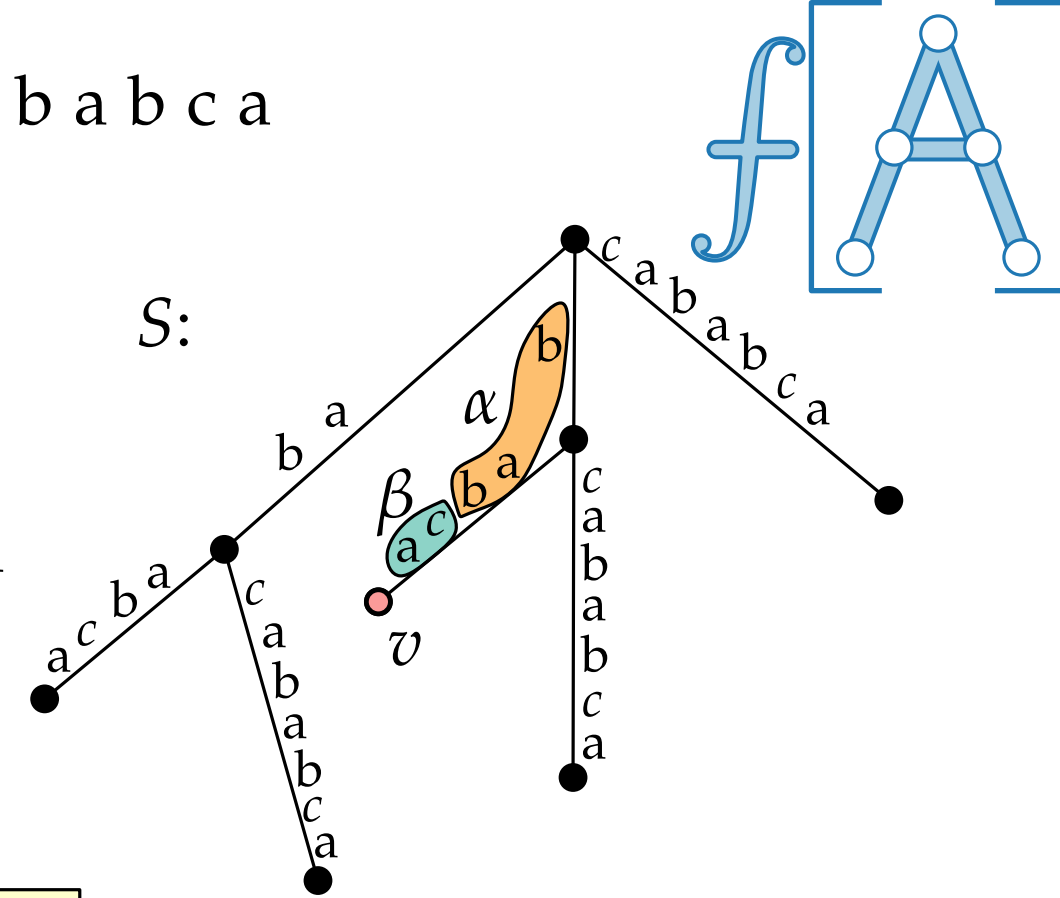
Idee. Repräsentiere T als Suchbaum.

Ein **Σ -Baum** ist ein Wurzelbaum $S = (V, E)$ mit Kantenbeschriftungen aus Σ , so dass für jeden $v \in V$

- Die Beschriftungen der Kanten zu den Kindern von v starten mit unterschiedlichen Elementen von Σ ;
- wenn v nicht die Wurzel ist, dann ist v ein Blatt oder hat mindestens 2 Kinder

Def. ■ $\bar{v}$ = Verkettung der Beschriftungen auf dem Pfad von der Wurzel zu v ;

- $d(v) = |\bar{v}|$ ist die **Stringtiefe** von v ;
- S **enthält** einen String α , wenn es ein $v \in V$ und einen String β gibt, so dass $\bar{v} = \alpha\beta$;
- $\text{wörter}(S) =$ Menge aller Strings, die in S enthalten sind.

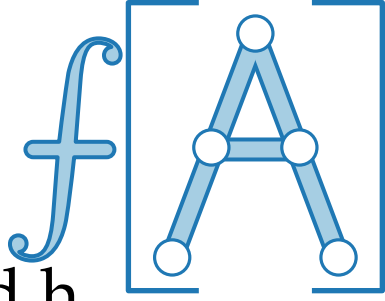


$$\overline{v} = \mathbf{b} \, \mathbf{a} \, \mathbf{b} \, \mathbf{c} \, \mathbf{a}$$

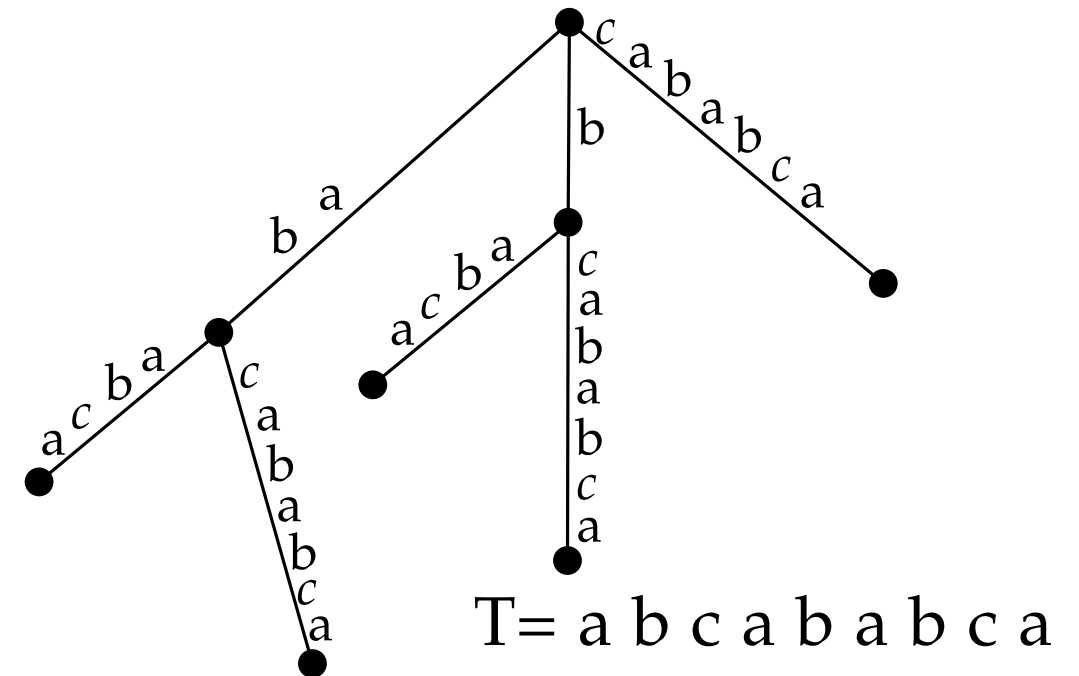
$$d(v) = |\overline{v}| = 5$$

S enthält $\alpha = \mathbf{b\ a\ b}$, weil
 $\bar{v} = \alpha\beta$ mit $\beta = \mathbf{c\ a}$.

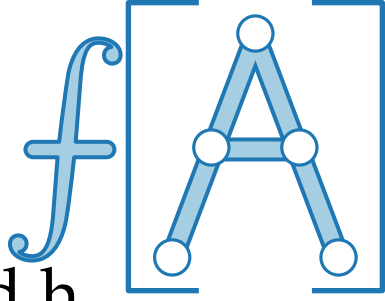
Suffixbäume – Struktur



Ein **Suffixbaum** S von T ist ein Σ -Baum, der genau die Infixe von T enthält, d.h.,
 $\text{wörter}(S) = \{T[i, j] \mid 1 \leq i \leq j \leq n\}$.

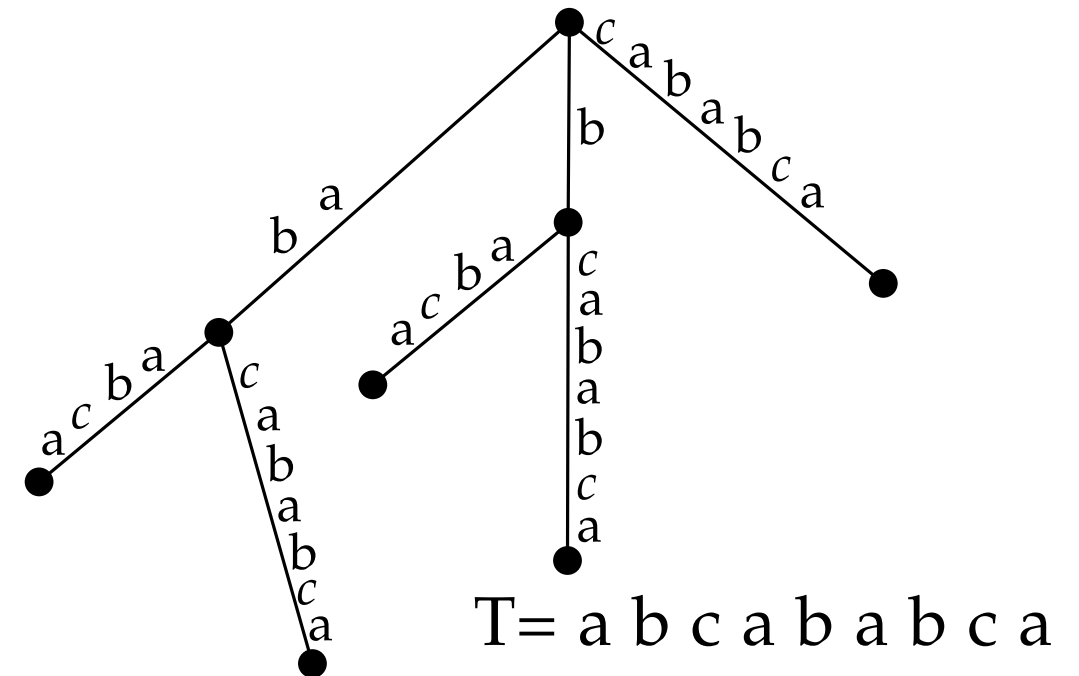


Suffixbäume – Struktur

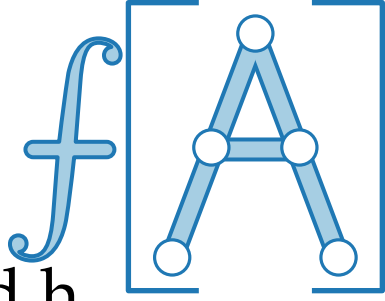


Ein **Suffixbaum** S von T ist ein Σ -Baum, der genau die Infixe von T enthält, d.h.,
 $\text{wörter}(S) = \{T[i, j] \mid 1 \leq i \leq j \leq n\}$.

Lemma. Für jedes Blatt v von S ist $\bar{v}$ ein Suffix von T .

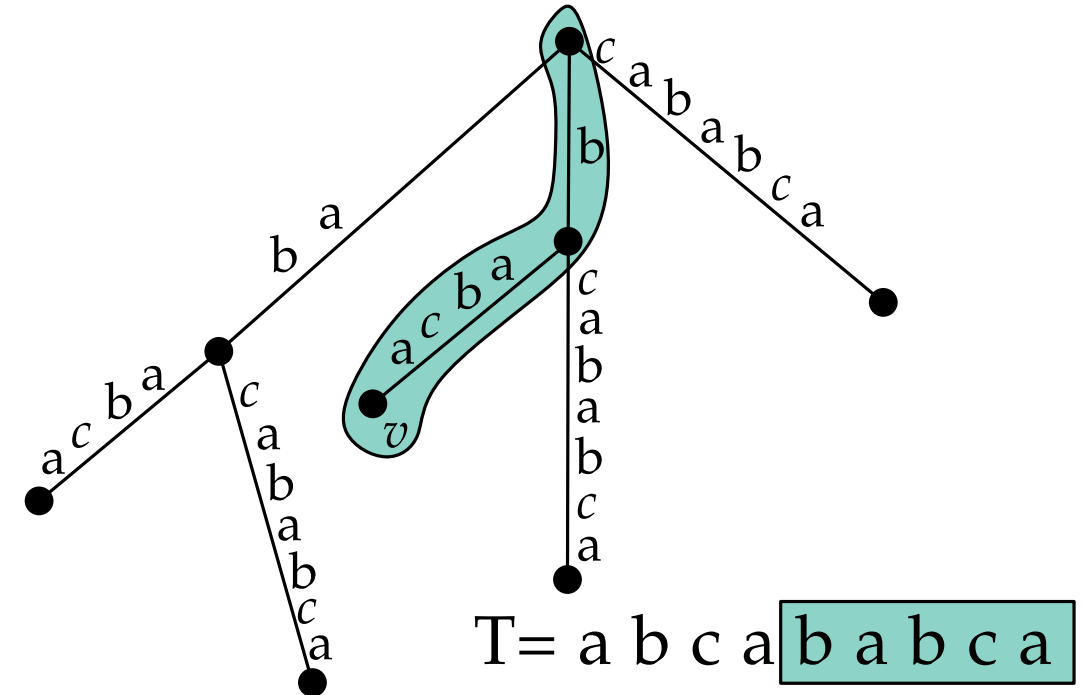


Suffixbäume – Struktur

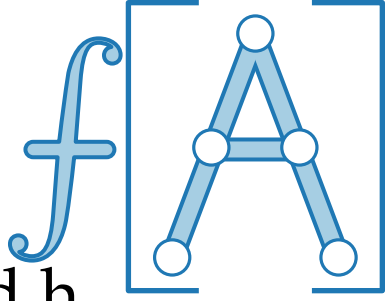


Ein **Suffixbaum** S von T ist ein Σ -Baum, der genau die Infixe von T enthält, d.h.,
 $\text{wörter}(S) = \{T[i, j] \mid 1 \leq i \leq j \leq n\}$.

Lemma. Für jedes Blatt v von S ist $\bar{v}$ ein Suffix von T .



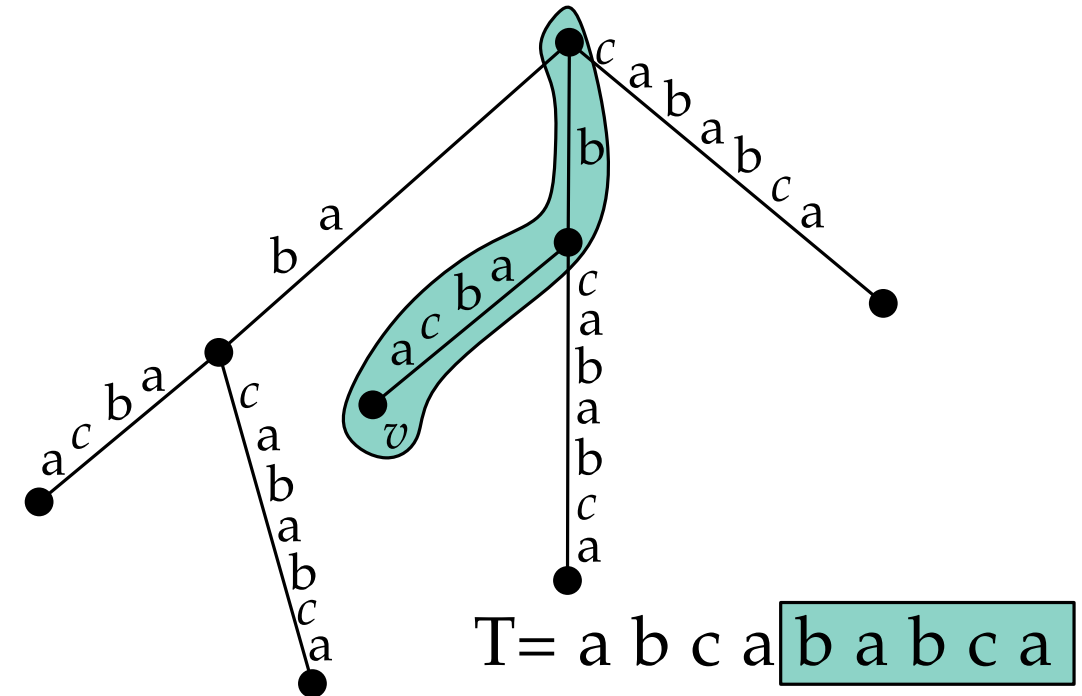
Suffixbäume – Struktur



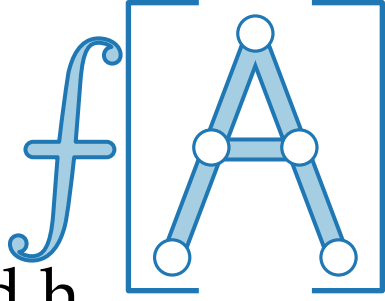
Ein **Suffixbaum** S von T ist ein Σ -Baum, der genau die Infixe von T enthält, d.h.,
 $\text{wörter}(S) = \{T[i, j] \mid 1 \leq i \leq j \leq n\}$.

Lemma. Für jedes Blatt v von S ist $\bar{v}$ ein Suffix von T .

Beweis. Angenommen, $\bar{v} = T[i, j]$ mit $j < n$.



Suffixbäume – Struktur

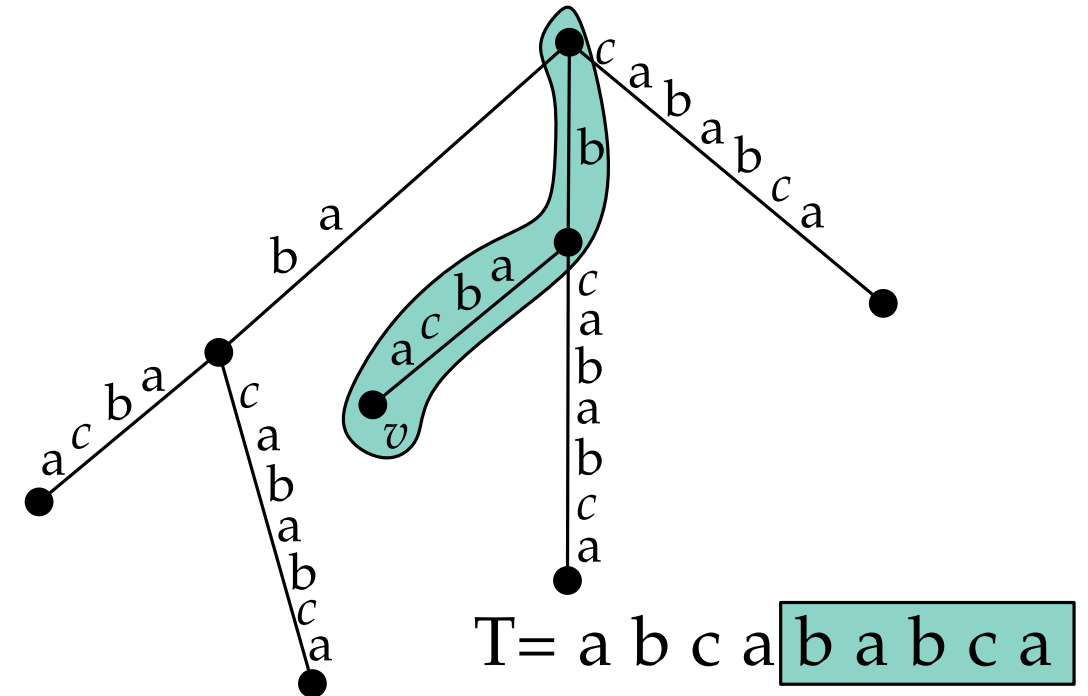


Ein **Suffixbaum** S von T ist ein Σ -Baum, der genau die Infixe von T enthält, d.h.,
 $\text{wörter}(S) = \{T[i, j] \mid 1 \leq i \leq j \leq n\}$.

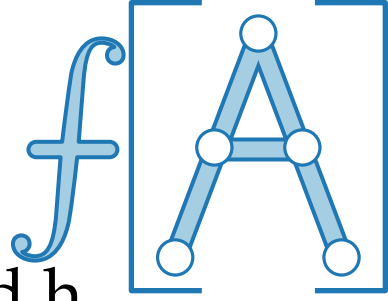
Lemma. Für jedes Blatt v von S ist $\bar{v}$ ein Suffix von T .

Beweis. Angenommen, $\bar{v} = T[i, j]$ mit $j < n$.

$\bar{v}$ ist ein Präfix von $T[i, n]$. Sei u ein Knoten, so dass $T[i, n]$ ein Präfix von $\bar{u}$ ist.



Suffixbäume – Struktur



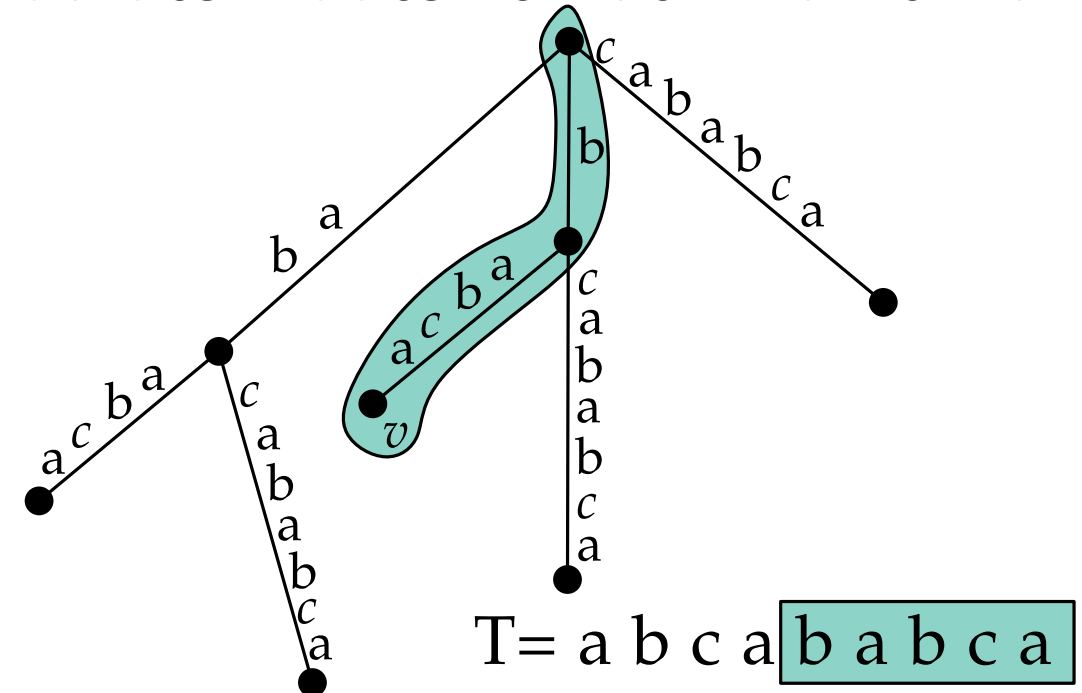
Ein **Suffixbaum** S von T ist ein Σ -Baum, der genau die Infixe von T enthält, d.h.,
 $\text{wörter}(S) = \{T[i, j] \mid 1 \leq i \leq j \leq n\}$.

Lemma. Für jedes Blatt v von S ist $\bar{v}$ ein Suffix von T .

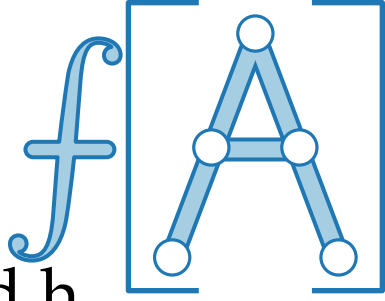
Beweis. Angenommen, $\bar{v} = T[i, j]$ mit $j < n$.

$\bar{v}$ ist ein Präfix von $T[i, n]$. Sei u ein Knoten, so dass $T[i, n]$ ein Präfix von $\bar{u}$ ist.

→ Der Pfad von der Wurzel zu v ist ein Teilpfad des Pfades von der Wurzel zu u .



Suffixbäume – Struktur



Ein **Suffixbaum** S von T ist ein Σ -Baum, der genau die Infixe von T enthält, d.h.,
 $\text{wörter}(S) = \{T[i, j] \mid 1 \leq i \leq j \leq n\}$.

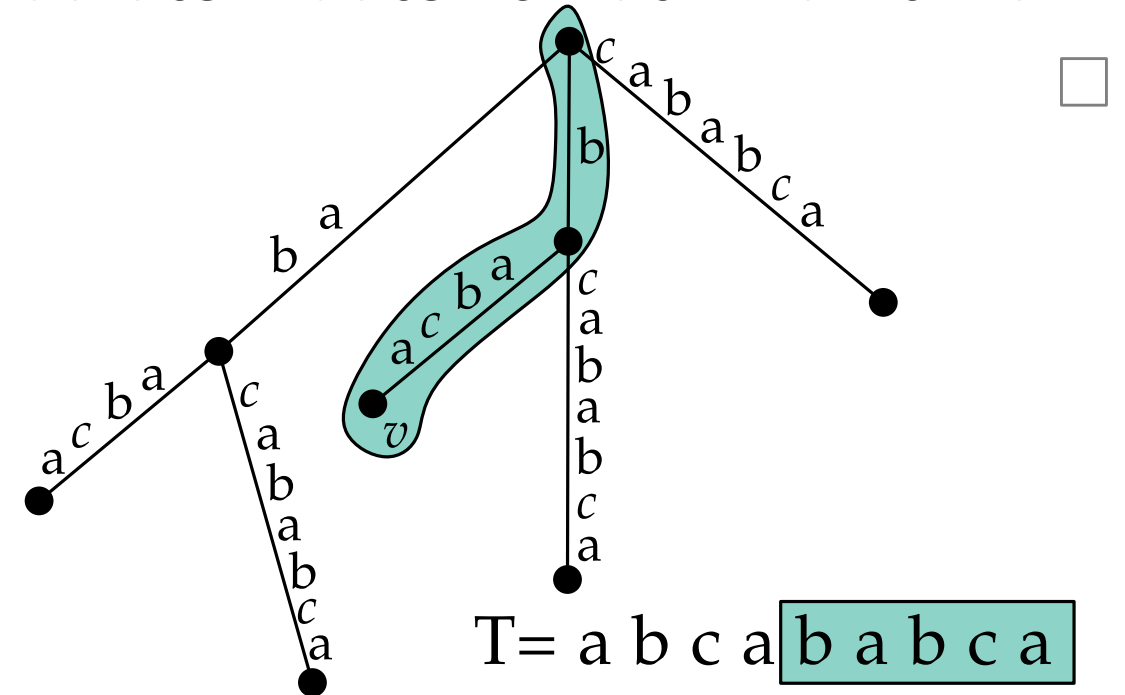
Lemma. Für jedes Blatt v von S ist $\bar{v}$ ein Suffix von T .

Beweis. Angenommen, $\bar{v} = T[i, j]$ mit $j < n$.

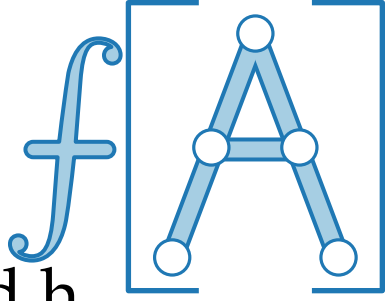
$\bar{v}$ ist ein Präfix von $T[i, n]$. Sei u ein Knoten, so dass $T[i, n]$ ein Präfix von $\bar{u}$ ist.

→ Der Pfad von der Wurzel zu v ist ein Teilpfad des Pfades von der Wurzel zu u .

→ v ist kein Blatt. ⚡

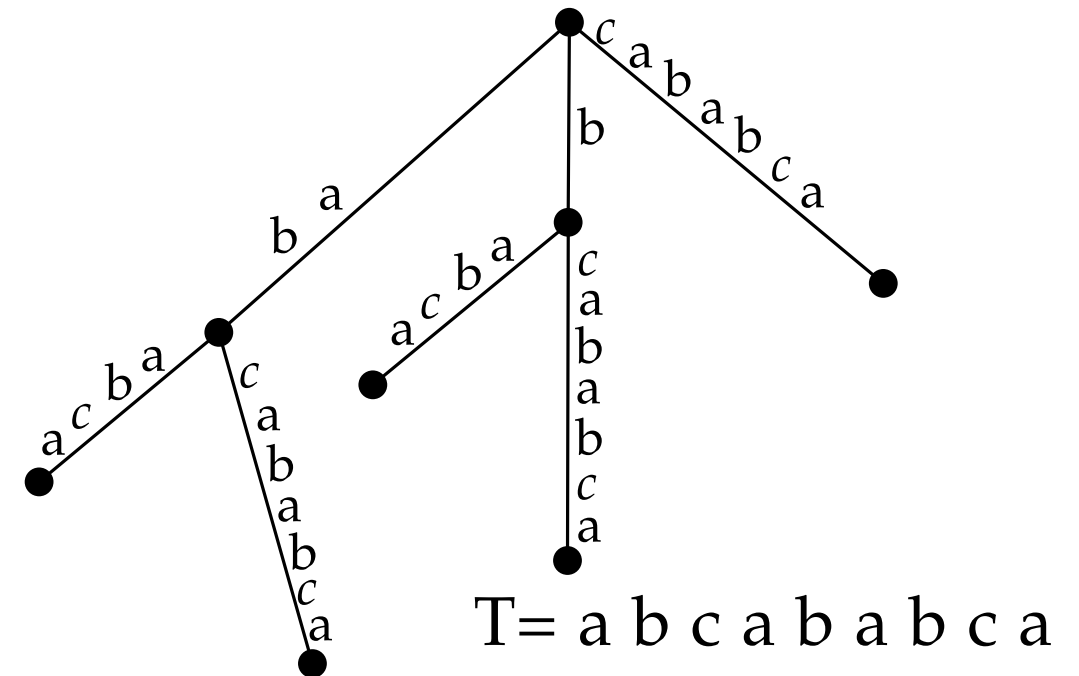


Suffixbäume – Struktur

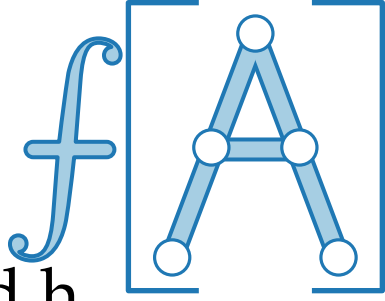


Ein **Suffixbaum** S von T ist ein Σ -Baum, der genau die Infixe von T enthält, d.h.,
 $\text{wörter}(S) = \{T[i, j] \mid 1 \leq i \leq j \leq n\}$.

Lemma. Für jedes Blatt v von S ist $\bar{v}$ ein Suffix von T .



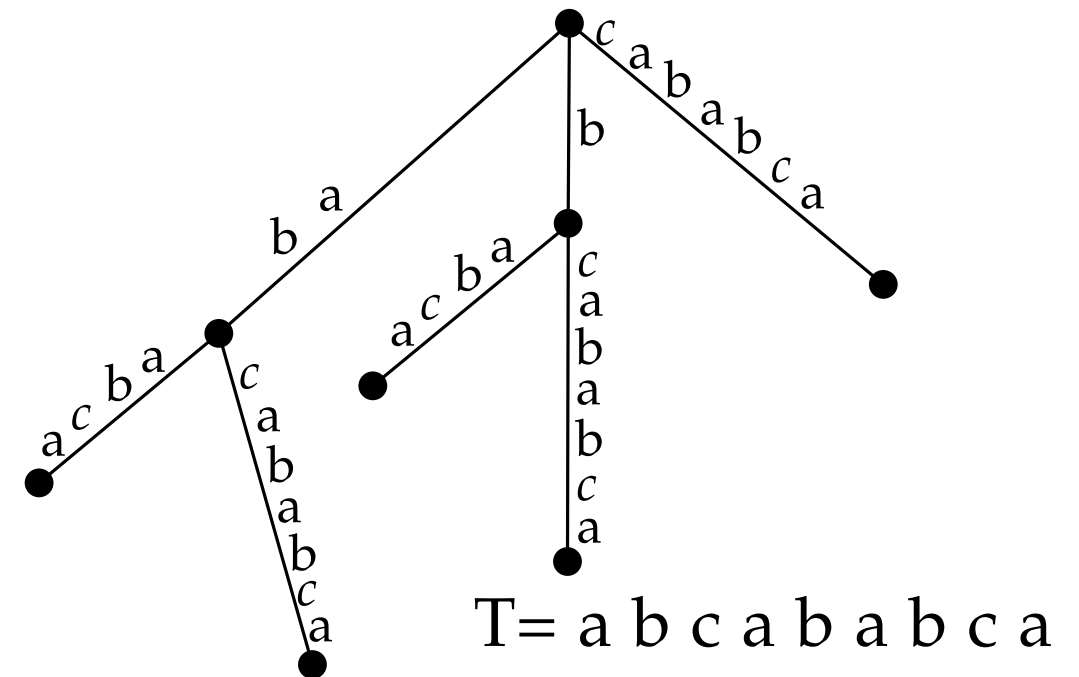
Suffixbäume – Struktur



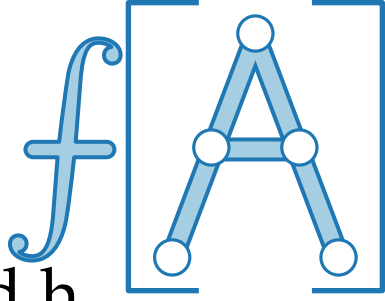
Ein **Suffixbaum** S von T ist ein Σ -Baum, der genau die Infixe von T enthält, d.h.,
 $\text{wörter}(S) = \{T[i, j] \mid 1 \leq i \leq j \leq n\}$.

Lemma. Für jedes Blatt v von S ist $\bar{v}$ ein Suffix von T .

Anmerkung. Die Rückrichtung gilt nicht, weil ein Suffix das Präfix eines anderen Suffix sein kann,



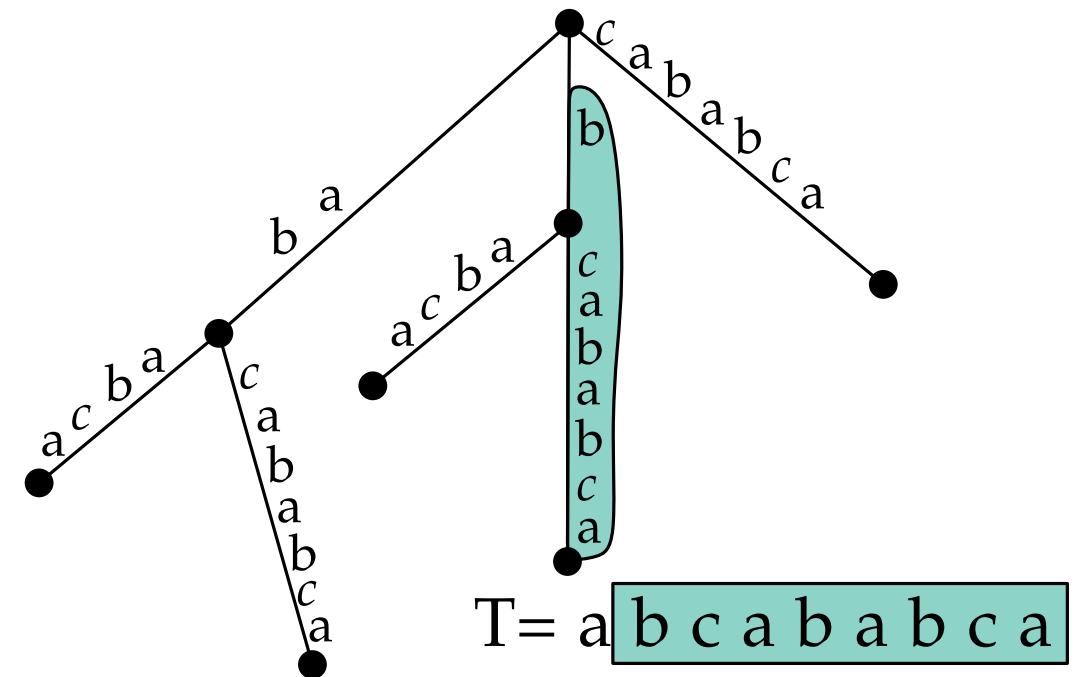
Suffixbäume – Struktur



Ein **Suffixbaum** S von T ist ein Σ -Baum, der genau die Infixe von T enthält, d.h.,
 $\text{wörter}(S) = \{T[i, j] \mid 1 \leq i \leq j \leq n\}$.

Lemma. Für jedes Blatt v von S ist $\bar{v}$ ein Suffix von T .

Anmerkung. Die Rückrichtung gilt nicht, weil ein Suffix das Präfix eines anderen Suffix sein kann,

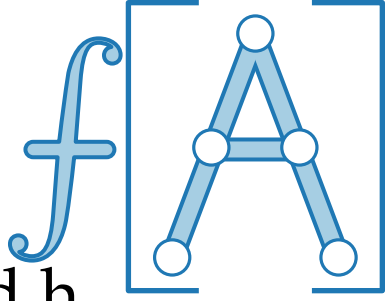


Lemma. Für jedes Blatt v von S ist $\bar{v}$ ein Suffix von T .

The diagram shows a Huffman tree for the string "abcababcbcab". The tree has a root node with three children. The left child has two children of its own, and the middle child has one child. The right child is a leaf node. The leaf nodes contain the characters 'a', 'c', 'b', 'a', 'b', 'c', 'b', 'c', 'a' respectively. The tree structure is as follows: Root () has left child () and right child (). The left child () has left child () and right child (). The left child () has left child () and right child (). The right child () has left child (). The leaf nodes contain the characters 'a', 'c', 'b', 'a', 'b', 'c', 'b', 'c', 'a' respectively.

T = a b c a b a b c a

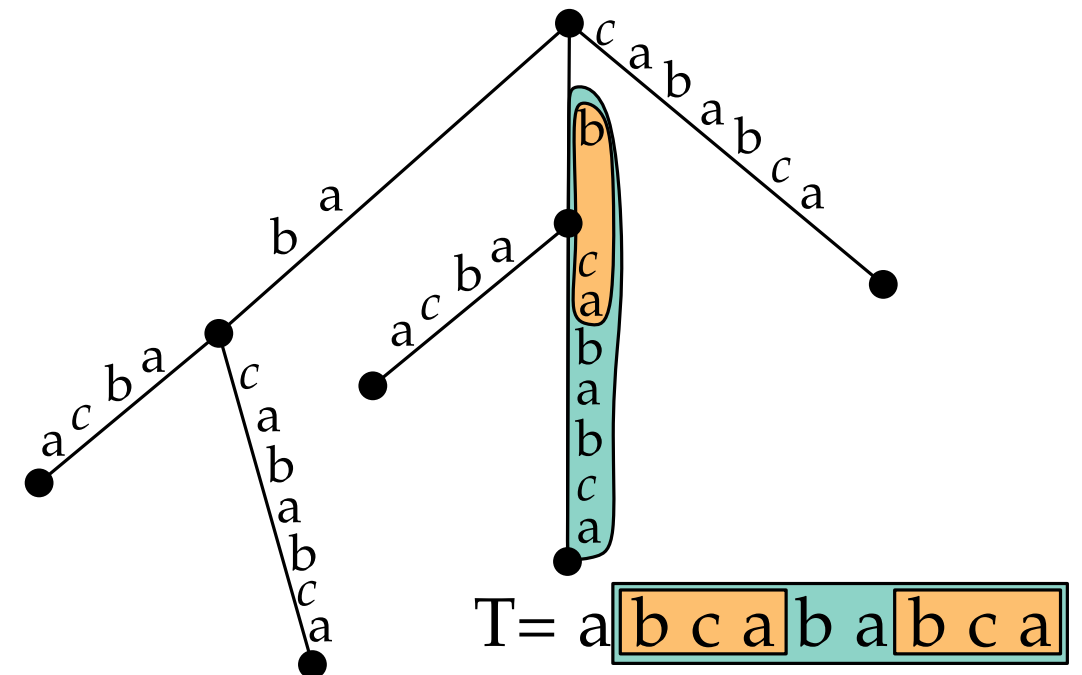
Suffixbäume – Struktur



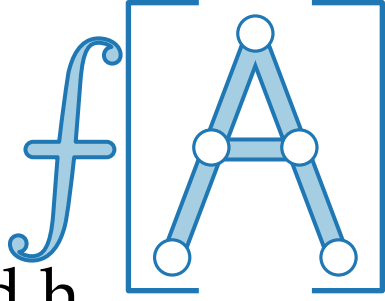
Ein **Suffixbaum** S von T ist ein Σ -Baum, der genau die Infixe von T enthält, d.h.,
 $\text{wörter}(S) = \{T[i, j] \mid 1 \leq i \leq j \leq n\}$.

Lemma. Für jedes Blatt v von S ist $\bar{v}$ ein Suffix von T .

Anmerkung. Die Rückrichtung gilt nicht, weil ein Suffix das Präfix eines anderen Suffix sein kann,



Suffixbäume – Struktur

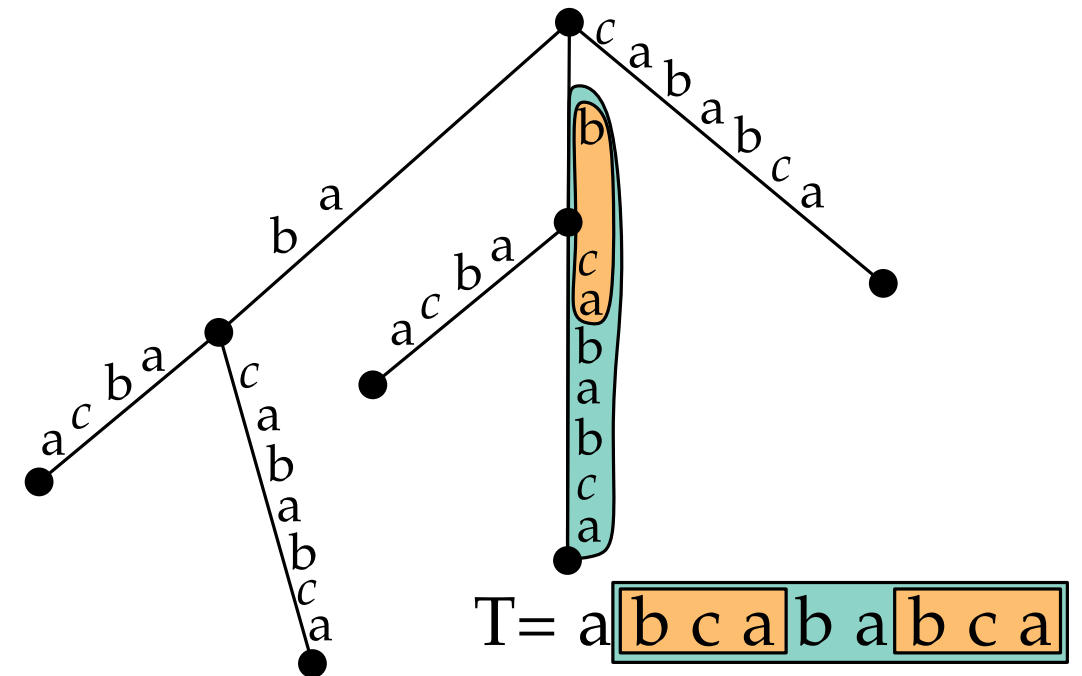


Ein **Suffixbaum** S von T ist ein Σ -Baum, der genau die Infixe von T enthält, d.h.,
 $\text{wörter}(S) = \{T[i, j] \mid 1 \leq i \leq j \leq n\}$.

Lemma. Für jedes Blatt v von S ist $\bar{v}$ ein Suffix von T .

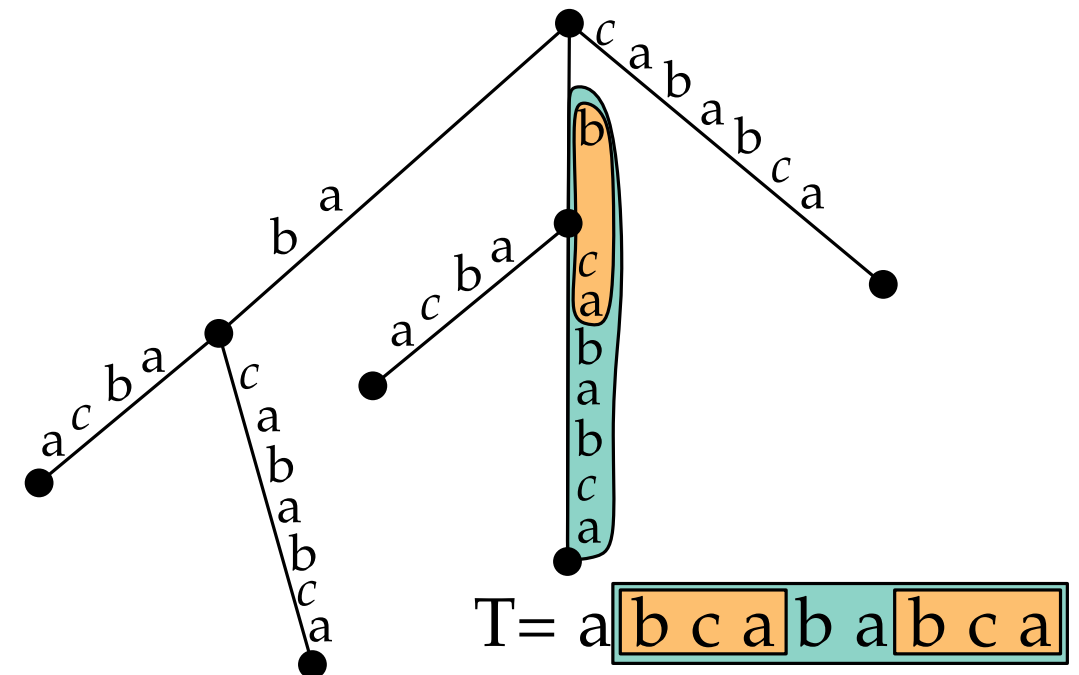
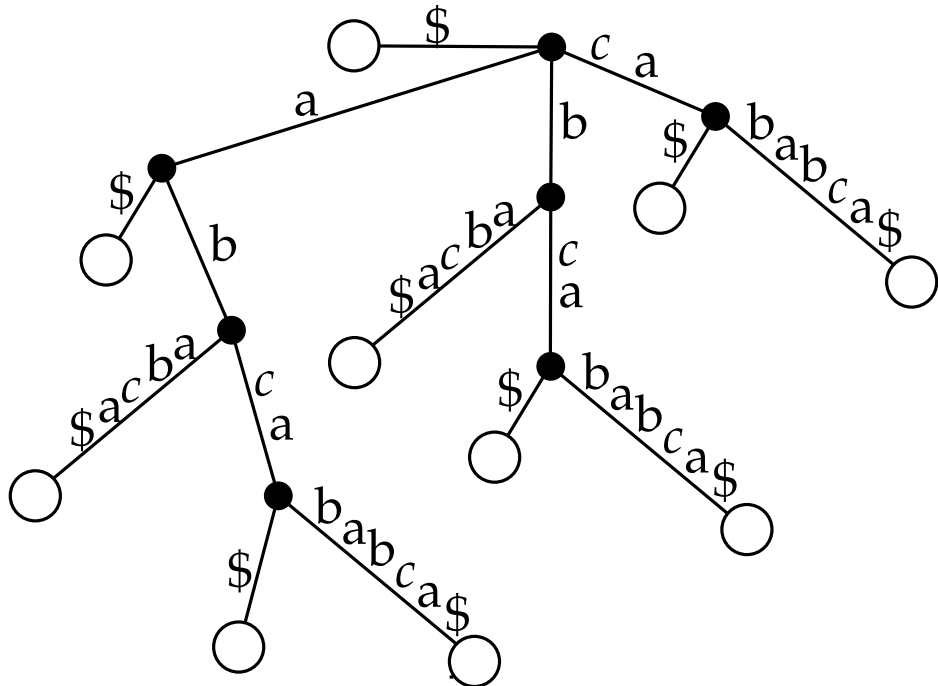
Anmerkung. Die Rückrichtung gilt nicht, weil ein Suffix das Präfix eines anderen Suffix sein kann,

Fix. Füge ein Symbol $\$ \notin \Sigma$ vor jedem Blatt hinzu
 $\Rightarrow$ die Blättern entsprechen bijektiv den Suffixen.



Lemma. Für jedes Blatt v von S ist $\bar{v}$ ein Suffix von T .

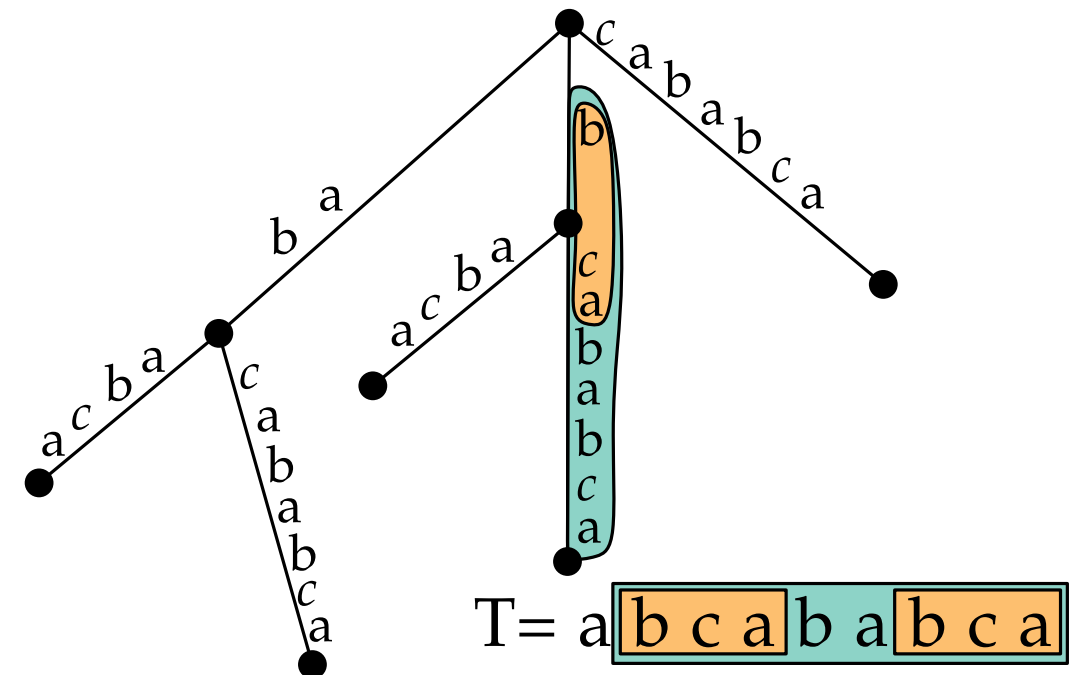
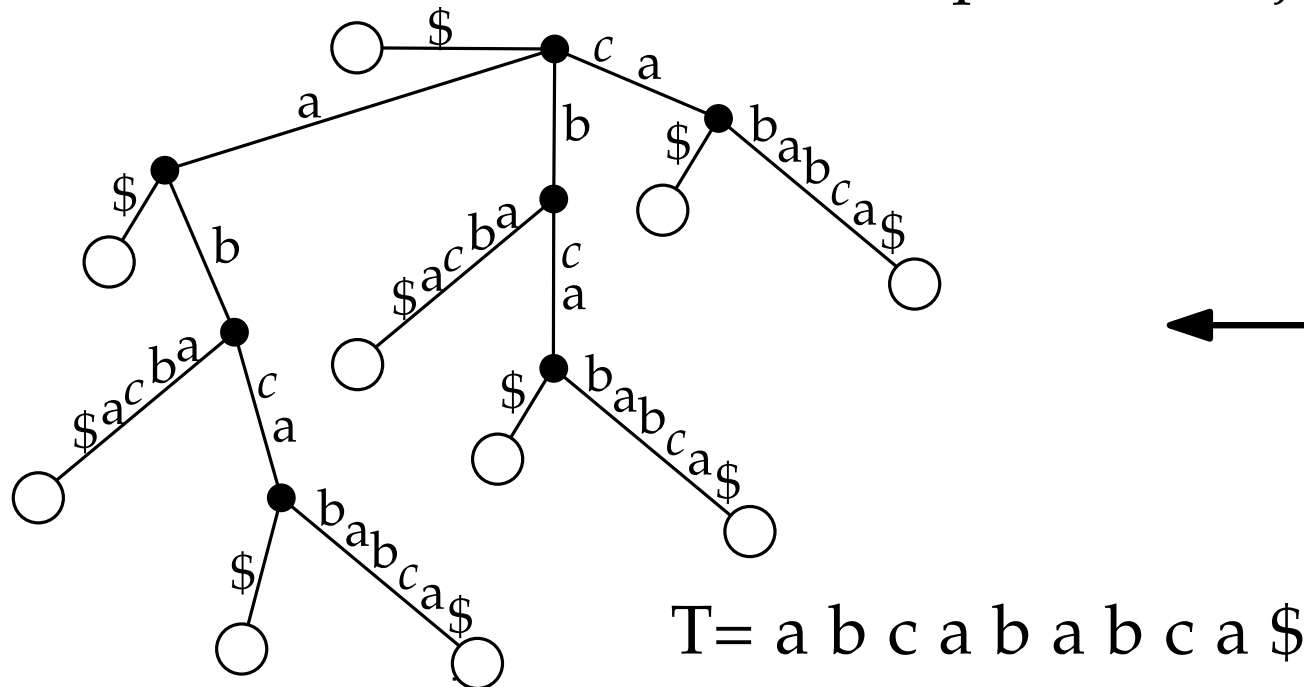
Fix. Füge ein Symbol $\$ \notin \Sigma$ vor jedem Blatt hinzu
 $\Rightarrow$ die Blätter entsprechen bijektiv den Suffixen.



Lemma. Für jedes Blatt v von S ist $\bar{v}$ ein Suffix von T .

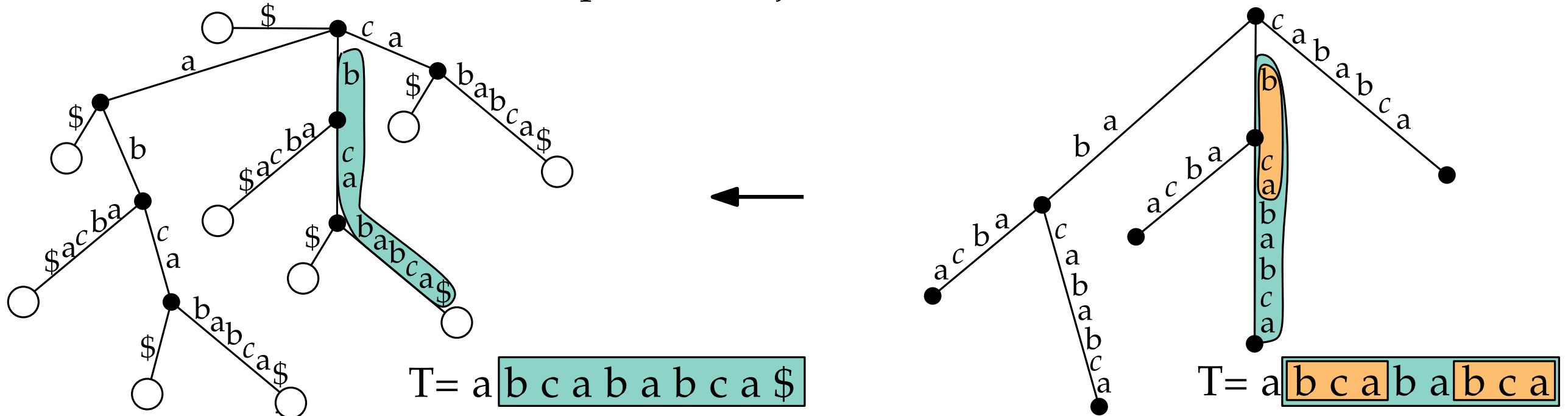
Anmerkung. Die Rückrichtung gilt nicht, weil ein Suffix das Präfix eines anderen Suffix sein kann,

Fix. Füge ein Symbol $\$ \notin \Sigma$ vor jedem Blatt hinzu
 $\Rightarrow$ die Blätter entsprechen bijektiv den Suffixen.



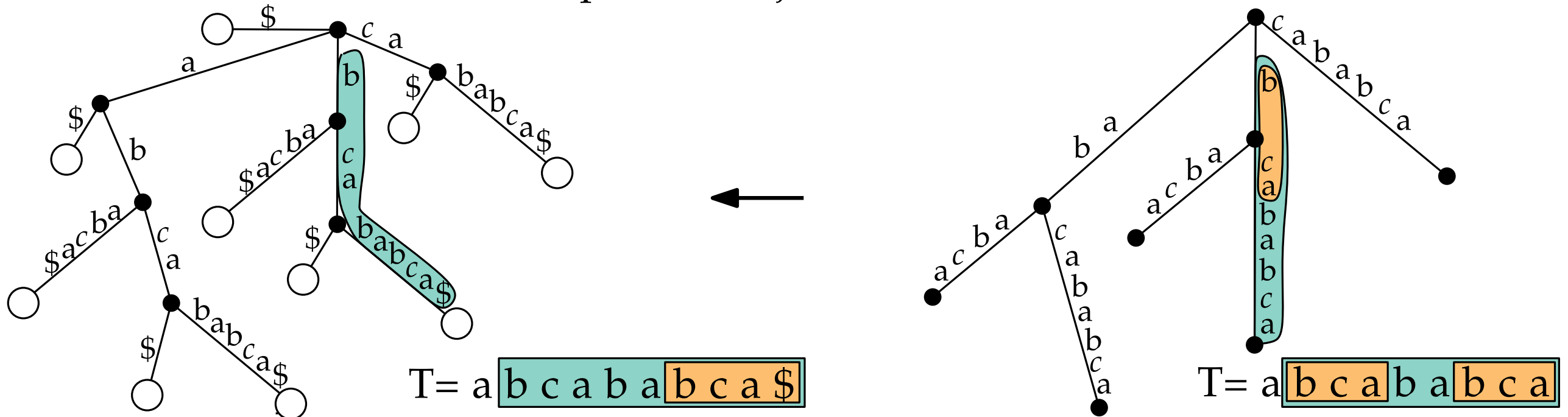
Lemma. Für jedes Blatt v von S ist $\bar{v}$ ein Suffix von T .

Fix. Füge ein Symbol $\$ \notin \Sigma$ vor jedem Blatt hinzu
 $\Rightarrow$ die Blätter entsprechen bijektiv den Suffixen.



Lemma. Für jedes Blatt v von S ist $\bar{v}$ ein Suffix von T .

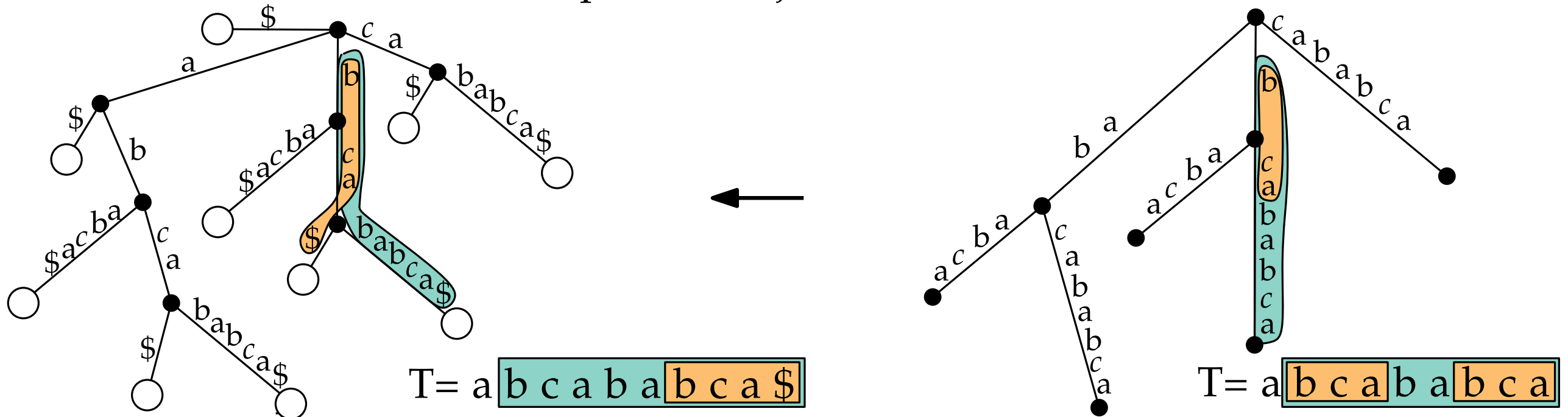
Fix. Füge ein Symbol $\$ \notin \Sigma$ vor jedem Blatt hinzu
 $\Rightarrow$ die Blätter entsprechen bijektiv den Suffixen.



Lemma. Für jedes Blatt v von S ist $\bar{v}$ ein Suffix von T .

Anmerkung. Die Rückrichtung gilt nicht, weil ein Suffix das Präfix eines anderen Suffix sein kann,

Fix. Füge ein Symbol $\$ \notin \Sigma$ vor jedem Blatt hinzu
 $\Rightarrow$ die Blätter entsprechen bijektiv den Suffixen.



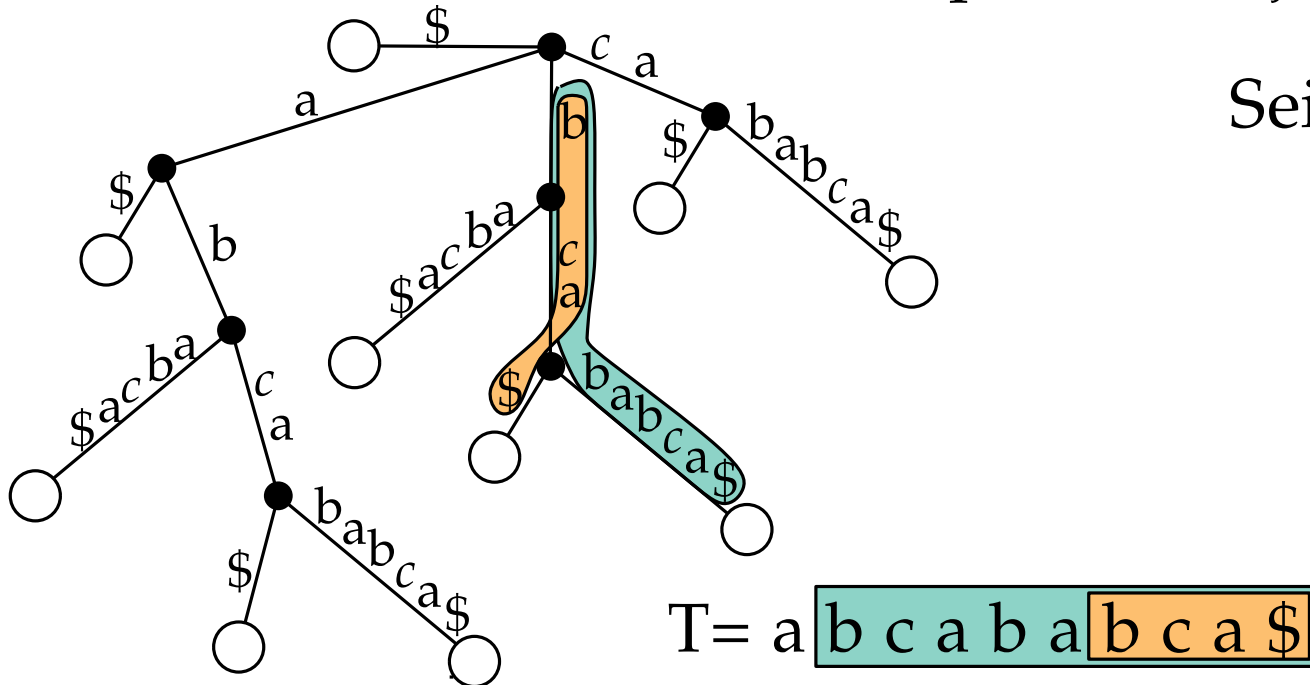
Ein **Suffixbaum** S von T ist ein Σ -Baum, der genau die Infixe von T enthält, d.h.,
 $\text{wörter}(S) = \{T[i, j] \mid 1 \leq i \leq j \leq n\}$.

Lemma. Für jedes Blatt v von S ist $\bar{v}$ ein Suffix von T .

Anmerkung. Die Rückrichtung gilt nicht, weil ein Suffix das Präfix eines anderen Suffix sein kann,

Fix. Füge ein Symbol $\$ \notin \Sigma$ vor jedem Blatt hinzu
 $\Rightarrow$ die Blätter entsprechen bijektiv den Suffixen.

Sei i das Blatt von S mit $\bar{i} = T[i, n]$.



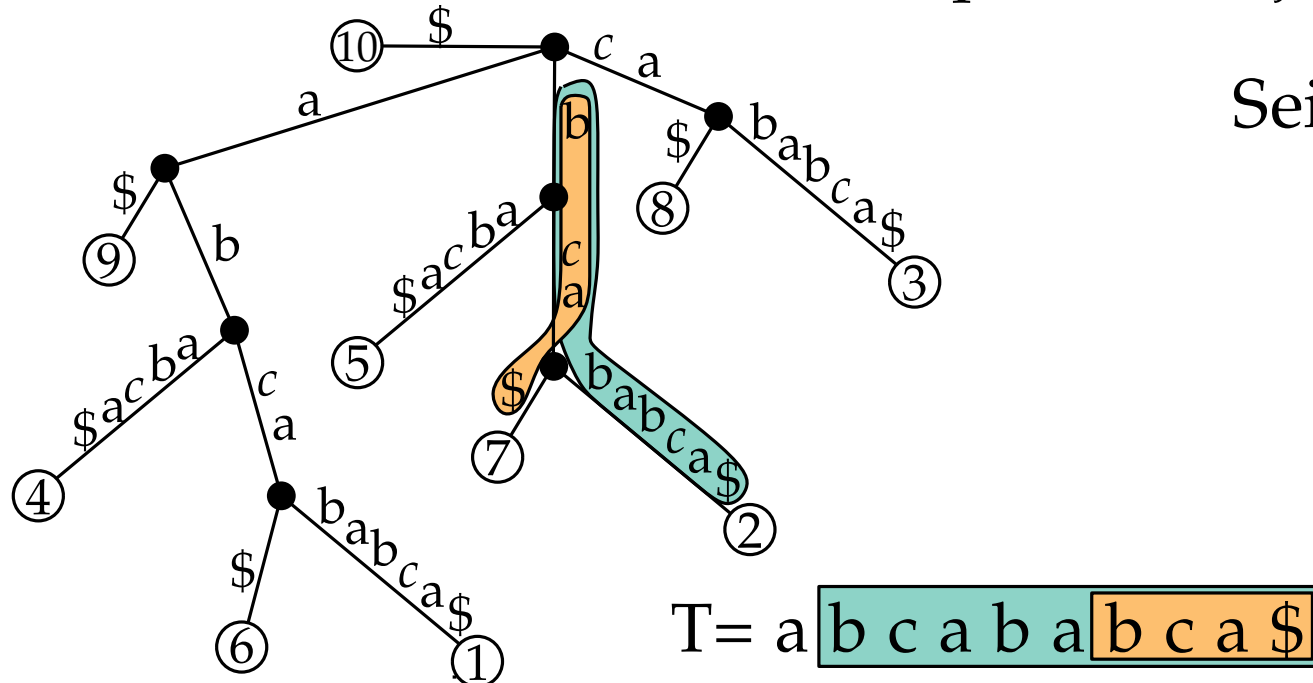
Ein **Suffixbaum** S von T ist ein Σ -Baum, der genau die Infixe von T enthält, d.h.,
 $\text{wörter}(S) = \{T[i, j] \mid 1 \leq i \leq j \leq n\}$.

Lemma. Für jedes Blatt v von S ist $\bar{v}$ ein Suffix von T .

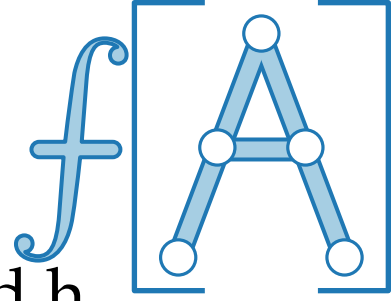
Anmerkung. Die Rückrichtung gilt nicht, weil ein Suffix das Präfix eines anderen Suffix sein kann,

Fix. Füge ein Symbol $\$ \notin \Sigma$ vor jedem Blatt hinzu
 $\Rightarrow$ die Blätter entsprechen bijektiv den Suffixen.

Sei i das Blatt von S mit $\bar{i} = T[i, n]$.



Suffixbäume – Struktur

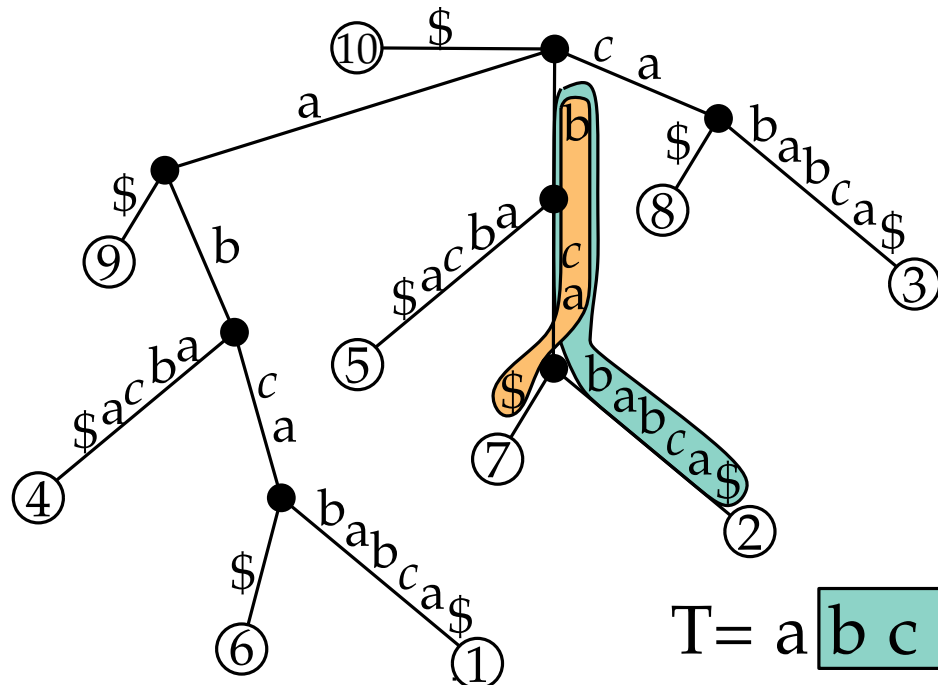


Ein **Suffixbaum** S von T ist ein Σ -Baum, der genau die Infixe von T enthält, d.h.,
 $\text{wörter}(S) = \{T[i, j] \mid 1 \leq i \leq j \leq n\}$.

Lemma. Für jedes Blatt v von S ist $\bar{v}$ ein Suffix von T .

Anmerkung. Die Rückrichtung gilt nicht, weil ein Suffix das Präfix eines anderen Suffix sein kann,

Fix. Füge ein Symbol $\$ \notin \Sigma$ vor jedem Blatt hinzu
 $\Rightarrow$ die Blättern entsprechen bijektiv den Suffixen.



Sei i das Blatt von S mit $\bar{i} = T[i, n]$.

Sei S_i

■ das i -te Suffix $T[i, n]$ von T ;

$T = a \text{ } \boxed{bca} \text{ } \boxed{bca} \$$

Lemma. Für jedes Blatt v von S ist $\bar{v}$ ein Suffix von T .

Fix. Füge ein Symbol $\$ \notin \Sigma$ vor jedem Blatt hinzu
 $\Rightarrow$ die Blätter entsprechen bijektiv den Suffixen.



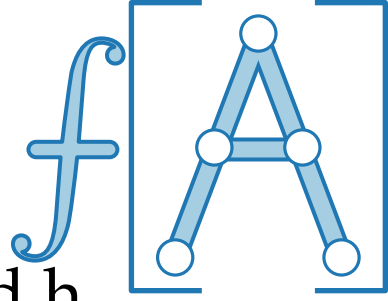
Sei S_i

- das i -te Suffix $T[i, n]$ von T ;

② S_2

T= a b c a b a b c a \$

Suffixbäume – Struktur

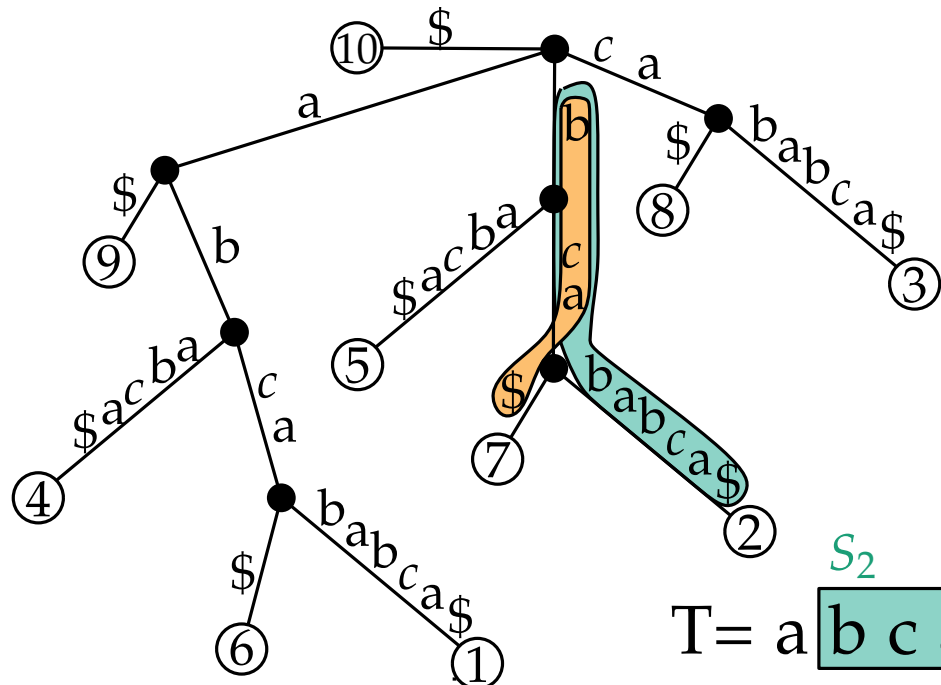


Ein **Suffixbaum** S von T ist ein Σ -Baum, der genau die Infixe von T enthält, d.h.,
 $\text{wörter}(S) = \{T[i, j] \mid 1 \leq i \leq j \leq n\}$.

Lemma. Für jedes Blatt v von S ist $\bar{v}$ ein Suffix von T .

Anmerkung. Die Rückrichtung gilt nicht, weil ein Suffix das Präfix eines anderen Suffix sein kann,

Fix. Füge ein Symbol $\$ \notin \Sigma$ vor jedem Blatt hinzu
 $\Rightarrow$ die Blättern entsprechen bijektiv den Suffixen.



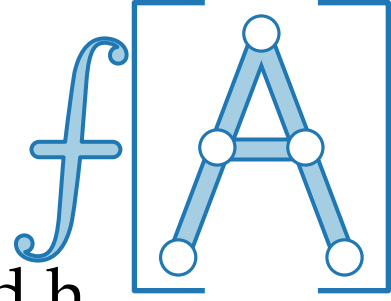
Sei i das Blatt von S mit $\bar{i} = T[i, n]$.

Sei S_i

■ das i -te Suffix $T[i, n]$ von T ;

$T = a \overset{S_2}{\boxed{b c a b a}} \overset{S_7}{\boxed{b c a \$}}$

Suffixbäume – Struktur

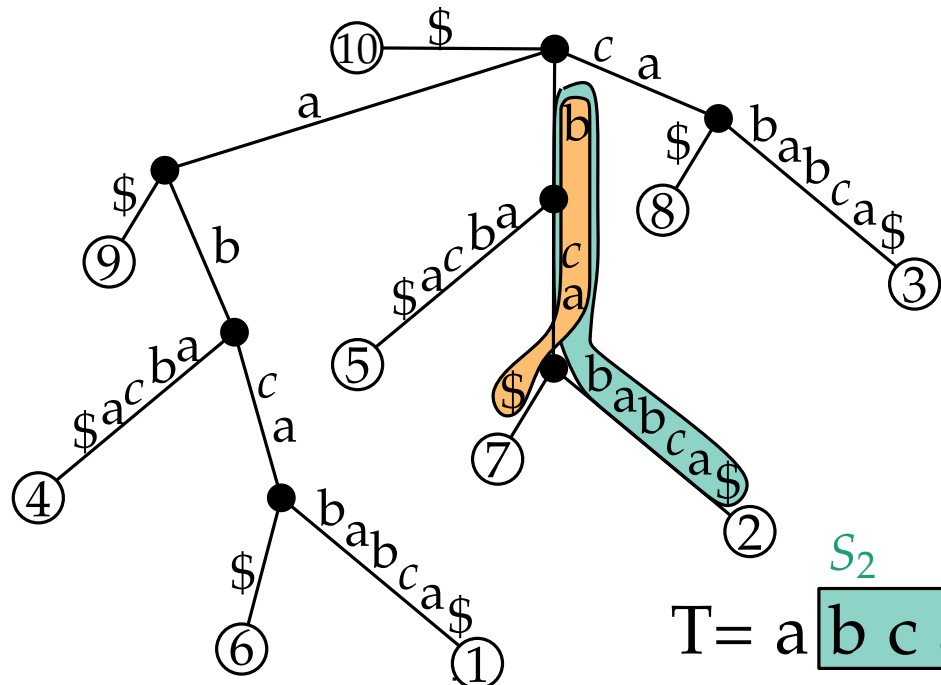


Ein **Suffixbaum** S von T ist ein Σ -Baum, der genau die Infixe von T enthält, d.h.,
 $\text{wörter}(S) = \{T[i, j] \mid 1 \leq i \leq j \leq n\}$.

Lemma. Für jedes Blatt v von S ist $\bar{v}$ ein Suffix von T .

Anmerkung. Die Rückrichtung gilt nicht, weil ein Suffix das Präfix eines anderen Suffix sein kann,

Fix. Füge ein Symbol $\$ \notin \Sigma$ vor jedem Blatt hinzu
 $\Rightarrow$ die Blättern entsprechen bijektiv den Suffixen.



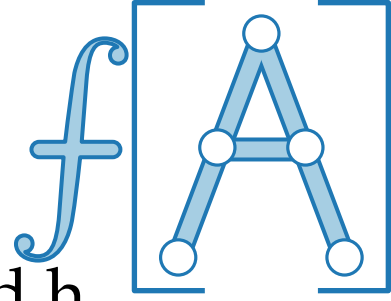
Sei i das Blatt von S mit $\bar{i} = T[i, n]$.

Sei S_i

- das i -te Suffix $T[i, n]$ von T ;
- der Pfad von der Wurzel von S nach i

$T = a \overset{S_2}{\boxed{b c a b a}} \overset{S_7}{\boxed{b c a \$}}$

Suffixbäume – Struktur

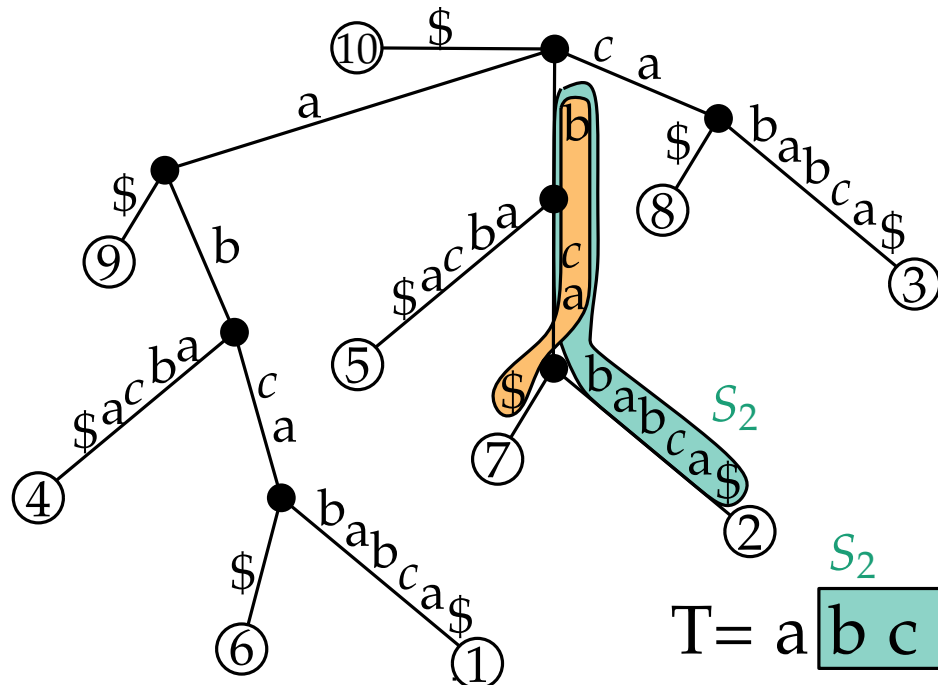


Ein **Suffixbaum** S von T ist ein Σ -Baum, der genau die Infixe von T enthält, d.h.,
 $\text{wörter}(S) = \{T[i, j] \mid 1 \leq i \leq j \leq n\}$.

Lemma. Für jedes Blatt v von S ist $\bar{v}$ ein Suffix von T .

Anmerkung. Die Rückrichtung gilt nicht, weil ein Suffix das Präfix eines anderen Suffix sein kann,

Fix. Füge ein Symbol $\$ \notin \Sigma$ vor jedem Blatt hinzu
 $\Rightarrow$ die Blättern entsprechen bijektiv den Suffixen.



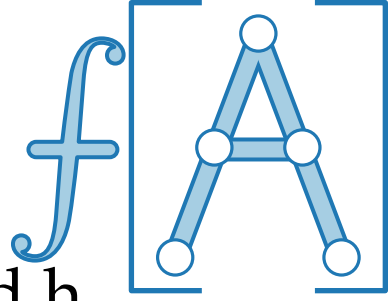
Sei i das Blatt von S mit $\bar{i} = T[i, n]$.

Sei S_i

- das i -te Suffix $T[i, n]$ von T ;
- der Pfad von der Wurzel von S nach i

$T = a \overset{S_2}{\boxed{b c a b a}} \overset{S_7}{\boxed{b c a \$}}$

Suffixbäume – Struktur

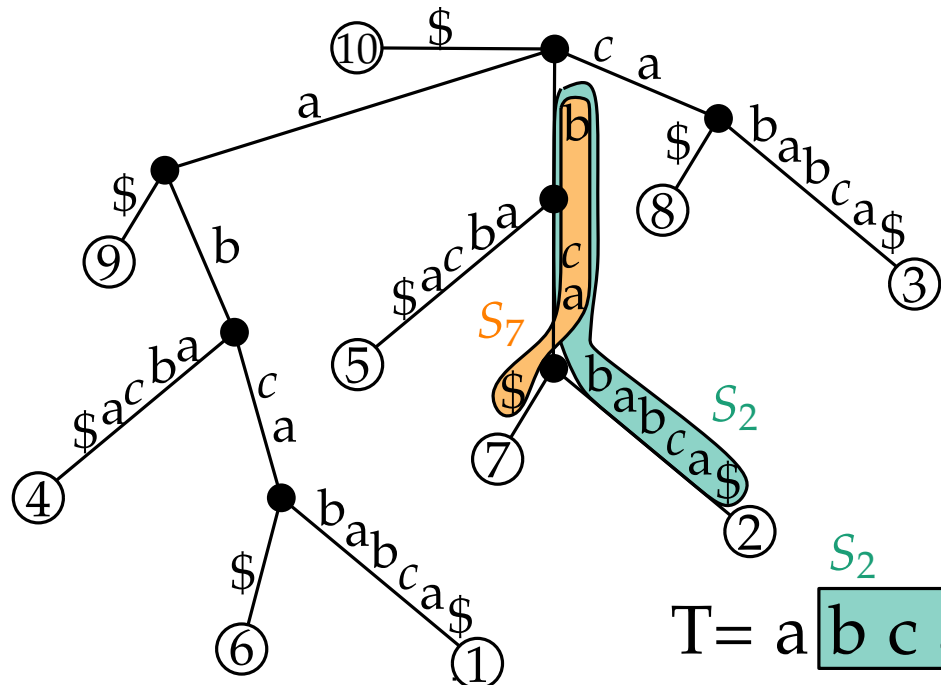


Ein **Suffixbaum** S von T ist ein Σ -Baum, der genau die Infixe von T enthält, d.h.,
 $\text{wörter}(S) = \{T[i, j] \mid 1 \leq i \leq j \leq n\}$.

Lemma. Für jedes Blatt v von S ist $\bar{v}$ ein Suffix von T .

Anmerkung. Die Rückrichtung gilt nicht, weil ein Suffix das Präfix eines anderen Suffix sein kann,

Fix. Füge ein Symbol $\$ \notin \Sigma$ vor jedem Blatt hinzu
 $\Rightarrow$ die Blättern entsprechen bijektiv den Suffixen.



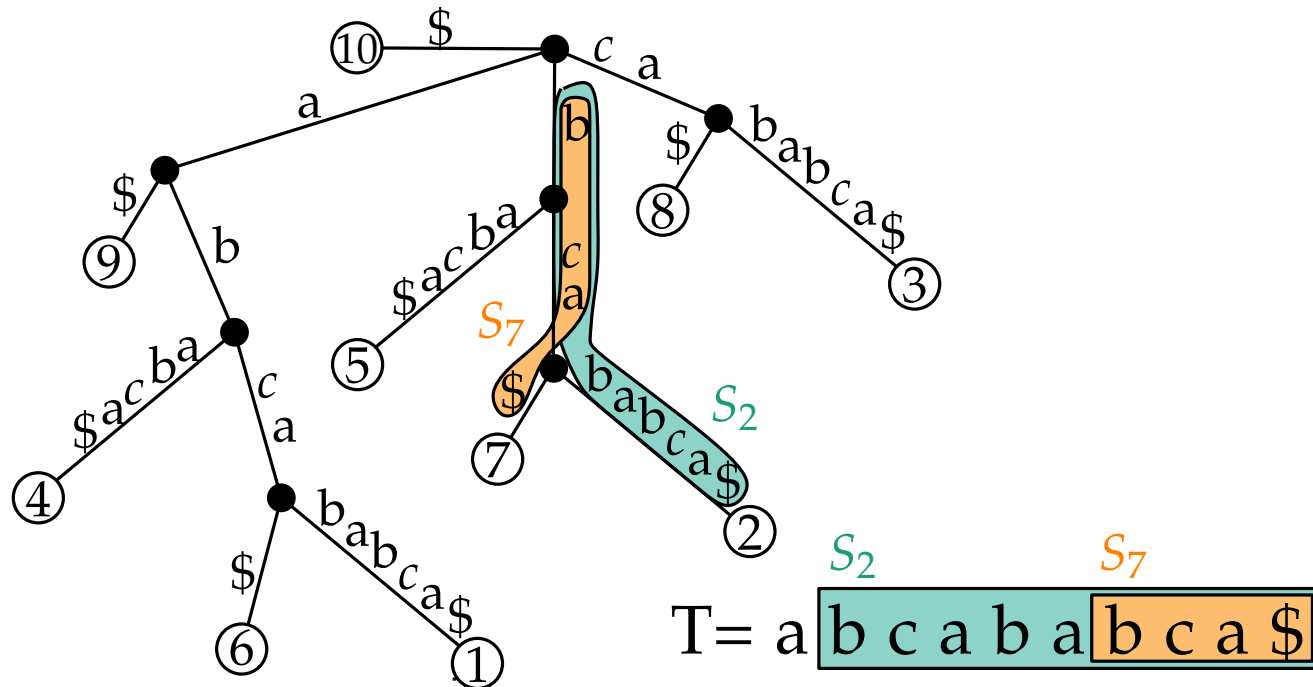
Sei i das Blatt von S mit $\bar{i} = T[i, n]$.

Sei S_i

- das i -te Suffix $T[i, n]$ von T ;
- der Pfad von der Wurzel von S nach i

$T = a \overset{S_2}{\boxed{b c a b a}} \overset{S_7}{\boxed{b c a \$}}$

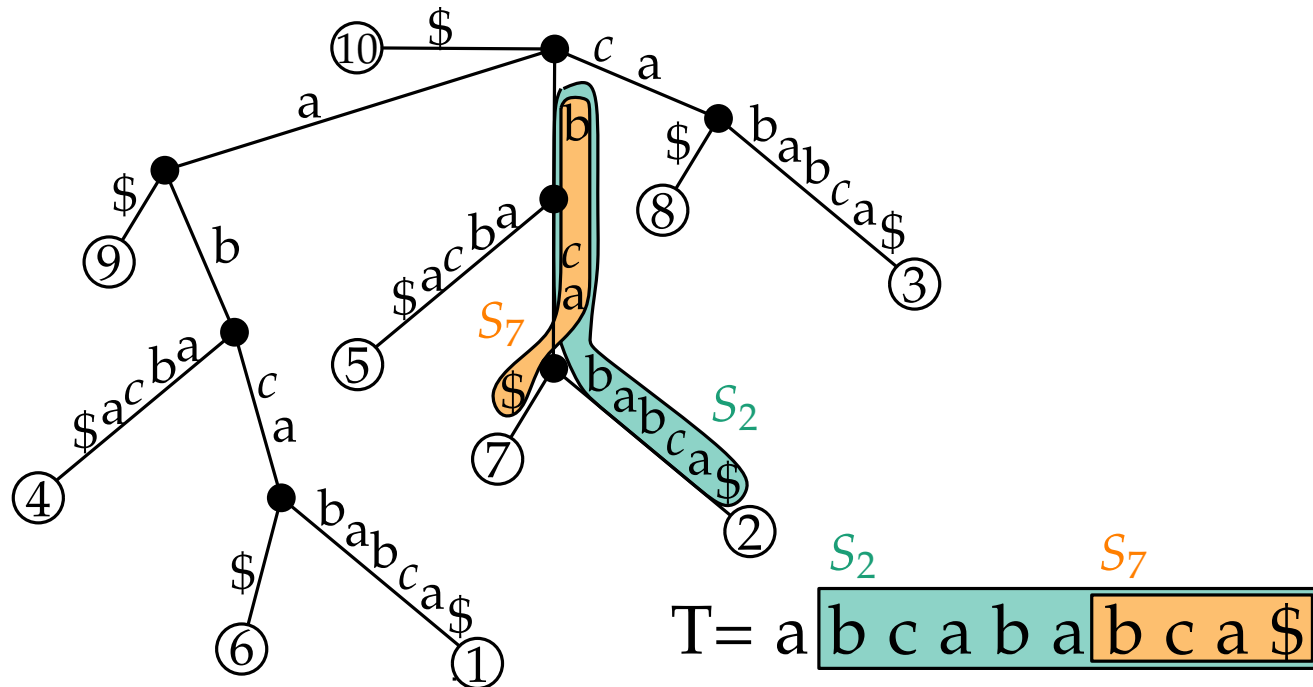
Suffixbäume – Implementierungsdetails



Suffixbäume – Implementierungsdetails



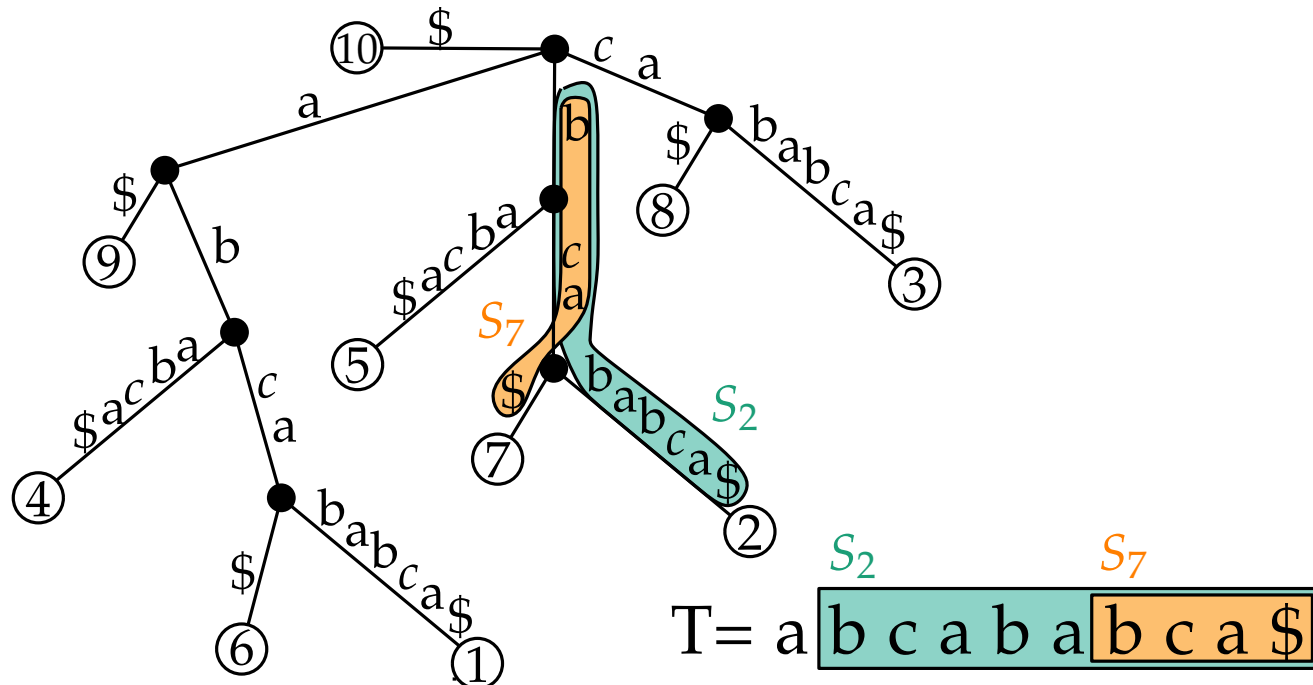
- Jede Kante ist mit einem Infix $T[i, j]$ beschriftet.



Suffixbäume – Implementierungsdetails



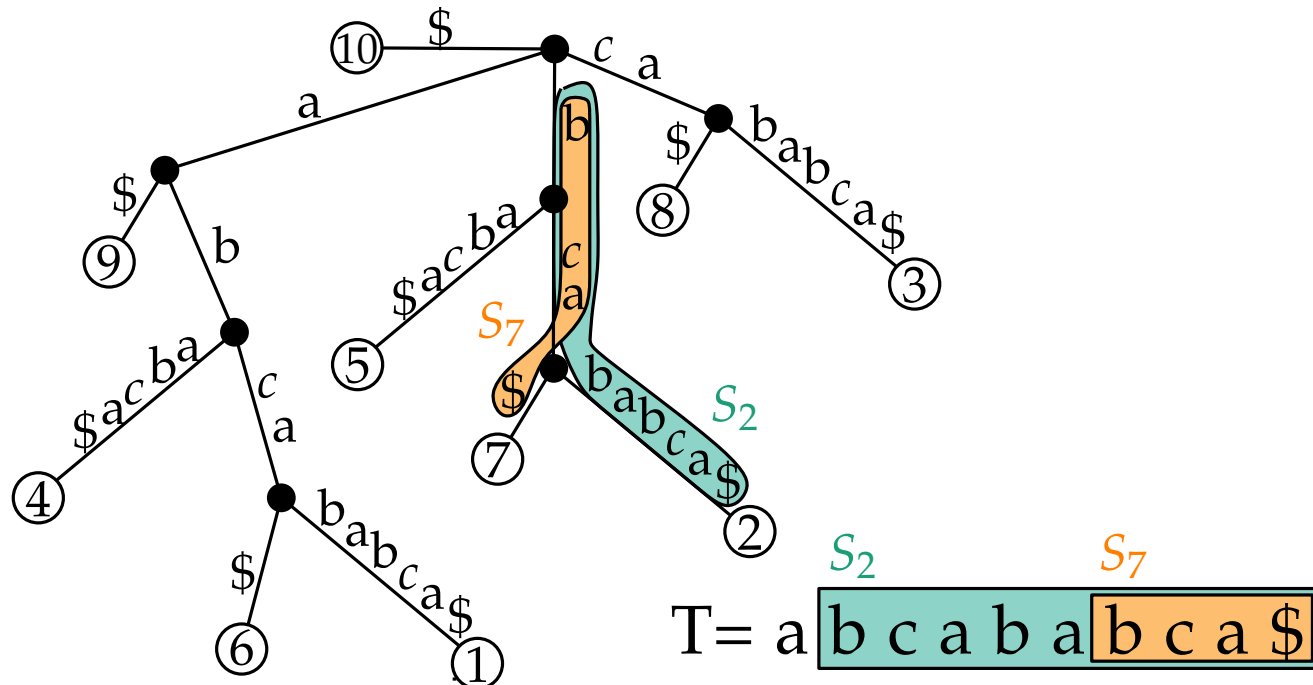
- Jede Kante ist mit einem Infix $T[i, j]$ beschriftet. Es reicht, die Indizes i und j zu speichern.



Suffixbäume – Implementierungsdetails

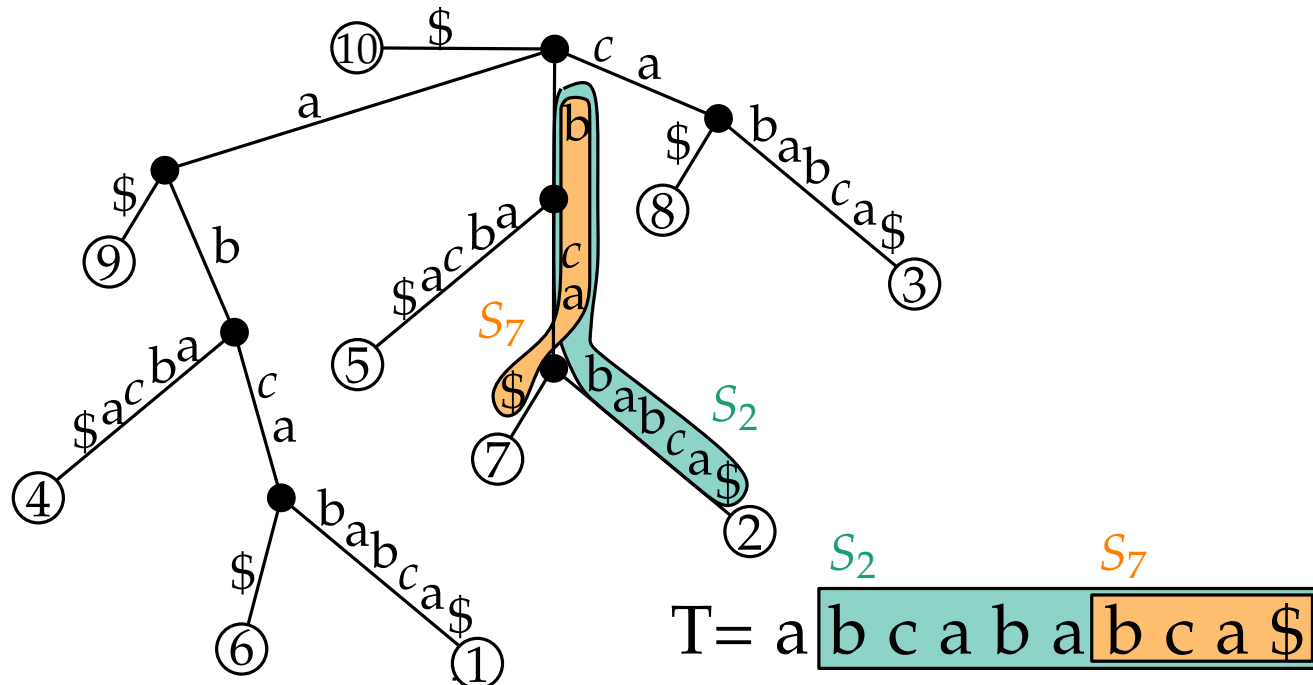


- Jede Kante ist mit einem Infix $T[i, j]$ beschriftet.
Es reicht, die Indizes i und j zu speichern.
➔ S benötigt $\mathcal{O}(n)$ Platz, weil $\# \text{Blätter} = \# \text{Suffixe} = n$.



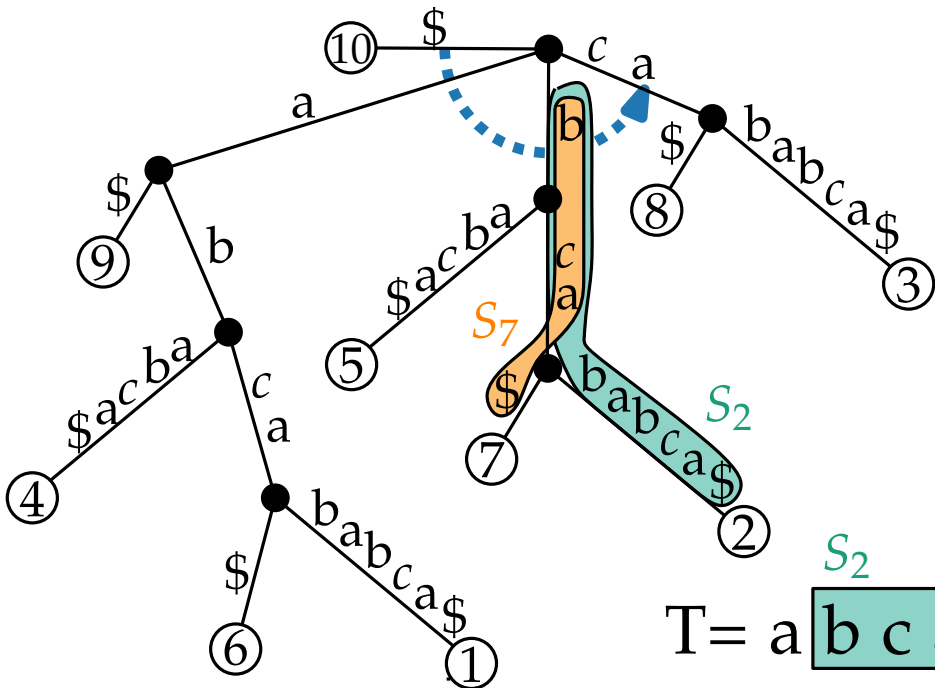
Suffixbäume – Implementierungsdetails

- Jede Kante ist mit einem Infix $T[i, j]$ beschriftet.
Es reicht, die Indizes i und j zu speichern.
➔ S benötigt $\mathcal{O}(n)$ Platz, weil $\# \text{Blätter} = \# \text{Suffixe} = n$.
- Speichere an jedem Knoten v die Kanten zu seinen k Kindern in einem Array der Länge k , sortiert nach dem ersten Buchstaben ihrer Beschriftungen.

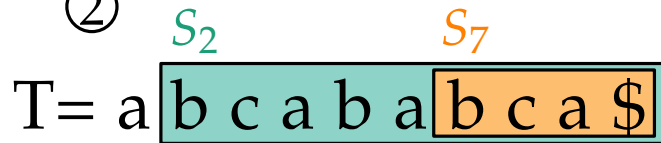


Suffixbäume – Implementierungsdetails

- Jede Kante ist mit einem Infix $T[i, j]$ beschriftet.
Es reicht, die Indizes i und j zu speichern.
→ S benötigt $\mathcal{O}(n)$ Platz, weil $\# \text{Blätter} = \# \text{Suffixe} = n$.
- Speichere an jedem Knoten v die Kanten zu seinen k Kindern in einem Array der Länge k , sortiert nach dem ersten Buchstaben ihrer Beschriftungen.



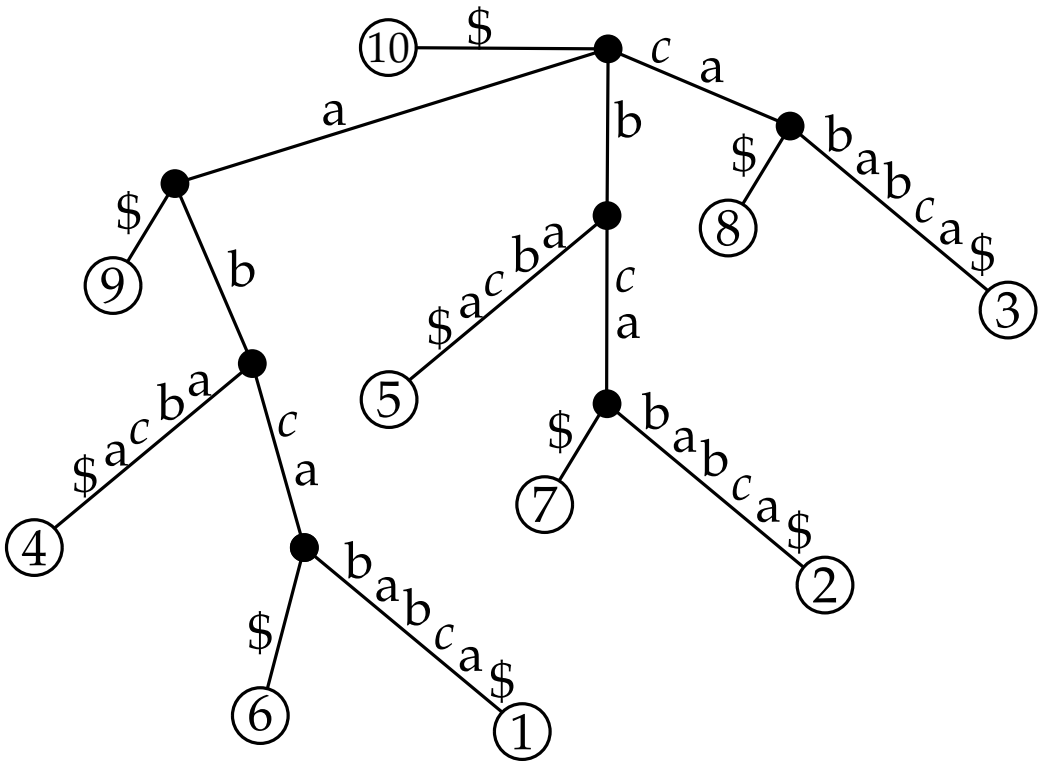
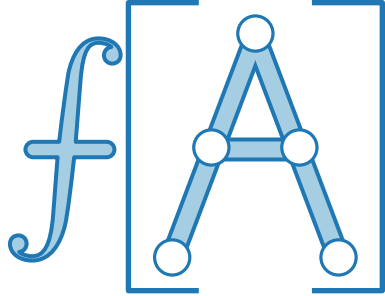
- ➡ erlaubt binäre Suche!



Suchen in Suffixbäumen

SUCHE(S,P)

$T = a b c a b a b c a$

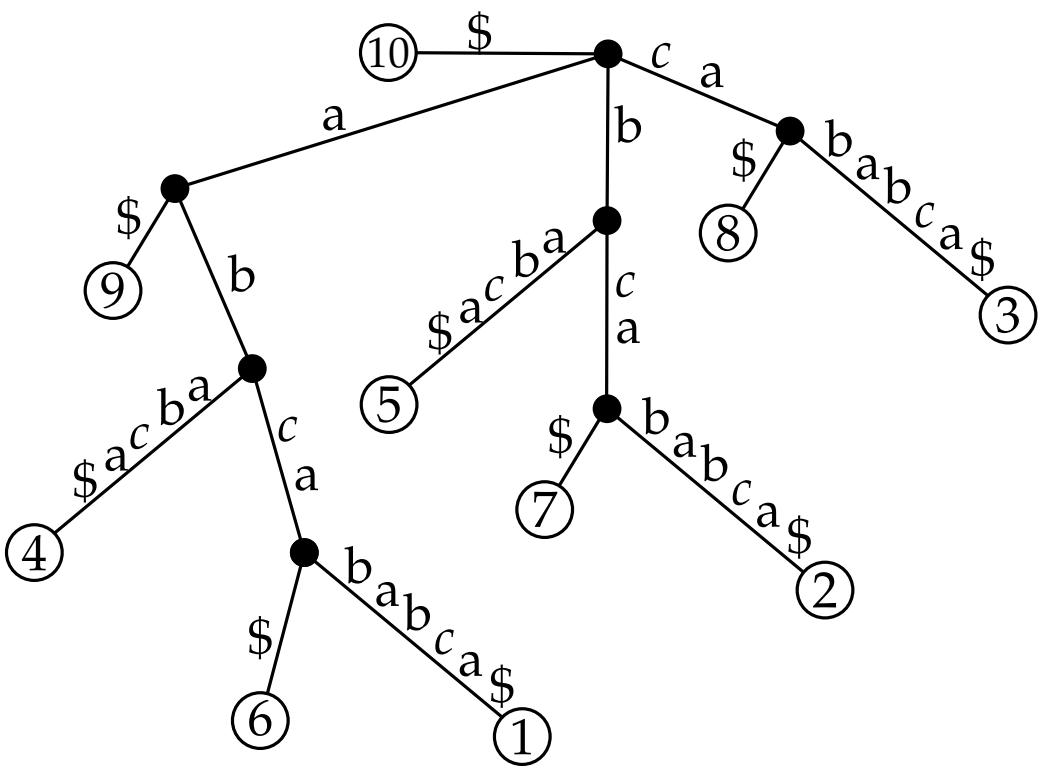
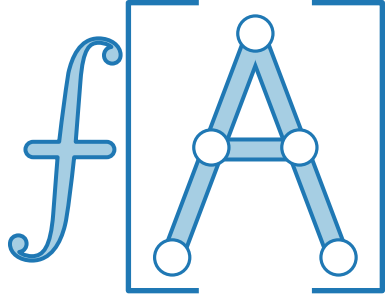


Beispiel. $M = \begin{matrix} a & b & c \\ 1 & 2 & 3 \end{matrix}$

Suchen in Suffixbäumen

SUCHE(S, P)
 u = Wurzel von S

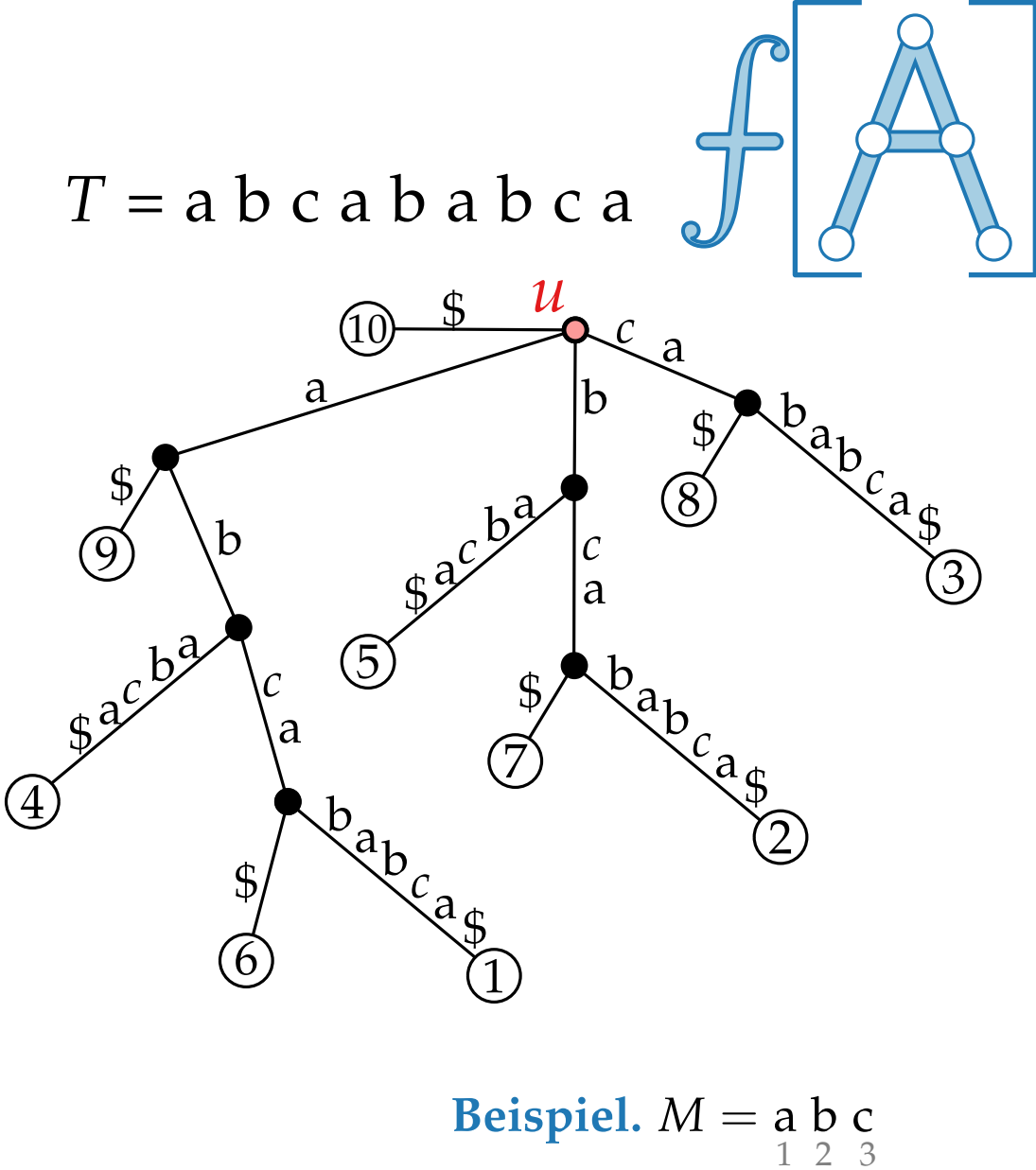
$T = a b c a b a b c a$



Beispiel. $M = \begin{matrix} a & b & c \\ 1 & 2 & 3 \end{matrix}$

Suchen in Suffixbäumen

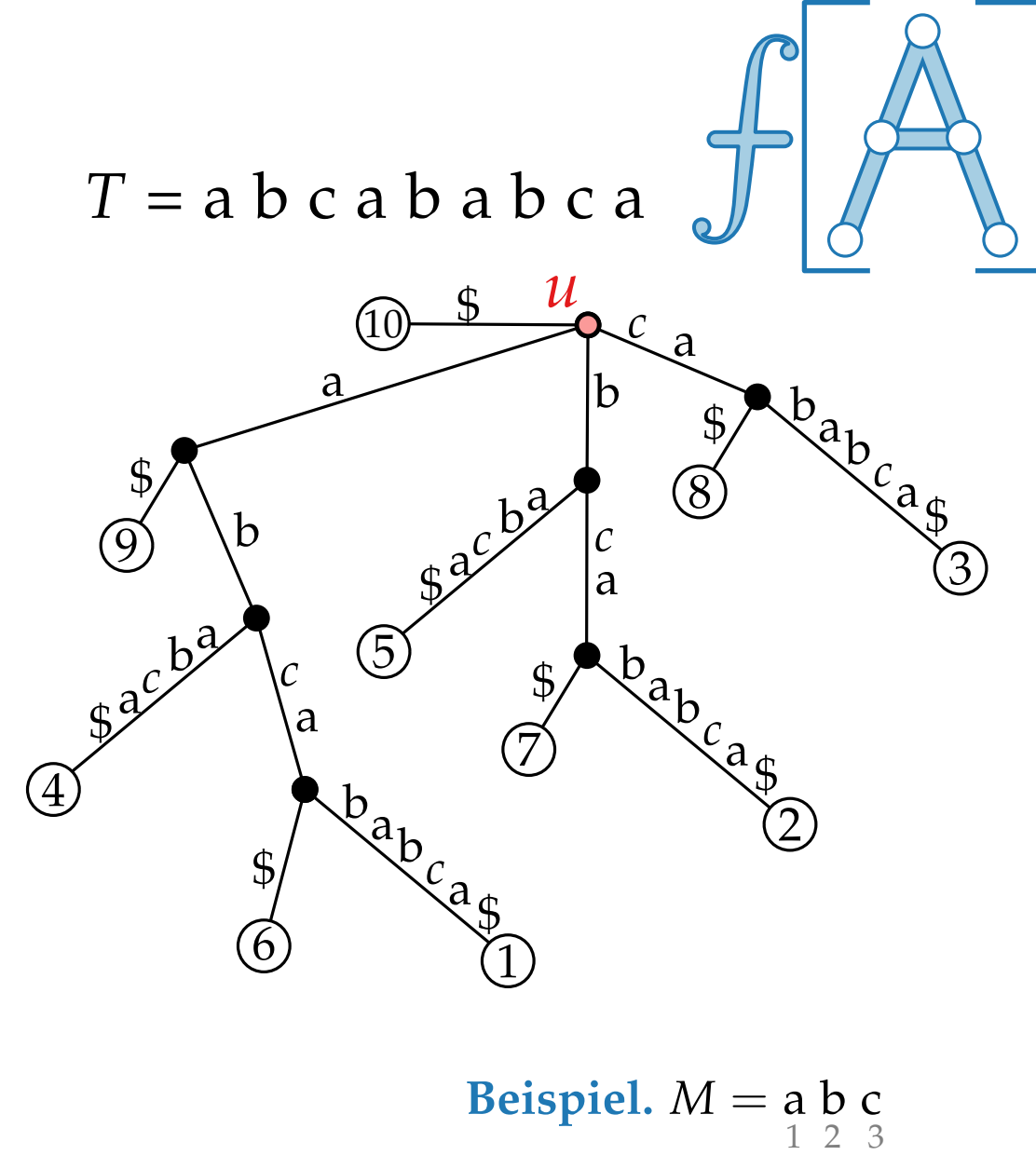
SUCHE(S,P)
 u = Wurzel von S



Suchen in Suffixbäumen

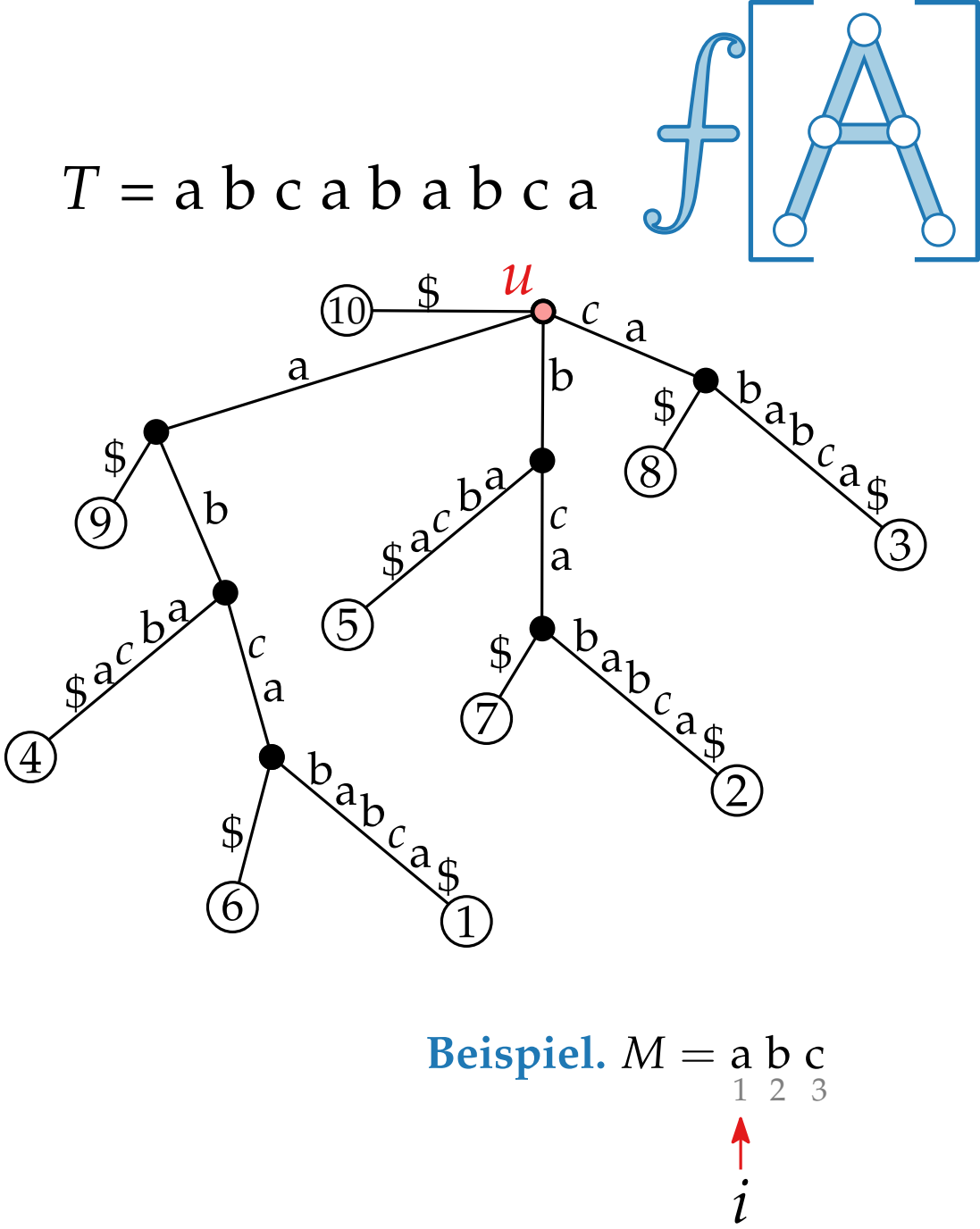
SUCHE(S, P)

u = Wurzel von S

$$i = 1$$


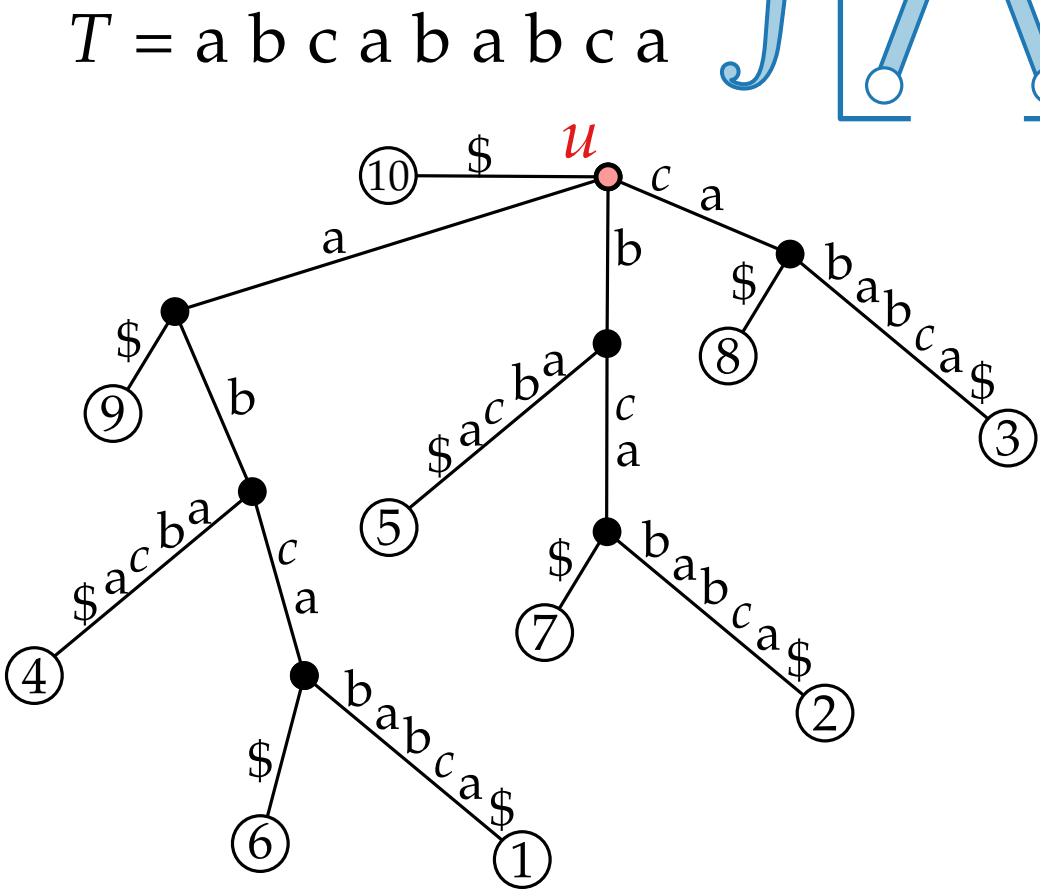
Suchen in Suffixbäumen

SUCHE(S,P)
 u = Wurzel von S
 $i = 1$



Suchen in Suffixbäumen

```
SUCHE(S,P)  
  u = Wurzel von S  
  i = 1  
  while u ist kein Blatt do
```



Suchen in Suffixbäumen

SUCHE(S, P)

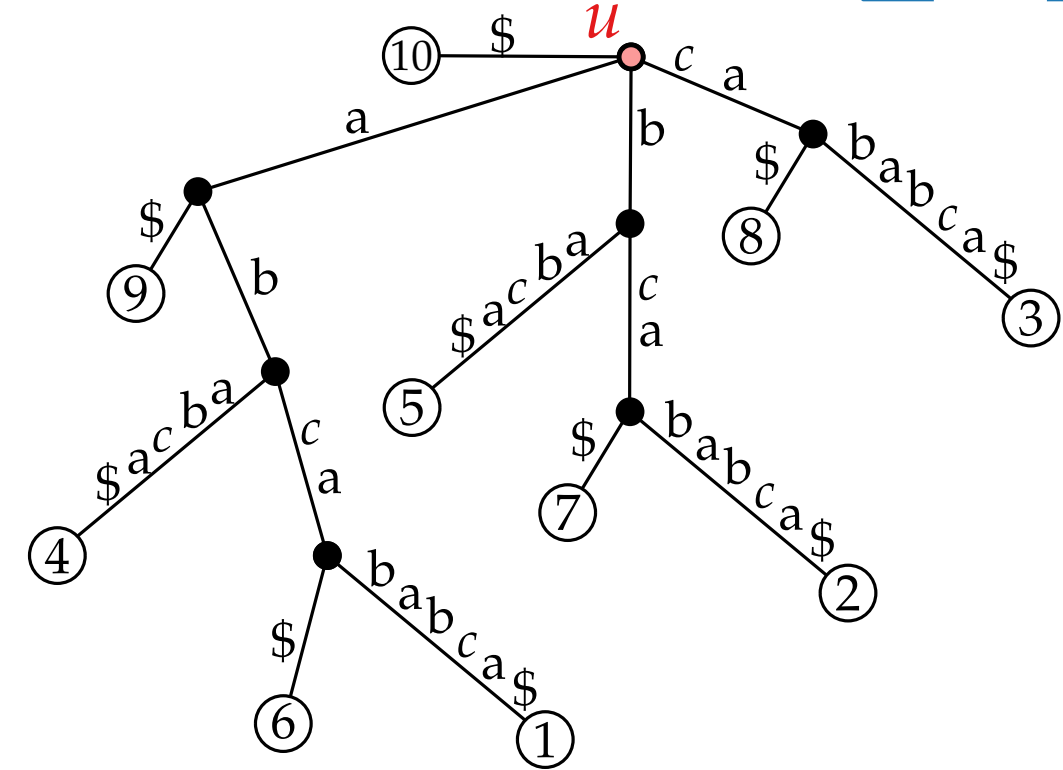
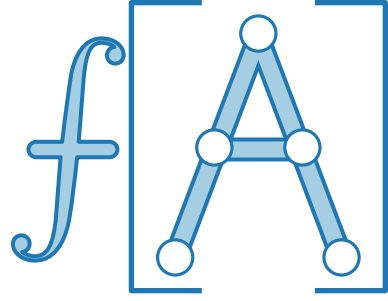
u = Wurzel von S

$i = 1$

while u ist kein Blatt **do**

 Suche Kante $e = (u, v)$, dessen Beschriftung B mit $M[i]$ beginnt.

$T = a b c a b a b c a$



Beispiel. $M = \begin{matrix} a & b & c \\ 1 & 2 & 3 \end{matrix}$
 $\uparrow$
 i

Suchen in Suffixbäumen

SUCHE(S, P)

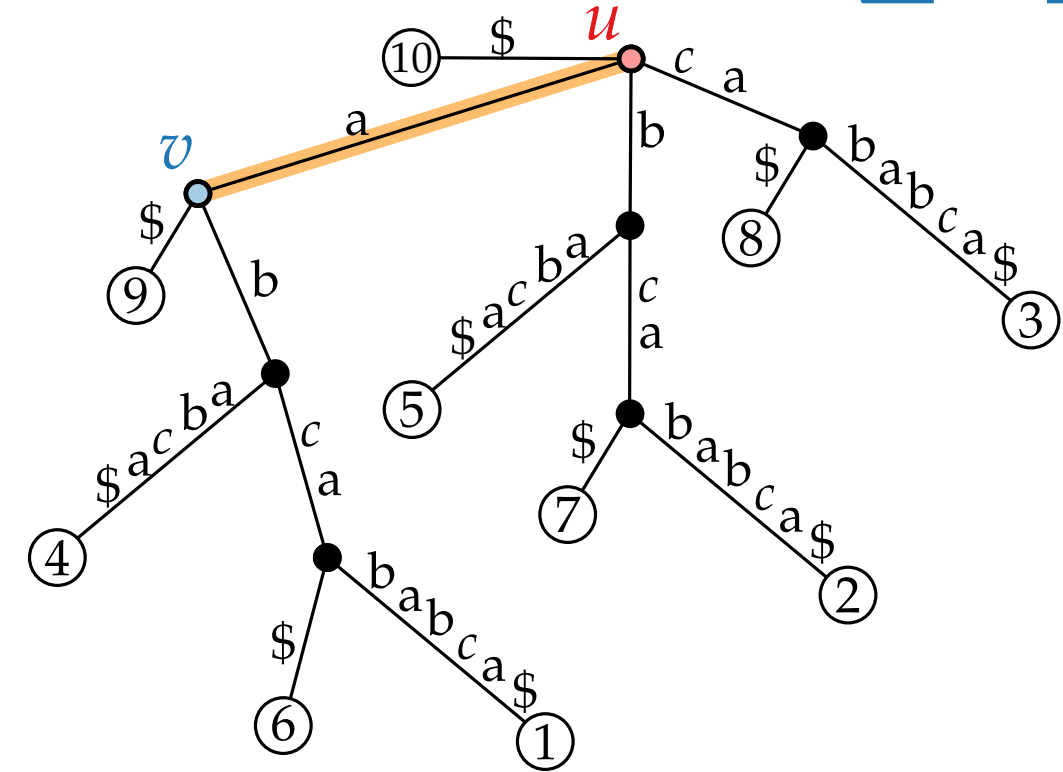
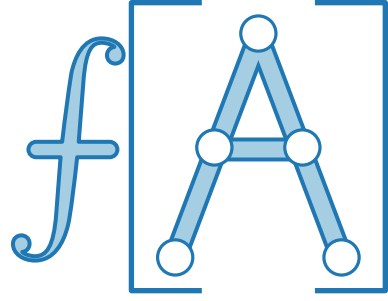
u = Wurzel von S

$i = 1$

while u ist kein Blatt **do**

 Suche Kante $e = (u, v)$, dessen Beschriftung B mit $M[i]$ beginnt.

$T = a b c a b a b c a$



Beispiel. $M = \begin{matrix} a & b & c \\ 1 & 2 & 3 \\ \uparrow \\ i \end{matrix}$

Suchen in Suffixbäumen

SUCHE(S, P)

u = Wurzel von S

$i = 1$

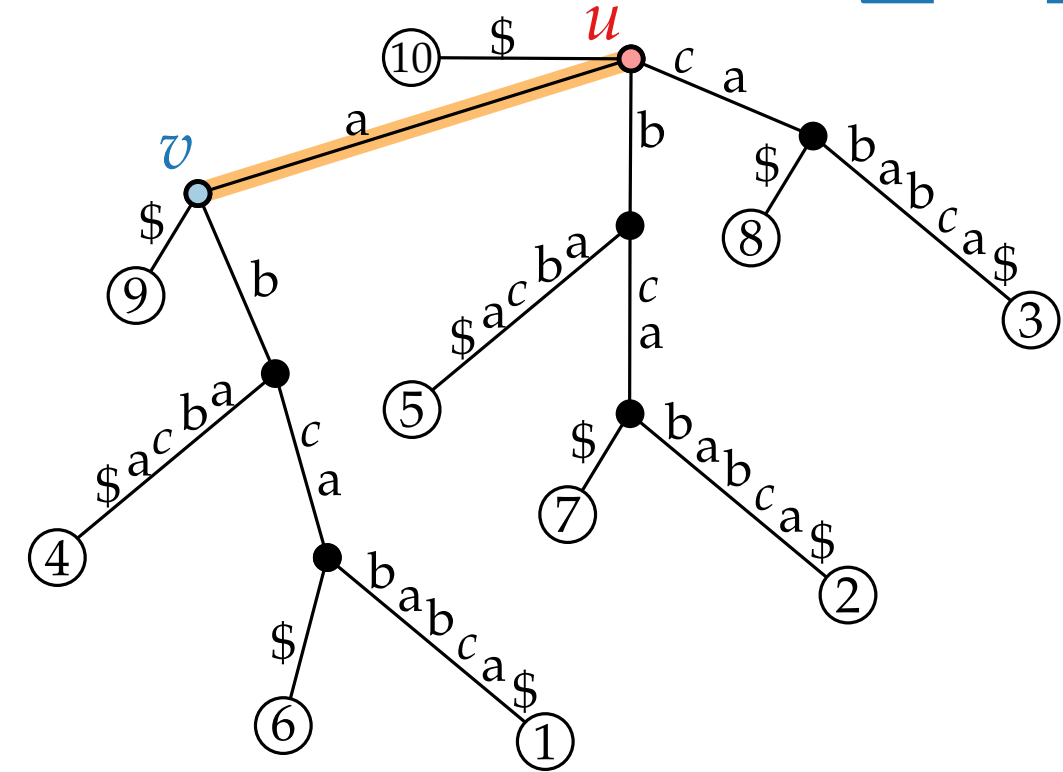
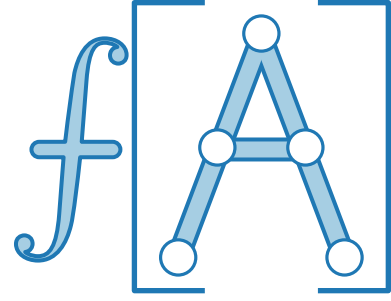
while u ist kein Blatt **do**

 Suche Kante $e = (u, v)$, dessen Beschriftung B mit $M[i]$ beginnt.

if e existiert nicht **then**

$\perp$ **return** $\emptyset$

$T = a b c a b a b c a$



Beispiel. $M = \begin{matrix} a & b & c \\ 1 & 2 & 3 \\ \uparrow \\ i \end{matrix}$

Suchen in Suffixbäumen

SUCHE(S, P)

u = Wurzel von S

$i = 1$

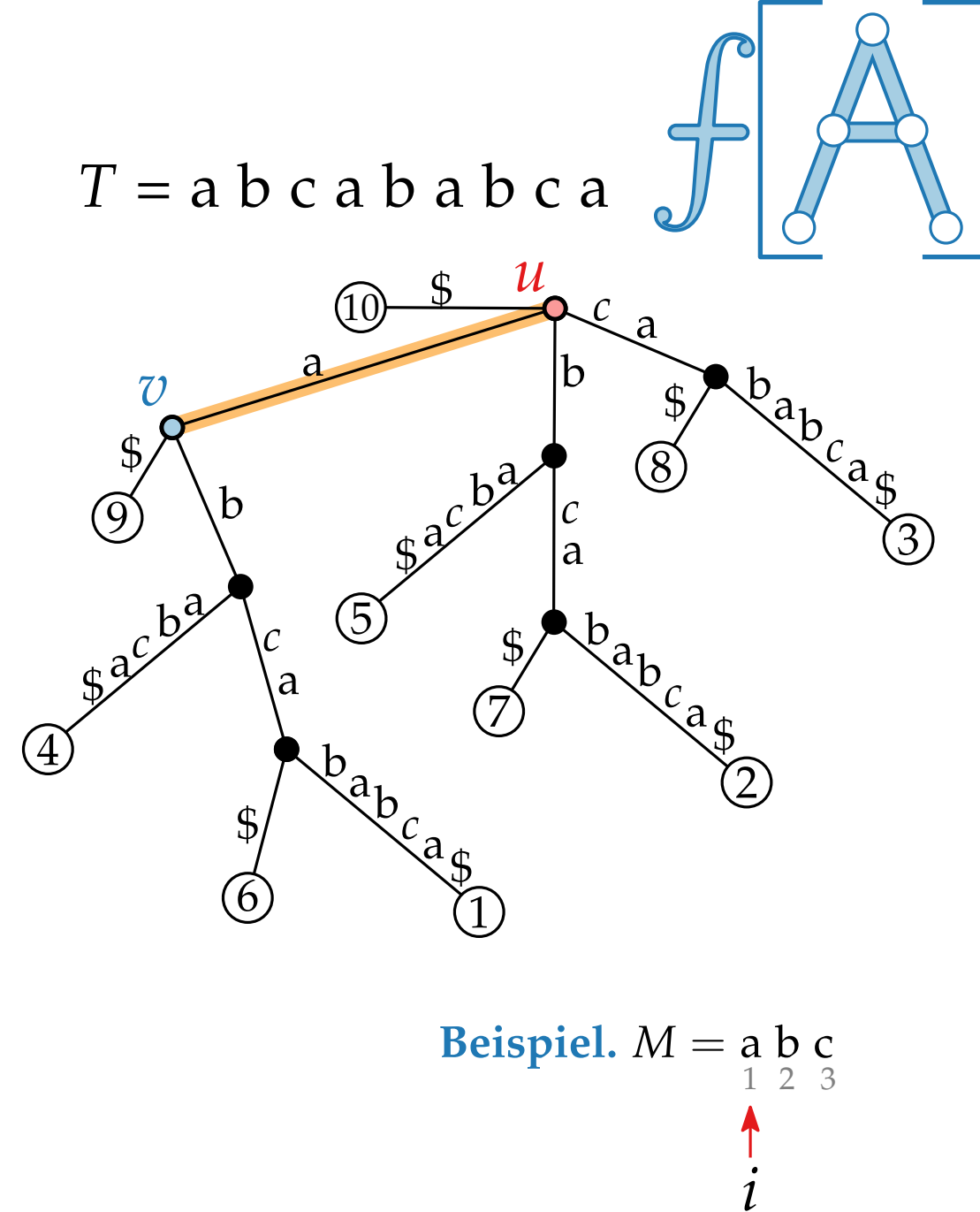
while u ist kein Blatt **do**

 Suche Kante $e = (u, v)$, dessen Beschriftung B mit $M[i]$ beginnt.

if e existiert nicht **then**

 └ **return** $\emptyset$

 Vergleiche B mit $M[i, m]$



Suchen in Suffixbäumen

SUCHE(S, P)

u = Wurzel von S

$$i = 1$$

while *u* ist kein Blatt **do**

Suche Kante $e = (u, v)$, dessen Beschriftung B mit $M[i]$ beginnt.

if e existiert nicht then

```

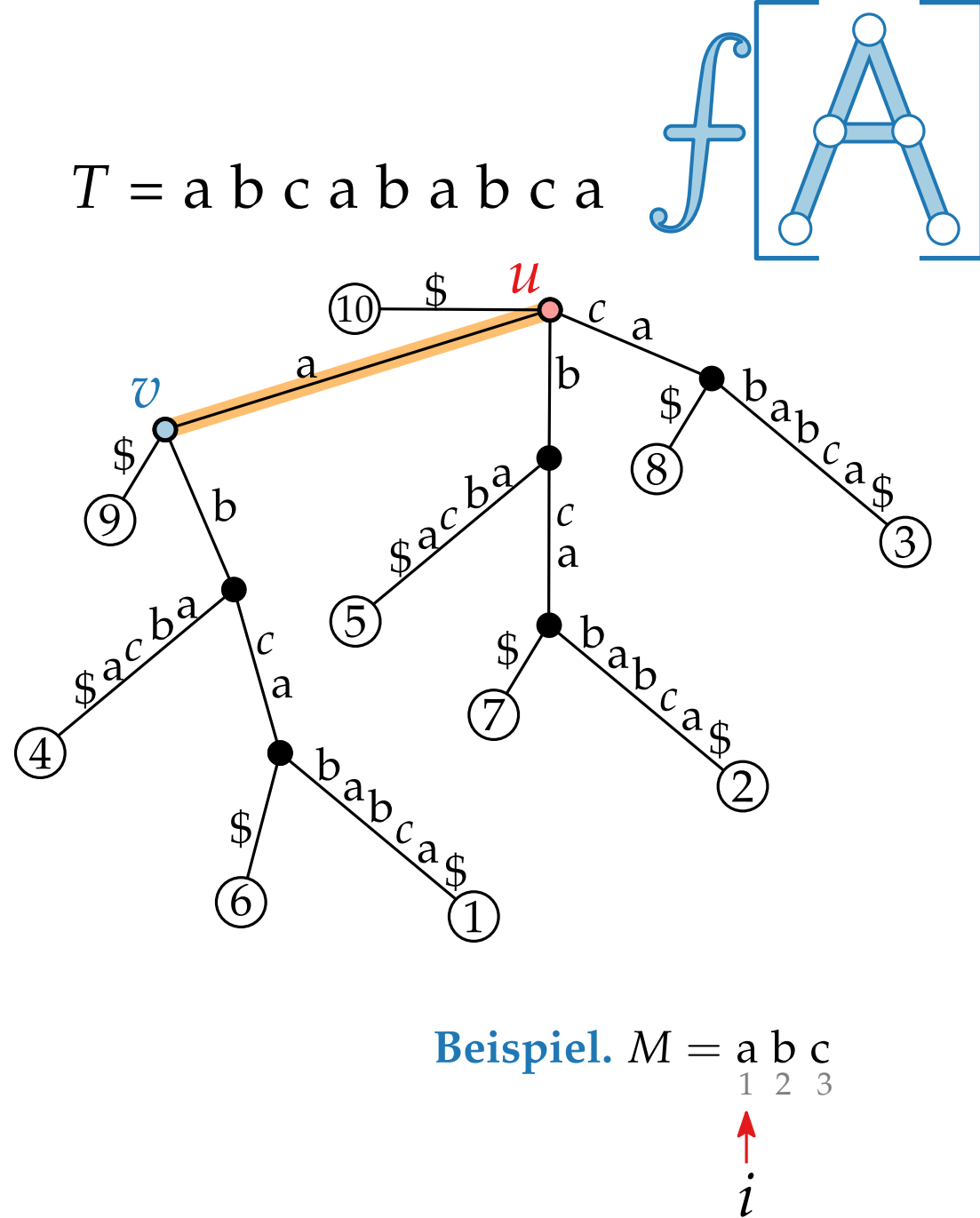
└─ return  $\emptyset$ 

```

Vergleiche B mit $M[i, m]$

if $M[i, m]$ ist Präfix von B then

return die Indizes aller Blätter im Teilbaum von v



Beispiel. $M = \begin{matrix} a & b & c \\ 1 & 2 & 3 \end{matrix}$

Suchen in Suffixbäumen

SUCHE(S, P)

u = Wurzel von S

$$i = 1$$

while *u* ist kein Blatt **do**

Suche Kante $e = (u, v)$, dessen Beschriftung B mit $M[i]$ beginnt.

if e existiert nicht then

```

└─ return  $\emptyset$ 

```

Vergleiche B mit $M[i, m]$

if $M[i, m]$ ist Präfix von B then

return die Indizes aller Blätter im Teilbaum von v

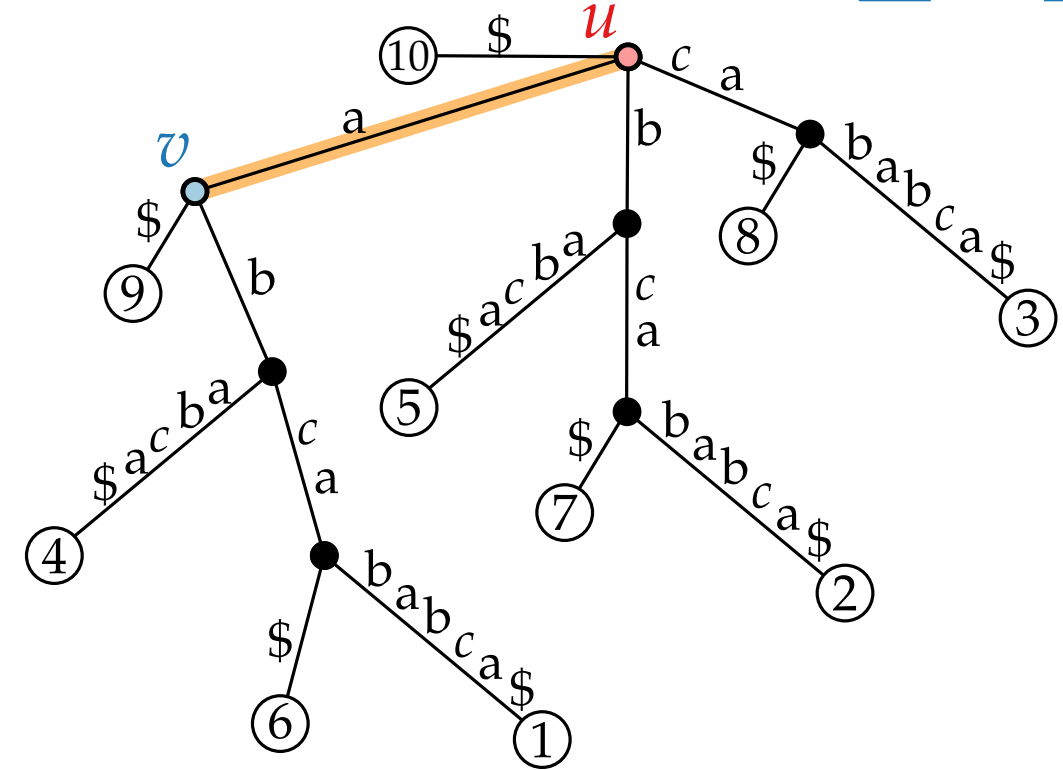
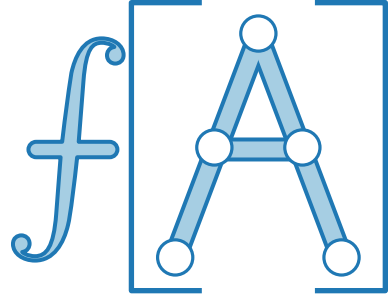
Übereinstimmung

$$B \quad \begin{array}{|c|c|c|c|c|} \hline \text{teal} & \text{teal} & \text{teal} & \text{white} & \text{white} \\ \hline \end{array}$$

|| || ||

 M

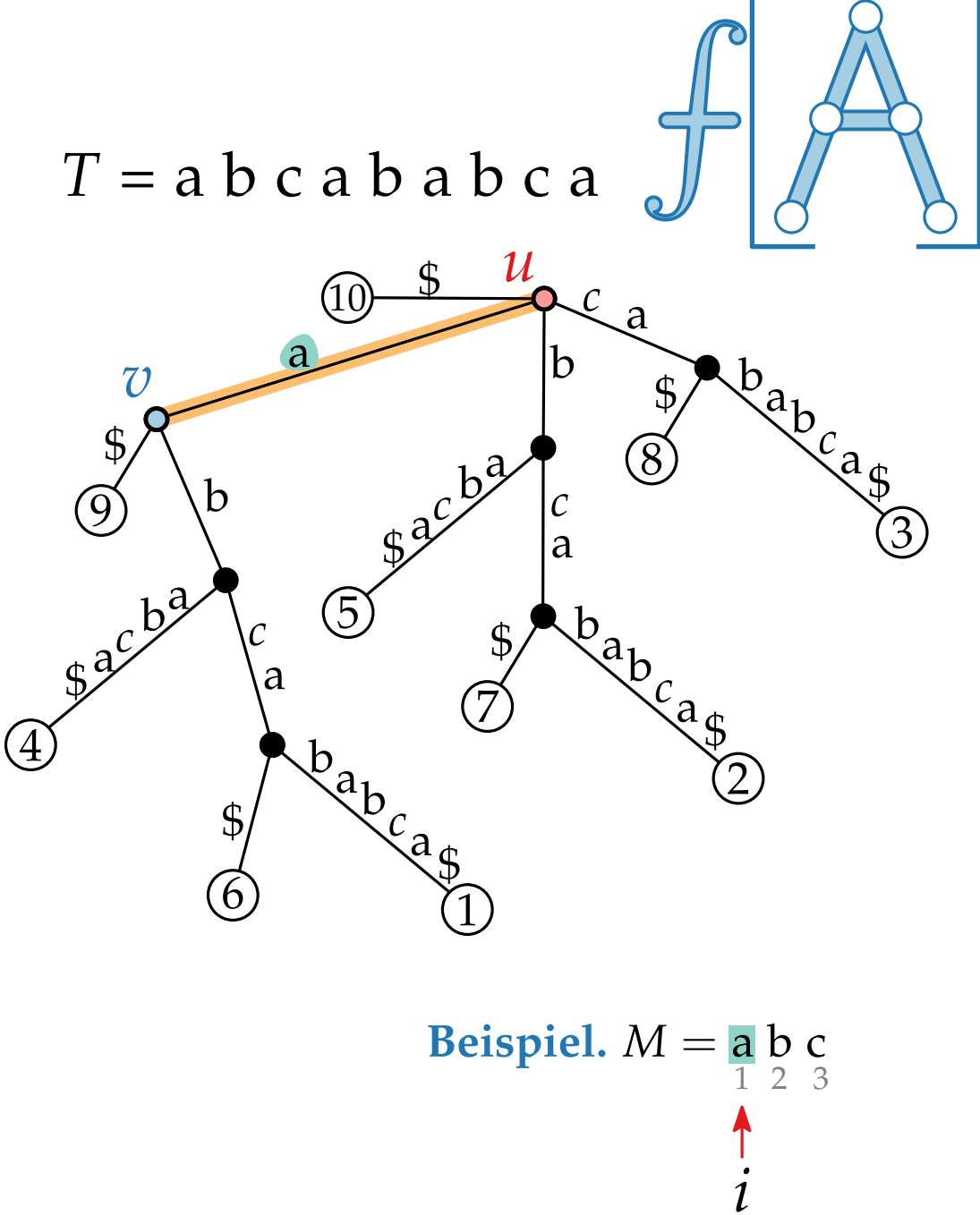
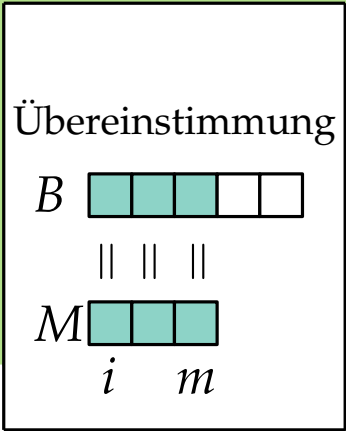
--	--	--

$$i \quad m$$
$$T = a \, b \, c \, a \, b \, a \, b \, c \, a$$


Beispiel. $M = \begin{matrix} & \mathbf{a} & \mathbf{b} & \mathbf{c} \\ & \mathbf{1} & \mathbf{2} & \mathbf{3} \\ & \uparrow \\ & i \end{matrix}$

Suchen in Suffixbäumen

```
SUCHE(S,P)
  u = Wurzel von S
  i = 1
  while u ist kein Blatt do
    Suche Kante e = (u,v), dessen Beschriftung B mit M[i] beginnt.
    if e existiert nicht then
      return ∅
    Vergleiche B mit M[i,m]
    if M[i,m] ist Präfix von B then
      return die Indizes aller Blätter im Teilbaum von v
```



Suchen in Suffixbäumen

SUCHE(S,P)

u = Wurzel von S

$i = 1$

while u ist kein Blatt **do**

 Suche Kante $e = (u,v)$, dessen Beschriftung B mit $M[i]$ beginnt.

if e existiert nicht **then**

return $\emptyset$

 Vergleiche B mit $M[i,m]$

if $M[i,m]$ ist Präfix von B **then**

return die Indizes aller Blätter im Teilbaum von v

else if $M[i,j] = B$ für ein $j < m$ **then**

$i = j + 1$

$u = v$

Übereinstimmung

B

--	--	--	--	--

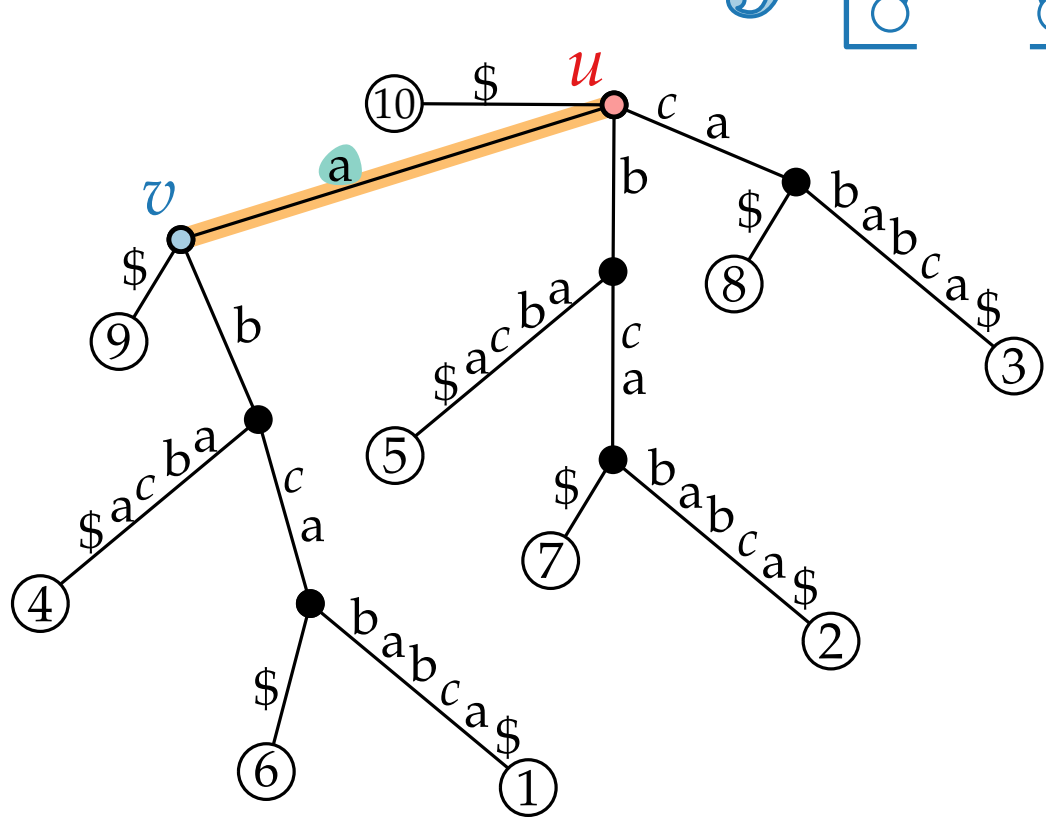
|| || ||

M

--	--	--

i m

$T = a b c a b a b c a$



Beispiel. $M = \begin{matrix} a & b & c \\ 1 & 2 & 3 \\ \uparrow \\ i \end{matrix}$

Suchen in Suffixbäumen

SUCHE(S,P)

u = Wurzel von S

$i = 1$

while u ist kein Blatt **do**

 Suche Kante $e = (u,v)$, dessen Beschriftung B mit $M[i]$ beginnt.

if e existiert nicht **then**

return $\emptyset$

 Vergleiche B mit $M[i,m]$

if $M[i,m]$ ist Präfix von B **then**

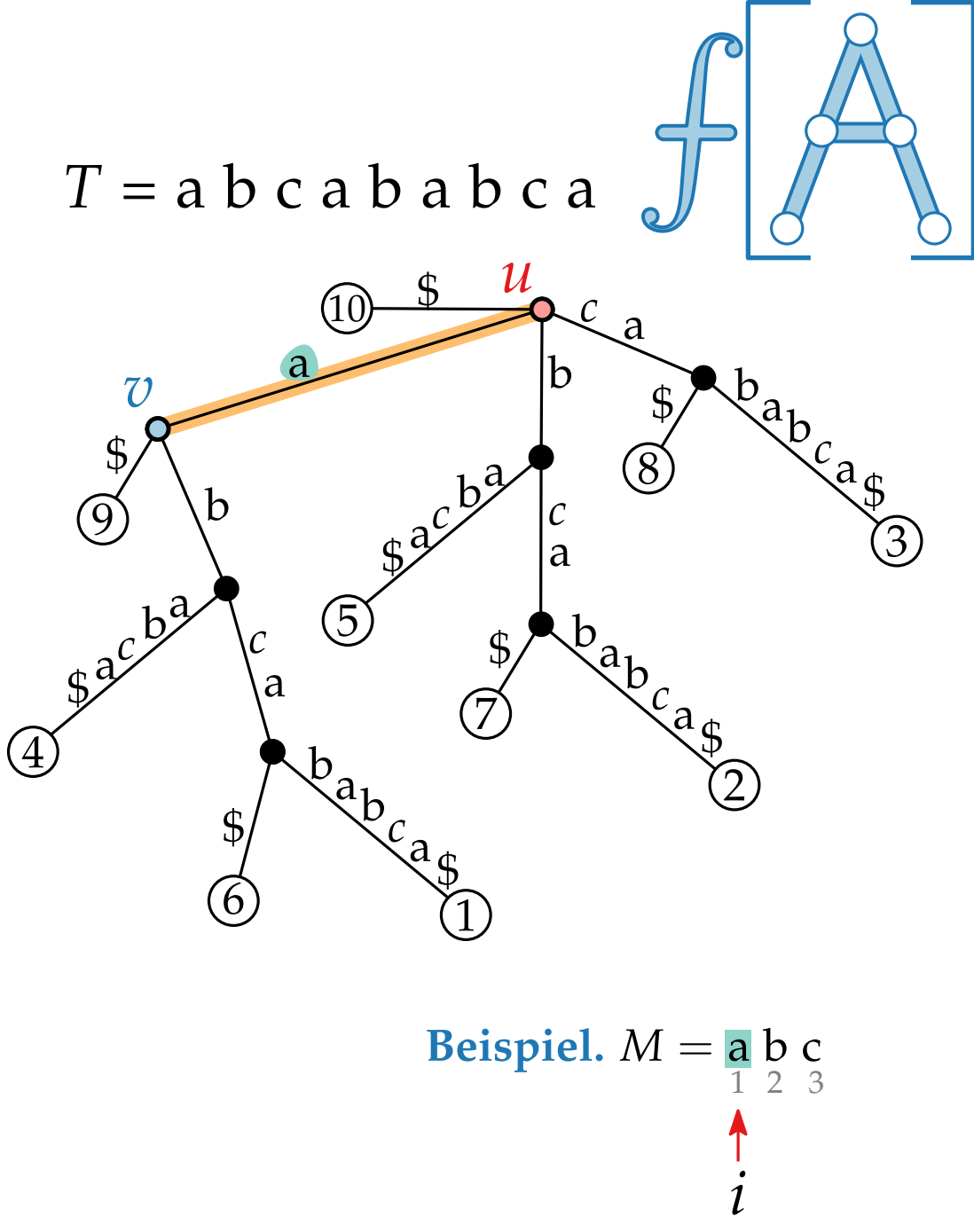
return die Indizes aller Blätter im Teilbaum von v

else if $M[i,j] = B$ für ein $j < m$ **then**

$i = j + 1$

$u = v$

	teilweise
Übereinstimmung	Übereinstimmung
B 	B 
M 	M 
i m	i j m



Suchen in Suffixbäumen

SUCHE(S,P)

u = Wurzel von S

$i = 1$

while u ist kein Blatt **do**

 Suche Kante $e = (u,v)$, dessen Beschriftung B mit $M[i]$ beginnt.

if e existiert nicht **then**

return $\emptyset$

 Vergleiche B mit $M[i,m]$

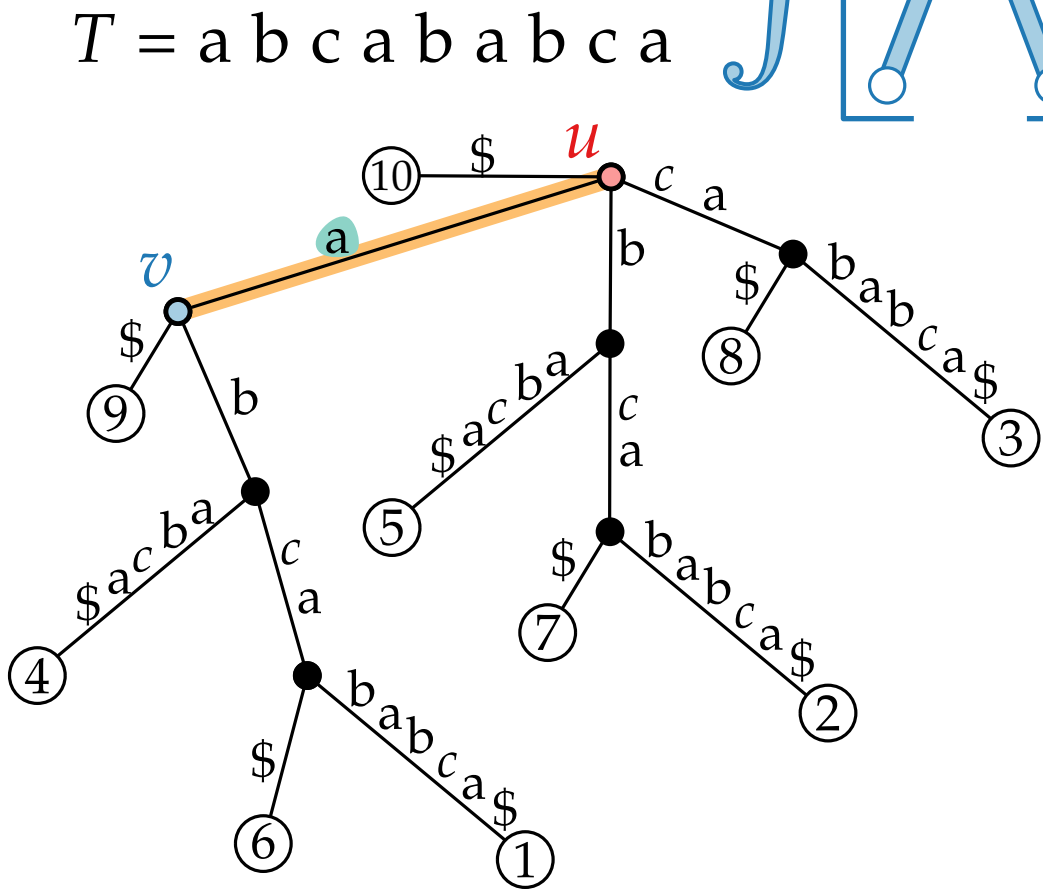
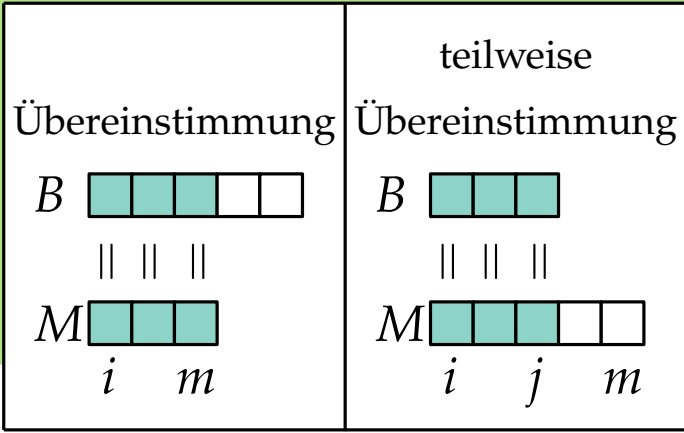
if $M[i,m]$ ist Präfix von B **then**

return die Indizes aller Blätter im Teilbaum von v

else if $M[i,j] = B$ für ein $j < m$ **then**

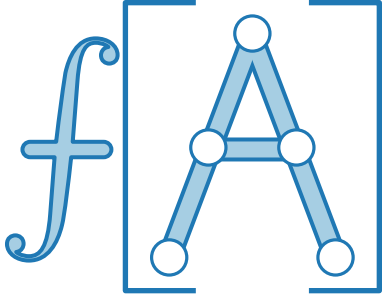
$i = j + 1$

$u = v$



Beispiel. $M = \begin{matrix} a & b & c \\ 1 & 2 & 3 \\ & \uparrow & \\ & i & \end{matrix}$

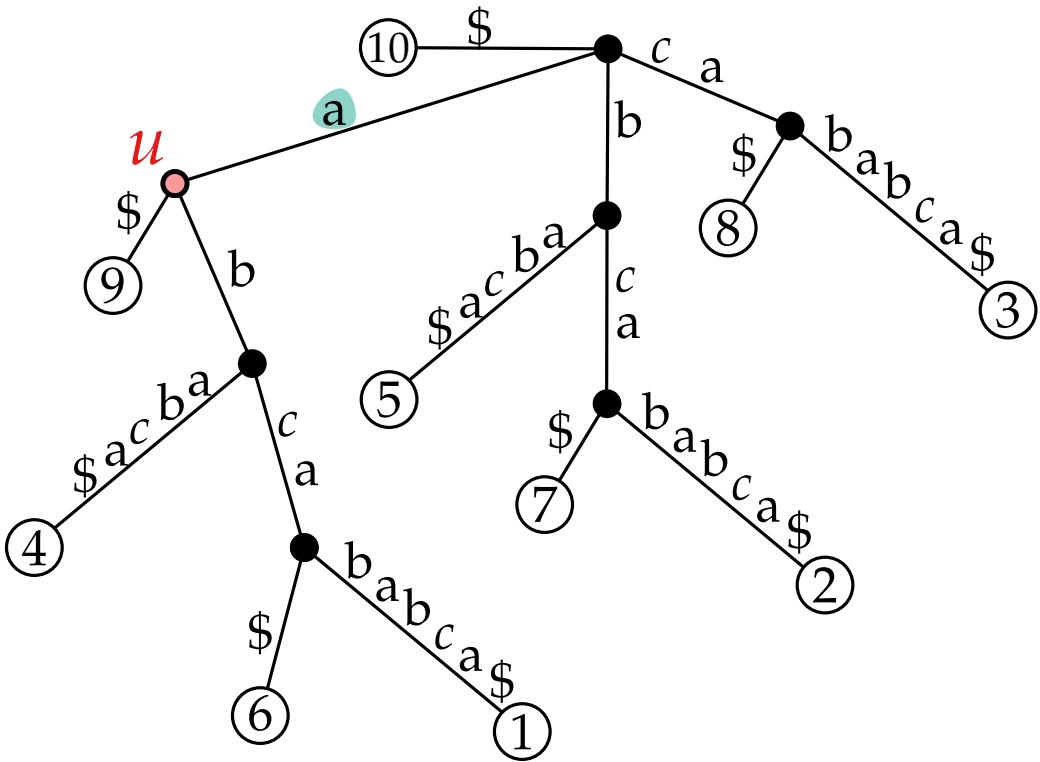
Suchen in Suffixbäumen



$T = a b c a b a b c a$

SUCHE(S,P)
 u = Wurzel von S
 $i = 1$
while u ist kein Blatt **do**
 Suche Kante $e = (u,v)$, dessen Beschriftung B mit $M[i]$ beginnt.
 if e existiert nicht **then**
 return $\emptyset$
 Vergleiche B mit $M[i,m]$
 if $M[i,m]$ ist Präfix von B **then**
 return die Indizes aller Blätter im Teilbaum von v
 else if $M[i,j] = B$ für ein $j < m$ **then**
 $i = j + 1$
 $u = v$

	teilweise
Übereinstimmung	Übereinstimmung
B	B
M	M
i m	i j m



Beispiel. $M = \begin{matrix} a & b & c \\ 1 & 2 & 3 \end{matrix}$
 $\uparrow$
 i

Suchen in Suffixbäumen

SUCHE(S,P)

u = Wurzel von S

$i = 1$

while u ist kein Blatt **do**

 Suche Kante $e = (u,v)$, dessen Beschriftung B mit $M[i]$ beginnt.

if e existiert nicht **then**

return $\emptyset$

 Vergleiche B mit $M[i,m]$

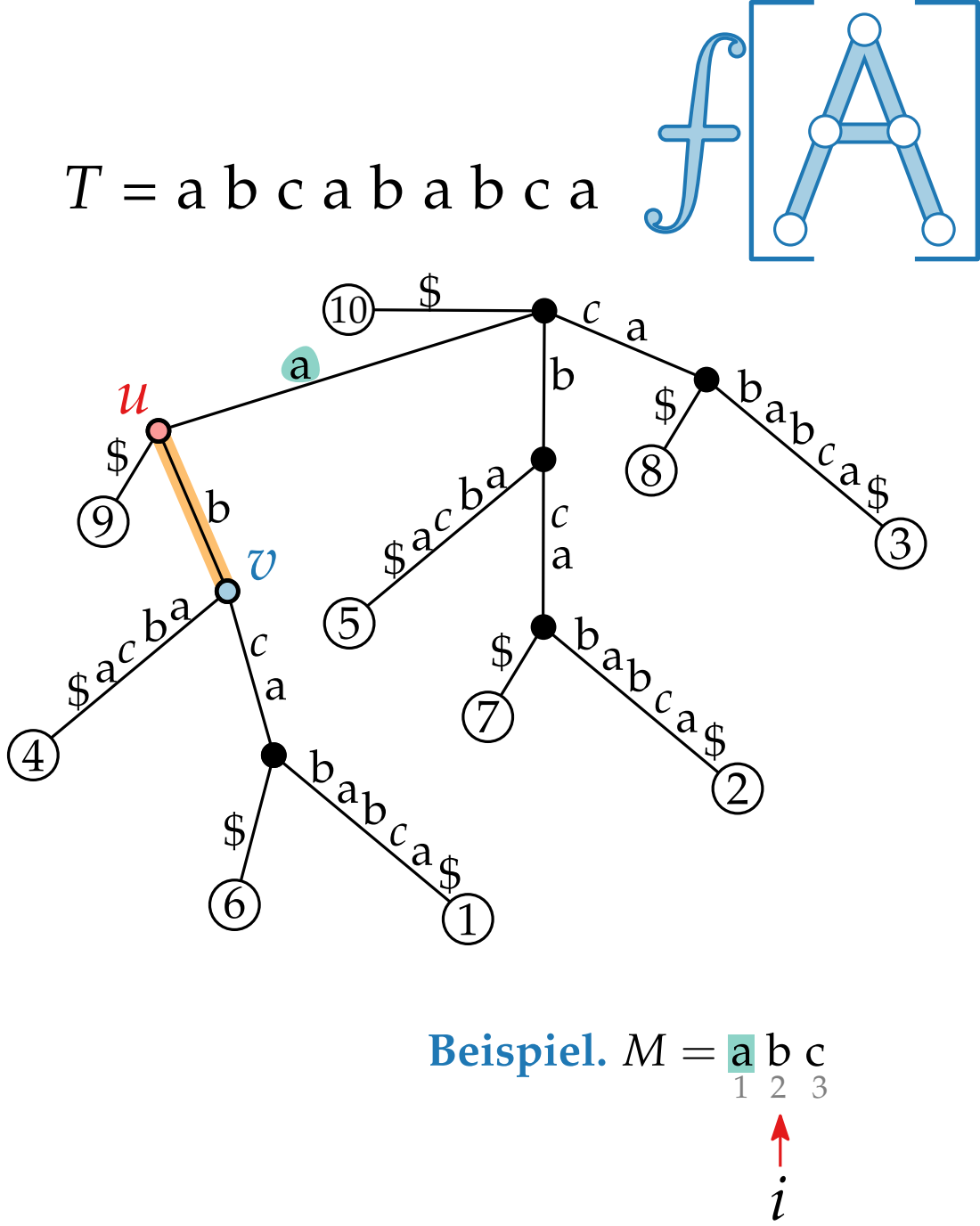
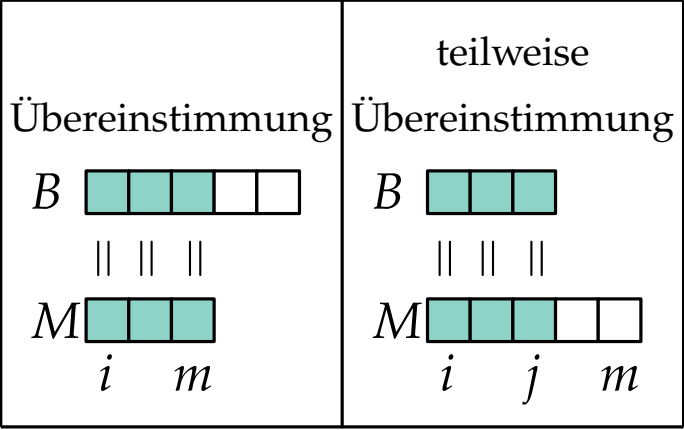
if $M[i,m]$ ist Präfix von B **then**

return die Indizes aller Blätter im Teilbaum von v

else if $M[i,j] = B$ für ein $j < m$ **then**

$i = j + 1$

$u = v$



Suchen in Suffixbäumen

SUCHE(S,P)

u = Wurzel von S

$i = 1$

while u ist kein Blatt **do**

 Suche Kante $e = (u,v)$, dessen Beschriftung B mit $M[i]$ beginnt.

if e existiert nicht **then**

return $\emptyset$

 Vergleiche B mit $M[i,m]$

if $M[i,m]$ ist Präfix von B **then**

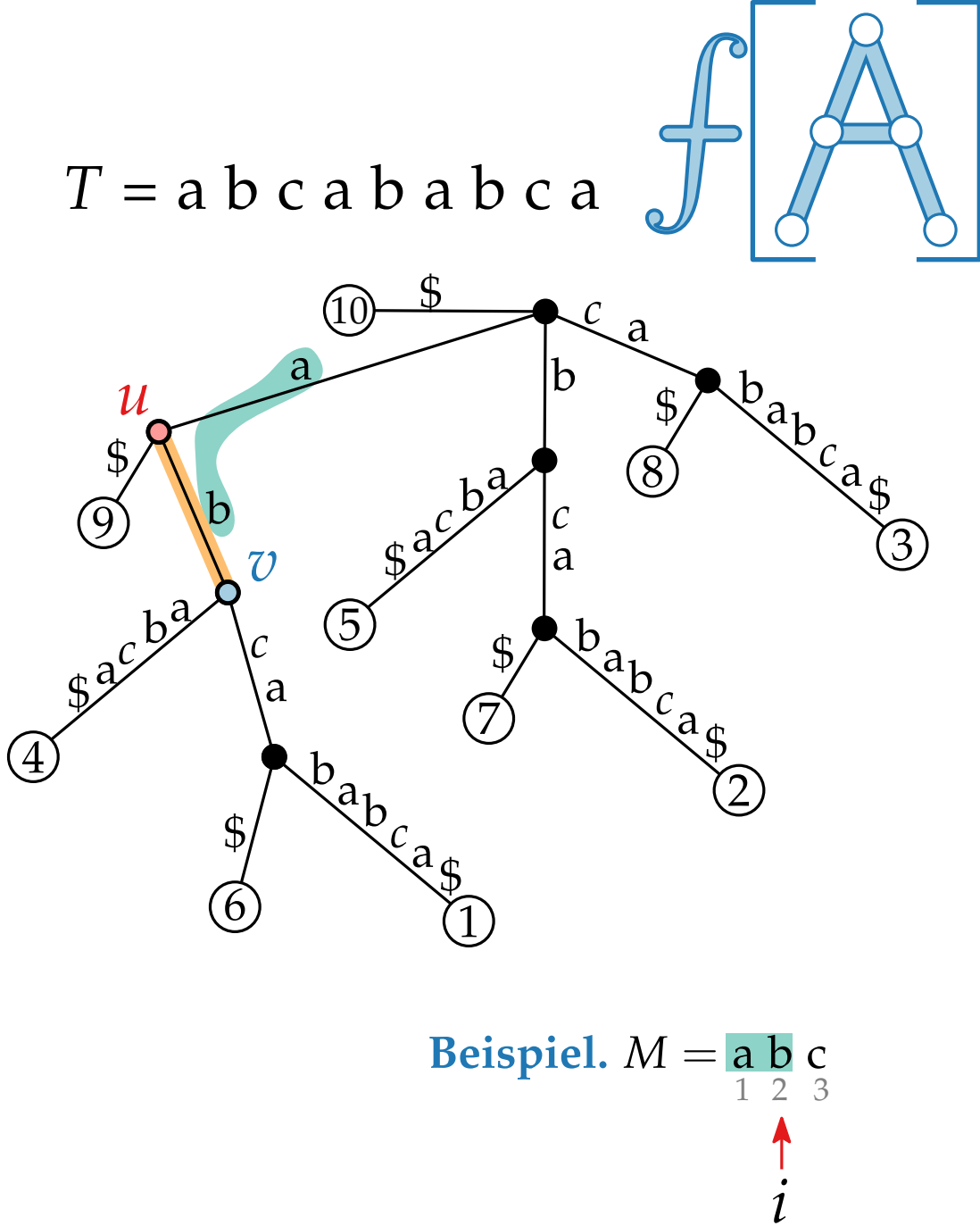
return die Indizes aller Blätter im Teilbaum von v

else if $M[i,j] = B$ für ein $j < m$ **then**

$i = j + 1$

$u = v$

Übereinstimmung	teilweise Übereinstimmung
B 	B 
$\parallel \parallel \parallel$	$\parallel \parallel \parallel$
M 	M 
$i \quad m$	$i \quad j \quad m$



Suchen in Suffixbäumen

SUCHE(S,P)

u = Wurzel von S

$i = 1$

while u ist kein Blatt **do**

 Suche Kante $e = (u,v)$, dessen Beschriftung B mit $M[i]$ beginnt.

if e existiert nicht **then**

return $\emptyset$

 Vergleiche B mit $M[i,m]$

if $M[i,m]$ ist Präfix von B **then**

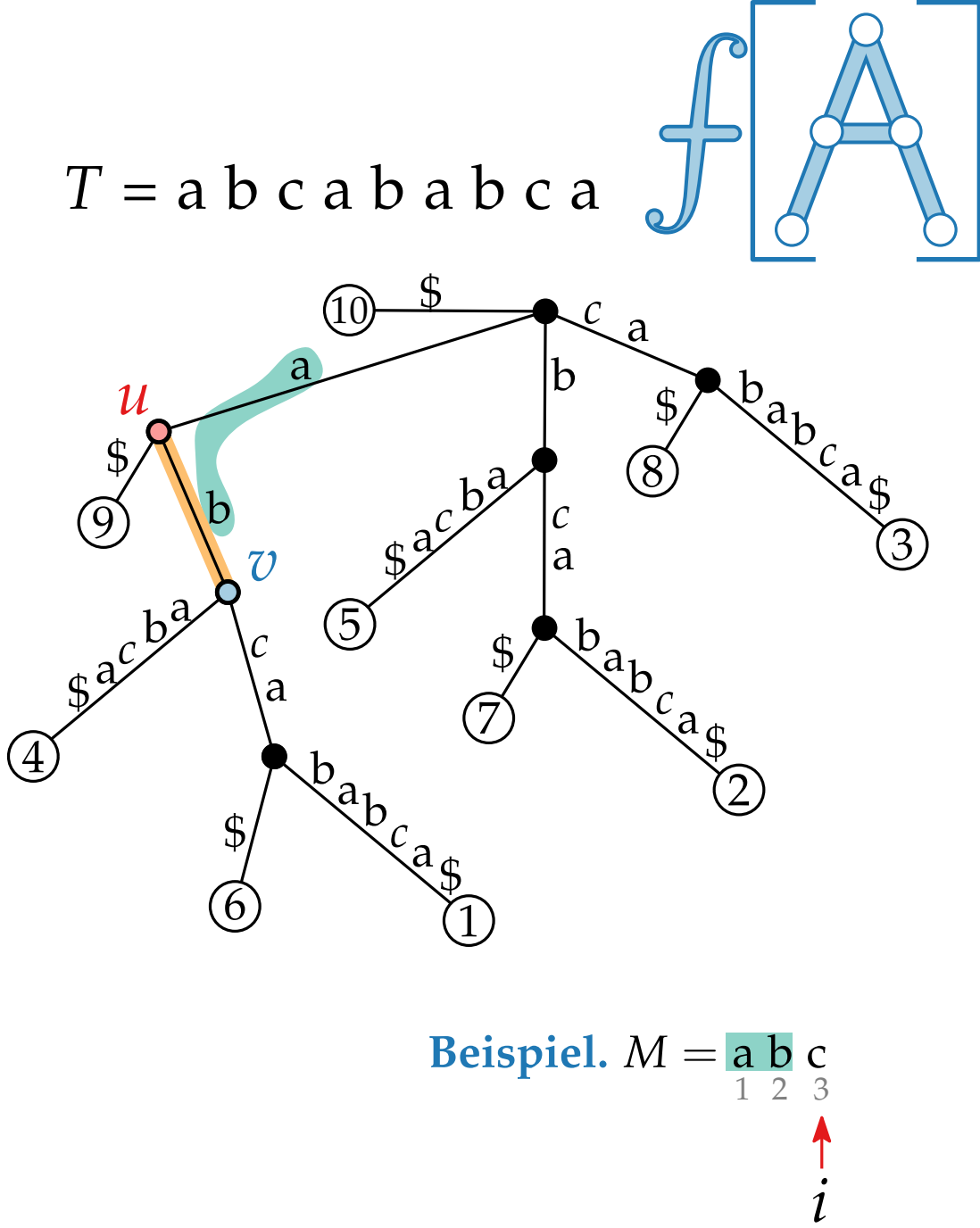
return die Indizes aller Blätter im Teilbaum von v

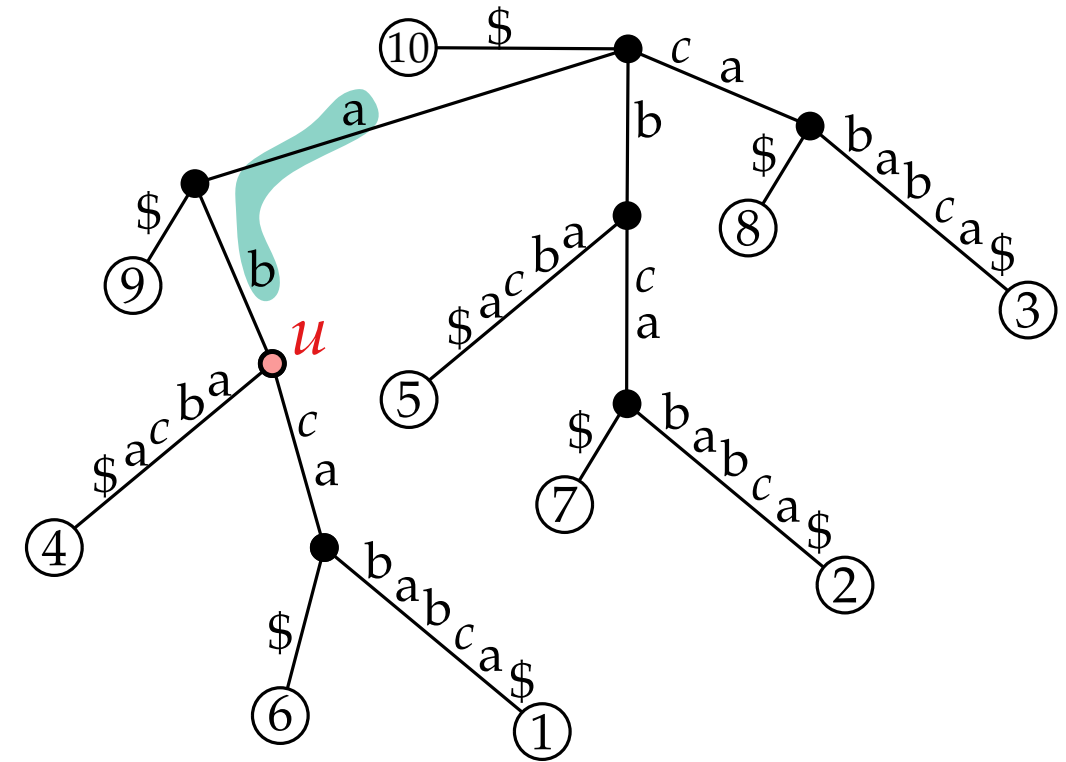
else if $M[i,j] = B$ für ein $j < m$ **then**

$i = j + 1$

$u = v$

Übereinstimmung	teilweise Übereinstimmung
B 	B 
$\parallel \parallel \parallel$	$\parallel \parallel \parallel$
M 	M 
$i \quad m$	$i \quad j \quad m$



$$T = a \, b \, c \, a \, b \, a \, b \, c \, a$$


Suchen in Suffixbäumen

SUCHE(S,P)

u = Wurzel von S

$i = 1$

while u ist kein Blatt **do**

 Suche Kante $e = (u,v)$, dessen Beschriftung B mit $M[i]$ beginnt.

if e existiert nicht **then**

return $\emptyset$

 Vergleiche B mit $M[i,m]$

if $M[i,m]$ ist Präfix von B **then**

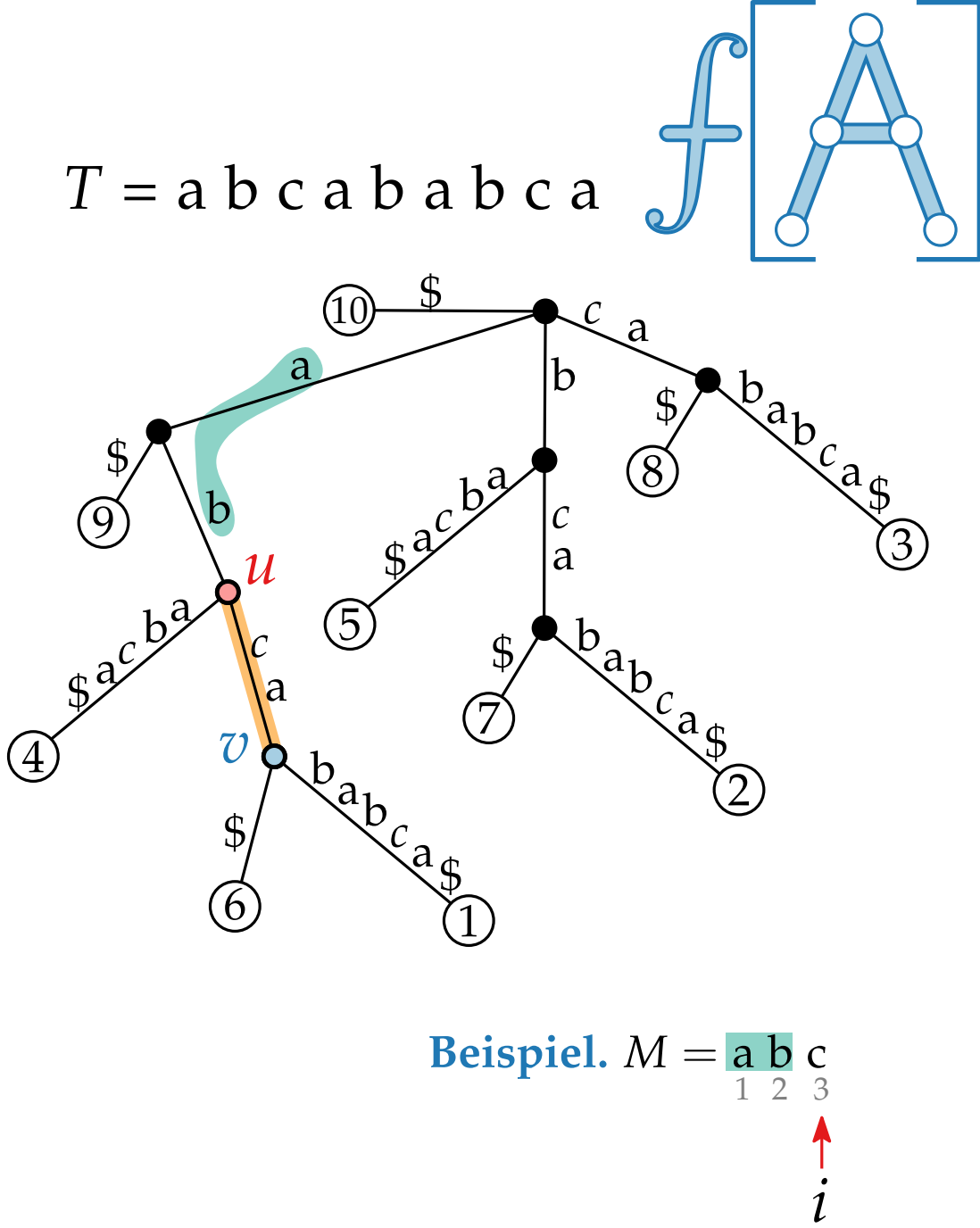
return die Indizes aller Blätter im Teilbaum von v

else if $M[i,j] = B$ für ein $j < m$ **then**

$i = j + 1$

$u = v$

Übereinstimmung	teilweise Übereinstimmung
B 	B 
$\parallel \parallel \parallel$	$\parallel \parallel \parallel$
M 	M 
$i \quad m$	$i \quad j \quad m$



Suchen in Suffixbäumen

SUCHE(S, P)

u = Wurzel von S

$$i = 1$$

while *u* ist kein Blatt **do**

Suche Kante $e = (u, v)$, dessen Beschriftung B mit $M[i]$ beginnt.

if e existiert nicht then

```

└─ return  $\emptyset$ 

```

Vergleiche B mit $M[i, m]$

if $M[i, m]$ ist Präfix von B then

return die Indizes aller Blätter im Teilbaum von v

else if $M[i, j] = B$ für ein $j < m$ then

$$i = j + 1$$
$$\mathcal{L} \quad u = v$$

Übereinstimmung

B

|| || ||

M

i m

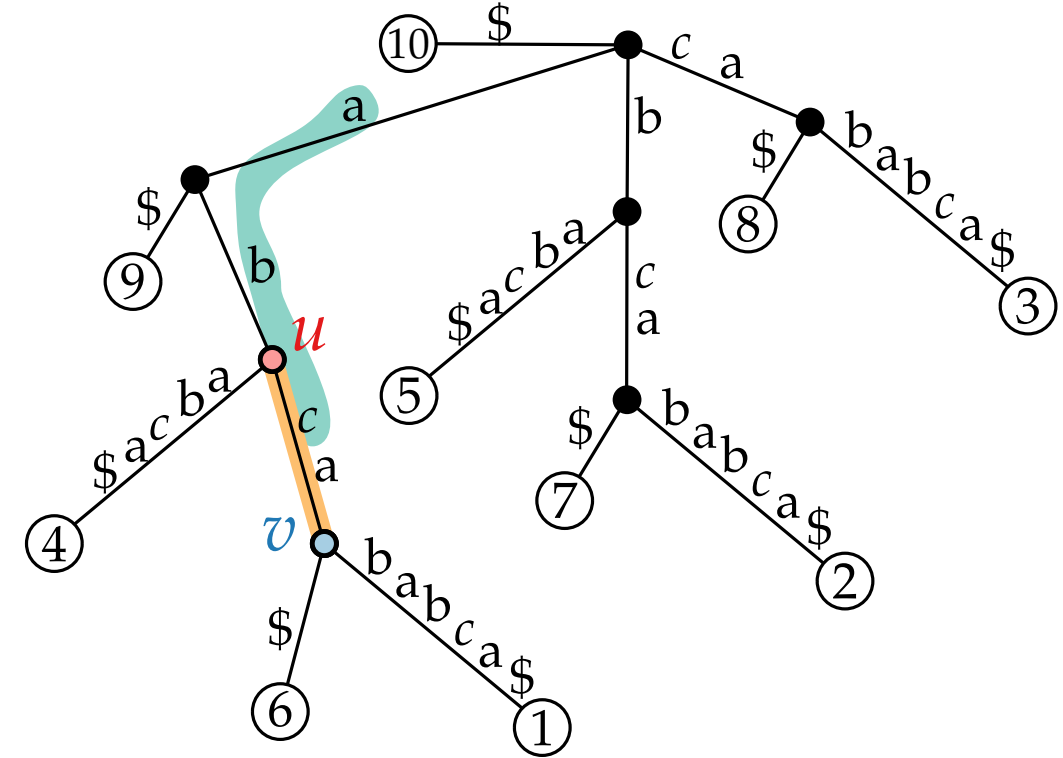
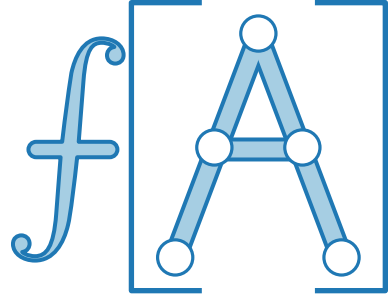
teilweise Übereinstimmung

B

|| || ||

M

i j m

$$T = a \, b \, c \, a \, b \, a \, b \, c \, a$$


Beispiel. $M = \begin{matrix} & \text{a} & \text{b} & \text{c} \\ \begin{matrix} 1 & 2 & 3 \end{matrix} & \end{matrix}$

i

Suchen in Suffixbäumen

SUCHE(S, P)

u = Wurzel von S

$$i = 1$$

while *u* ist kein Blatt **do**

Suche Kante $e = (u, v)$, dessen Beschriftung B mit $M[i]$ beginnt.

if e existiert nicht then

```

└─ return  $\emptyset$ 

```

Vergleiche B mit $M[i, m]$

if $M[i, m]$ ist Präfix von B then

```
return die Indizes aller Blätter im Teilbaum von  $v$ 
```

else if $M[i, j] = B$ für ein $j < m$ then

$$i = j + 1$$
$$\mathcal{L} \quad u = v$$

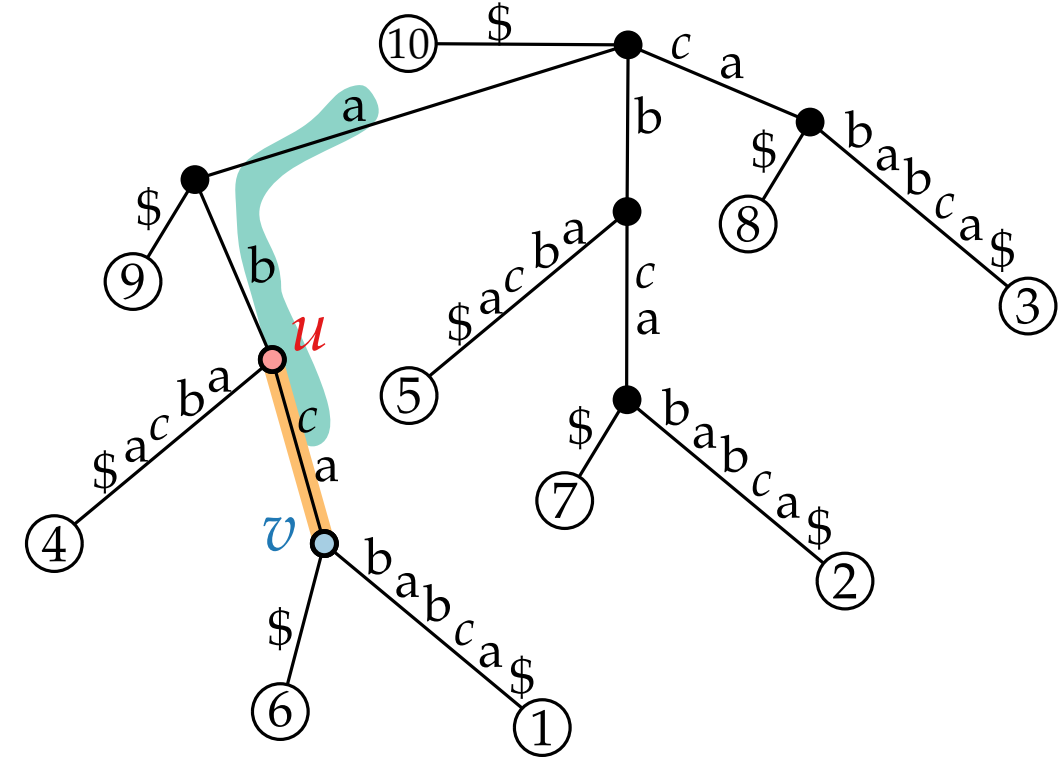
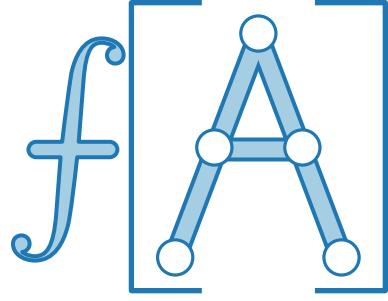
else

```

| return  $\emptyset$ 

```

Übereinstimmung	teilweise Übereinstimmung
B 	B 
M 	M 
i m	i j m

$$T = a \, b \, c \, a \, b \, a \, b \, c \, a$$


Beispiel. $M = \begin{matrix} & \text{a} & \text{b} & \text{c} \\ \begin{matrix} 1 & 2 & 3 \end{matrix} & \end{matrix}$

$\uparrow$
 i

$$T = a \, b \, c \, a \, b \, a \, b \, c \, a$$
$$i = 1$$

Suche Kante $e = (u, v)$, dessen Beschriftung B mit $M[i]$ beginnt.

```

└─ return  $\emptyset$ 

```

if $M[i, m]$ ist Präfix von B then

return die Indizes aller Blätter im Teilbaum von v

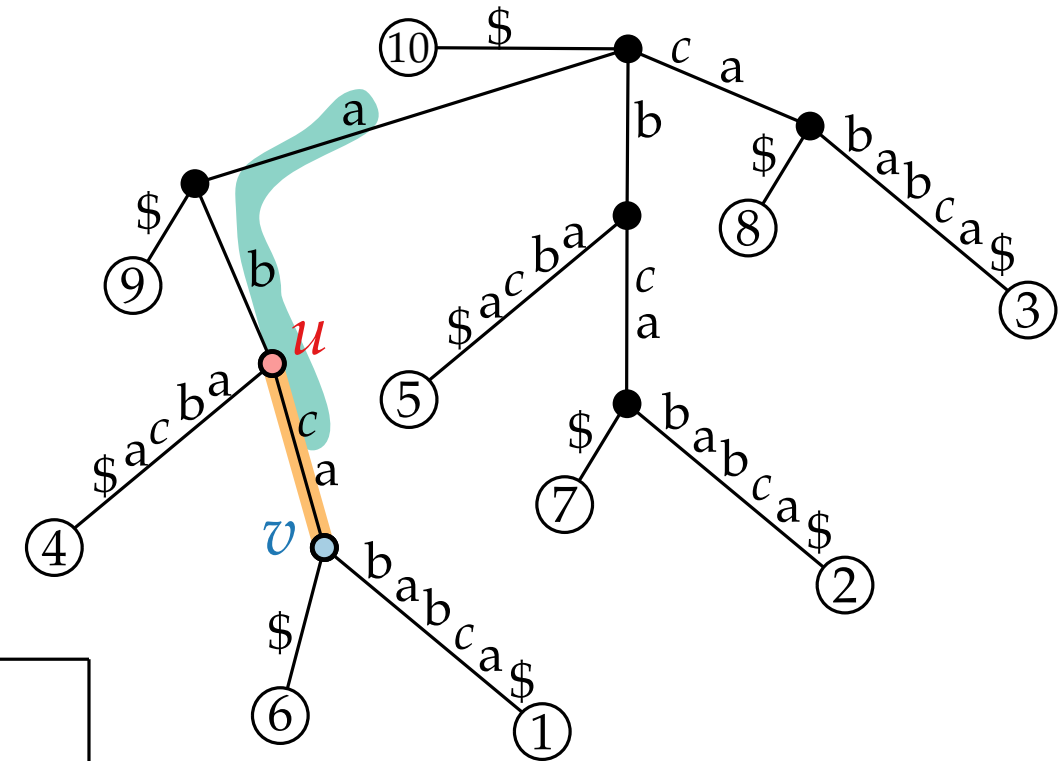
$$| \quad i = j + 1$$
$$u = v$$

```

| return  $\emptyset$ 

```

	teilweise	keine
Übereinstimmung	Übereinstimmung	Übereinstimmung
B   M  i m	B   M  i j m	B   M  i



Beispiel. $M = \begin{matrix} \text{a} & \text{b} & \text{c} \\ 1 & 2 & 3 \end{matrix}$


 i

$$T = a \, b \, c \, a \, b \, a \, b \, c \, a$$
$$i = 1$$

Suche Kante $e = (u, v)$, dessen Beschriftung B mit $M[i]$ beginnt.

```

└─ return  $\emptyset$ 

```

if $M[i, m]$ ist Präfix von B then

return die Indizes aller Blätter im Teilbaum von v

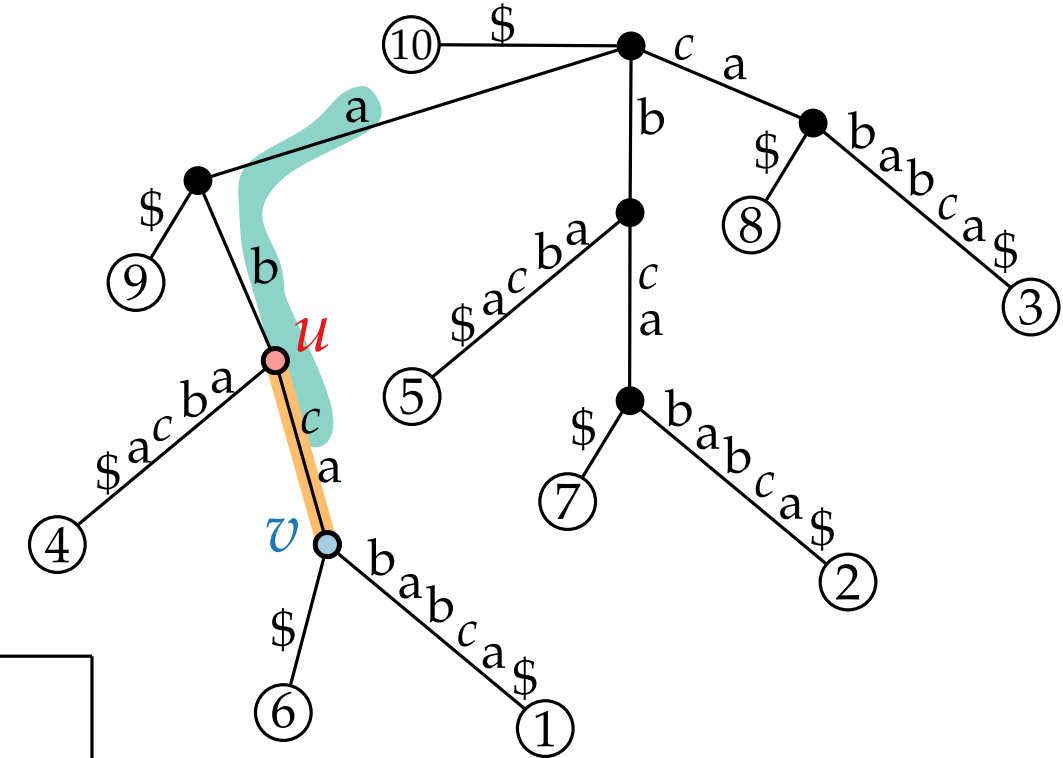
$$|i = j + 1|$$
$$u = v$$

```

|   return  $\emptyset$ 

```

return $\emptyset$



Beispiel. $M = \begin{matrix} & \text{a} & \text{b} & \text{c} \\ \begin{matrix} 1 & 2 & 3 \end{matrix} & \end{matrix}$

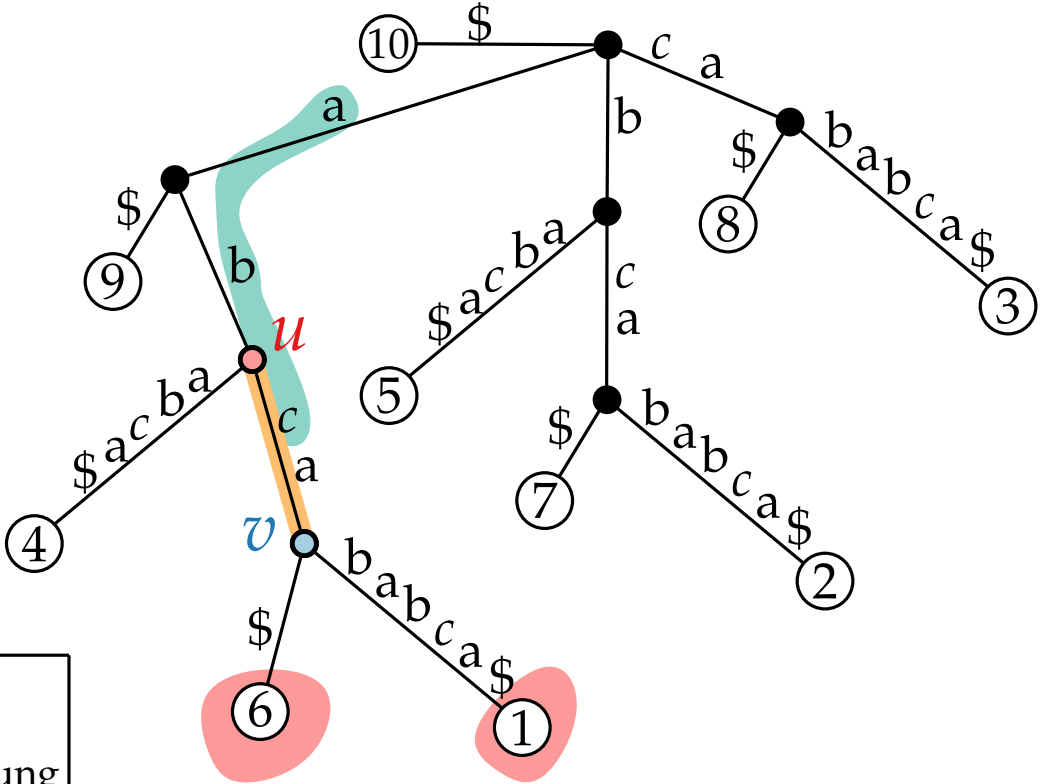
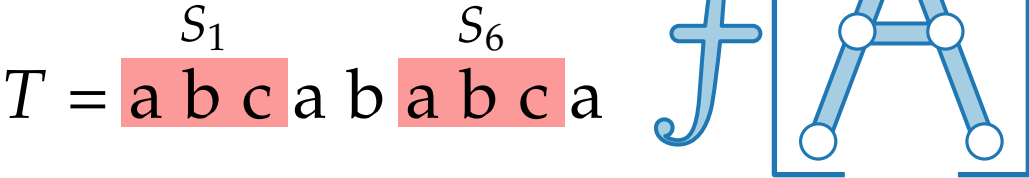


	teilweise	keine
Übereinstimmung	Übereinstimmung	Übereinstimmung
B   M  i m	B   M  i j m	B   M  i

Suchen in Suffixbäumen

```
SUCHE(S,P)
  u = Wurzel von S
  i = 1
  while u ist kein Blatt do
    Suche Kante e = (u,v), dessen Beschriftung B mit M[i] beginnt.
    if e existiert nicht then
      return ∅
    Vergleiche B mit M[i,m]
    if M[i,m] ist Präfix von B then
      return die Indizes aller Blätter im Teilbaum von v
    else if M[i,j] = B für ein j < m then
      i = j + 1
      u = v
    else
      return ∅
  return ∅
```

	Übereinstimmung	teilweise Übereinstimmung	keine Übereinstimmung
B			
			
M			
	i m	i j m	i

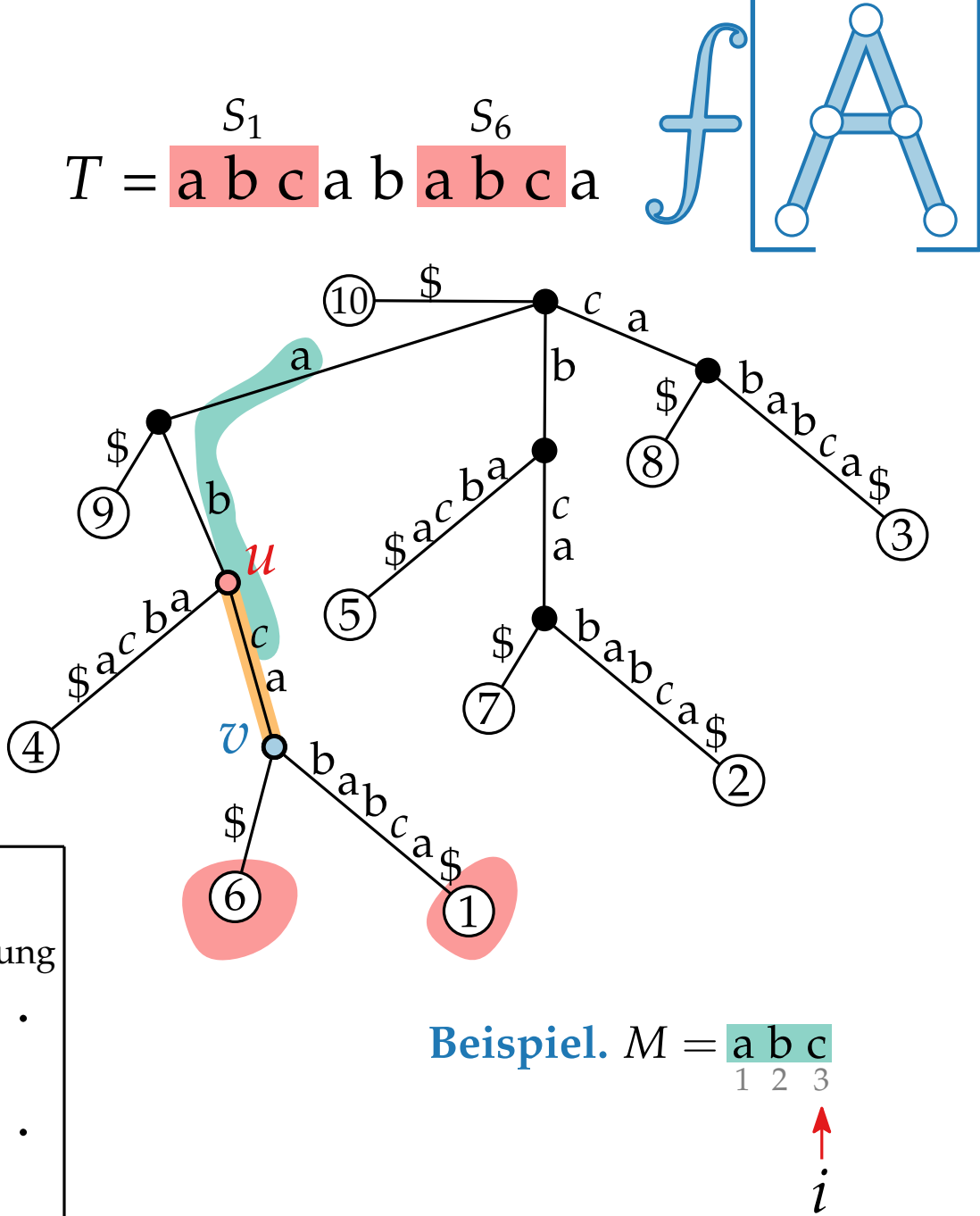


Beispiel. $M = \text{a b c}$
 1 2 3
 ↑
 i

Suchen in Suffixbäumen

```
SUCHE(S,P)
  u = Wurzel von S
  i = 1
  while u ist kein Blatt do
    Suche Kante e = (u,v), dessen Beschriftung B mit M[i] beginnt.
    if e existiert nicht then
      return ∅
    Vergleiche B mit M[i,m]
    if M[i,m] ist Präfix von B then
      return die Indizes aller Blätter im Teilbaum von v
    else if M[i,j] = B für ein j < m then
      i = j + 1
      u = v
    else
      return ∅
  return ∅
```

	Übereinstimmung	teilweise Übereinstimmung	keine Übereinstimmung
B			
			
M			
	i m	i j m	i



Korrektheit. Jedes Vorkommen von M ist ein Präfix von genau einem Suffix von T .
Wir geben alle Suffixe mit M als Präfix zurück.

	teilweise	keine
Übereinstimmung	Übereinstimmung	Übereinstimmung
B   M  i m	B   M  i j m	B   M  i

i

Korrektheit.

Suchen in Suffixbäumen

SUCHE(S, P)

u = Wurzel von S

$$i = 1$$

while *u* ist kein Blatt **do**

$$\mathcal{O}(\log |\Sigma|)$$

Suche Kante $e = (u, v)$, dessen Beschriftung B mit $M[i]$ beginnt.

if e existiert nicht then

```

└─ return  $\emptyset$ 

```

Vergleiche B mit $M[i, m]$

if $M[i, m]$ ist Präfix von B then

return die Indizes aller Blätter im Teilbaum von v

 $\mathcal{O}(k)$ insgesamt

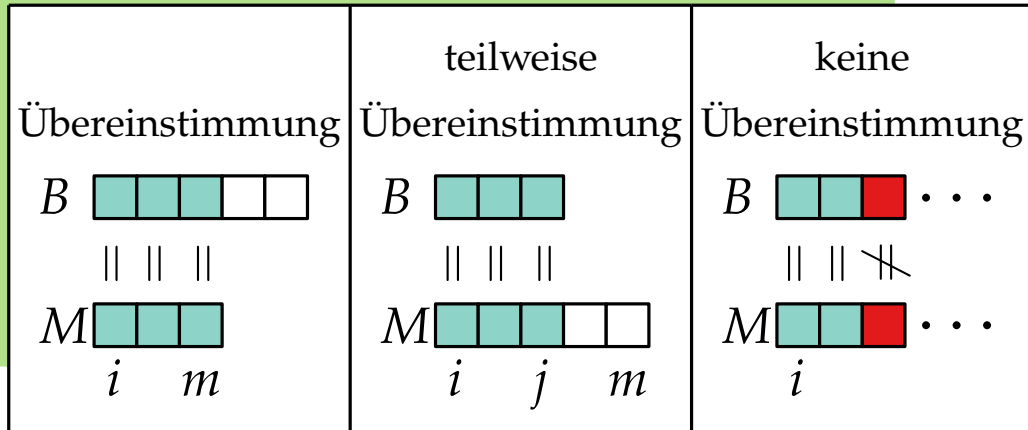
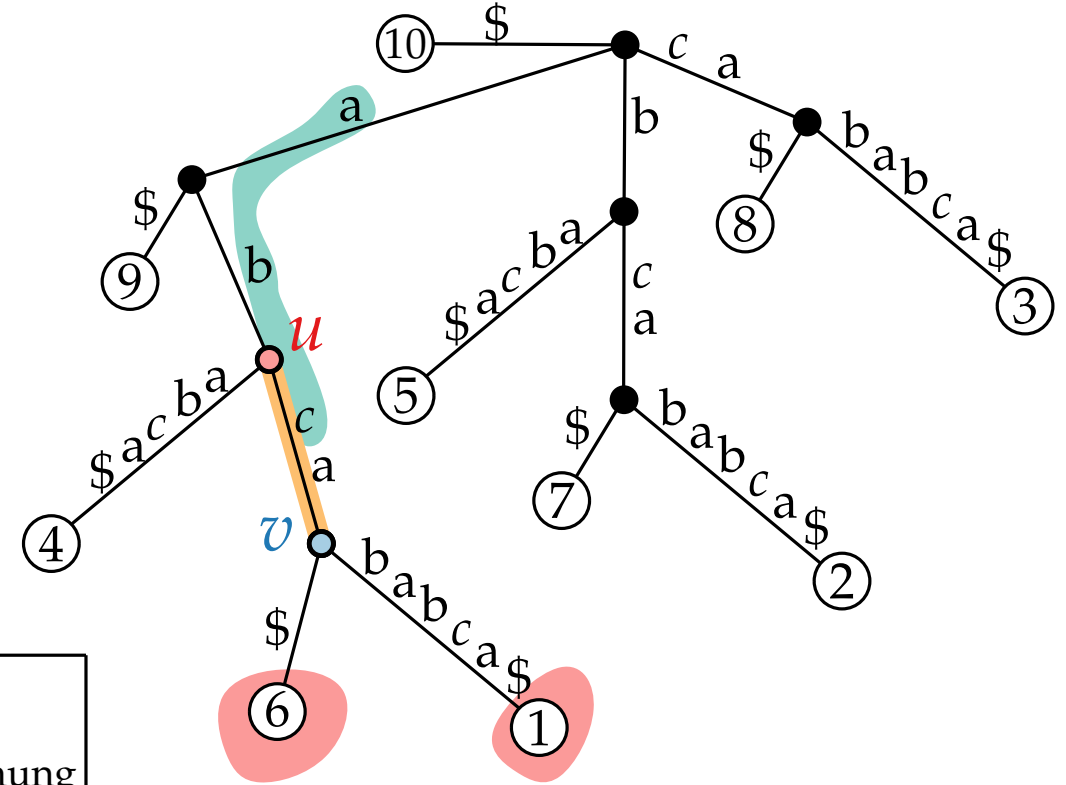
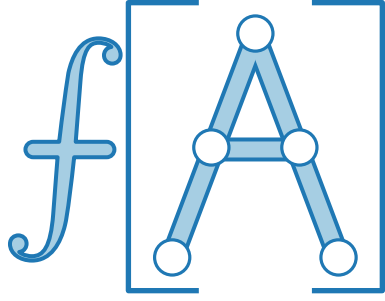
else if $M[i, j] = B$ für ein $j < m$ then

$$i = j + 1$$
$$u = v$$

else

```
return  $\emptyset$ 
```

return $\emptyset$


$$T = \overset{S_1}{\text{a b c}} \text{ a b } \overset{S_6}{\text{a b c}} \text{ a}$$


Beispiel. $M = \begin{matrix} & \text{a} & \text{b} & \text{c} \\ \begin{matrix} 1 & 2 & 3 \end{matrix} & \end{matrix}$

●

Korrektheit. Jedes Vorkommen von M ist ein Präfix von genau einem Suffix von T . Wir geben alle Suffixe mit M als Präfix zurück.

Suchen in Suffixbäumen

SUCHE(S,P)

u = Wurzel von S

$i = 1$

while u ist kein Blatt **do** $O(\log |\Sigma|)$

Suche Kante $e = (u,v)$, dessen Beschriftung B mit $M[i]$ beginnt.

if e existiert nicht **then**

└ **return** $\emptyset$

Vergleiche B mit $M[i,m]$ m Vergleiche insgesamt

if $M[i,m]$ ist Präfix von B **then**

└ **return** die Indizes aller Blätter im Teilbaum von v

else if $M[i,j] = B$ für ein $j < m$ **then** $O(k)$ insgesamt

└ $i = j + 1$

└ $u = v$

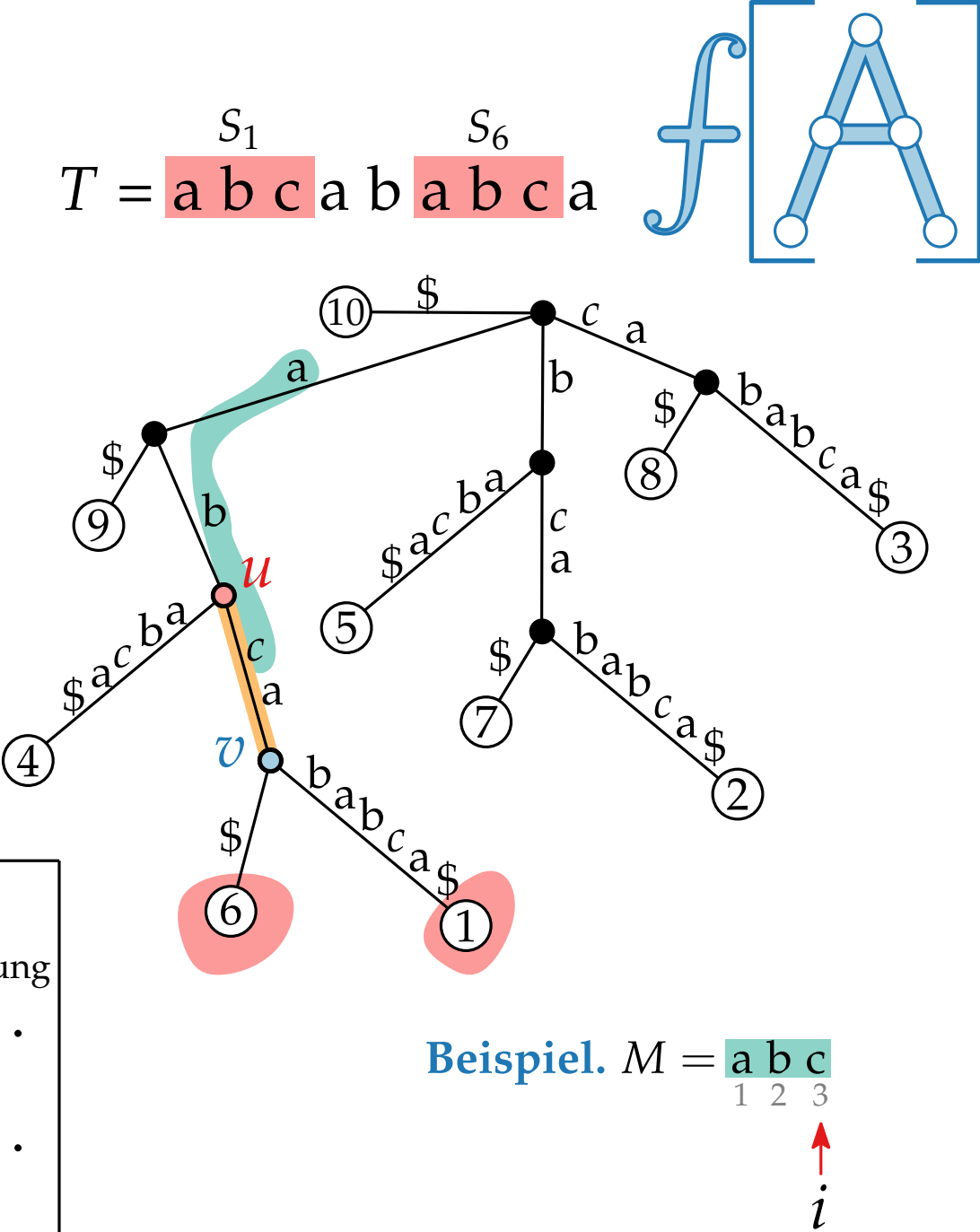
else

└ **return** $\emptyset$

return $\emptyset$

Übereinstimmung	teilweise Übereinstimmung	keine Übereinstimmung														
B <table><tr><td></td><td></td><td></td><td></td><td></td></tr></table>						B <table><tr><td></td><td></td><td></td><td></td></tr></table>					B <table><tr><td></td><td></td><td></td><td></td><td></td></tr></table>					
M <table><tr><td></td><td></td><td></td><td></td></tr></table>					M <table><tr><td></td><td></td><td></td><td></td><td></td></tr></table>						M <table><tr><td></td><td></td><td></td><td></td><td></td></tr></table>					
i m	i j m	i														

Korrektheit. Jedes Vorkommen von M ist ein Präfix von genau einem Suffix von T .
Wir geben alle Suffixe mit M als Präfix zurück.



$\mathcal{O}(k)$ insgesamt

Beispiel. $M = \begin{matrix} \text{a} & \text{b} & \text{c} \\ 1 & 2 & 3 \end{matrix}$


 i

Korrektheit. Jedes Vorkommen von M ist ein Präfix von genau einem Suffix von T .
Wir geben alle Suffixe mit M als Präfix zurück.

Suchen in Suffixbäumen

SUCHE(S,P)

u = Wurzel von S

$i = 1$

while u ist kein Blatt **do**

Suche Kante $e = (u,v)$, dessen Beschriftung B mit $M[i]$ beginnt.

if e existiert nicht **then**

└ **return** $\emptyset$

Vergleiche B mit $M[i,m]$

if $M[i,m]$ ist Präfix von B **then**

└ **return** die Indizes aller Blätter im Teilbaum von v

else if $M[i,j] = B$ für ein $j < m$ **then**

└ $i = j + 1$

└ $u = v$

else

└ **return** $\emptyset$

return $\emptyset$

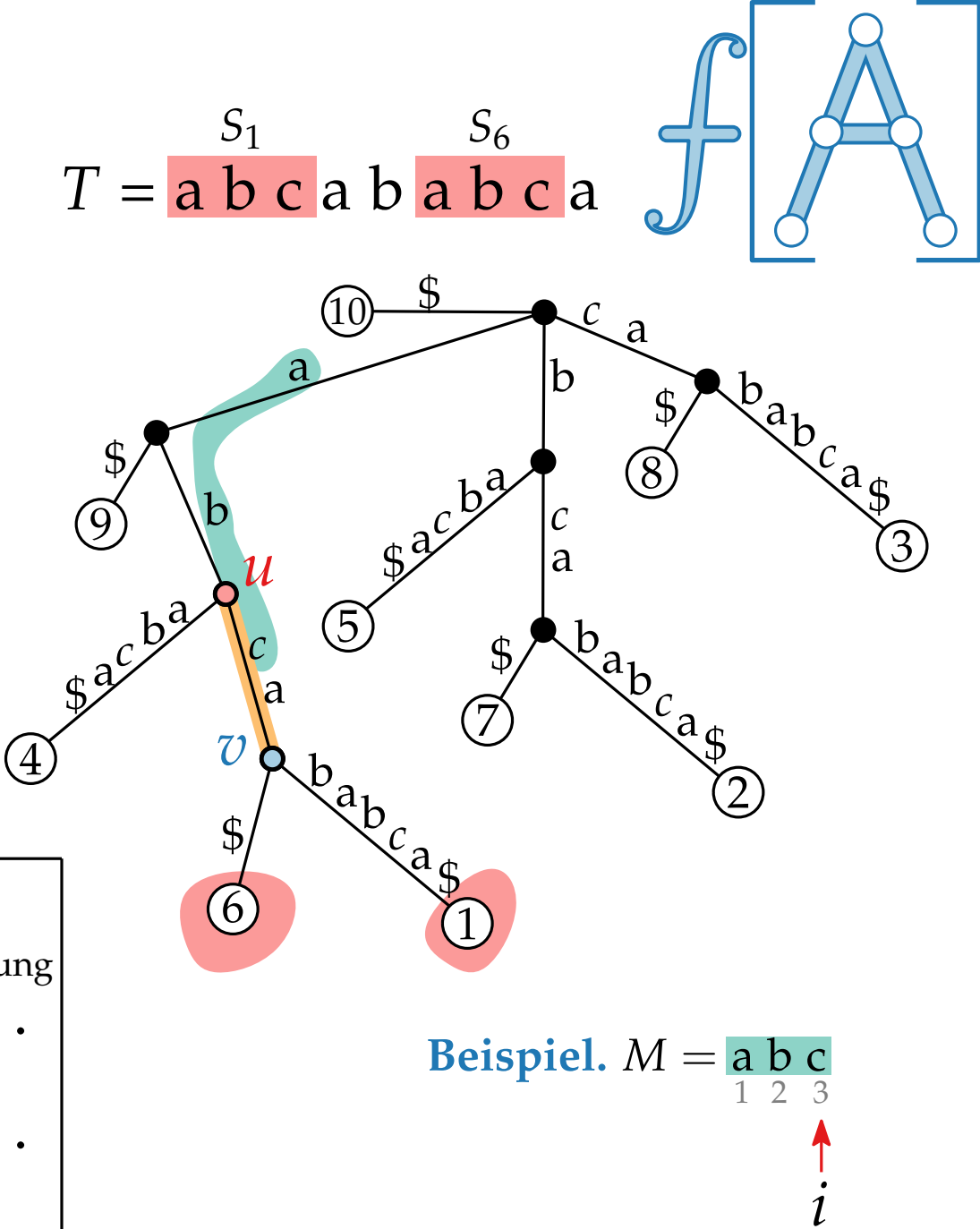
$\leq m$ Iterationen

$\mathcal{O}(\log |\Sigma|)$

m Vergleiche insgesamt

$\mathcal{O}(k)$ insgesamt

Übereinstimmung	teilweise Übereinstimmung	keine Übereinstimmung
B M i m	B M i j m	B M i



Korrektheit. Jedes Vorkommen von M ist ein Präfix von genau einem Suffix von T . Wir geben alle Suffixe mit M als Präfix zurück.

Laufzeit. $\mathcal{O}(m \log |\Sigma| + k)$, wobei k die Anzahl der Vorkommen von M in T ist.

Suchen in Suffixbäumen

SUCHE(S,P)

u = Wurzel von S

$i = 1$

while u ist kein Blatt **do**

Suche Kante $e = (u,v)$, dessen Beschriftung B mit $M[i]$ beginnt.

if e existiert nicht **then**

└ **return** $\emptyset$

Vergleiche B mit $M[i,m]$

if $M[i,m]$ ist Präfix von B **then**

└ **return** die Indizes aller Blätter im Teilbaum von v

else if $M[i,j] = B$ für ein $j < m$ **then**

└ $i = j + 1$

└ $u = v$

else

└ **return** $\emptyset$

return $\emptyset$

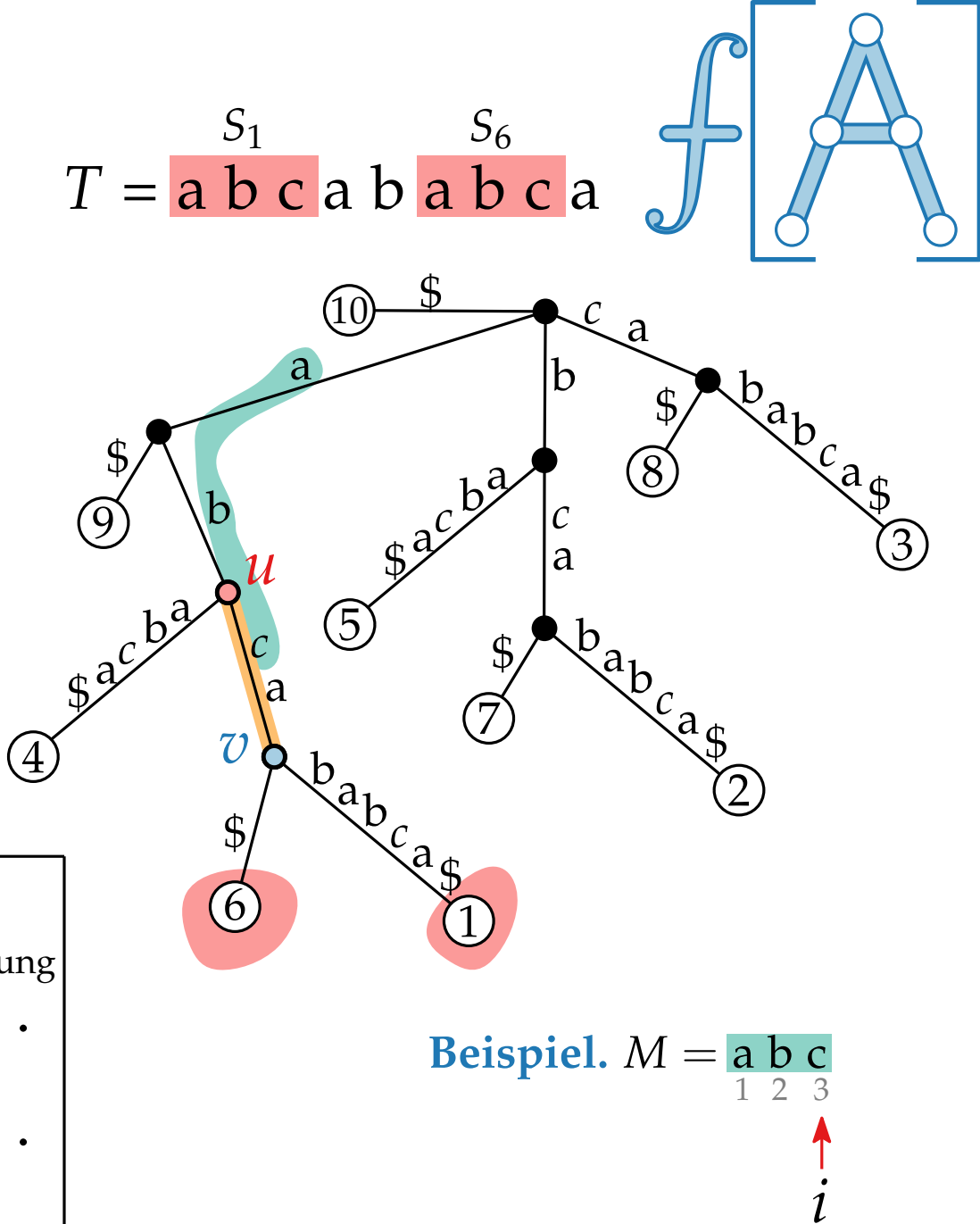
$\leq m$ Iterationen

$\mathcal{O}(\log |\Sigma|)$

m Vergleiche insgesamt

$\mathcal{O}(k)$ insgesamt

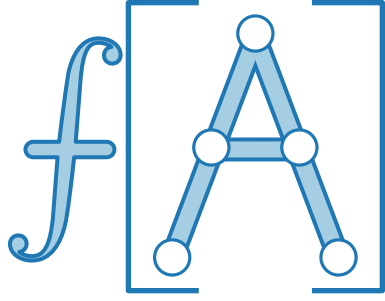
	Übereinstimmung	teilweise Übereinstimmung	keine Übereinstimmung
B			
			
M			
	i m	i j m	i



- Korrektheit.** Jedes Vorkommen von M ist ein Präfix von genau einem Suffix von T .
Wir geben alle Suffixe mit M als Präfix zurück.
- Laufzeit.** $\mathcal{O}(m \log |\Sigma| + k)$, wobei k die Anzahl der Vorkommen von M in T ist.
- ausgabesensitiv

Konstruktion eines Suffixbaums

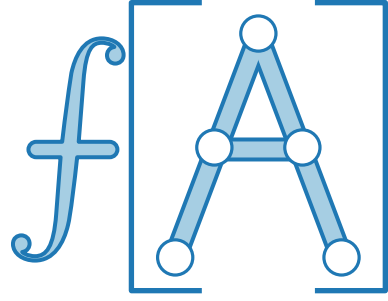
Geg. Ein String T mit $n = |T|$ über Alphabet Σ



$T = a\ b\ c\ a\ b\ a\ b\ c\ a\ \$$

Konstruktion eines Suffixbaums

Geg. Ein String T mit $n = |T|$ über Alphabet Σ
Ges. Ein Suffixbaum S für T .



$T = a\ b\ c\ a\ b\ a\ b\ c\ a\ \$$

Konstruktion eines Suffixbaums



Geg. Ein String T mit $n = |T|$ über Alphabet Σ

Ges. Ein Suffixbaum S für T .

Idee. Erstelle Σ -Bäume $N_1, N_2, \dots, N_n$, so dass N_i die Suffixe $S_1, S_2, \dots, S_i$ enthält.

$T = a\ b\ c\ a\ b\ a\ b\ c\ a\ \$$

Konstruktion eines Suffixbaums



Geg. Ein String T mit $n = |T|$ über Alphabet Σ

Ges. Ein Suffixbaum S für T .

Idee. Erstelle Σ -Bäume $N_1, N_2, \dots, N_n$, so dass N_i die Suffixe $S_1, S_2, \dots, S_i$ enthält.

Initialisierung. N_1 enthält eine einzige Kante, beschriftet mit S_1 .

$T = a\ b\ c\ a\ b\ a\ b\ c\ a\ \$$

Konstruktion eines Suffixbaums



Geg. Ein String T mit $n = |T|$ über Alphabet Σ

Ges. Ein Suffixbaum S für T .

Idee. Erstelle Σ -Bäume $N_1, N_2, \dots, N_n$, so dass N_i die Suffixe $S_1, S_2, \dots, S_i$ enthält.

Initialisierung. N_1 enthält eine einzige Kante, beschriftet mit S_1 .

$T = a b c a b a b c a \$$



Konstruktion eines Suffixbaums



Geg. Ein String T mit $n = |T|$ über Alphabet Σ

Ges. Ein Suffixbaum S für T .

Idee. Erstelle Σ -Bäume $N_1, N_2, \dots, N_n$, so dass N_i die Suffixe $S_1, S_2, \dots, S_i$ enthält.

Initialisierung. N_1 enthält eine einzige Kante, beschriftet mit S_1 .

N_{i+1} **aus** N_i **erstellen.** Suche das längste Präfix P von S_{i+1} , das in N_i enthalten ist.

$T = a b c a b a b c a \$$



Konstruktion eines Suffixbaums



Geg. Ein String T mit $n = |T|$ über Alphabet Σ

Ges. Ein Suffixbaum S für T .

Idee. Erstelle Σ -Bäume $N_1, N_2, \dots, N_n$, so dass N_i die Suffixe $S_1, S_2, \dots, S_i$ enthält.

Initialisierung. N_1 enthält eine einzige Kante, beschriftet mit S_1 .

N_{i+1} aus N_i erstellen. Suche das längste Präfix P von S_{i+1} , das in N_i enthalten ist.

Fall 1. P endet in der Mitte einer Kante e . Unterteile e und füge eine neue Kante ein.

$T = a b c a b a b c a \$$



Konstruktion eines Suffixbaums



Geg. Ein String T mit $n = |T|$ über Alphabet Σ

Ges. Ein Suffixbaum S für T .

Idee. Erstelle Σ -Bäume $N_1, N_2, \dots, N_n$, so dass N_i die Suffixe $S_1, S_2, \dots, S_i$ enthält.

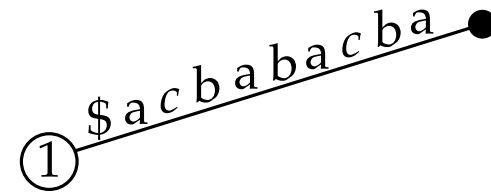
Initialisierung. N_1 enthält eine einzige Kante, beschriftet mit S_1 .

N_{i+1} aus N_i erstellen. Suche das längste Präfix P von S_{i+1} , das in N_i enthalten ist.

Fall 1. P endet in der Mitte einer Kante e . Unterteile e und füge eine neue Kante ein.

Fall 2. P endet in einem Knoten v . Füge eine neue Kante an der richtigen Position ein.

$T = a b c a b a b c a \$$



Konstruktion eines Suffixbaums



Geg. Ein String T mit $n = |T|$ über Alphabet Σ

Ges. Ein Suffixbaum S für T .

Idee. Erstelle Σ -Bäume $N_1, N_2, \dots, N_n$, so dass N_i die Suffixe $S_1, S_2, \dots, S_i$ enthält.

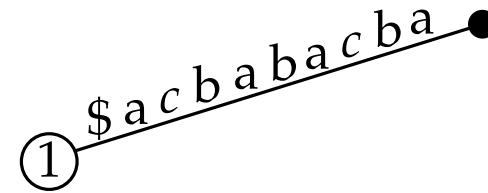
Initialisierung. N_1 enthält eine einzige Kante, beschriftet mit S_1 .

N_{i+1} aus N_i erstellen. Suche das längste Präfix P von S_{i+1} , das in N_i enthalten ist.

Fall 1. P endet in der Mitte einer Kante e . Unterteile e und füge eine neue Kante ein.

Fall 2. P endet in einem Knoten v . Füge eine neue Kante an der richtigen Position ein.

$T = a \mathbf{b c a b a b c a} \$$
 S_2



Nächster Schritt:

Füge $S_2 = b c a b a b c a \$$ hinzu:

Konstruktion eines Suffixbaums



Geg. Ein String T mit $n = |T|$ über Alphabet Σ

Ges. Ein Suffixbaum S für T .

Idee. Erstelle Σ -Bäume $N_1, N_2, \dots, N_n$, so dass N_i die Suffixe $S_1, S_2, \dots, S_i$ enthält.

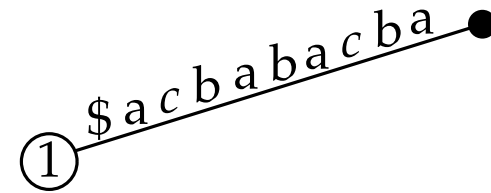
Initialisierung. N_1 enthält eine einzige Kante, beschriftet mit S_1 .

N_{i+1} aus N_i erstellen. Suche das längste Präfix P von S_{i+1} , das in N_i enthalten ist.

Fall 1. P endet in der Mitte einer Kante e . Unterteile e und füge eine neue Kante ein.

Fall 2. P endet in einem Knoten v . Füge eine neue Kante an der richtigen Position ein.

$T = a \mathbf{b c a b a b c a} \$$
 S_2



Nächster Schritt:

Füge $S_2 = b c a b a b c a \$$ hinzu:

Präfix endet an der Wurzel

Konstruktion eines Suffixbaums



Geg. Ein String T mit $n = |T|$ über Alphabet Σ

Ges. Ein Suffixbaum S für T .

Idee. Erstelle Σ -Bäume $N_1, N_2, \dots, N_n$, so dass N_i die Suffixe $S_1, S_2, \dots, S_i$ enthält.

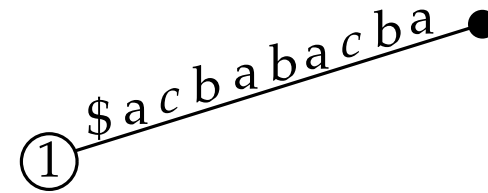
Initialisierung. N_1 enthält eine einzige Kante, beschriftet mit S_1 .

N_{i+1} aus N_i erstellen. Suche das längste Präfix P von S_{i+1} , das in N_i enthalten ist.

Fall 1. P endet in der Mitte einer Kante e . Unterteile e und füge eine neue Kante ein.

Fall 2. P endet in einem Knoten v . Füge eine neue Kante an der richtigen Position ein.

$T = a \mathbf{b c a b a b c a} \$$
 S_2



Nächster Schritt:

Füge $S_2 = b c a b a b c a \$$ hinzu:

Präfix endet an der Wurzel

➔ Fall 2.

Konstruktion eines Suffixbaums



Geg. Ein String T mit $n = |T|$ über Alphabet Σ

Ges. Ein Suffixbaum S für T .

Idee. Erstelle Σ -Bäume $N_1, N_2, \dots, N_n$, so dass N_i die Suffixe $S_1, S_2, \dots, S_i$ enthält.

Initialisierung. N_1 enthält eine einzige Kante, beschriftet mit S_1 .

N_{i+1} aus N_i erstellen. Suche das längste Präfix P von S_{i+1} , das in N_i enthalten ist.

Fall 1. P endet in der Mitte einer Kante e . Unterteile e und füge eine neue Kante ein.

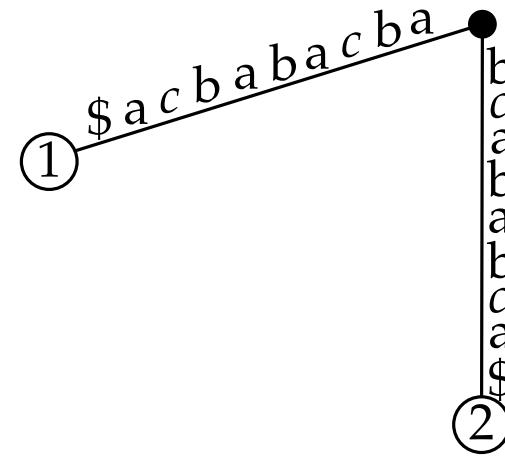
Fall 2. P endet in einem Knoten v . Füge eine neue Kante an der richtigen Position ein.

$T = a \mathbf{b c a b a b c a} \$$
 S_2

Nächster Schritt:

Füge $S_2 = b c a b a b c a \$$ hinzu:
Präfix endet an der Wurzel

➔ Fall 2.



Konstruktion eines Suffixbaums



Geg. Ein String T mit $n = |T|$ über Alphabet Σ

Ges. Ein Suffixbaum S für T .

Idee. Erstelle Σ -Bäume $N_1, N_2, \dots, N_n$, so dass N_i die Suffixe $S_1, S_2, \dots, S_i$ enthält.

Initialisierung. N_1 enthält eine einzige Kante, beschriftet mit S_1 .

N_{i+1} aus N_i erstellen. Suche das längste Präfix P von S_{i+1} , das in N_i enthalten ist.

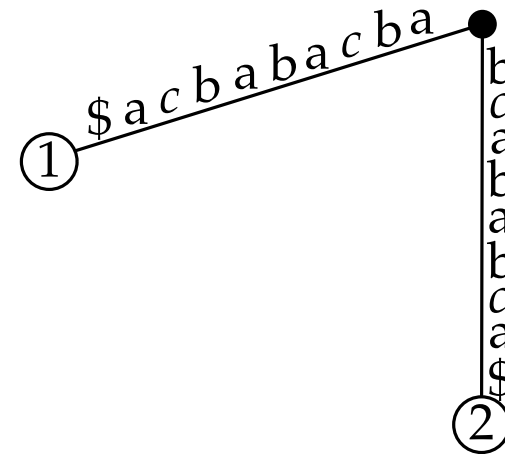
Fall 1. P endet in der Mitte einer Kante e . Unterteile e und füge eine neue Kante ein.

Fall 2. P endet in einem Knoten v . Füge eine neue Kante an der richtigen Position ein.

$T = a b \text{ c a b a b c a } \$$
 S_3

Nächster Schritt:

Füge $S_3 = c a b a b c a \$$ hinzu:



Konstruktion eines Suffixbaums



Geg. Ein String T mit $n = |T|$ über Alphabet Σ

Ges. Ein Suffixbaum S für T .

Idee. Erstelle Σ -Bäume $N_1, N_2, \dots, N_n$, so dass N_i die Suffixe $S_1, S_2, \dots, S_i$ enthält.

Initialisierung. N_1 enthält eine einzige Kante, beschriftet mit S_1 .

N_{i+1} aus N_i erstellen. Suche das längste Präfix P von S_{i+1} , das in N_i enthalten ist.

Fall 1. P endet in der Mitte einer Kante e . Unterteile e und füge eine neue Kante ein.

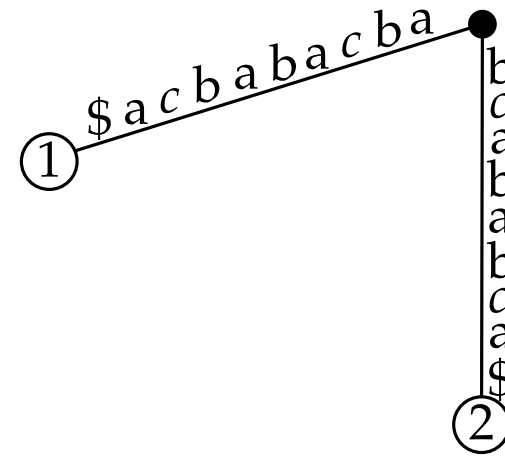
Fall 2. P endet in einem Knoten v . Füge eine neue Kante an der richtigen Position ein.

$T = a b \text{ c a b a b c a \$}$
 S_3

Nächster Schritt:

Füge $S_3 = c a b a b c a \$$ hinzu:
Präfix endet an der Wurzel

➔ Fall 2.



Konstruktion eines Suffixbaums



Geg. Ein String T mit $n = |T|$ über Alphabet Σ

Ges. Ein Suffixbaum S für T .

Idee. Erstelle Σ -Bäume $N_1, N_2, \dots, N_n$, so dass N_i die Suffixe $S_1, S_2, \dots, S_i$ enthält.

Initialisierung. N_1 enthält eine einzige Kante, beschriftet mit S_1 .

N_{i+1} aus N_i erstellen. Suche das längste Präfix P von S_{i+1} , das in N_i enthalten ist.

Fall 1. P endet in der Mitte einer Kante e . Unterteile e und füge eine neue Kante ein.

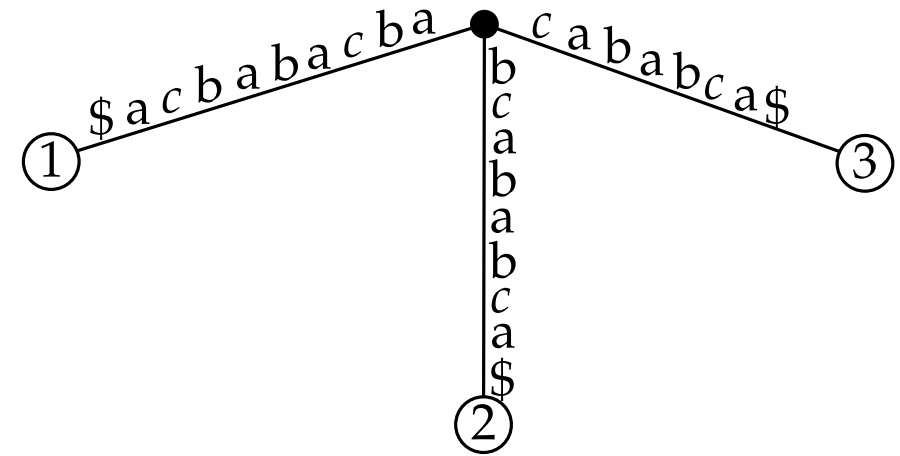
Fall 2. P endet in einem Knoten v . Füge eine neue Kante an der richtigen Position ein.

$T = a b \text{ c a b a b c a } \$$
 S_3

Nächster Schritt:

Füge $S_3 = c a b a b c a \$$ hinzu:
Präfix endet an der Wurzel

➔ Fall 2.



Konstruktion eines Suffixbaums



Geg. Ein String T mit $n = |T|$ über Alphabet Σ

Ges. Ein Suffixbaum S für T .

Idee. Erstelle Σ -Bäume $N_1, N_2, \dots, N_n$, so dass N_i die Suffixe $S_1, S_2, \dots, S_i$ enthält.

Initialisierung. N_1 enthält eine einzige Kante, beschriftet mit S_1 .

N_{i+1} aus N_i erstellen. Suche das längste Präfix P von S_{i+1} , das in N_i enthalten ist.

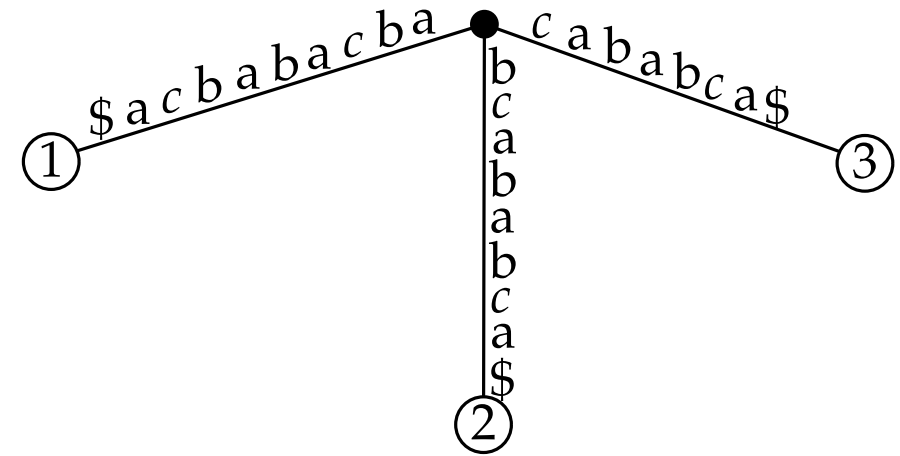
Fall 1. P endet in der Mitte einer Kante e . Unterteile e und füge eine neue Kante ein.

Fall 2. P endet in einem Knoten v . Füge eine neue Kante an der richtigen Position ein.

$T = a b c \text{ } \underline{a b a b c a \$}$
 S_4

Nächster Schritt:

Füge $S_4 = a b a b c a \$$ hinzu:



Konstruktion eines Suffixbaums



Geg. Ein String T mit $n = |T|$ über Alphabet Σ

Ges. Ein Suffixbaum S für T .

Idee. Erstelle Σ -Bäume $N_1, N_2, \dots, N_n$, so dass N_i die Suffixe $S_1, S_2, \dots, S_i$ enthält.

Initialisierung. N_1 enthält eine einzige Kante, beschriftet mit S_1 .

N_{i+1} aus N_i erstellen. Suche das längste Präfix P von S_{i+1} , das in N_i enthalten ist.

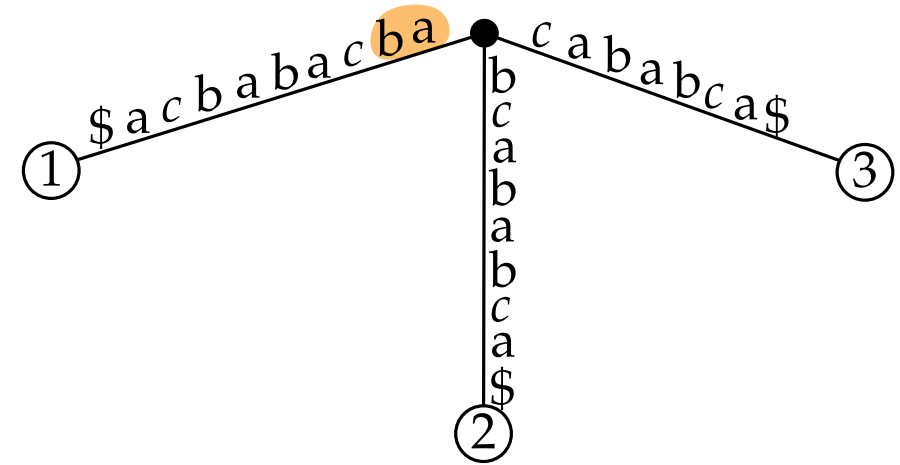
Fall 1. P endet in der Mitte einer Kante e . Unterteile e und füge eine neue Kante ein.

Fall 2. P endet in einem Knoten v . Füge eine neue Kante an der richtigen Position ein.

$T = a b c \text{ } \underline{a b a b c a \$}$
 S_4

Nächster Schritt:

Füge $S_4 = a b a b c a \$$ hinzu:



Konstruktion eines Suffixbaums



Geg. Ein String T mit $n = |T|$ über Alphabet Σ

Ges. Ein Suffixbaum S für T .

Idee. Erstelle Σ -Bäume $N_1, N_2, \dots, N_n$, so dass N_i die Suffixe $S_1, S_2, \dots, S_i$ enthält.

Initialisierung. N_1 enthält eine einzige Kante, beschriftet mit S_1 .

N_{i+1} aus N_i erstellen. Suche das längste Präfix P von S_{i+1} , das in N_i enthalten ist.

Fall 1. P endet in der Mitte einer Kante e . Unterteile e und füge eine neue Kante ein.

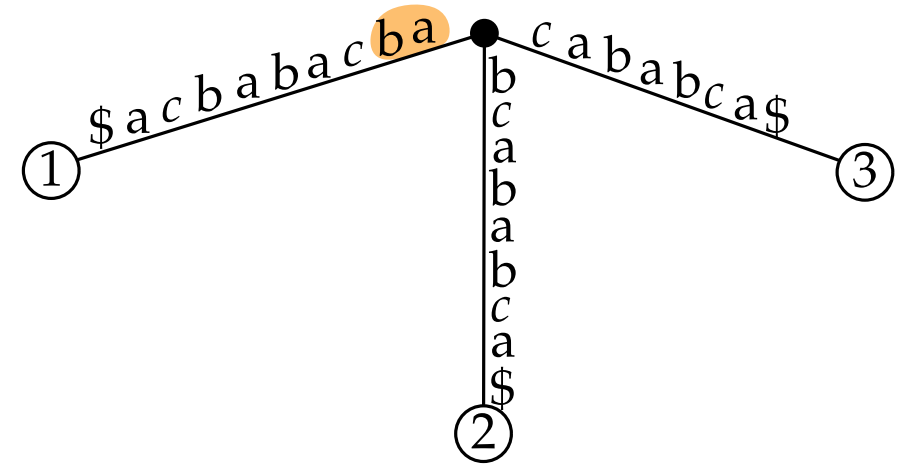
Fall 2. P endet in einem Knoten v . Füge eine neue Kante an der richtigen Position ein.

$T = a b c \text{ } \underline{a b a b c a \$}$
 S_4

Nächster Schritt:

Füge $S_4 = a b a b c a \$$ hinzu:

Präfix endet entlang S_1 nach 2 Symbolen



Konstruktion eines Suffixbaums



Geg. Ein String T mit $n = |T|$ über Alphabet Σ

Ges. Ein Suffixbaum S für T .

Idee. Erstelle Σ -Bäume $N_1, N_2, \dots, N_n$, so dass N_i die Suffixe $S_1, S_2, \dots, S_i$ enthält.

Initialisierung. N_1 enthält eine einzige Kante, beschriftet mit S_1 .

N_{i+1} aus N_i erstellen. Suche das längste Präfix P von S_{i+1} , das in N_i enthalten ist.

Fall 1. P endet in der Mitte einer Kante e . Unterteile e und füge eine neue Kante ein.

Fall 2. P endet in einem Knoten v . Füge eine neue Kante an der richtigen Position ein.

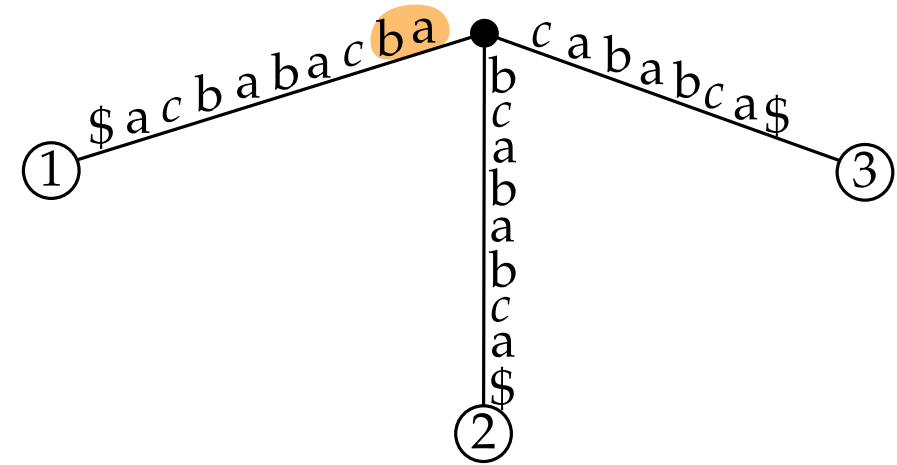
$T = a b c \mathbf{a b a b c a \$}$
 S_4

Nächster Schritt:

Füge $S_4 = a b a b c a \$$ hinzu:

Präfix endet entlang S_1 nach 2 Symbolen

➔ Fall 1.



Konstruktion eines Suffixbaums



Geg. Ein String T mit $n = |T|$ über Alphabet Σ

Ges. Ein Suffixbaum S für T .

Idee. Erstelle Σ -Bäume $N_1, N_2, \dots, N_n$, so dass N_i die Suffixe $S_1, S_2, \dots, S_i$ enthält.

Initialisierung. N_1 enthält eine einzige Kante, beschriftet mit S_1 .

N_{i+1} aus N_i erstellen. Suche das längste Präfix P von S_{i+1} , das in N_i enthalten ist.

Fall 1. P endet in der Mitte einer Kante e . Unterteile e und füge eine neue Kante ein.

Fall 2. P endet in einem Knoten v . Füge eine neue Kante an der richtigen Position ein.

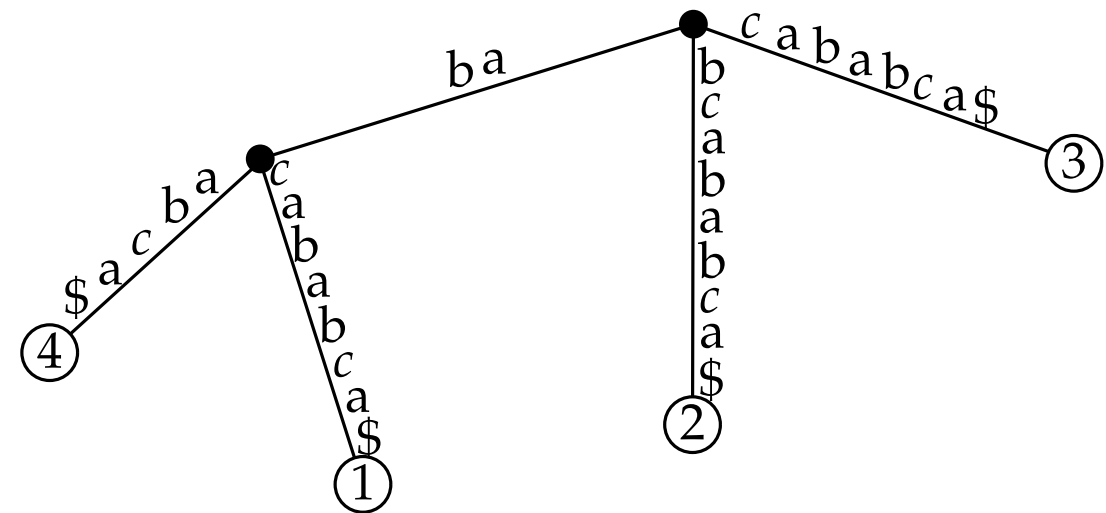
$T = a b c \text{ a b a b c a } \$$
 S_4

Nächster Schritt:

Füge $S_4 = a b a b c a \$$ hinzu:

Präfix endet entlang S_1 nach 2 Symbolen

➔ Fall 1.



Fall 2. P endet in einem Knoten v . Füge eine neue Kante an der richtigen Position ein.

A tree diagram representing a search space for a 3-letter word. The root node has three children: node 1 (left), node 2 (middle), and node 3 (right). Node 1 has two children: node 4 (left) and node 1 (right). Node 2 has one child: node 2 (bottom). Node 3 has no children. The edges are labeled with letters 'a', 'b', and 'c'. The leaf nodes are labeled with numbers 1, 2, 3, and 4.

Konstruktion eines Suffixbaums



Geg. Ein String T mit $n = |T|$ über Alphabet Σ

Ges. Ein Suffixbaum S für T .

Idee. Erstelle Σ -Bäume $N_1, N_2, \dots, N_n$, so dass N_i die Suffixe $S_1, S_2, \dots, S_i$ enthält.

Initialisierung. N_1 enthält eine einzige Kante, beschriftet mit S_1 .

N_{i+1} aus N_i erstellen. Suche das längste Präfix P von S_{i+1} , das in N_i enthalten ist.

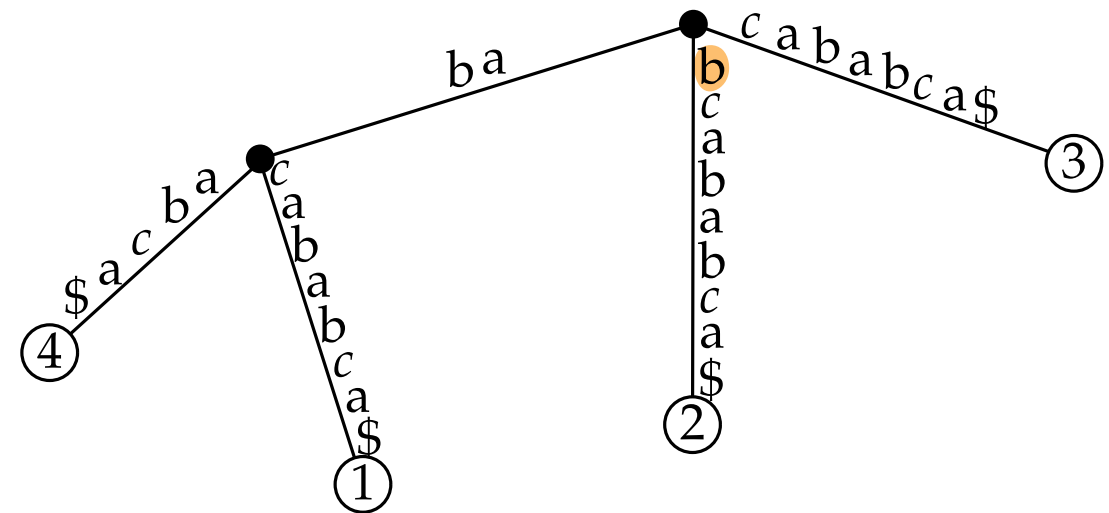
Fall 1. P endet in der Mitte einer Kante e . Unterteile e und füge eine neue Kante ein.

Fall 2. P endet in einem Knoten v . Füge eine neue Kante an der richtigen Position ein.

$T = a b c a \mathbf{b a b c a \$}$
 S_5

Nächster Schritt:

Füge $S_5 = b a b c a \$$ hinzu:



Konstruktion eines Suffixbaums



Geg. Ein String T mit $n = |T|$ über Alphabet Σ

Ges. Ein Suffixbaum S für T .

Idee. Erstelle Σ -Bäume $N_1, N_2, \dots, N_n$, so dass N_i die Suffixe $S_1, S_2, \dots, S_i$ enthält.

Initialisierung. N_1 enthält eine einzige Kante, beschriftet mit S_1 .

N_{i+1} aus N_i erstellen. Suche das längste Präfix P von S_{i+1} , das in N_i enthalten ist.

Fall 1. P endet in der Mitte einer Kante e . Unterteile e und füge eine neue Kante ein.

Fall 2. P endet in einem Knoten v . Füge eine neue Kante an der richtigen Position ein.

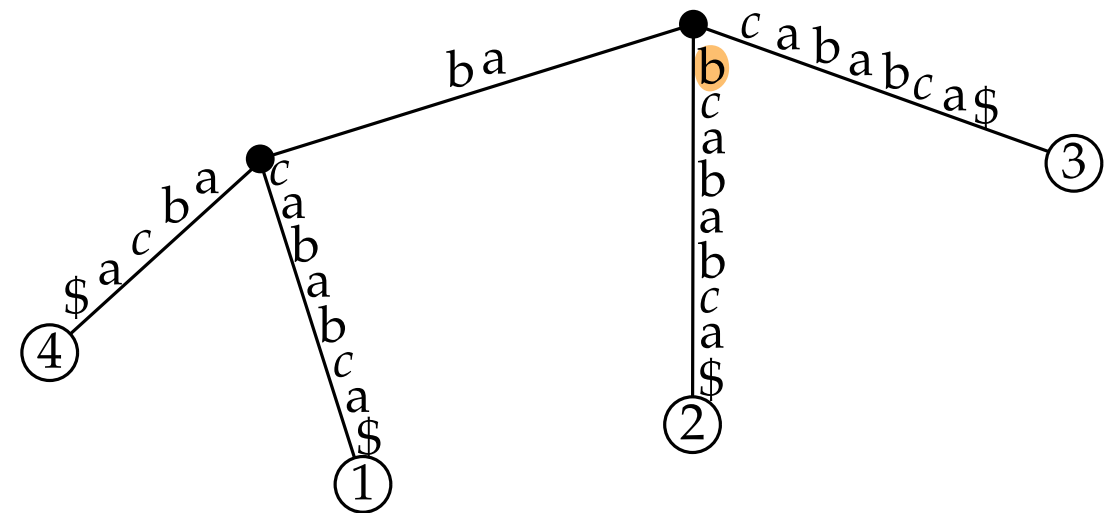
$T = a b c a \mathbf{b a b c a \$}$
 S_5

Nächster Schritt:

Füge $S_5 = b a b c a \$$ hinzu:

Präfix endet entlang S_2 nach 1 Symbol

➔ Fall 1.



Konstruktion eines Suffixbaums



Geg. Ein String T mit $n = |T|$ über Alphabet Σ

Ges. Ein Suffixbaum S für T .

Idee. Erstelle Σ -Bäume $N_1, N_2, \dots, N_n$, so dass N_i die Suffixe $S_1, S_2, \dots, S_i$ enthält.

Initialisierung. N_1 enthält eine einzige Kante, beschriftet mit S_1 .

N_{i+1} aus N_i erstellen. Suche das längste Präfix P von S_{i+1} , das in N_i enthalten ist.

Fall 1. P endet in der Mitte einer Kante e . Unterteile e und füge eine neue Kante ein.

Fall 2. P endet in einem Knoten v . Füge eine neue Kante an der richtigen Position ein.

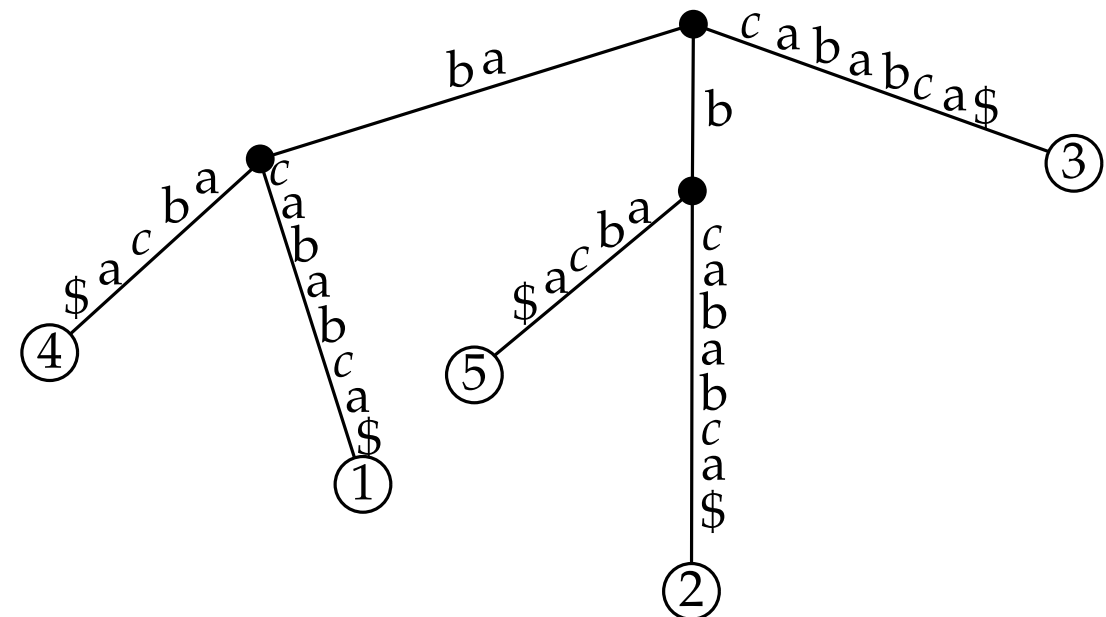
$T = a b c a \mathbf{b a b c a \$}$
 S_5

Nächster Schritt:

Füge $S_5 = b a b c a \$$ hinzu:

Präfix endet entlang S_2 nach 1 Symbol

➔ Fall 1.



Konstruktion eines Suffixbaums



Geg. Ein String T mit $n = |T|$ über Alphabet Σ

Ges. Ein Suffixbaum S für T .

Idee. Erstelle Σ -Bäume $N_1, N_2, \dots, N_n$, so dass N_i die Suffixe $S_1, S_2, \dots, S_i$ enthält.

Initialisierung. N_1 enthält eine einzige Kante, beschriftet mit S_1 .

N_{i+1} aus N_i erstellen. Suche das längste Präfix P von S_{i+1} , das in N_i enthalten ist.

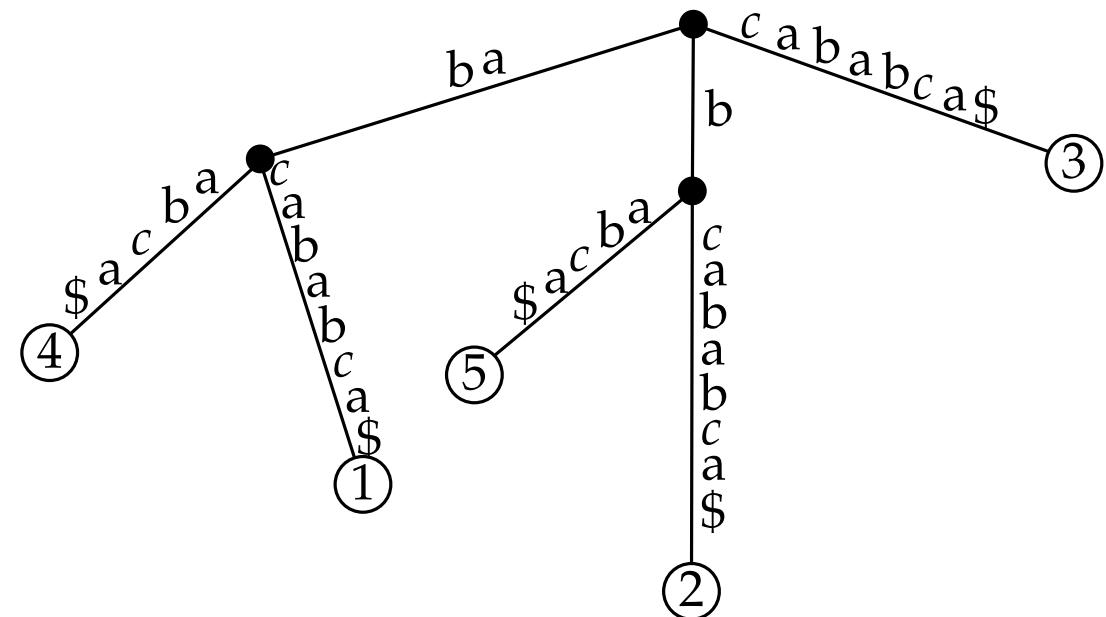
Fall 1. P endet in der Mitte einer Kante e . Unterteile e und füge eine neue Kante ein.

Fall 2. P endet in einem Knoten v . Füge eine neue Kante an der richtigen Position ein.

$T = a b c a b \mathbf{a b c a \$}$
 S_6

Nächster Schritt:

Füge $S_6 = a b c a \$$ hinzu:



Konstruktion eines Suffixbaums



Geg. Ein String T mit $n = |T|$ über Alphabet Σ

Ges. Ein Suffixbaum S für T .

Idee. Erstelle Σ -Bäume $N_1, N_2, \dots, N_n$, so dass N_i die Suffixe $S_1, S_2, \dots, S_i$ enthält.

Initialisierung. N_1 enthält eine einzige Kante, beschriftet mit S_1 .

N_{i+1} aus N_i erstellen. Suche das längste Präfix P von S_{i+1} , das in N_i enthalten ist.

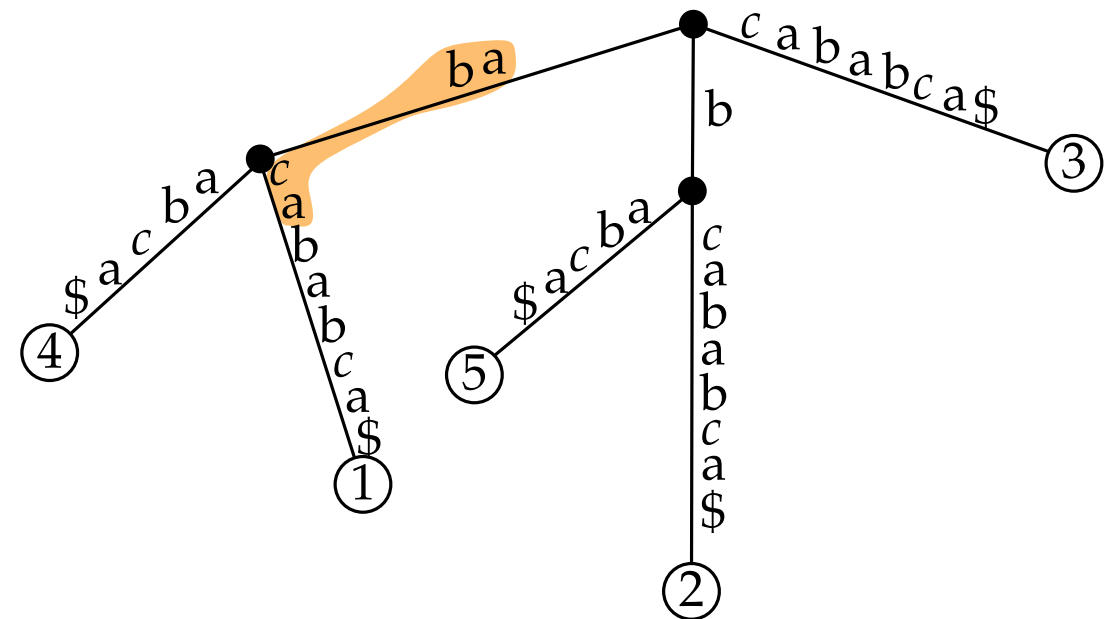
Fall 1. P endet in der Mitte einer Kante e . Unterteile e und füge eine neue Kante ein.

Fall 2. P endet in einem Knoten v . Füge eine neue Kante an der richtigen Position ein.

$T = a b c a b \underline{a b c a \$}$
 S_6

Nächster Schritt:

Füge $S_6 = a b c a \$$ hinzu:



Konstruktion eines Suffixbaums



Geg. Ein String T mit $n = |T|$ über Alphabet Σ

Ges. Ein Suffixbaum S für T .

Idee. Erstelle Σ -Bäume $N_1, N_2, \dots, N_n$, so dass N_i die Suffixe $S_1, S_2, \dots, S_i$ enthält.

Initialisierung. N_1 enthält eine einzige Kante, beschriftet mit S_1 .

N_{i+1} aus N_i erstellen. Suche das längste Präfix P von S_{i+1} , das in N_i enthalten ist.

Fall 1. P endet in der Mitte einer Kante e . Unterteile e und füge eine neue Kante ein.

Fall 2. P endet in einem Knoten v . Füge eine neue Kante an der richtigen Position ein.

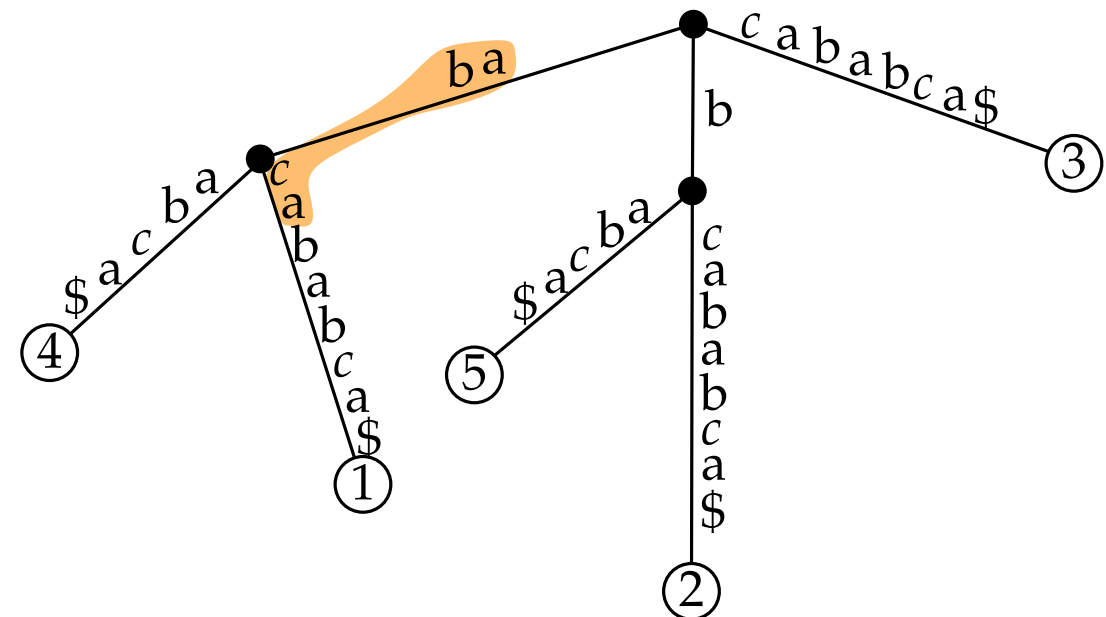
$T = a b c a b \underline{a b c a \$}$
 S_6

Nächster Schritt:

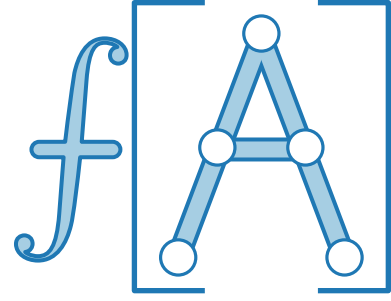
Füge $S_6 = a b c a \$$ hinzu:

Präfix endet entlang S_1 nach 4 Symbolen

➔ Fall 1.



Konstruktion eines Suffixbaums



Geg. Ein String T mit $n = |T|$ über Alphabet Σ

Ges. Ein Suffixbaum S für T .

Idee. Erstelle Σ -Bäume $N_1, N_2, \dots, N_n$, so dass N_i die Suffixe $S_1, S_2, \dots, S_i$ enthält.

Initialisierung. N_1 enthält eine einzige Kante, beschriftet mit S_1 .

N_{i+1} aus N_i erstellen. Suche das längste Präfix P von S_{i+1} , das in N_i enthalten ist.

Fall 1. P endet in der Mitte einer Kante e . Unterteile e und füge eine neue Kante ein.

Fall 2. P endet in einem Knoten v . Füge eine neue Kante an der richtigen Position ein.

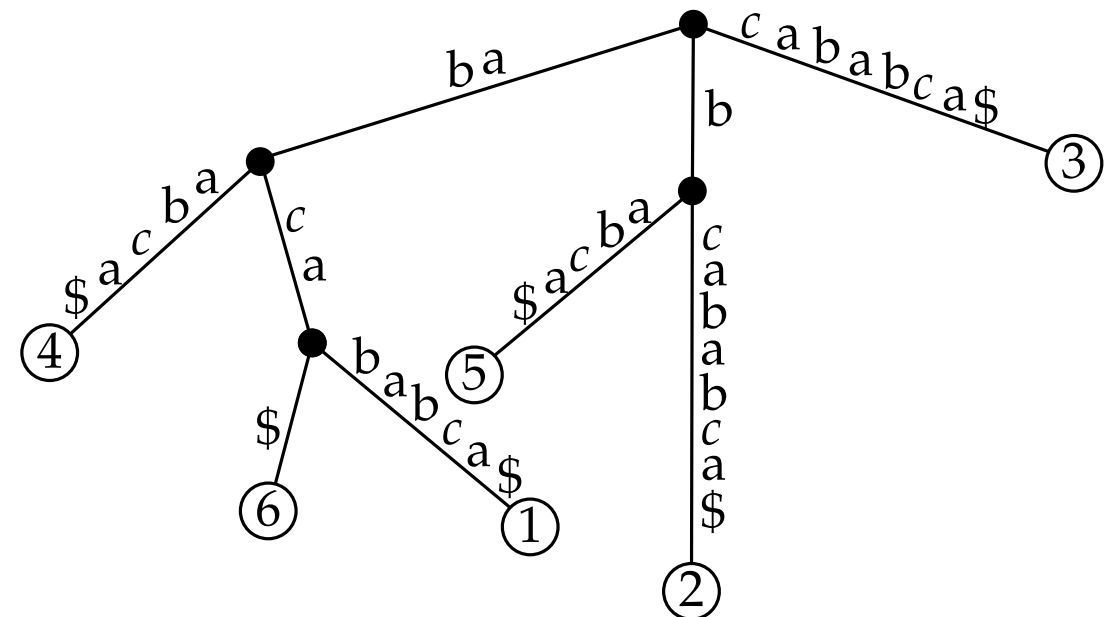
$T = a b c a b \mathbf{a b c a \$}$
 S_6

Nächster Schritt:

Füge $S_6 = a b c a \$$ hinzu:

Präfix endet entlang S_1 nach 4 Symbolen

➔ Fall 1.



Konstruktion eines Suffixbaums



Geg. Ein String T mit $n = |T|$ über Alphabet Σ

Ges. Ein Suffixbaum S für T .

Idee. Erstelle Σ -Bäume $N_1, N_2, \dots, N_n$, so dass N_i die Suffixe $S_1, S_2, \dots, S_i$ enthält.

Initialisierung. N_1 enthält eine einzige Kante, beschriftet mit S_1 .

N_{i+1} aus N_i erstellen. Suche das längste Präfix P von S_{i+1} , das in N_i enthalten ist.

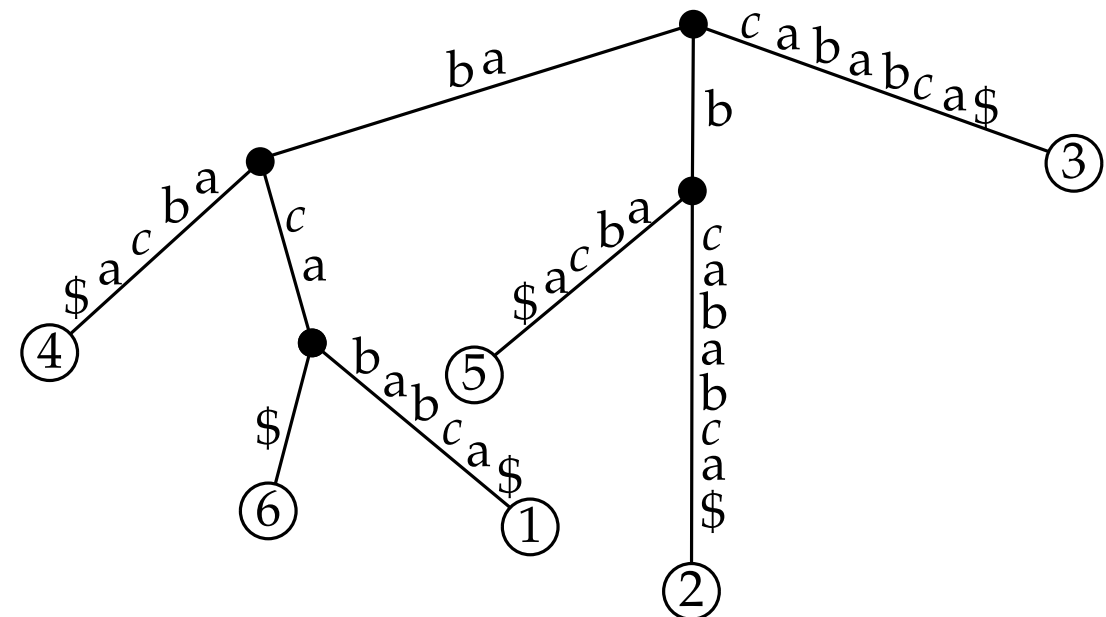
Fall 1. P endet in der Mitte einer Kante e . Unterteile e und füge eine neue Kante ein.

Fall 2. P endet in einem Knoten v . Füge eine neue Kante an der richtigen Position ein.

$T = a b c a b a \mathbf{b c a \$}$
 S_7

Nächster Schritt:

Füge $S_7 = b c a \$$ hinzu:



Konstruktion eines Suffixbaums



Geg. Ein String T mit $n = |T|$ über Alphabet Σ

Ges. Ein Suffixbaum S für T .

Idee. Erstelle Σ -Bäume $N_1, N_2, \dots, N_n$, so dass N_i die Suffixe $S_1, S_2, \dots, S_i$ enthält.

Initialisierung. N_1 enthält eine einzige Kante, beschriftet mit S_1 .

N_{i+1} aus N_i erstellen. Suche das längste Präfix P von S_{i+1} , das in N_i enthalten ist.

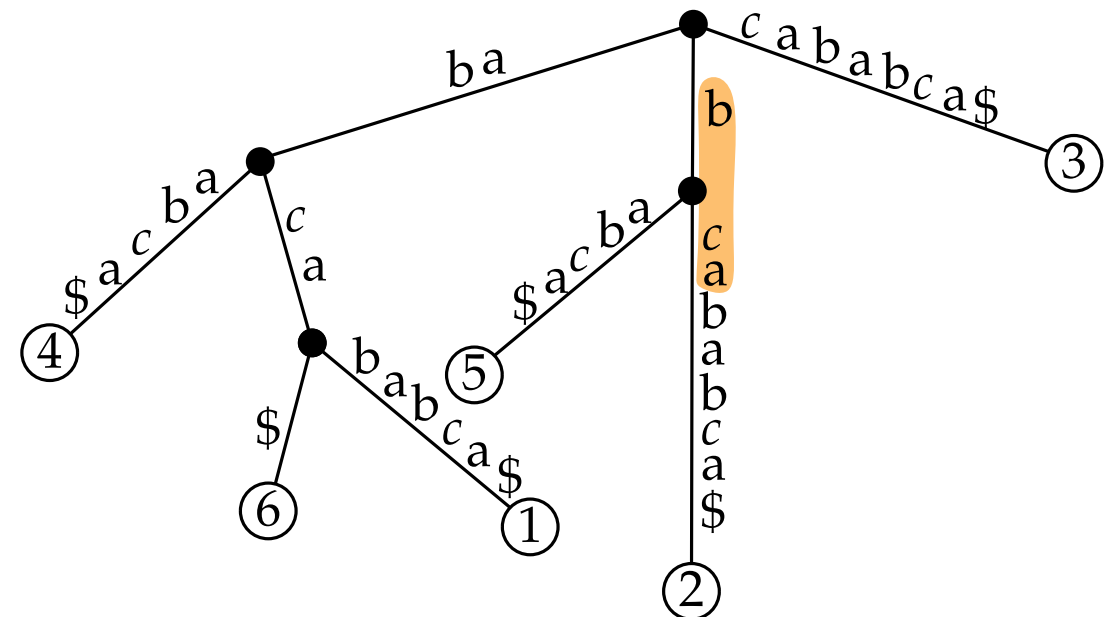
Fall 1. P endet in der Mitte einer Kante e . Unterteile e und füge eine neue Kante ein.

Fall 2. P endet in einem Knoten v . Füge eine neue Kante an der richtigen Position ein.

$T = a b c a b a \mathbf{b c a \$}$
 S_7

Nächster Schritt:

Füge $S_7 = b c a \$$ hinzu:



Konstruktion eines Suffixbaums



Geg. Ein String T mit $n = |T|$ über Alphabet Σ

Ges. Ein Suffixbaum S für T .

Idee. Erstelle Σ -Bäume $N_1, N_2, \dots, N_n$, so dass N_i die Suffixe $S_1, S_2, \dots, S_i$ enthält.

Initialisierung. N_1 enthält eine einzige Kante, beschriftet mit S_1 .

N_{i+1} aus N_i erstellen. Suche das längste Präfix P von S_{i+1} , das in N_i enthalten ist.

Fall 1. P endet in der Mitte einer Kante e . Unterteile e und füge eine neue Kante ein.

Fall 2. P endet in einem Knoten v . Füge eine neue Kante an der richtigen Position ein.

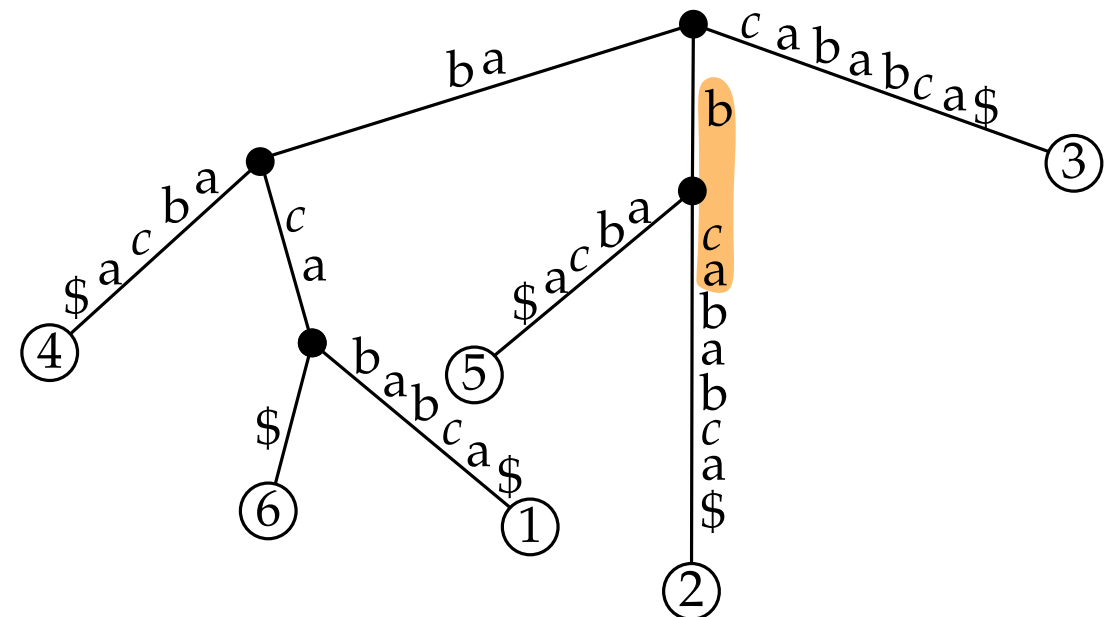
$T = a b c a b a \mathbf{b c a \$}$
 S_7

Nächster Schritt:

Füge $S_7 = b c a \$$ hinzu:

Präfix endet entlang S_2 nach 3 Symbolen

➔ Fall 1.



Konstruktion eines Suffixbaums



Geg. Ein String T mit $n = |T|$ über Alphabet Σ

Ges. Ein Suffixbaum S für T .

Idee. Erstelle Σ -Bäume $N_1, N_2, \dots, N_n$, so dass N_i die Suffixe $S_1, S_2, \dots, S_i$ enthält.

Initialisierung. N_1 enthält eine einzige Kante, beschriftet mit S_1 .

N_{i+1} aus N_i erstellen. Suche das längste Präfix P von S_{i+1} , das in N_i enthalten ist.

Fall 1. P endet in der Mitte einer Kante e . Unterteile e und füge eine neue Kante ein.

Fall 2. P endet in einem Knoten v . Füge eine neue Kante an der richtigen Position ein.

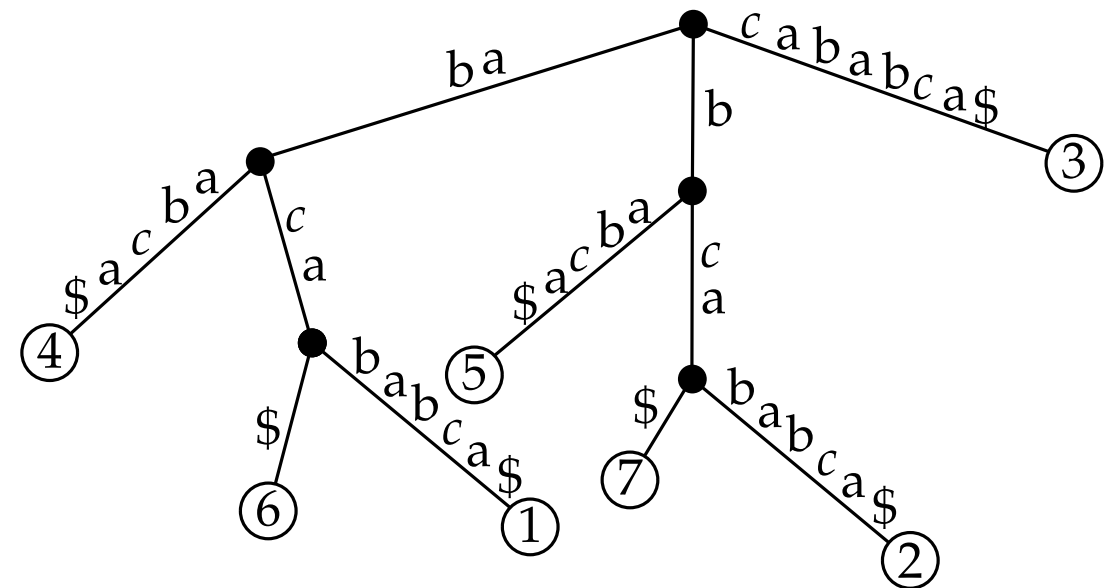
$T = a b c a b a \mathbf{b c a \$}$
 S_7

Nächster Schritt:

Füge $S_7 = b c a \$$ hinzu:

Präfix endet entlang S_2 nach 3 Symbolen

➔ Fall 1.



Konstruktion eines Suffixbaums



Geg. Ein String T mit $n = |T|$ über Alphabet Σ

Ges. Ein Suffixbaum S für T .

Idee. Erstelle Σ -Bäume $N_1, N_2, \dots, N_n$, so dass N_i die Suffixe $S_1, S_2, \dots, S_i$ enthält.

Initialisierung. N_1 enthält eine einzige Kante, beschriftet mit S_1 .

N_{i+1} aus N_i erstellen. Suche das längste Präfix P von S_{i+1} , das in N_i enthalten ist.

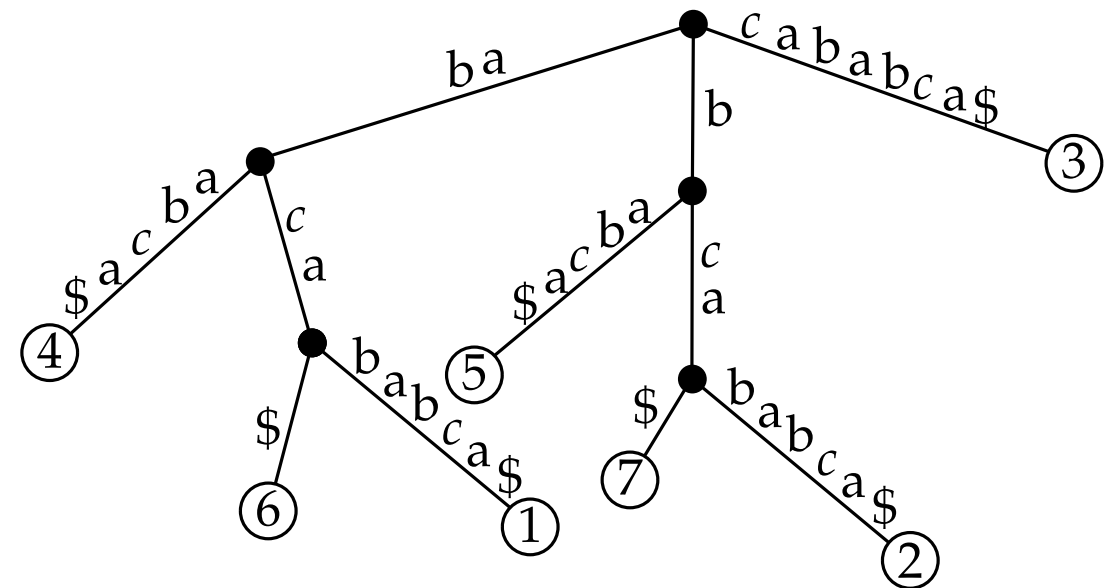
Fall 1. P endet in der Mitte einer Kante e . Unterteile e und füge eine neue Kante ein.

Fall 2. P endet in einem Knoten v . Füge eine neue Kante an der richtigen Position ein.

$T = a b c a b a b \text{ } \boxed{c a \$}$
 S_8

Nächster Schritt:

Füge $S_8 = c a \$$ hinzu:



Konstruktion eines Suffixbaums



Geg. Ein String T mit $n = |T|$ über Alphabet Σ

Ges. Ein Suffixbaum S für T .

Idee. Erstelle Σ -Bäume $N_1, N_2, \dots, N_n$, so dass N_i die Suffixe $S_1, S_2, \dots, S_i$ enthält.

Initialisierung. N_1 enthält eine einzige Kante, beschriftet mit S_1 .

N_{i+1} aus N_i erstellen. Suche das längste Präfix P von S_{i+1} , das in N_i enthalten ist.

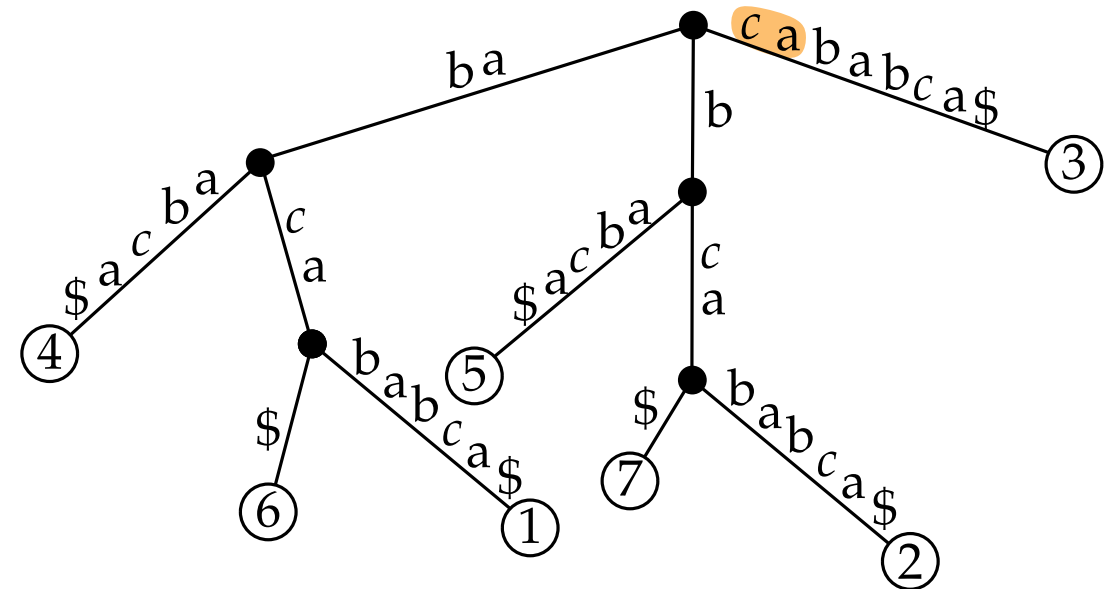
Fall 1. P endet in der Mitte einer Kante e . Unterteile e und füge eine neue Kante ein.

Fall 2. P endet in einem Knoten v . Füge eine neue Kante an der richtigen Position ein.

$T = a b c a b a b \text{ } \boxed{c a \$}$
 S_8

Nächster Schritt:

Füge $S_8 = c a \$$ hinzu:



Konstruktion eines Suffixbaums



Geg. Ein String T mit $n = |T|$ über Alphabet Σ

Ges. Ein Suffixbaum S für T .

Idee. Erstelle Σ -Bäume $N_1, N_2, \dots, N_n$, so dass N_i die Suffixe $S_1, S_2, \dots, S_i$ enthält.

Initialisierung. N_1 enthält eine einzige Kante, beschriftet mit S_1 .

N_{i+1} aus N_i erstellen. Suche das längste Präfix P von S_{i+1} , das in N_i enthalten ist.

Fall 1. P endet in der Mitte einer Kante e . Unterteile e und füge eine neue Kante ein.

Fall 2. P endet in einem Knoten v . Füge eine neue Kante an der richtigen Position ein.

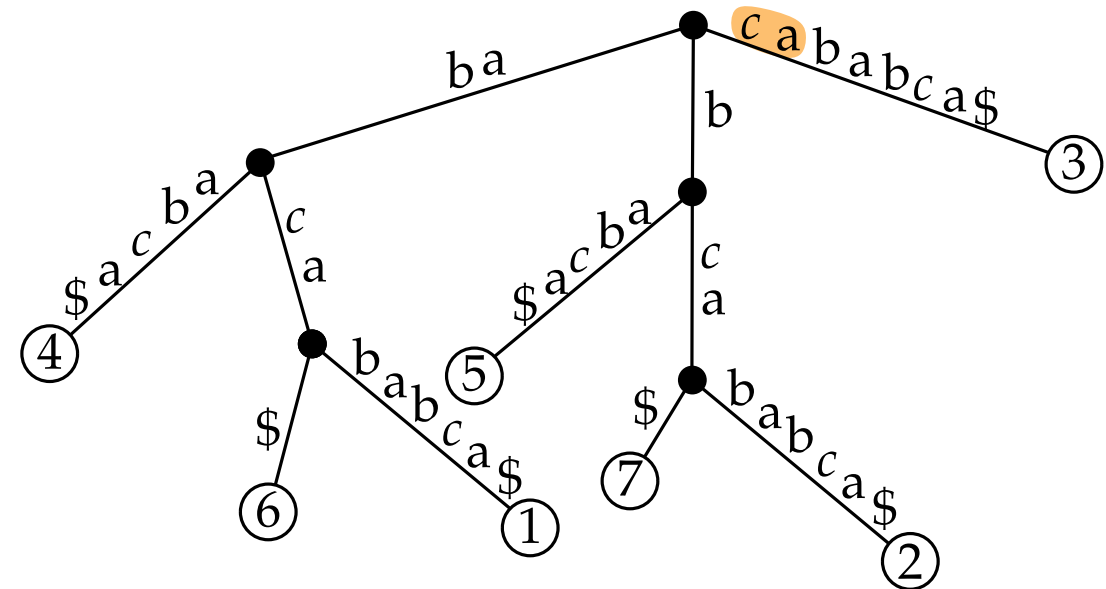
$T = a b c a b a b \text{ } \boxed{c a \$}$
 S_8

Nächster Schritt:

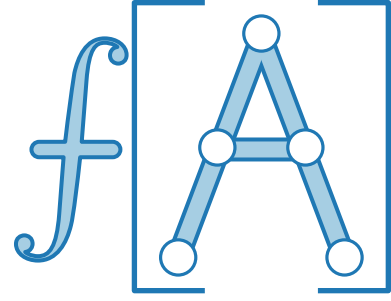
Füge $S_8 = c a \$$ hinzu:

Präfix endet entlang S_3 nach 2 Symbolen

➔ Fall 1.



Konstruktion eines Suffixbaums



Geg. Ein String T mit $n = |T|$ über Alphabet Σ

Ges. Ein Suffixbaum S für T .

Idee. Erstelle Σ -Bäume $N_1, N_2, \dots, N_n$, so dass N_i die Suffixe $S_1, S_2, \dots, S_i$ enthält.

Initialisierung. N_1 enthält eine einzige Kante, beschriftet mit S_1 .

N_{i+1} aus N_i erstellen. Suche das längste Präfix P von S_{i+1} , das in N_i enthalten ist.

Fall 1. P endet in der Mitte einer Kante e . Unterteile e und füge eine neue Kante ein.

Fall 2. P endet in einem Knoten v . Füge eine neue Kante an der richtigen Position ein.

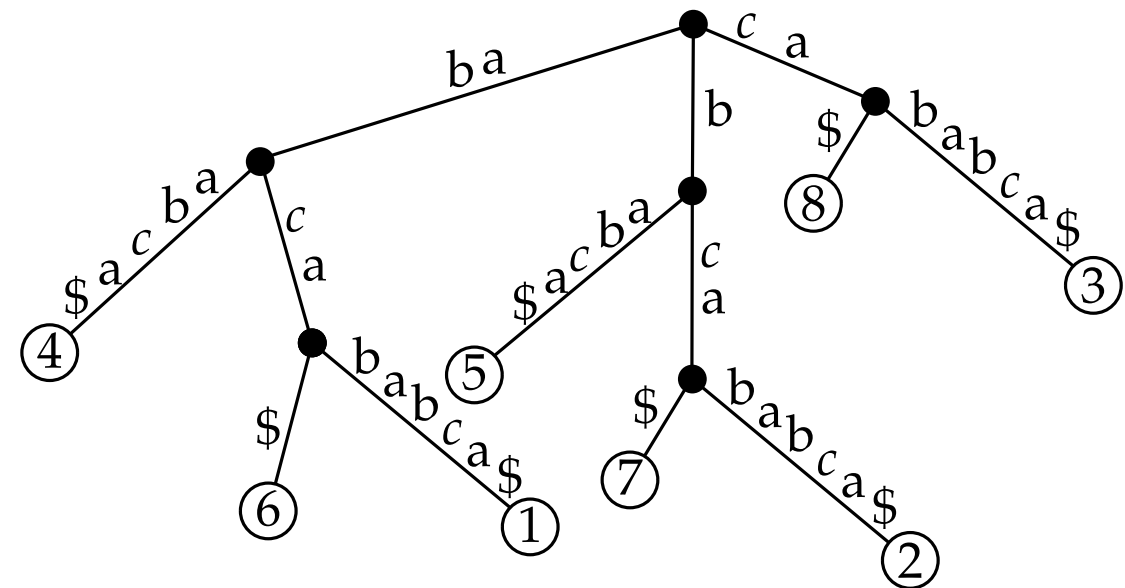
$T = a b c a b a b \text{ } \boxed{c a \$}$
 S_8

Nächster Schritt:

Füge $S_8 = c a \$$ hinzu:

Präfix endet entlang S_3 nach 2 Symbolen

➔ Fall 1.



Konstruktion eines Suffixbaums



Geg. Ein String T mit $n = |T|$ über Alphabet Σ

Ges. Ein Suffixbaum S für T .

Idee. Erstelle Σ -Bäume $N_1, N_2, \dots, N_n$, so dass N_i die Suffixe $S_1, S_2, \dots, S_i$ enthält.

Initialisierung. N_1 enthält eine einzige Kante, beschriftet mit S_1 .

N_{i+1} aus N_i erstellen. Suche das längste Präfix P von S_{i+1} , das in N_i enthalten ist.

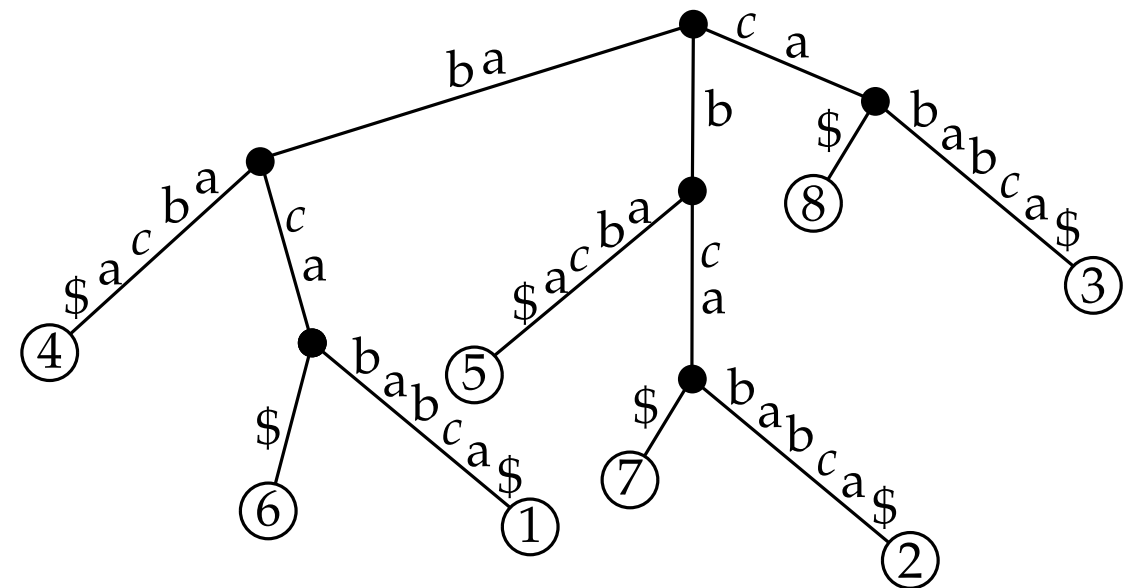
Fall 1. P endet in der Mitte einer Kante e . Unterteile e und füge eine neue Kante ein.

Fall 2. P endet in einem Knoten v . Füge eine neue Kante an der richtigen Position ein.

$T = a b c a b a b c \mathbf{a \$}$
 S_9

Nächster Schritt:

Füge $S_9 = a \$$ hinzu:



Konstruktion eines Suffixbaums



Geg. Ein String T mit $n = |T|$ über Alphabet Σ

Ges. Ein Suffixbaum S für T .

Idee. Erstelle Σ -Bäume $N_1, N_2, \dots, N_n$, so dass N_i die Suffixe $S_1, S_2, \dots, S_i$ enthält.

Initialisierung. N_1 enthält eine einzige Kante, beschriftet mit S_1 .

N_{i+1} aus N_i erstellen. Suche das längste Präfix P von S_{i+1} , das in N_i enthalten ist.

Fall 1. P endet in der Mitte einer Kante e . Unterteile e und füge eine neue Kante ein.

Fall 2. P endet in einem Knoten v . Füge eine neue Kante an der richtigen Position ein.

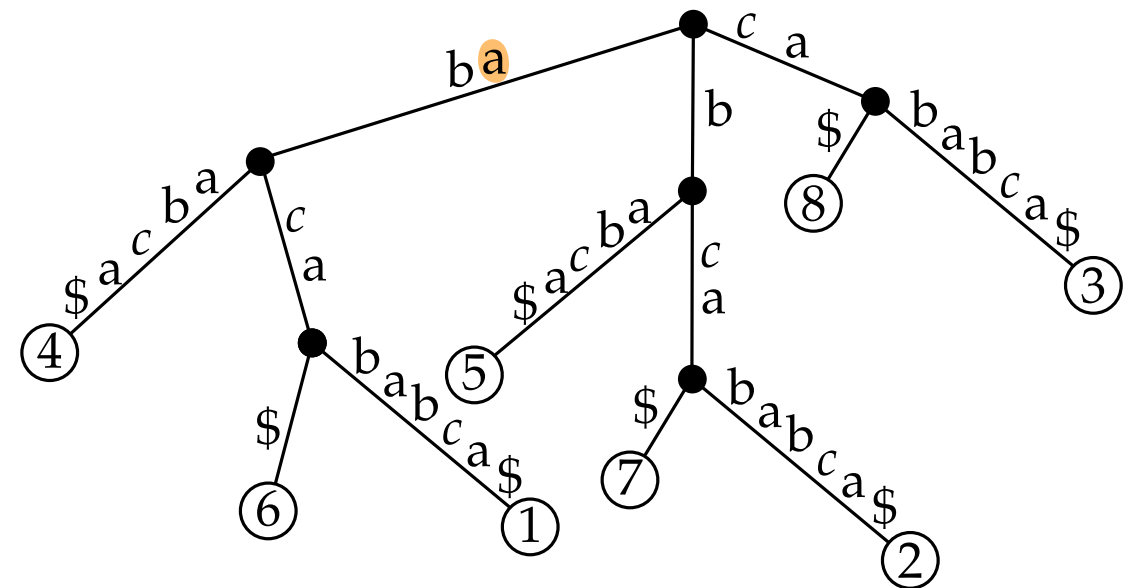
$T = a b c a b a b c \underline{a \$}$
 S_9

Nächster Schritt:

Füge $S_9 = a \$$ hinzu:

Präfix endet entlang S_4 nach 1 Symbol

➔ Fall 1.



Konstruktion eines Suffixbaums



Geg. Ein String T mit $n = |T|$ über Alphabet Σ

Ges. Ein Suffixbaum S für T .

Idee. Erstelle Σ -Bäume $N_1, N_2, \dots, N_n$, so dass N_i die Suffixe $S_1, S_2, \dots, S_i$ enthält.

Initialisierung. N_1 enthält eine einzige Kante, beschriftet mit S_1 .

N_{i+1} aus N_i erstellen. Suche das längste Präfix P von S_{i+1} , das in N_i enthalten ist.

Fall 1. P endet in der Mitte einer Kante e . Unterteile e und füge eine neue Kante ein.

Fall 2. P endet in einem Knoten v . Füge eine neue Kante an der richtigen Position ein.

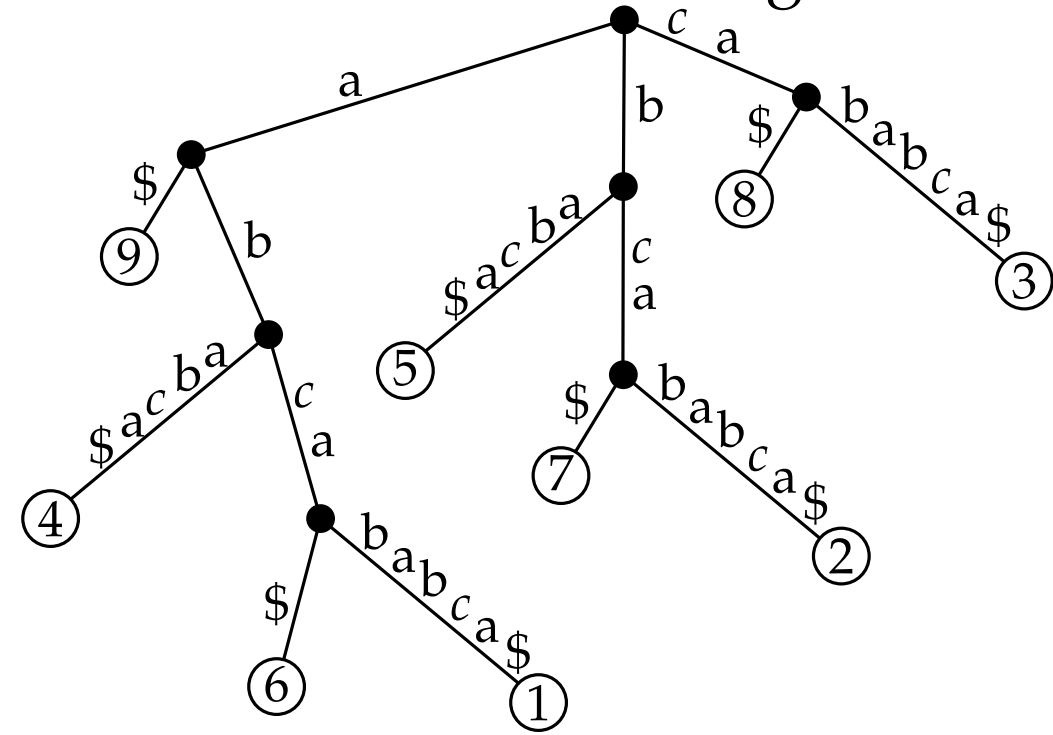
$T = a b c a b a b c \mathbf{a \$}$
 S_9

Nächster Schritt:

Füge $S_9 = a \$$ hinzu:

Präfix endet entlang S_4 nach 1 Symbol

➔ Fall 1.



Konstruktion eines Suffixbaums



Geg. Ein String T mit $n = |T|$ über Alphabet Σ

Ges. Ein Suffixbaum S für T .

Idee. Erstelle Σ -Bäume $N_1, N_2, \dots, N_n$, so dass N_i die Suffixe $S_1, S_2, \dots, S_i$ enthält.

Initialisierung. N_1 enthält eine einzige Kante, beschriftet mit S_1 .

N_{i+1} aus N_i erstellen. Suche das längste Präfix P von S_{i+1} , das in N_i enthalten ist.

Fall 1. P endet in der Mitte einer Kante e . Unterteile e und füge eine neue Kante ein.

Fall 2. P endet in einem Knoten v . Füge eine neue Kante an der richtigen Position ein.

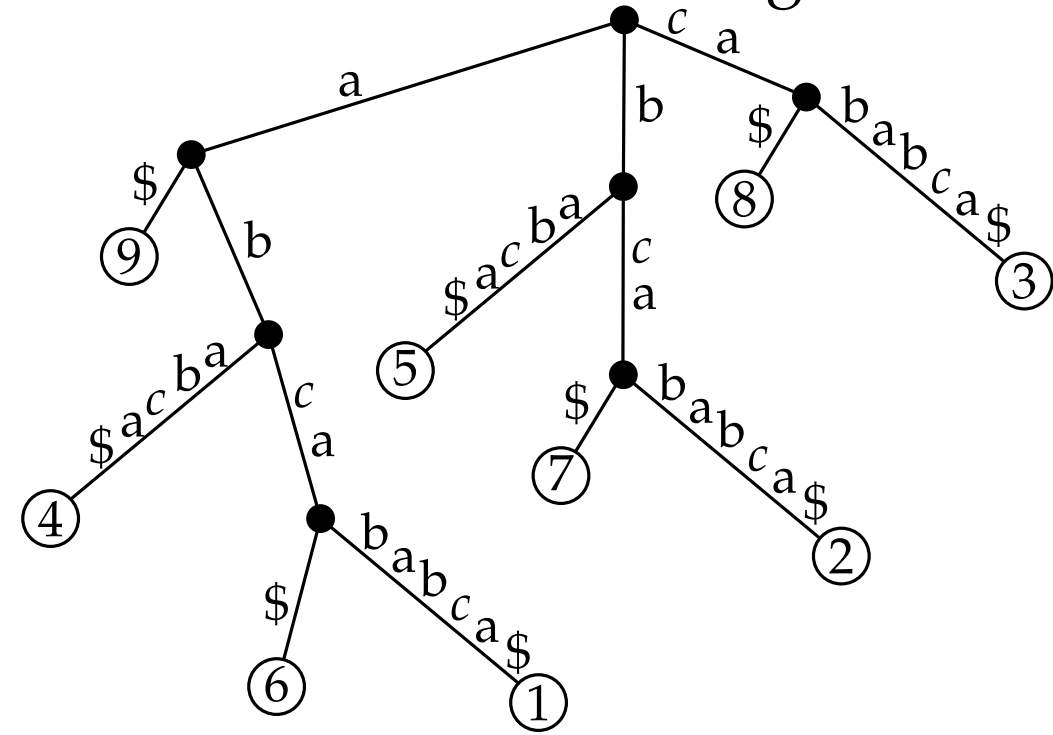
$T = a b c a b a b c a \$$
 S_{10}

Letzter Schritt:

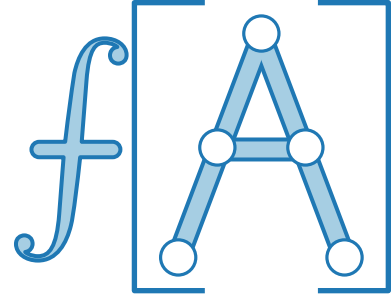
Füge $S_{10} = \$$ hinzu:

Präfix endet an der Wurzel

➔ Fall 2.



Konstruktion eines Suffixbaums



Geg. Ein String T mit $n = |T|$ über Alphabet Σ

Ges. Ein Suffixbaum S für T .

Idee. Erstelle Σ -Bäume $N_1, N_2, \dots, N_n$, so dass N_i die Suffixe $S_1, S_2, \dots, S_i$ enthält.

Initialisierung. N_1 enthält eine einzige Kante, beschriftet mit S_1 .

N_{i+1} aus N_i erstellen. Suche das längste Präfix P von S_{i+1} , das in N_i enthalten ist.

Fall 1. P endet in der Mitte einer Kante e . Unterteile e und füge eine neue Kante ein.

Fall 2. P endet in einem Knoten v . Füge eine neue Kante an der richtigen Position ein.

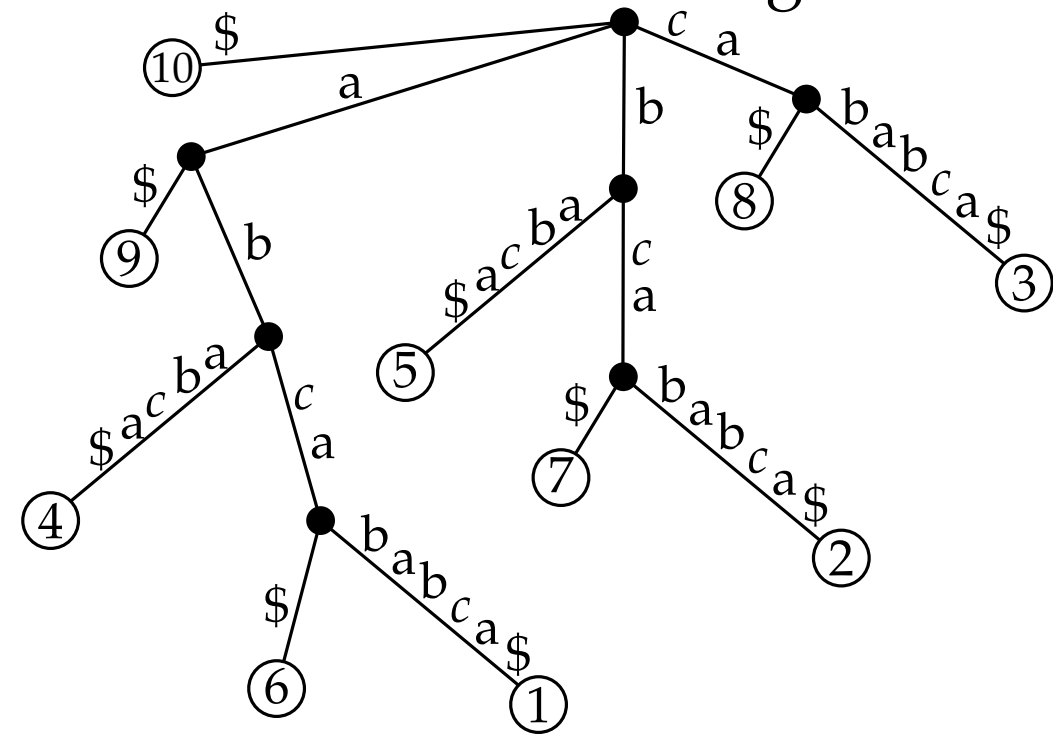
$T = a b c a b a b c a \$$
 S_{10}

Letzter Schritt:

Füge $S_{10} = \$$ hinzu:

Präfix endet an der Wurzel

➔ Fall 2.



Konstruktion eines Suffixbaums



Geg. Ein String T mit $n = |T|$ über Alphabet Σ

Ges. Ein Suffixbaum S für T .

Idee. Erstelle Σ -Bäume $N_1, N_2, \dots, N_n$, so dass N_i die Suffixe $S_1, S_2, \dots, S_i$ enthält.

Initialisierung. N_1 enthält eine einzige Kante, beschriftet mit S_1 .

N_{i+1} aus N_i erstellen. Suche das längste Präfix P von S_{i+1} , das in N_i enthalten ist.

Fall 1. P endet in der Mitte einer Kante e . Unterteile e und füge eine neue Kante ein.

Fall 2. P endet in einem Knoten v . Füge eine neue Kante an der richtigen Position ein.

Laufzeit.

$\mathcal{O}(\quad)$

Konstruktion eines Suffixbaums



Geg. Ein String T mit $n = |T|$ über Alphabet Σ

Ges. Ein Suffixbaum S für T .

Idee. Erstelle Σ -Bäume $N_1, N_2, \dots, N_n$, so dass N_i die Suffixe $S_1, S_2, \dots, S_i$ enthält.

Initialisierung. N_1 enthält eine einzige Kante, beschriftet mit S_1 .

N_{i+1} aus N_i erstellen. Suche das längste Präfix P von S_{i+1} , das in N_i enthalten ist.

Fall 1. P endet in der Mitte einer Kante e . Unterteile e und füge eine neue Kante ein.

Fall 2. P endet in einem Knoten v . Füge eine neue Kante an der richtigen Position ein.

Laufzeit.

$$\mathcal{O}\left(\underbrace{((n-1) + (n-2) + \dots + 1)}_{P \text{ suchen}} \log |\Sigma|\right)$$

Konstruktion eines Suffixbaums



Geg. Ein String T mit $n = |T|$ über Alphabet Σ

Ges. Ein Suffixbaum S für T .

Idee. Erstelle Σ -Bäume $N_1, N_2, \dots, N_n$, so dass N_i die Suffixe $S_1, S_2, \dots, S_i$ enthält.

Initialisierung. N_1 enthält eine einzige Kante, beschriftet mit S_1 .

N_{i+1} aus N_i erstellen. Suche das längste Präfix P von S_{i+1} , das in N_i enthalten ist.

Fall 1. P endet in der Mitte einer Kante e . Unterteile e und füge eine neue Kante ein.

Fall 2. P endet in einem Knoten v . Füge eine neue Kante an der richtigen Position ein.

Laufzeit.

$$\mathcal{O}\left(\underbrace{((n-1) + (n-2) + \dots + 1)}_{P \text{ suchen}} \log |\Sigma| + n|\Sigma|\right)$$

P suchen

← Richtige Position für neue Kante finden

Konstruktion eines Suffixbaums



Geg. Ein String T mit $n = |T|$ über Alphabet Σ

Ges. Ein Suffixbaum S für T .

Idee. Erstelle Σ -Bäume $N_1, N_2, \dots, N_n$, so dass N_i die Suffixe $S_1, S_2, \dots, S_i$ enthält.

Initialisierung. N_1 enthält eine einzige Kante, beschriftet mit S_1 .

N_{i+1} aus N_i erstellen. Suche das längste Präfix P von S_{i+1} , das in N_i enthalten ist.

Fall 1. P endet in der Mitte einer Kante e . Unterteile e und füge eine neue Kante ein.

Fall 2. P endet in einem Knoten v . Füge eine neue Kante an der richtigen Position ein.

Laufzeit.

$$\mathcal{O}\left(\underbrace{((n-1) + (n-2) + \dots + 1)}_{P \text{ suchen}} \log |\Sigma| + \underbrace{n|\Sigma|}_{\text{Richtige Position für neue Kante finden}}\right) \subseteq \mathcal{O}(n^2 \log |\Sigma|)$$

P suchen

Richtige Position für neue Kante finden

Konstruktion eines Suffixbaums



Geg. Ein String T mit $n = |T|$ über Alphabet Σ

Ges. Ein Suffixbaum S für T .

Idee. Erstelle Σ -Bäume $N_1, N_2, \dots, N_n$, so dass N_i die Suffixe $S_1, S_2, \dots, S_i$ enthält.

Initialisierung. N_1 enthält eine einzige Kante, beschriftet mit S_1 .

N_{i+1} aus N_i erstellen. Suche das längste Präfix P von S_{i+1} , das in N_i enthalten ist.

Fall 1. P endet in der Mitte einer Kante e . Unterteile e und füge eine neue Kante ein.

Fall 2. P endet in einem Knoten v . Füge eine neue Kante an der richtigen Position ein.

Laufzeit.

$$\mathcal{O}\left(\left((n-1) + (n-2) + \dots + 1\right) \log |\Sigma| + n|\Sigma|\right) \subseteq \mathcal{O}(n^2 \log |\Sigma|)$$

Man kann Suffixbäume auch in $\mathcal{O}(n)$ Zeit konstruieren

Konstruktion eines Suffixbaums



Geg. Ein String T mit $n = |T|$ über Alphabet Σ

Ges. Ein Suffixbaum S für T .

Idee. Erstelle Σ -Bäume $N_1, N_2, \dots, N_n$, so dass N_i die Suffixe $S_1, S_2, \dots, S_i$ enthält.

Initialisierung. N_1 enthält eine einzige Kante, beschriftet mit S_1 .

N_{i+1} aus N_i erstellen. Suche das längste Präfix P von S_{i+1} , das in N_i enthalten ist.

Fall 1. P endet in der Mitte einer Kante e . Unterteile e und füge eine neue Kante ein.

Fall 2. P endet in einem Knoten v . Füge eine neue Kante an der richtigen Position ein.

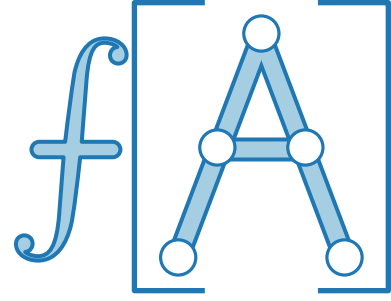
Laufzeit.

$$\mathcal{O}\left(\left((n-1) + (n-2) + \dots + 1\right) \log |\Sigma| + n|\Sigma|\right) \subseteq \mathcal{O}(n^2 \log |\Sigma|)$$

Man kann Suffixbäume auch in $\mathcal{O}(n)$ Zeit konstruieren

■ direkt, z.B. mit einem Algorithmus von Farach (1997); oder

Konstruktion eines Suffixbaums



Geg. Ein String T mit $n = |T|$ über Alphabet Σ

Ges. Ein Suffixbaum S für T .

Idee. Erstelle Σ -Bäume $N_1, N_2, \dots, N_n$, so dass N_i die Suffixe $S_1, S_2, \dots, S_i$ enthält.

Initialisierung. N_1 enthält eine einzige Kante, beschriftet mit S_1 .

N_{i+1} aus N_i erstellen. Suche das längste Präfix P von S_{i+1} , das in N_i enthalten ist.

Fall 1. P endet in der Mitte einer Kante e . Unterteile e und füge eine neue Kante ein.

Fall 2. P endet in einem Knoten v . Füge eine neue Kante an der richtigen Position ein.

Laufzeit.

$$\mathcal{O}\left(\left((n-1) + (n-2) + \dots + 1\right) \log |\Sigma| + n|\Sigma|\right) \subseteq \mathcal{O}(n^2 \log |\Sigma|)$$

Man kann Suffixbäume auch in $\mathcal{O}(n)$ Zeit konstruieren

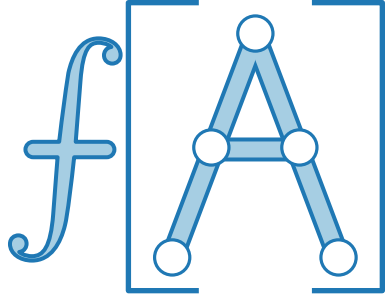
- direkt, z.B. mit einem Algorithmus von Farach (1997); oder
- indirekt, indem man zuerst ein **Suffixarray** erstellt, z.B. mit einem Algorithmus von Kärkkäinen und Sanders (2003).

Suffixarrays

Ein **Suffixarray** A eines Textes T mit $n = |T|$ speichert eine Permutation der Indizes $\{1, 2, \dots, n\}$, so dass $S_{A[i]}$ **das i -kleinste Suffix** von T in lexikografischer Reihenfolge ist.

$T = a\ b\ c\ a\ b\ a\ b\ c\ a\ \$$
 $A =$

--	--	--	--	--	--	--	--	--	--

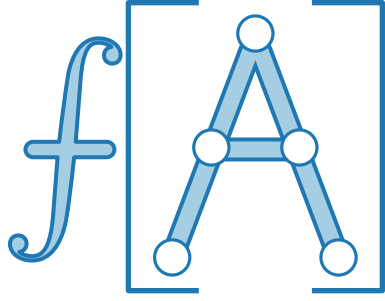


Suffixarrays

Ein **Suffixarray** A eines Textes T mit $n = |T|$ speichert eine Permutation der Indizes $\{1, 2, \dots, n\}$, so dass $S_{A[i]}$ **das i -kleinste Suffix** von T in lexikografischer Reihenfolge ist.

$$S_{A[i-1]} < S_{A[i]} \text{ für jedes } 1 < i \leq n$$

$T = a b c a b a b c a \$$
 $A = \boxed{} \boxed{} \boxed{} \boxed{} \boxed{} \boxed{} \boxed{} \boxed{} \boxed{} \boxed{}$



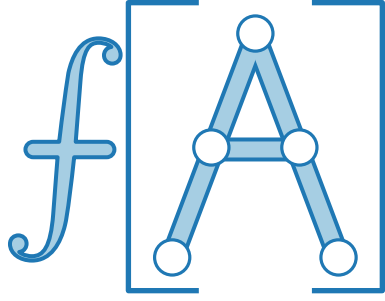
Suffixarrays

Ein **Suffixarray** A eines Textes T mit $n = |T|$ speichert eine Permutation der Indizes $\{1, 2, \dots, n\}$, so dass $S_{A[i]}$ **das i -kleinste Suffix** von T in lexikografischer Reihenfolge ist.

$$S_{A[i-1]} < S_{A[i]} \text{ f\"ur jedes } 1 < i \leq n$$

Konvention. \$ ist der kleinste Buchstabe

$T = a b c a b a b c a \$$
 $A = \boxed{} \boxed{} \boxed{} \boxed{} \boxed{} \boxed{} \boxed{} \boxed{} \boxed{} \boxed{}$



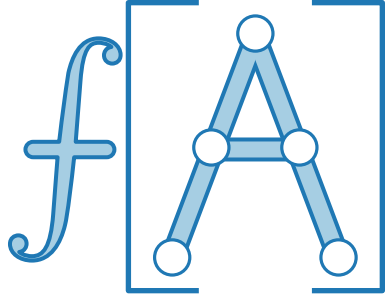
Suffixarrays

Ein **Suffixarray** A eines Textes T mit $n = |T|$ speichert eine Permutation der Indizes $\{1, 2, \dots, n\}$, so dass $S_{A[i]}$ **das i -kleinste Suffix** von T in lexikografischer Reihenfolge ist.

$$S_{A[i-1]} < S_{A[i]} \text{ für jedes } 1 < i \leq n$$

Konvention. \$ ist der kleinste Buchstabe

$T = a \ b \ c \ a \ b \ a \ b \ c \ a \ \$$
 $A = \begin{array}{|c|c|c|c|c|c|c|c|c|} \hline 10 & & & & & & & & \\ \hline \end{array}$
\$



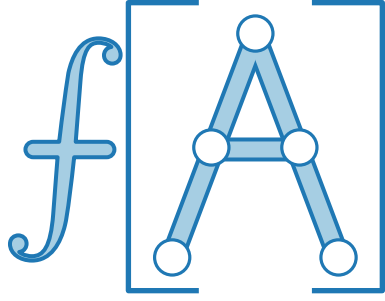
Suffixarrays

Ein **Suffixarray** A eines Textes T mit $n = |T|$ speichert eine Permutation der Indizes $\{1, 2, \dots, n\}$, so dass $S_{A[i]}$ **das i -kleinste Suffix** von T in lexikografischer Reihenfolge ist.

$$S_{A[i-1]} < S_{A[i]} \text{ f\"ur jedes } 1 < i \leq n$$

Konvention. \$ ist der kleinste Buchstabe

$T = a b c a b a b c a \$$
 $A = \begin{array}{|c|c|c|c|c|c|c|c|c|c|} \hline 10 & 9 & & & & & & & & \\ \hline \end{array}$
\$ a
\$



Suffixarrays

Ein **Suffixarray** A eines Textes T mit $n = |T|$ speichert eine Permutation der Indizes $\{1, 2, \dots, n\}$, so dass $S_{A[i]}$ **das i -kleinste Suffix** von T in lexikografischer Reihenfolge ist.

$$S_{A[i-1]} < S_{A[i]} \text{ f\"ur jedes } 1 < i \leq n$$

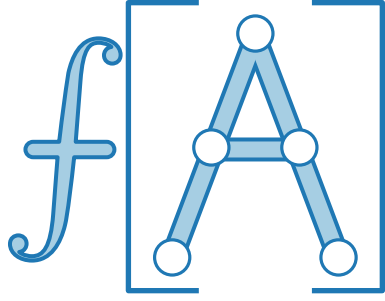
Konvention. \$ ist der kleinste Buchstabe

$T = a \ b \ c \ a \ b \ a \ b \ c \ a \ \$$

$A =$

10	9	4							
----	---	---	--	--	--	--	--	--	--

\$ a a
\$ b
a
b
c
a
\$



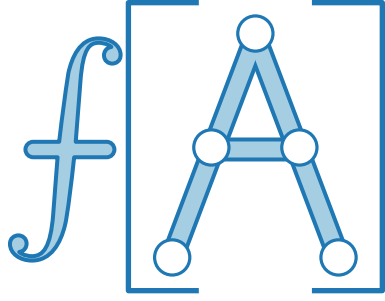
Suffixarrays

Ein **Suffixarray** A eines Textes T mit $n = |T|$ speichert eine Permutation der Indizes $\{1, 2, \dots, n\}$, so dass $S_{A[i]}$ **das i -kleinste Suffix** von T in lexikografischer Reihenfolge ist.

$$S_{A[i-1]} < S_{A[i]} \text{ f\"ur jedes } 1 < i \leq n$$

Konvention. \$ ist der kleinste Buchstabe

$T = a\ b\ c\ a\ b\ a\ b\ c\ a\ \$$										
$A =$	10	9	4	6						
	\$	a	a	a						
		\$	b	b						
			a	c						
			b	a						
			c	\$						
			a							
			\$							



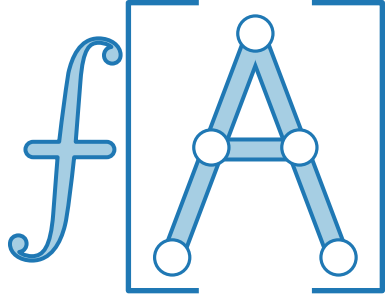
Suffixarrays

Ein **Suffixarray** A eines Textes T mit $n = |T|$ speichert eine Permutation der Indizes $\{1, 2, \dots, n\}$, so dass $S_{A[i]}$ **das i -kleinste Suffix** von T in lexikografischer Reihenfolge ist.

$$S_{A[i-1]} < S_{A[i]} \text{ f\"ur jedes } 1 < i \leq n$$

Konvention. \$ ist der kleinste Buchstabe

$T = a\ b\ c\ a\ b\ a\ b\ c\ a\ \$$										
$A =$	10	9	4	6	1					
	\$	a	a	a	a					
		\$	b	b	b					
			a	c	c					
			b	a	a					
			c	\$	b					
			a		a					
			\$		b					
					c					
					a					
					\$					



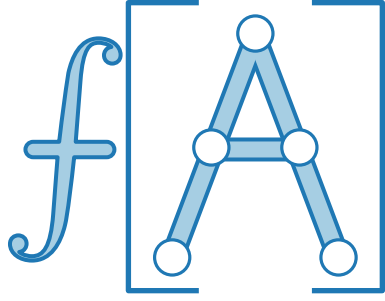
Suffixarrays

Ein **Suffixarray** A eines Textes T mit $n = |T|$ speichert eine Permutation der Indizes $\{1, 2, \dots, n\}$, so dass $S_{A[i]}$ **das i -kleinste Suffix** von T in lexikografischer Reihenfolge ist.

$$S_{A[i-1]} < S_{A[i]} \text{ f\"ur jedes } 1 < i \leq n$$

Konvention. \$ ist der kleinste Buchstabe

$T = a\ b\ c\ a\ b\ a\ b\ c\ a\ \$$									
$A =$	10	9	4	6	1	5			
	\$	a	a	a	a	b			
		\$	b	b	b	a			
			a	c	c	b			
			b	a	a	c			
			c	\$	b	a			
			a		a	\$			
			\$		b				
					c				
					a				
					\$				



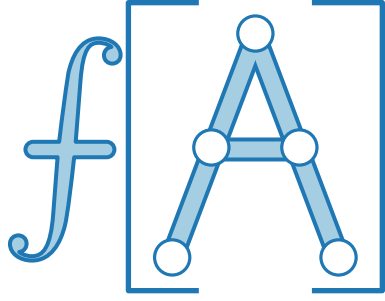
Suffixarrays

Ein **Suffixarray** A eines Textes T mit $n = |T|$ speichert eine Permutation der Indizes $\{1, 2, \dots, n\}$, so dass $S_{A[i]}$ **das i -kleinste Suffix** von T in lexikografischer Reihenfolge ist.

$$S_{A[i-1]} < S_{A[i]} \text{ f\"ur jedes } 1 < i \leq n$$

Konvention. \$ ist der kleinste Buchstabe

$T = a\ b\ c\ a\ b\ a\ b\ c\ a\ \$$									
$A =$	10	9	4	6	1	5	7		
	\$	a	a	a	a	b	b		
		\$	b	b	b	a	c		
			a	c	c	b	a		
			b	a	a	c	\$		
			c	\$	b	a			
			a		a	\$			
			\$		b				
					c				
					a				
					\$				



Suffixarrays

Ein **Suffixarray** A eines Textes T mit $n = |T|$ speichert eine Permutation der Indizes $\{1, 2, \dots, n\}$, so dass $S_{A[i]}$ **das i -kleinste Suffix** von T in lexikografischer Reihenfolge ist.

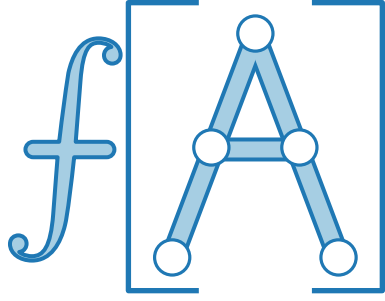
$$S_{A[i-1]} < S_{A[i]} \text{ f\"ur jedes } 1 < i \leq n$$

Konvention. \$ ist der kleinste Buchstabe

$$T = a \ b \ c \ a \ b \ a \ b \ c \ a \ \$$$

$$A = \begin{array}{|c|c|c|c|c|c|c|c|c|c|} \hline 10 & 9 & 4 & 6 & 1 & 5 & 7 & 2 & & \\ \hline \end{array}$$

\$	a	a	a	a	b	b	b		
	\$	b	b	b	a	c	c		
		a	c	c	b	a	a		
		b	a	a	c	\$	b		
		c	\$	b	a		a		
		a		a	\$		b		
		\$		b			c		
				c			a		
				a			\$		



Suffixarrays

Ein **Suffixarray** A eines Textes T mit $n = |T|$ speichert eine Permutation der Indizes $\{1, 2, \dots, n\}$, so dass $S_{A[i]}$ **das i -kleinste Suffix** von T in lexikografischer Reihenfolge ist.

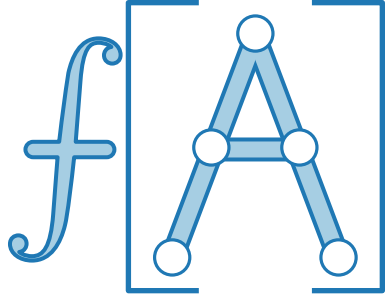
$$S_{A[i-1]} < S_{A[i]} \text{ f\"ur jedes } 1 < i \leq n$$

Konvention. \$ ist der kleinste Buchstabe

$$T = a \ b \ c \ a \ b \ a \ b \ c \ a \ \$$$

$$A = \begin{array}{|c|c|c|c|c|c|c|c|c|c|} \hline 10 & 9 & 4 & 6 & 1 & 5 & 7 & 2 & 8 & \\ \hline \end{array}$$

\$	a	a	a	a	b	b	b	c	
	\$	b	b	b	a	c	c	a	
		a	c	c	b	a	a	\$	
		b	a	a	c	\$	b		
		c	\$	b	a		a		
		a		a	\$		b		
		\$		b			c		
				c			a		
				a			\$		



Suffixarrays

Ein **Suffixarray** A eines Textes T mit $n = |T|$ speichert eine Permutation der Indizes $\{1, 2, \dots, n\}$, so dass $S_{A[i]}$ **das i -kleinste Suffix** von T in lexikografischer Reihenfolge ist.

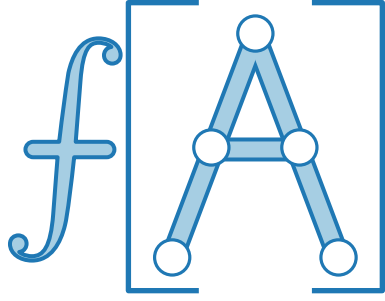
$$S_{A[i-1]} < S_{A[i]} \text{ f\"ur jedes } 1 < i \leq n$$

Konvention. \$ ist der kleinste Buchstabe

$$T = a \ b \ c \ a \ b \ a \ b \ c \ a \ \$$$

$$A = \begin{array}{|c|c|c|c|c|c|c|c|c|c|} \hline 10 & 9 & 4 & 6 & 1 & 5 & 7 & 2 & 8 & 3 \\ \hline \end{array}$$

\$	a	a	a	a	b	b	b	c	c
	\$	b	b	b	a	c	c	a	a
		a	c	c	b	a	a	\$	b
		b	a	a	c	\$	b		a
		c	\$	b	a		a		b
		a		a	\$		b		c
		\$		b			c		a
				c			a		\$
				a			\$		



Suffixarrays

Ein **Suffixarray** A eines Textes T mit $n = |T|$ speichert eine Permutation der Indizes $\{1, 2, \dots, n\}$, so dass $S_{A[i]}$ **das i -kleinste Suffix** von T in lexikografischer Reihenfolge ist.

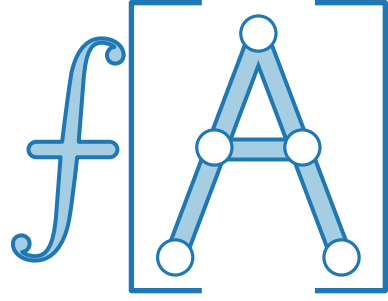
$$S_{A[i-1]} < S_{A[i]} \text{ f\"ur jedes } 1 < i \leq n$$

Konvention. \$ ist der kleinste Buchstabe

Eigenschaften.

- Die Einträge von A entsprechen einer lexikografischen Sortierung der Suffixe von T .

$T = a\ b\ c\ a\ b\ a\ b\ c\ a\ \$$										
$A =$	10	9	4	6	1	5	7	2	8	3
\$	a	a	a	a	b	b	b	c	c	
	\$	b	b	b	a	c	c	a	a	
		a	c	c	b	a	a	\$	b	
		b	a	a	c	\$	b		a	
		c	\$	b	a		a		b	
		a		a	\$		b		c	
		\$		b			c		a	
				c			a		\$	
				a			\$			



Suffixarrays

Ein **Suffixarray** A eines Textes T mit $n = |T|$ speichert eine Permutation der Indizes $\{1, 2, \dots, n\}$, so dass $S_{A[i]}$ **das i -kleinste Suffix** von T in lexikografischer Reihenfolge ist.

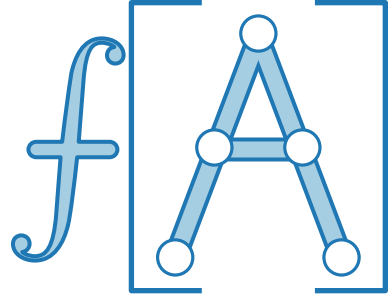
$$S_{A[i-1]} < S_{A[i]} \text{ für jedes } 1 < i \leq n$$

Konvention. \$ ist der kleinste Buchstabe

Eigenschaften.

- Die Einträge von A entsprechen einer lexikografischen Sortierung der Suffixe von T .
- Die Einträge von A entsprechen der Reihenfolge, in der die Blätter eines Suffixbaums S von T in einem inorder-Durchlauf besucht werden.

$T = a\ b\ c\ a\ b\ a\ b\ c\ a\ \$$										
$A =$	10	9	4	6	1	5	7	2	8	3
\$	a	a	a	a	b	b	b	c	c	
	\$	b	b	b	a	c	c	a	a	
		a	c	c	b	a	a	\$	b	
		b	a	a	c	\$	b		a	
		c	\$	b	a		a		b	
		a		a	\$		b		c	
		\$		b			c		a	
				c			a		\$	
				a			\$			



Suffixarrays

Ein **Suffixarray** A eines Textes T mit $n = |T|$ speichert eine Permutation der Indizes $\{1, 2, \dots, n\}$, so dass $S_{A[i]}$ **das i -kleinste Suffix** von T in lexikografischer Reihenfolge ist.

$$S_{A[i-1]} < S_{A[i]} \text{ f\"ur jedes } 1 < i \leq n$$

Konvention. $\$$ ist der kleinste Buchstabe

Eigenschaften.

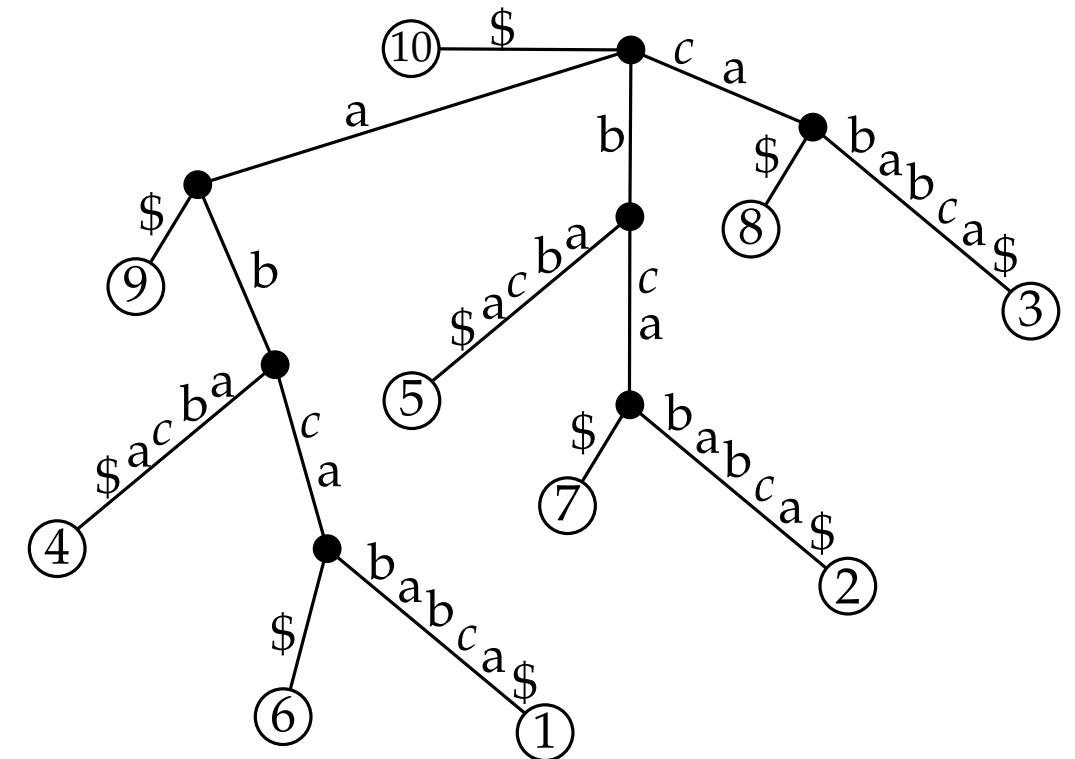
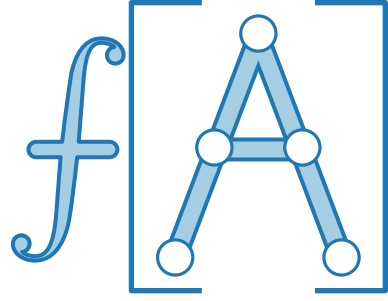
- Die Einträge von A entsprechen einer lexikografischen Sortierung der Suffixe von T .
- Die Einträge von A entsprechen der Reihenfolge, in der die Blätter eines Suffixbaums S von T in einem inorder-Durchlauf besucht werden.

$T = a b c a b a b c a \$$

$A =$

10	9	4	6	1	5	7	2	8	3
----	---	---	---	---	---	---	---	---	---

$\$$ a a a a b b b c c
\$ b b b a c c a a
a c c b a a \$ b
b a a c \$ b a b
c \$ b a a b c a
a b a \$ b c a \$
\$ b c a \$



Suffixarrays

Ein **Suffixarray** A eines Textes T mit $n = |T|$ speichert eine Permutation der Indizes $\{1, 2, \dots, n\}$, so dass $S_{A[i]}$ **das i -kleinste Suffix** von T in lexikografischer Reihenfolge ist.

$$S_{A[i-1]} < S_{A[i]} \text{ f\"ur jedes } 1 < i \leq n$$

Konvention. $\$$ ist der kleinste Buchstabe

Eigenschaften.

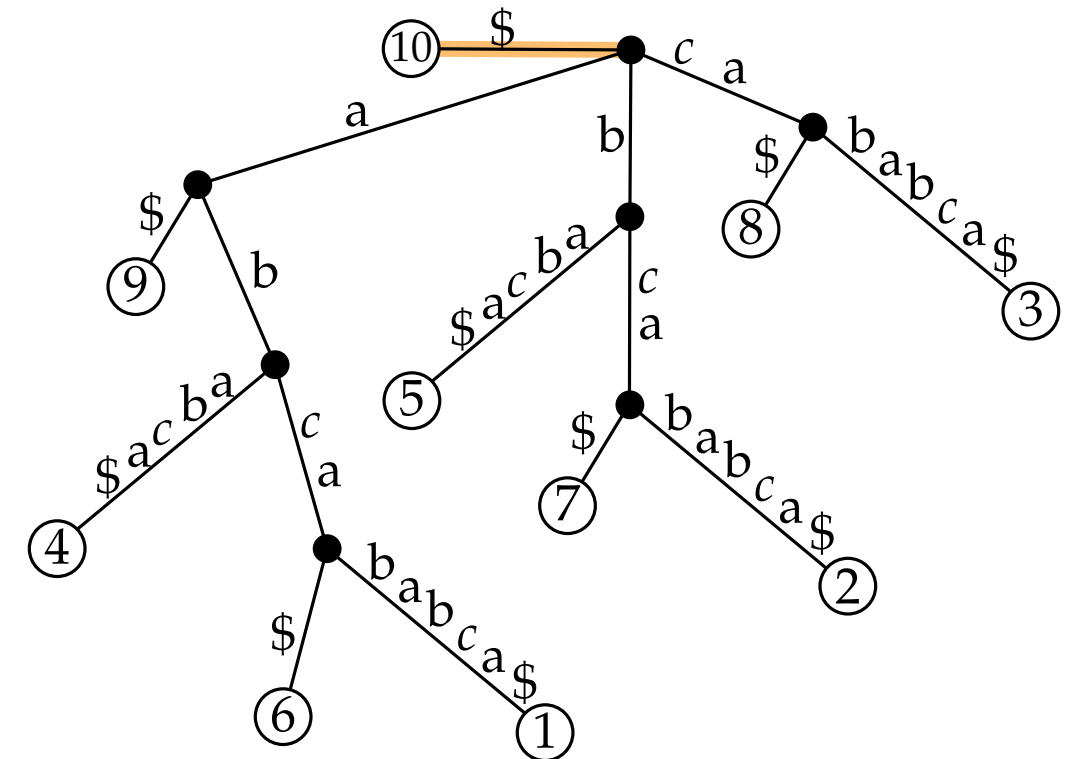
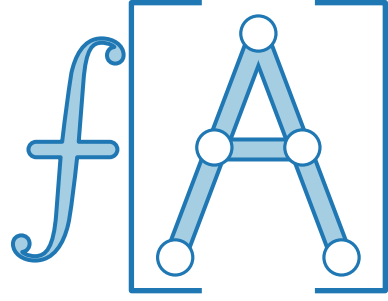
- Die Einträge von A entsprechen einer lexikografischen Sortierung der Suffixe von T .
- Die Einträge von A entsprechen der Reihenfolge, in der die Blätter eines Suffixbaums S von T in einem inorder-Durchlauf besucht werden.

$T = a b c a b a b c a \$$

$A =$

10	9	4	6	1	5	7	2	8	3
----	---	---	---	---	---	---	---	---	---

$\$$ a a a a b b b c c
 $\$$ b b b a c c a a
 a c c b a a $\$$ b
 b a a c $\$$ b a b
 c $\$$ b a a b c a
 a $\$$ b a b c a $\$$
 $\$$ b c a $\$$



Suffixarrays

Ein **Suffixarray** A eines Textes T mit $n = |T|$ speichert eine Permutation der Indizes $\{1, 2, \dots, n\}$, so dass $S_{A[i]}$ **das i -kleinste Suffix** von T in lexikografischer Reihenfolge ist.

$$\left[S_{A[i-1]} < S_{A[i]} \text{ f\"ur jedes } 1 < i \leq n \right]$$

Konvention. $\$$ ist der kleinste Buchstabe

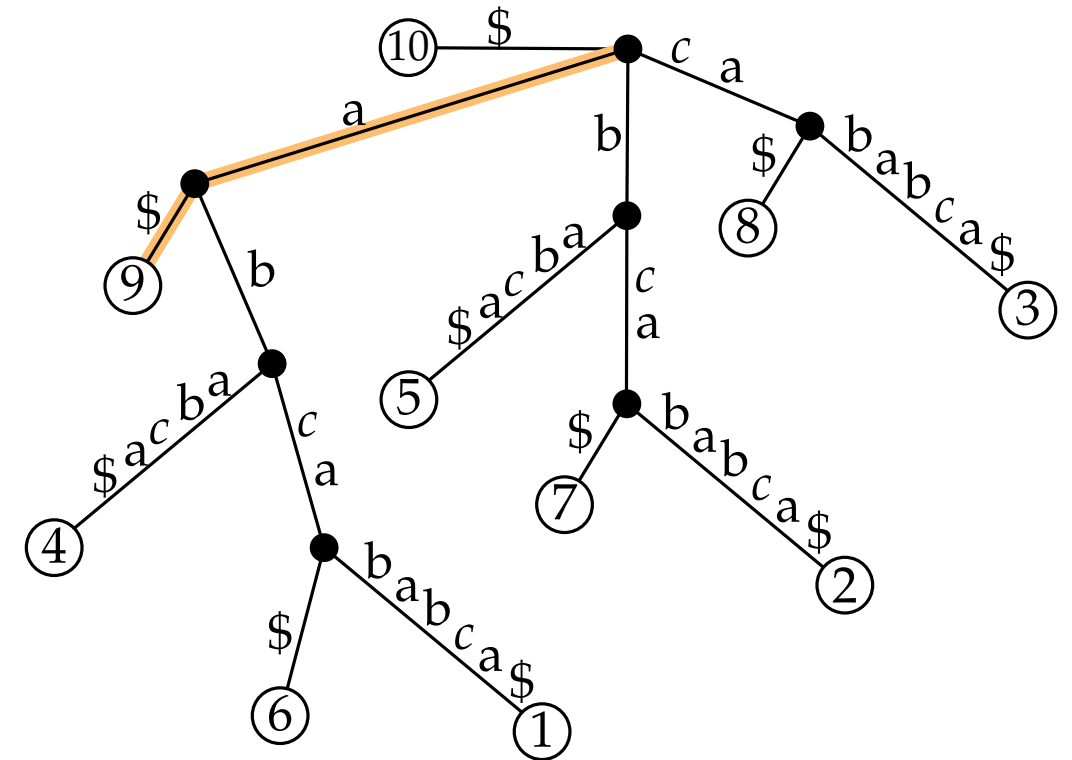
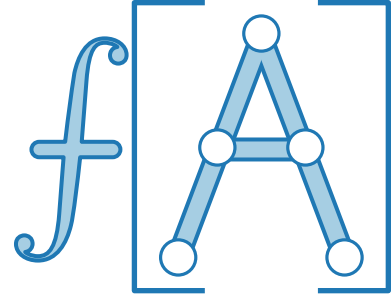
Eigenschaften.

- Die Einträge von A entsprechen einer lexikografischen Sortierung der Suffixe von T .
- Die Einträge von A entsprechen der Reihenfolge, in der die Blätter eines Suffixbaums S von T in einem inorder-Durchlauf besucht werden.

$T = a\,b\,c\,a\,b\,a\,b\,c\,a\,\$$

$$A = \begin{array}{|c|c|c|c|c|c|c|c|c|c|} \hline 10 & 9 & 4 & 6 & 1 & 5 & 7 & 2 & 8 & 3 \\ \hline \end{array}$$

\$	a	a	a	a	b	b	b	c	c
\$	b	b	b	b	a	c	c	a	a
	a	c	a	c	b	a	a	\$	b
	b	a	a	a	c	b	b		a
	c	\$	b	b	a	a	a		b
	a		a	b	\$	c	c		c
	\$		c	c		a	a		a
				a			\$		\$



Suffixarrays

Ein **Suffixarray** A eines Textes T mit $n = |T|$ speichert eine Permutation der Indizes $\{1, 2, \dots, n\}$, so dass $S_{A[i]}$ **das i -kleinste Suffix** von T in lexikografischer Reihenfolge ist.

$$\left[S_{A[i-1]} < S_{A[i]} \text{ f\"ur jedes } 1 < i \leq n \right]$$

Konvention. $\$$ ist der kleinste Buchstabe

Eigenschaften.

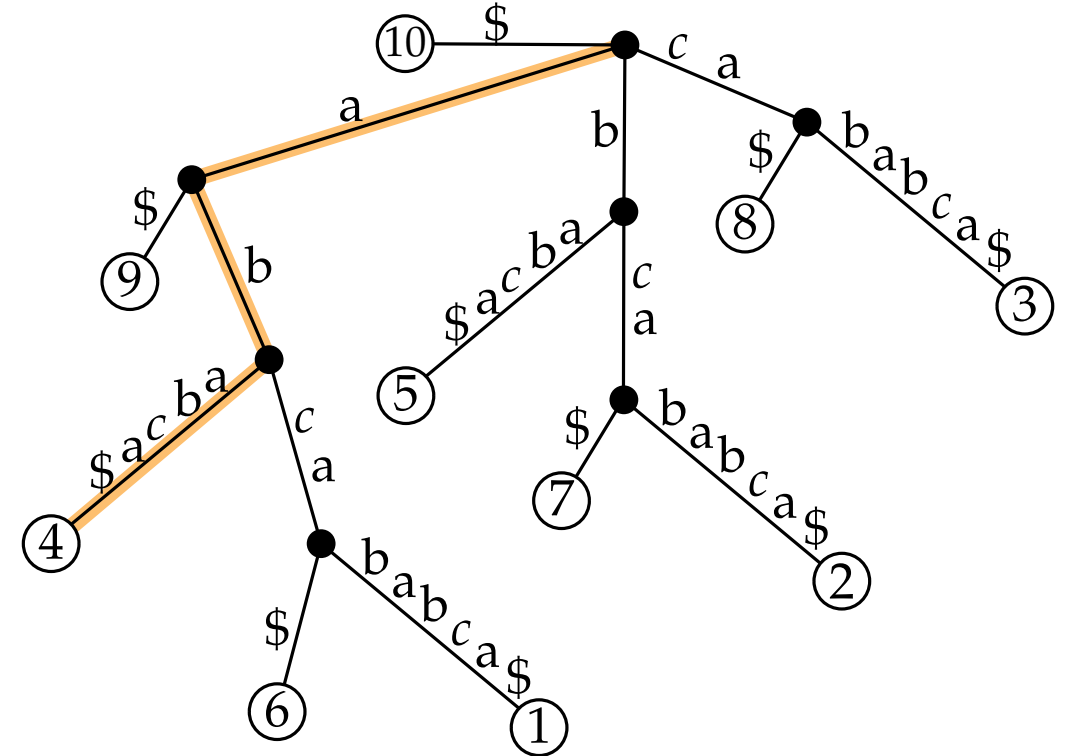
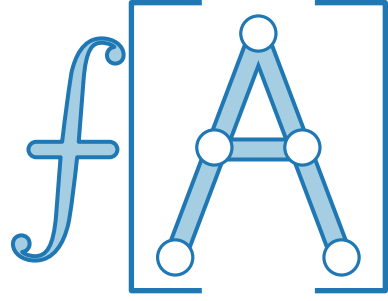
- Die Einträge von A entsprechen einer lexikografischen Sortierung der Suffixe von T .
- Die Einträge von A entsprechen der Reihenfolge, in der die Blätter eines Suffixbaums S von T in einem inorder-Durchlauf besucht werden.

$T = a\ b\ c\ a\ b\ a\ b\ c\ a\ \$$

$A =$

10	9	4	6	1	5	7	2	8	3
----	---	---	---	---	---	---	---	---	---

\$ a a a a b b b c c
\$ b b b a c a
a c c b \$
b a c \$
c a b a b c a \$
\$



Suffixarrays

Ein **Suffixarray** A eines Textes T mit $n = |T|$ speichert eine Permutation der Indizes $\{1, 2, \dots, n\}$, so dass $S_{A[i]}$ **das i -kleinste Suffix** von T in lexikografischer Reihenfolge ist.

$$S_{A[i-1]} < S_{A[i]} \text{ f\"ur jedes } 1 < i \leq n$$

Konvention. $\$$ ist der kleinste Buchstabe

Eigenschaften.

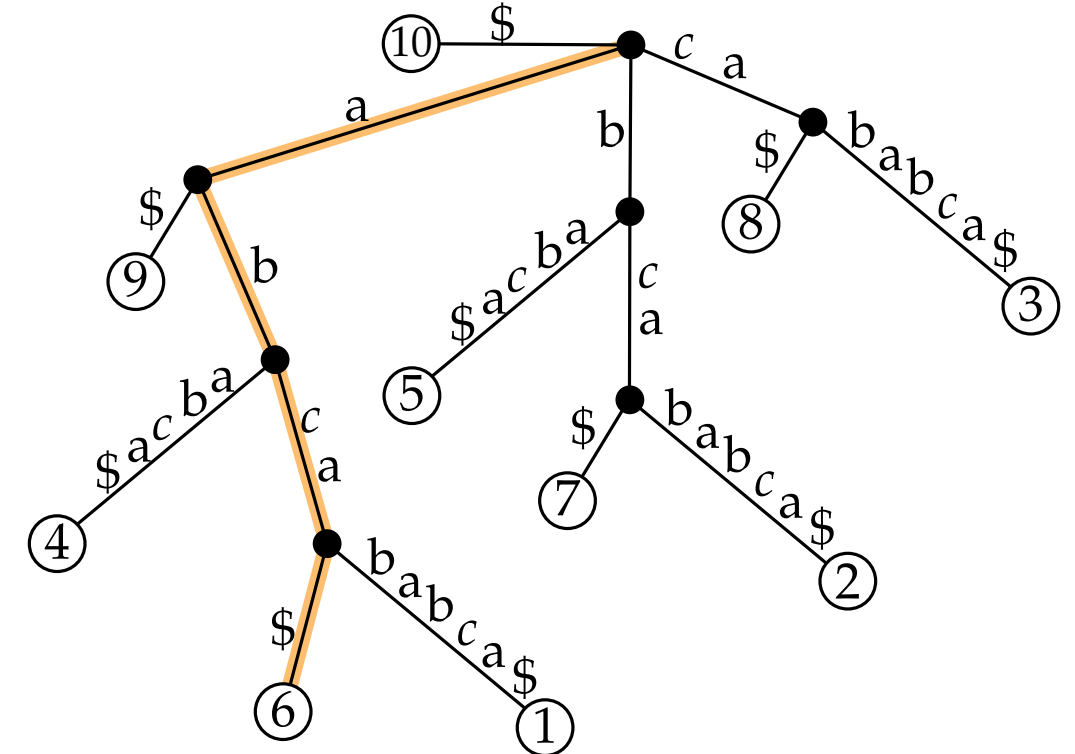
- Die Einträge von A entsprechen einer lexikografischen Sortierung der Suffixe von T .
- Die Einträge von A entsprechen der Reihenfolge, in der die Blätter eines Suffixbaums S von T in einem inorder-Durchlauf besucht werden.

$T = a\ b\ c\ a\ b\ a\ b\ c\ a\ \$$

$A =$

10	9	4	6	1	5	7	2	8	3
----	---	---	---	---	---	---	---	---	---

\$ a a a a b b b c c
\$ b b c c a a
a c c b \$
b a \$ b a b c a \$
c a \$ b c a \$
\$ b a \$ b c a \$
\$ b c a \$
\$ b c a \$
\$ b c a \$
\$ b c a \$
\$ b c a \$



Suffixarrays

Ein **Suffixarray** A eines Textes T mit $n = |T|$ speichert eine Permutation der Indizes $\{1, 2, \dots, n\}$, so dass $S_{A[i]}$ **das i -kleinste Suffix** von T in lexikografischer Reihenfolge ist.

$$[S_{A[i-1]} < S_{A[i]} \text{ f\"ur jedes } 1 < i \leq n]$$

Konvention. s ist der kleinste Buchstabe

Eigenschaften.

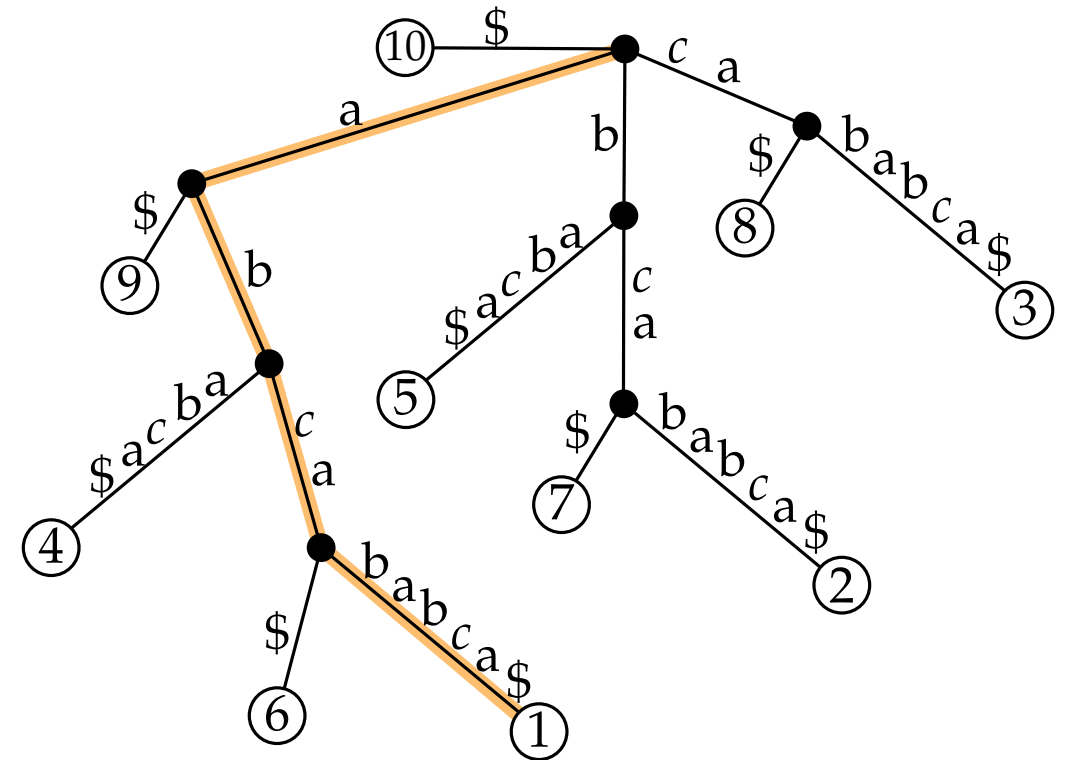
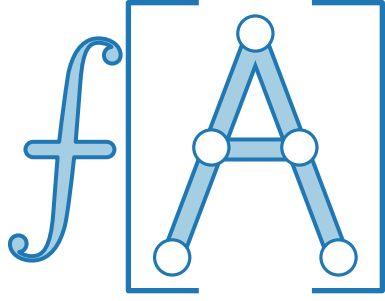
- Die Einträge von A entsprechen einer lexikografischen Sortierung der Suffixe von T .
- Die Einträge von A entsprechen der Reihenfolge, in der die Blätter eines Suffixbaums S von T in einem inorder-Durchlauf besucht werden.

$T = a\ b\ c\ a\ b\ a\ b\ c\ a\ \$$

$A =$

10	9	4	6	1	5	7	2	8	3
----	---	---	---	---	---	---	---	---	---

\$	a	a	a	a	b	b	b	c	c
	\$	b	b	b	a	c	c	a	a
		a	c	c	b	a	a	\$	b
		b	a	a	c	\$	b		a
		c	\$	b	a		a		b
		a		a	\$		b		c
		\$		b			c		a
				c			a		\$
				a			\$		
				\$					



Suffixarrays

Ein **Suffixarray** A eines Textes T mit $n = |T|$ speichert eine Permutation der Indizes $\{1, 2, \dots, n\}$, so dass $S_{A[i]}$ **das i -kleinste Suffix** von T in lexikografischer Reihenfolge ist.

$$S_{A[i-1]} < S_{A[i]} \text{ für jedes } 1 < i \leq n$$

Konvention. \$ ist der kleinste Buchstabe

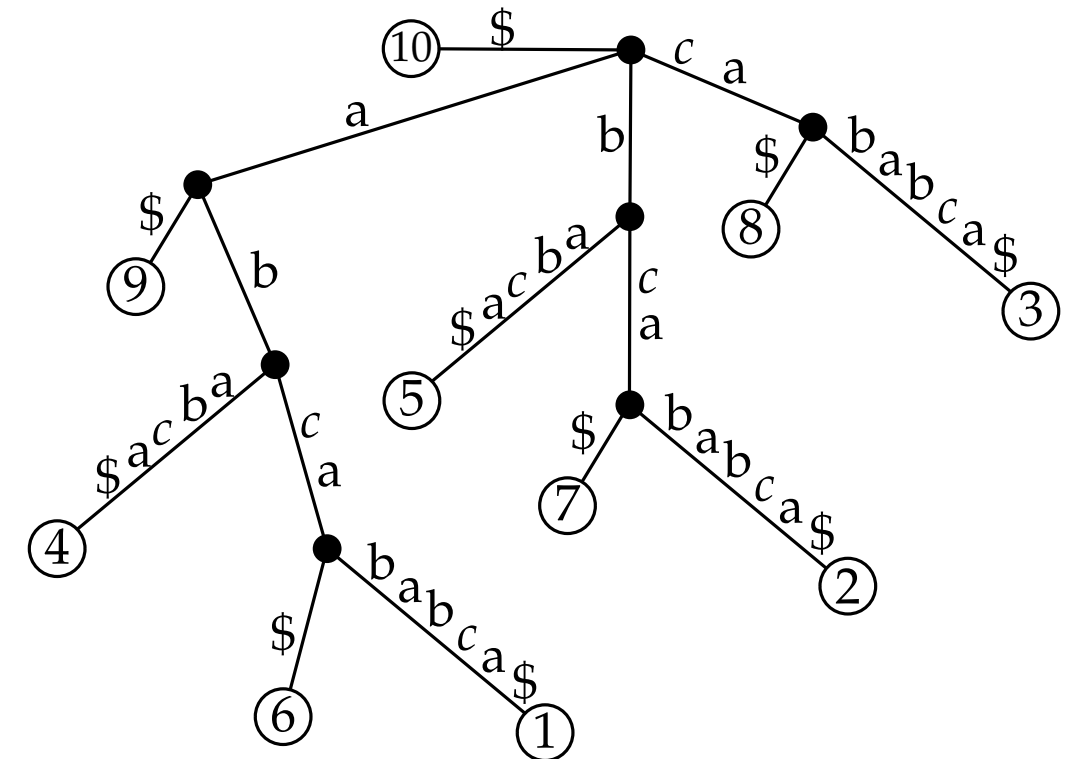
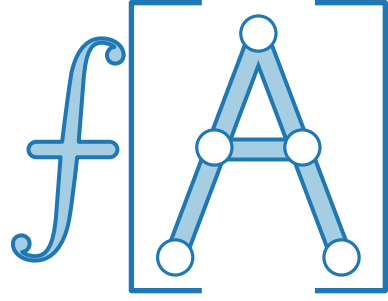
Eigenschaften.

- Die Einträge von A entsprechen einer lexikografischen Sortierung der Suffixe von T .
- Die Einträge von A entsprechen der Reihenfolge, in der die Blätter eines Suffixbaums S von T in einem inorder-Durchlauf besucht werden.

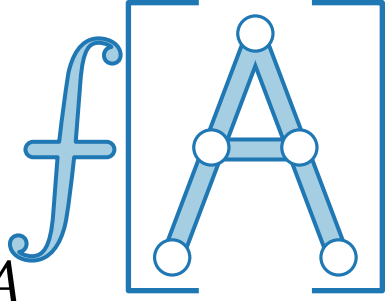
$T = a b c a b a b c a \$$

$A = \boxed{10} \boxed{9} \boxed{4} \boxed{6} \boxed{1} \boxed{5} \boxed{7} \boxed{2} \boxed{8} \boxed{3}$

\$	a	a	a	a	b	b	b	c	c
\$	\$	b	b	b	a	c	c	a	a
		a	c	c	b	a	a	\$	b
		b	a	a	c	\$	b		a
		c	\$	b	a		a		b
		a		a	\$		b		c
		\$		b	c		a		a
				c	a		\$		\$



Suchen in Suffixarrays



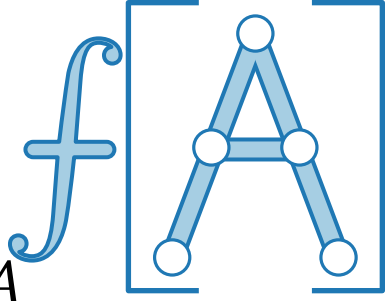
Beobachtung. Die Vorkommen eines Musters M in T bilden ein Intervall in A .

$T = a\ b\ c\ a\ b\ a\ b\ c\ a\ \$$

$A =$

10	9	4	6	1	5	7	2	8	3
\$	a	a	a	a	b	b	b	c	c
	\$	b	b	b	a	c	c	a	a
		a	c	c	b	a	a	\$	b
		b	a	a	c	\$	b		a
		c	\$	b	a		a		b
		a		a	\$		b		c
		\$		b			c		a
				c			a		\$
				a			\$		

Suchen in Suffixarrays



Beobachtung. Die Vorkommen eines Musters M in T bilden ein Intervall in A .

$T = a\ b\ c\ a\ b\ a\ b\ c\ a\ \$$

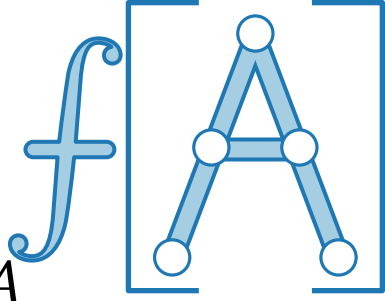
$A =$

10	9	4	6	1	5	7	2	8	3
----	---	---	---	---	---	---	---	---	---

\$ a a a a b b b c c
\$ b b b a c c a a
a c c b a a \$ b
b a a c \$ b a b
c \$ b a \$ b c a
a b a \$ b c a \$
\$ b c a \$

$M = a\ b$

Suchen in Suffixarrays



Beobachtung. Die Vorkommen eines Musters M in T bilden ein Intervall in A .

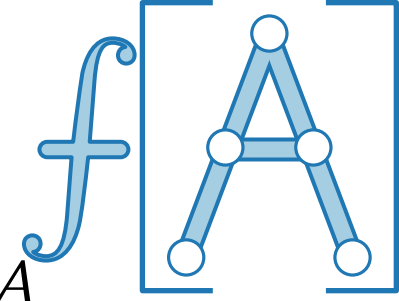
$T = a\ b\ c\ a\ b\ a\ b\ c\ a\ \$$

$A =$

10	9	4	6	1	5	7	2	8	3
\$	a	a	a	a	b	b	b	c	c
	\$	b	b	b	a	c	c	a	a
		a	c	c	b	a	a	\$	b
		b	a	a	c	\$	b		a
		c	\$	b	a		a		b
		a		a	\$		b		c
		\$		b			c		a
				c			a		\$
				a			\$		

$M = a\ b$

Suchen in Suffixarrays



Beobachtung. Die Vorkommen eines Musters M in T bilden ein Intervall in A .

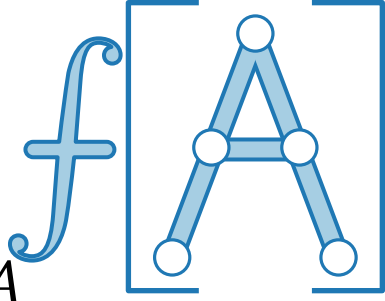
$T = a\ b\ c\ a\ b\ a\ b\ c\ a\ \$$

$A =$

10	9	4	6	1	5	7	2	8	3
\$	a	a	a	a	b	b	b	c	c
	\$	b	b	b	a	c	c	a	a
		a	c	c	b	a	a	\$	b
		b	a	a	c	\$	b		a
		c	\$	b	a		a		b
		a		a	\$		b		c
		\$		b			c		a
				c			a		\$
				a			\$		

$M = a\ b$

Suchen in Suffixarrays



Beobachtung. Die Vorkommen eines Musters M in T bilden ein Intervall in A .

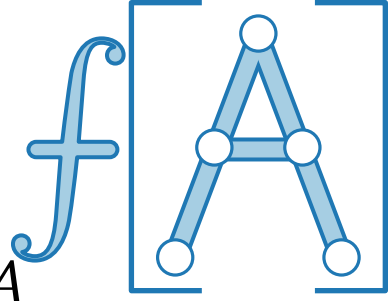
$T = a\ b\ c\ a\ b\ a\ b\ c\ a\ \$$

$A =$

10	9	4	6	1	5	7	2	8	3
\$	a	a	a	a	b	b	b	c	c
	\$	b	b	b	a	c	c	a	a
		a	c	c	b	a	a	\$	b
		b	a	a	c	\$	b		a
		c	\$	b	a		a		b
		a		a	\$		b		c
		\$		b			c		a
				c			a		\$
				a			\$		

$M = a\ b$

Suchen in Suffixarrays



Beobachtung. Die Vorkommen eines Musters M in T bilden ein Intervall in A .

Idee. Finde den linken und rechten Rand des Intervalles durch binäre Suche.

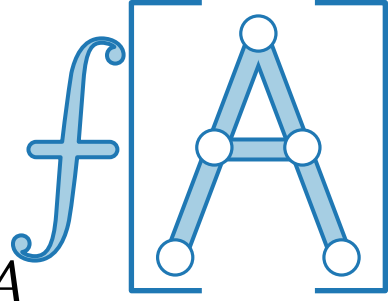
$T = a\ b\ c\ a\ b\ a\ b\ c\ a\ \$$

$A =$

10	9	4	6	1	5	7	2	8	3
\$	a	a	a	a	b	b	b	c	c
	\$	b	b	b	a	c	c	a	a
		a	c	c	b	a	a	\$	b
		b	a	a	c	\$	b		a
		c	\$	b	a		a		b
		a		a	\$		b		c
		\$		b			c		a
				c			a		\$
				a			\$		

$M = a\ b$

Suchen in Suffixarrays



Beobachtung. Die Vorkommen eines Musters M in T bilden ein Intervall in A .

Idee. Finde den linken und rechten Rand des Intervalles durch binäre Suche.

Gebe alle Einträge im Intervall zurück!

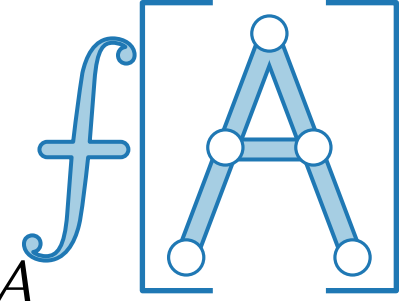
$T = a\ b\ c\ a\ b\ a\ b\ c\ a\ \$$

$A =$

10	9	4	6	1	5	7	2	8	3
\$	a	a	a	a	b	b	b	c	c
	\$	b	b	b	a	c	c	a	a
		a	c	c	b	a	a	\$	b
		b	a	a	c	\$	b		a
		c	\$	b	a		a		b
		a		a	\$		b		c
		\$		b			c		a
				c			a		\$
				a			\$		

$M = a\ b$

Suchen in Suffixarrays



Beobachtung. Die Vorkommen eines Musters M in T bilden ein Intervall in A .

Idee. Finde den linken und rechten Rand des Intervalles durch binäre Suche.

Gebe alle Einträge im Intervall zurück!

FINDELINKENRAND($A, M, \ell = 1, r = n$)

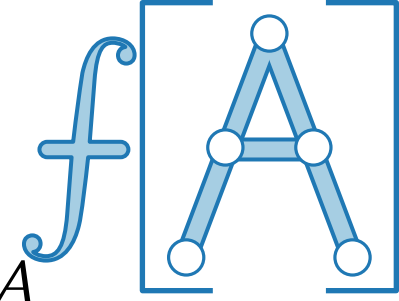
$T = a\ b\ c\ a\ b\ a\ b\ c\ a\ \$$

$A =$

10	9	4	6	1	5	7	2	8	3
\$	a	a	a	a	b	b	b	c	c
	\$	b	b	b	a	c	c	a	a
		a	c	c	b	a	a	\$	b
		b	a	a	c	\$	b		a
		c	\$	b	a		a		b
		a		a	\$		b		c
		\$		b			c		a
				c			a		\$
				a			\$		

$M = a\ b$

Suchen in Suffixarrays



Beobachtung. Die Vorkommen eines Musters M in T bilden ein Intervall in A .

Idee. Finde den linken und rechten Rand des Intervalles durch binäre Suche.

Gebe alle Einträge im Intervall zurück!

FINDELINKENRAND($A, M, \ell = 1, r = n$)

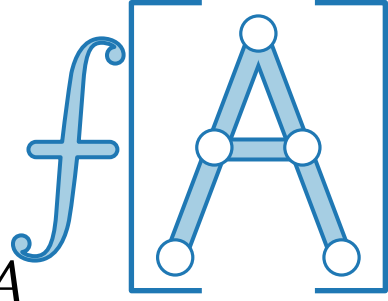
$T = a\ b\ c\ a\ b\ a\ b\ c\ a\ \$$

$A =$

10	9	4	6	1	5	7	2	8	3
\$	a	a	a	a	b	b	b	c	c
	\$	b	b	b	a	c	c	a	a
		a	c	c	b	a	a	\$	b
		b	a	a	c	\$	b		a
		c	\$	b	a		a		b
		a		a	\$		b		c
		\$		b			c		a
				c			a		\$
				a			\$		

$M = a\ b$

Suchen in Suffixarrays



Beobachtung. Die Vorkommen eines Musters M in T bilden ein Intervall in A .

Idee. Finde den linken und rechten Rand des Intervalles durch binäre Suche.

Gebe alle Einträge im Intervall zurück!

```
FINDELINKENRAND( $A, M, \ell = 1, r = n$ )
```

```
  if  $\ell == r$  then
```

```
    // Abbruchbedingung
```

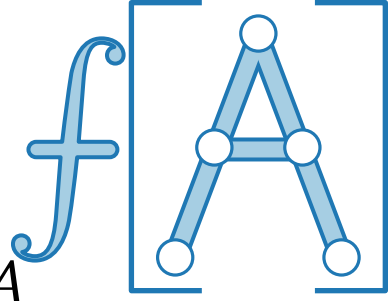
$T = a\ b\ c\ a\ b\ a\ b\ c\ a\ \$$

$A =$

10	9	4	6	1	5	7	2	8	3
\$	a	a	a	a	b	b	b	c	c
	\$	b	b	b	a	c	c	a	a
		a	c	c	b	a	a	\$	b
		b	a	a	c	\$	b		a
		c	\$	b	a		a		b
		a		a	\$		b		c
		\$		b			c		a
				c			a		\$
				a			\$		

$M = a\ b$

Suchen in Suffixarrays



Beobachtung. Die Vorkommen eines Musters M in T bilden ein Intervall in A .

Idee. Finde den linken und rechten Rand des Intervalles durch binäre Suche.

Gebe alle Einträge im Intervall zurück!

```
FINDELINKENRAND( $A, M, \ell = 1, r = n$ )
```

```
  if  $\ell == r$  then
```

```
    // Abbruchbedingung
```

```
   $m = \lfloor (\ell + r) / 2 \rfloor$ 
```

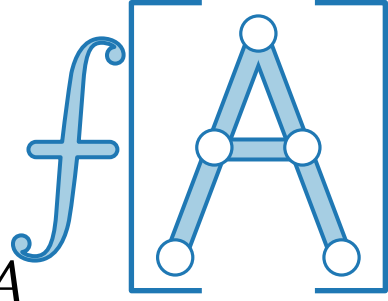
$T = a\ b\ c\ a\ b\ a\ b\ c\ a\ \$$

$A =$

10	9	4	6	1	5	7	2	8	3
\$	a	a	a	a	b	b	b	c	c
	\$	b	b	b	a	c	c	a	a
		a	c	c	b	a	a	\$	b
		b	a	a	c	\$	b		a
		c	\$	b	a		a		b
		a		a	\$		b		c
		\$		b			c		a
				c			a		\$
				a			\$		

$M = a\ b$

Suchen in Suffixarrays



Beobachtung. Die Vorkommen eines Musters M in T bilden ein Intervall in A .

Idee. Finde den linken und rechten Rand des Intervalles durch binäre Suche.

Gebe alle Einträge im Intervall zurück!

```
FINDELINKENRAND( $A, M, \ell = 1, r = n$ )
```

```
  if  $\ell == r$  then
```

```
    // Abbruchbedingung
```

```
     $m = \lfloor (\ell + r) / 2 \rfloor$ 
```

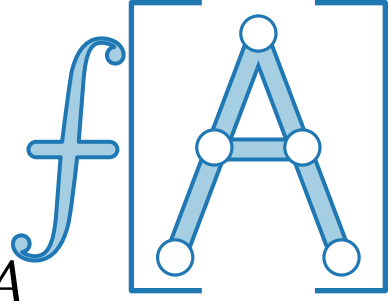
$T = a\ b\ c\ a\ b\ a\ b\ c\ a\ \$$

$A =$

10	9	4	6	1	5	7	2	8	3
\$	a	a	a	a	b	b	b	c	c
	\$	b	b	b	a	c	c	a	a
		a	c	c	b	a	a	\$	b
		b	a	a	c	\$	b		a
		c	\$	b	a		a		b
		a		a	\$		b		c
		\$		b			c		a
				c			a		\$
				a			\$		

$M = a\ b$

Suchen in Suffixarrays



Beobachtung. Die Vorkommen eines Musters M in T bilden ein Intervall in A .

Idee. Finde den linken und rechten Rand des Intervalles durch binäre Suche.

Gebe alle Einträge im Intervall zurück!

```
FINDELINKENRAND( $A, M, \ell = 1, r = n$ )  
  if  $\ell == r$  then  
    // Abbruchbedingung  
    
   $m = \lfloor (\ell + r) / 2 \rfloor$   
  if  $M > S_{A[i]}[1, m]$  then // rechte Hälfte  
    return FINDELINKENRAND( $A, M, m + 1, r$ )
```

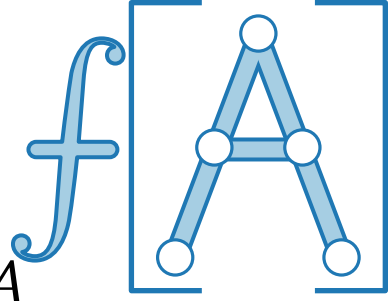
$T = a\ b\ c\ a\ b\ a\ b\ c\ a\ \$$

$A =$

10	9	4	6	1	5	7	2	8	3
\$	a	a	a	a	b	b	b	c	c
	\$	b	b	b	a	c	c	a	a
		a	c	c	b	a	a	\$	b
		b	a	a	c	\$	b		a
		c	\$	b	a		a		b
		a		a	\$		b		c
		\$		b			c		a
				c			a		\$
				a			\$		

$M = a\ b$

Suchen in Suffixarrays



Beobachtung. Die Vorkommen eines Musters M in T bilden ein Intervall in A .

Idee. Finde den linken und rechten Rand des Intervalles durch binäre Suche.

Gebe alle Einträge im Intervall zurück!

```
FINDELINKENRAND( $A, M, \ell = 1, r = n$ )  
  if  $\ell == r$  then  
    |  
    | // Abbruchbedingung  
    |  
   $m = \lfloor (\ell + r) / 2 \rfloor$   
  if  $M > S_{A[i]}[1, m]$  then // rechte Hälfte  
    | return FINDELINKENRAND( $A, M, m + 1, r$ )  
  else // linke Hälfte  
    | return FINDELINKENRAND( $A, M, \ell, m$ )
```

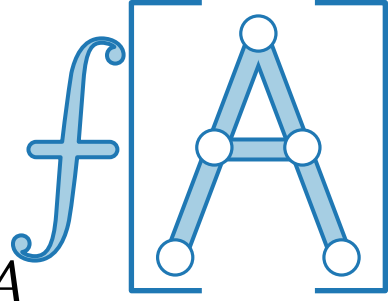
$T = a\ b\ c\ a\ b\ a\ b\ c\ a\ \$$

$A =$

10	9	4	6	1	5	7	2	8	3
\$	a	a	a	a	b	b	b	c	c
	\$	b	b	b	a	c	c	a	a
		a	c	c	b	a	a	\$	b
		b	a	a	c	\$	b		a
		c	\$	b	a		a		b
		a		a	\$		b		c
		\$		b			c		a
				c			a		\$
				a			\$		

$M = a\ b$

Suchen in Suffixarrays



Beobachtung. Die Vorkommen eines Musters M in T bilden ein Intervall in A .

Idee. Finde den linken und rechten Rand des Intervalles durch binäre Suche.

Gebe alle Einträge im Intervall zurück!

```
FINDELINKENRAND( $A, M, \ell = 1, r = n$ )
  if  $\ell == r$  then
    // Abbruchbedingung
  else
     $m = \lfloor (\ell + r) / 2 \rfloor$ 
    if  $M > S_{A[i]}[1, m]$  then // rechte Hälfte
      return FINDELINKENRAND( $A, M, m + 1, r$ )
    else // linke Hälfte
      return FINDELINKENRAND( $A, M, \ell, m$ )
```

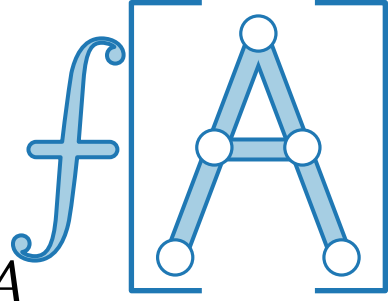
$T = a\ b\ c\ a\ b\ a\ b\ c\ a\ \$$

$A =$

10	9	4	6	1	5	7	2	8	3
\$	a	a	a	a	b	b	b	c	c
	\$	b	b	b	a	c	c	a	a
		a	c	c	b	a	a	\$	b
		b	a	a	c	\$	b		a
		c	\$	b	a		a		b
		a		a	\$		b		c
		\$		b			c		a
				c			a		\$
				a			\$		

$M = a\ b$

Suchen in Suffixarrays



Beobachtung. Die Vorkommen eines Musters M in T bilden ein Intervall in A .

Idee. Finde den linken und rechten Rand des Intervalles durch binäre Suche.

Gebe alle Einträge im Intervall zurück!

```
FINDELINKENRAND( $A, M, \ell = 1, r = n$ )  
  if  $\ell == r$  then  
    |  
    | // Abbruchbedingung  
    |  
     $m = \lfloor (\ell + r) / 2 \rfloor$   
    if  $M > S_{A[i]}[1, m]$  then // rechte Hälfte  
      | return FINDELINKENRAND( $A, M, m + 1, r$ )  
    else // linke Hälfte  
      | return FINDELINKENRAND( $A, M, \ell, m$ )
```

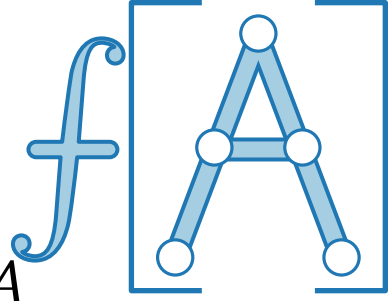
$T = a\ b\ c\ a\ b\ a\ b\ c\ a\ \$$

$A =$

10	9	4	6	1	5	7	2	8	3
\$	a	a	a	a	b	b	b	c	c
	\$	b	b	b	a	c	c	a	a
		a	c	c	b	a	a	\$	b
		b	a	a	c	\$	b		a
		c	\$	b	a		a		b
		a		a	\$		b		c
		\$		b			c		a
				c			a		\$
				a			\$		

$M = a\ b$

Suchen in Suffixarrays



Beobachtung. Die Vorkommen eines Musters M in T bilden ein Intervall in A .

Idee. Finde den linken und rechten Rand des Intervalles durch binäre Suche.

Gebe alle Einträge im Intervall zurück!

```
FINDELINKENRAND( $A, M, \ell = 1, r = n$ )  
  if  $\ell == r$  then  
    |  
    | // Abbruchbedingung  
    |  
   $m = \lfloor (\ell + r) / 2 \rfloor$   
  if  $M > S_{A[i]}[1, m]$  then // rechte Hälfte  
    | return FINDELINKENRAND( $A, M, m + 1, r$ )  
  else // linke Hälfte  
    | return FINDELINKENRAND( $A, M, \ell, m$ )
```

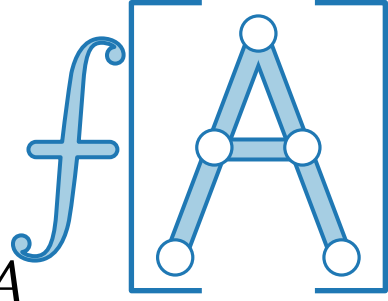
$T = a\ b\ c\ a\ b\ a\ b\ c\ a\ \$$

$A =$

10	9	4	6	1	5	7	2	8	3
\$	a	a	a	a	b	b	b	c	c
	\$	b	b	b	a	c	c	a	a
		a	c	c	b	a	a	\$	b
		b	a	a	c	\$	b		a
		c	\$	b	a		a		b
		a		a	\$		b		c
		\$		b			c		a
				c			a		\$
				a			\$		

$M = a\ b$

Suchen in Suffixarrays



Beobachtung. Die Vorkommen eines Musters M in T bilden ein Intervall in A .

Idee. Finde den linken und rechten Rand des Intervalles durch binäre Suche.

Gebe alle Einträge im Intervall zurück!

```
FINDELINKENRAND( $A, M, \ell = 1, r = n$ )  
  if  $\ell == r$  then  
    |  
    | // Abbruchbedingung  
    |  
     $m = \lfloor (\ell + r) / 2 \rfloor$   
    if  $M > S_{A[i]}[1, m]$  then // rechte Hälfte  
      | return FINDELINKENRAND( $A, M, m + 1, r$ )  
    else // linke Hälfte  
      | return FINDELINKENRAND( $A, M, \ell, m$ )
```

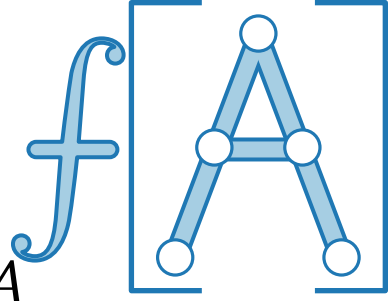
$T = a\ b\ c\ a\ b\ a\ b\ c\ a\ \$$

$A =$

10	9	4	6	1	5	7	2	8	3
\$	a	a	a	a	b	b	b	c	c
	\$	b	b	b	a	c	c	a	a
		a	c	c	b	a	a	\$	b
		b	a	a	c	\$	b		a
		c	\$	b	a		a		b
		a		a	\$		b		c
		\$		b			c		a
				c			a		\$
				a			\$		

$M = a\ b$

Suchen in Suffixarrays



Beobachtung. Die Vorkommen eines Musters M in T bilden ein Intervall in A .

Idee. Finde den linken und rechten Rand des Intervalles durch binäre Suche.

Gebe alle Einträge im Intervall zurück!

```
FINDELINKENRAND( $A, M, \ell = 1, r = n$ )  
  if  $\ell == r$  then  
    |  
    | // Abbruchbedingung  
    |  
   $m = \lfloor (\ell + r) / 2 \rfloor$   
  if  $M > S_{A[i]}[1, m]$  then // rechte Hälfte  
    | return FINDELINKENRAND( $A, M, m + 1, r$ )  
  else // linke Hälfte  
    | return FINDELINKENRAND( $A, M, \ell, m$ )
```

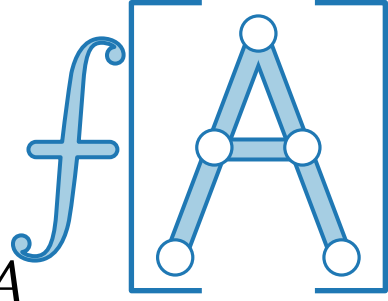
$T = a\ b\ c\ a\ b\ a\ b\ c\ a\ \$$

$A =$

10	9	4	6	1	5	7	2	8	3
\$	a	a	a	a	b	b	b	c	c
	\$	b	b	b	a	c	c	a	a
		a	c	c	b	a	a	\$	b
		b	a	a	c	\$	b		a
		c	\$	b	a		a		b
		a		a	\$		b		c
		\$		b			c		a
				c			a		\$
				a			\$		

$M = a\ b$

Suchen in Suffixarrays



Beobachtung. Die Vorkommen eines Musters M in T bilden ein Intervall in A .

Idee. Finde den linken und rechten Rand des Intervalles durch binäre Suche.

Gebe alle Einträge im Intervall zurück!

```
FINDELINKENRAND( $A, M, \ell = 1, r = n$ )
  if  $\ell == r$  then
    |
    | // Abbruchbedingung
    |
   $m = \lfloor (\ell + r) / 2 \rfloor$ 
  if  $M > S_{A[i]}[1, m]$  then // rechte Hälfte
    | return FINDELINKENRAND( $A, M, m + 1, r$ )
  else // linke Hälfte
    | return FINDELINKENRAND( $A, M, \ell, m$ )
```

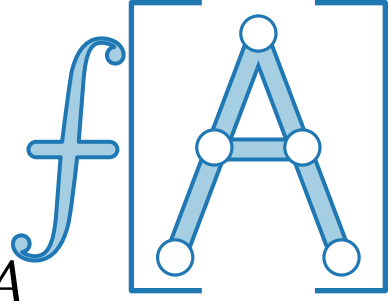
$T = a\ b\ c\ a\ b\ a\ b\ c\ a\ \$$

$A =$

10	9	4	6	1	5	7	2	8	3
\$	a	a	a	a	b	b	b	c	c
	\$	b	b	b	a	c	c	a	a
		a	c	c	b	a	a	\$	b
		b	a	a	c	\$	b		a
		c	\$	b	a		a		b
		a		a	\$		b		c
		\$		b			c		a
				c			a		\$
				a			\$		

$M = ab$

Suchen in Suffixarrays



Beobachtung. Die Vorkommen eines Musters M in T bilden ein Intervall in A .

Idee. Finde den linken und rechten Rand des Intervalles durch binäre Suche.

Gebe alle Einträge im Intervall zurück!

```
FINDELINKENRAND( $A, M, \ell = 1, r = n$ )  
  if  $\ell == r$  then  
    if  $M$  ist Präfix von  $A[\ell]$  then  
      return  $\ell$   
   $m = \lfloor (\ell + r) / 2 \rfloor$   
  if  $M > S_{A[m]}[1, m]$  then // rechte Hälfte  
    return FINDELINKENRAND( $A, M, m + 1, r$ )  
  else // linke Hälfte  
    return FINDELINKENRAND( $A, M, \ell, m$ )
```

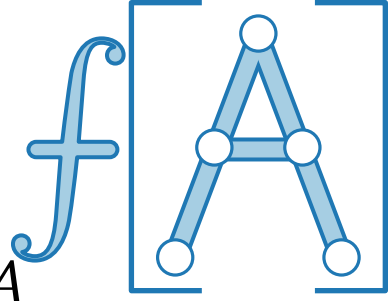
$T = a\ b\ c\ a\ b\ a\ b\ c\ a\ \$$

$A =$

10	9	4	6	1	5	7	2	8	3
\$	a	a	a	a	b	b	b	c	c
	\$	b	b	b	a	c	c	a	a
		a	c	c	b	a	a	\$	b
		b	a	a	c	\$	b		a
		c	\$	b	a		a		b
		a		a	\$		b		c
		\$		b			c		a
				c			a		\$
				a			\$		

$M = ab$

Suchen in Suffixarrays



Beobachtung. Die Vorkommen eines Musters M in T bilden ein Intervall in A .

Idee. Finde den linken und rechten Rand des Intervalles durch binäre Suche.

Gebe alle Einträge im Intervall zurück!

```
FINDELINKENRAND( $A, M, \ell = 1, r = n$ )  
  if  $\ell == r$  then  
    | if  $M$  ist Präfix von  $A[\ell]$  then  
    |   return  $\ell$   
    |  
   $m = \lfloor (\ell + r) / 2 \rfloor$   
  if  $M > S_{A[i]}[1, m]$  then           // rechte Hälfte  
    | return FINDELINKENRAND( $A, M, m + 1, r$ )  
  else                                 // linke Hälfte  
    | return FINDELINKENRAND( $A, M, \ell, m$ )
```

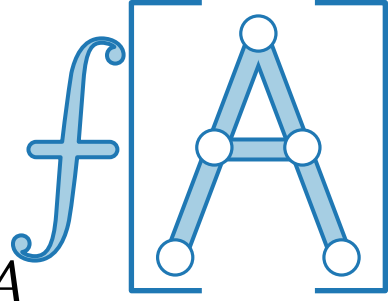
$T = a\ b\ c\ a\ b\ a\ b\ c\ a\ \$$

$A =$

10	9	4	6	1	5	7	2	8	3
\$	a	a	a	a	b	b	b	c	c
	\$	b	b	b	a	c	c	a	a
		a	c	c	b	a	a	\$	b
		b	a	a	c	\$	b		a
		c	\$	b	a		a		b
		a		a	\$		b		c
		\$		b			c		a
				c			a		\$
				a			\$		

$M = ab$

Suchen in Suffixarrays



Beobachtung. Die Vorkommen eines Musters M in T bilden ein Intervall in A .

Idee. Finde den linken und rechten Rand des Intervalles durch binäre Suche.

Gebe alle Einträge im Intervall zurück!

```
FINDELINKENRAND( $A, M, \ell = 1, r = n$ )
  if  $\ell == r$  then
    if  $M$  ist Präfix von  $A[\ell]$  then
      return  $\ell$ 
    else
      return  $\emptyset$ 
   $m = \lfloor (\ell + r) / 2 \rfloor$ 
  if  $M > S_{A[m]}[1, m]$  then           // rechte Hälfte
    return FINDELINKENRAND( $A, M, m + 1, r$ )
  else                                 // linke Hälfte
    return FINDELINKENRAND( $A, M, \ell, m$ )
```

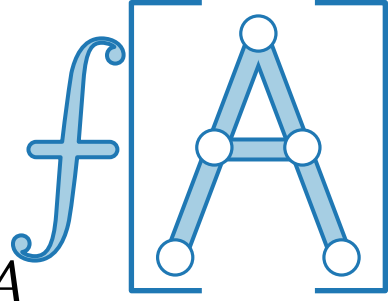
$T = a\ b\ c\ a\ b\ a\ b\ c\ a\ \$$

$A =$

10	9	4	6	1	5	7	2	8	3
\$	a	a	a	a	b	b	b	c	c
	\$	b	b	b	a	c	c	a	a
		a	c	c	b	a	a	\$	b
		b	a	a	c	\$	b		a
		c	\$	b	a		a		b
		a		a	\$		b		c
		\$		b			c		a
				c			a		\$
				a			\$		

$M = ab$

Suchen in Suffixarrays



Beobachtung. Die Vorkommen eines Musters M in T bilden ein Intervall in A .

Idee. Finde den linken und rechten Rand des Intervalles durch binäre Suche.

Gebe alle Einträge im Intervall zurück!

```
FINDERECHTENRAND( $A, M, \ell = 1, r = n$ )
  if  $\ell == r$  then
    if  $M$  ist Präfix von  $A[\ell]$  then
      return  $\ell$ 
    else
      return  $\emptyset$ 
   $m = \lceil (\ell + r) / 2 \rceil$ 
  if  $M < S_{A[m]}[1, m]$  then           // linke Hälfte
    return FINDERECHTENRAND( $A, M, \ell, m - 1$ )
  else                                   // rechte Hälfte
    return FINDERECHTENRAND( $A, M, m, r$ )
```

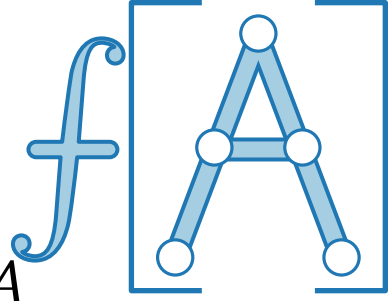
$T = a\ b\ c\ a\ b\ a\ b\ c\ a\ \$$

$A =$

10	9	4	6	1	5	7	2	8	3
\$	a	a	a	a	b	b	b	c	c
	\$	b	b	b	a	c	c	a	a
		a	c	c	b	a	a	\$	b
		b	a	a	c	\$	b		a
		c	\$	b	a		a		b
		a		a	\$		b		c
		\$		b			c		a
				c			a		\$
				a			\$		

$M = a\ b$

Suchen in Suffixarrays



Beobachtung. Die Vorkommen eines Musters M in T bilden ein Intervall in A .

Idee. Finde den linken und rechten Rand des Intervalles durch binäre Suche.

Gebe alle Einträge im Intervall zurück!

```
FINDERECHTENRAND( $A, M, \ell = 1, r = n$ )  
  if  $\ell == r$  then  
    if  $M$  ist Präfix von  $A[\ell]$  then  
      return  $\ell$   
    else  
      return  $\emptyset$   
   $m = \lceil (\ell + r) / 2 \rceil$   
  if  $M < S_{A[i]}[1, m]$  then // linke Hälfte  
    return FINDERECHTENRAND( $A, M, \ell, m - 1$ )  
  else // rechte Hälfte  
    return FINDERECHTENRAND( $A, M, m, r$ )
```

$T = a\ b\ c\ a\ b\ a\ b\ c\ a\ \$$

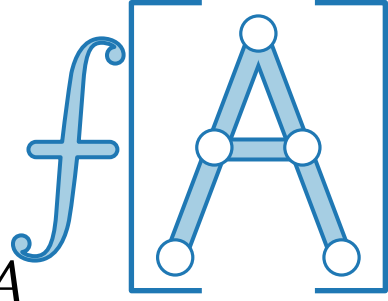
$A =$

10	9	4	6	1	5	7	2	8	3
\$	a	a	a	a	b	b	b	c	c
	\$	b	b	b	a	c	c	a	a
		a	c	c	b	a	a	\$	b
		b	a	a	c	\$	b		a
		c	\$	b	a		a		b
		a		a	\$		b		c
		\$		b			c		a
				c			a		\$
				a			\$		

$M = ab$

Jeder lexikografische Vergleich kann in $\mathcal{O}(m)$ Zeit durchgeführt werden.

Suchen in Suffixarrays



Beobachtung. Die Vorkommen eines Musters M in T bilden ein Intervall in A .

Idee. Finde den linken und rechten Rand des Intervalles durch binäre Suche.

Gebe alle Einträge im Intervall zurück!

```
FINDERECHTENRAND( $A, M, \ell = 1, r = n$ )  
  if  $\ell == r$  then  
    if  $M$  ist Präfix von  $A[\ell]$  then  
      return  $\ell$   
    else  
      return  $\emptyset$   
   $m = \lceil (\ell + r) / 2 \rceil$   
  if  $M < S_{A[i]}[1, m]$  then // linke Hälfte  
    return FINDERECHTENRAND( $A, M, \ell, m - 1$ )  
  else // rechte Hälfte  
    return FINDERECHTENRAND( $A, M, m, r$ )
```

$T = a\ b\ c\ a\ b\ a\ b\ c\ a\ \$$

$A =$	10	9	4	6	1	5	7	2	8	3
	\$	a	a	a	a	b	b	b	c	c
		\$	b	b	b	a	c	c	a	a
			a	c	c	b	a	a	\$	b
			b	a	a	c	\$	b		a
			c	\$	b	a		a		b
			a		a	\$		b		c
			\$		b			c		a
					c			a		\$
					\$					

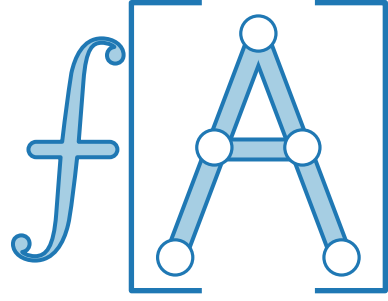
$M = a\ b$

Jeder lexikografische Vergleich kann in $\mathcal{O}(m)$ Zeit durchgeführt werden.

➔ Die k Vorkommen von M können in $\mathcal{O}(m \log n + k)$ Zeit gefunden werden.

Konstruktion von Suffixarrays – Idee

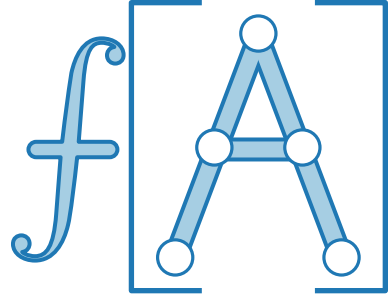
Geg. Ein String T mit $n = |T|$ über Alphabet Σ .



Konstruktion von Suffixarrays – Idee

Geg. Ein String T mit $n = |T|$ über Alphabet Σ .

Ges. Ein Suffixarray A von T .

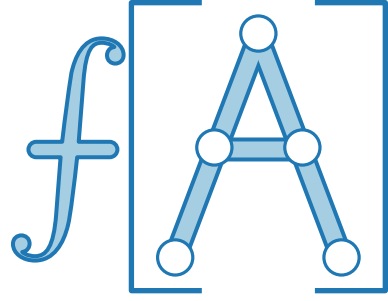


Konstruktion von Suffixarrays – Idee

Geg. Ein String T mit $n = |T|$ über Alphabet Σ .

Ges. Ein Suffixarray A von T .

Idee.



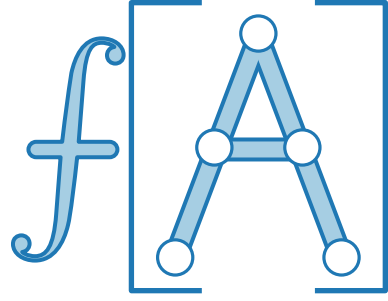
Konstruktion von Suffixarrays – Idee

Geg. Ein String T mit $n = |T|$ über Alphabet Σ .

Ges. Ein Suffixarray A von T .

Idee.

- Wenn $n \in \mathcal{O}(1)$, dann benutze Brute-Force.



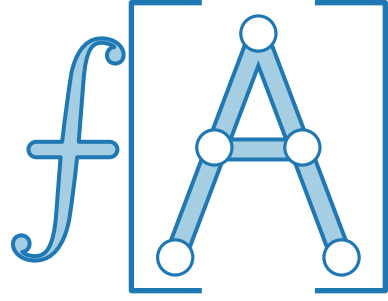
Konstruktion von Suffixarrays – Idee

Geg. Ein String T mit $n = |T|$ über Alphabet Σ .

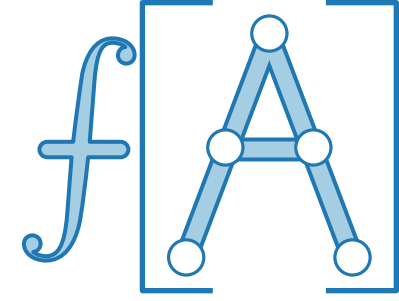
Ges. Ein Suffixarray A von T .

Idee.

- Wenn $n \in \mathcal{O}(1)$, dann benutze Brute-Force.
- Ansonsten zerlege T in Dreiergruppen.



Konstruktion von Suffixarrays – Idee



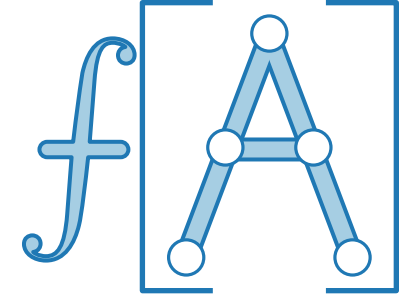
Geg. Ein String T mit $n = |T|$ über Alphabet Σ .

Ges. Ein Suffixarray A von T .

Idee.

- Wenn $n \in \mathcal{O}(1)$, dann benutze Brute-Force.
- Ansonsten zerlege T in Dreiergruppen.
- Interpretiere die Dreiergruppen als Buchstaben über einem Alphabet $\Sigma' \subseteq \Sigma^3$.

Konstruktion von Suffixarrays – Idee



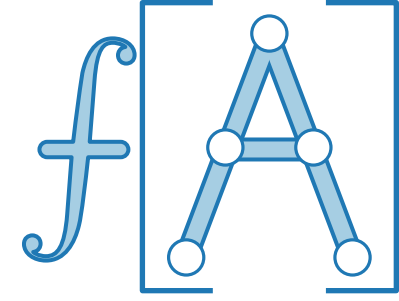
Geg. Ein String T mit $n = |T|$ über Alphabet Σ .

Ges. Ein Suffixarray A von T .

Idee.

- Wenn $n \in \mathcal{O}(1)$, dann benutze Brute-Force.
- Ansonsten zerlege T in Dreiergruppen.
- Interpretiere die Dreiergruppen als Buchstaben über einem Alphabet $\Sigma' \subseteq \Sigma^3$.
- Interpretiere T als String R über Σ' mit $|R| = \lceil n/3 \rceil$.

Konstruktion von Suffixarrays – Idee



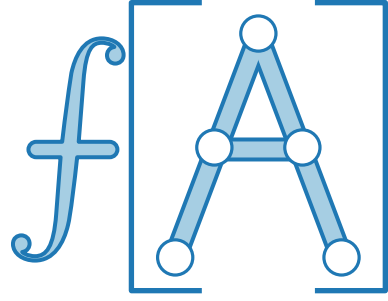
Geg. Ein String T mit $n = |T|$ über Alphabet Σ .

Ges. Ein Suffixarray A von T .

Idee.

- Wenn $n \in \mathcal{O}(1)$, dann benutze Brute-Force.
- Ansonsten zerlege T in Dreiergruppen.
- Interpretiere die Dreiergruppen als Buchstaben über einem Alphabet $\Sigma' \subseteq \Sigma^3$.
- Interpretiere T als String R über Σ' mit $|R| = \lceil n/3 \rceil$.
- Rekursion!

Konstruktion von Suffixarrays – Idee



Geg. Ein String T mit $n = |T|$ über Alphabet Σ .

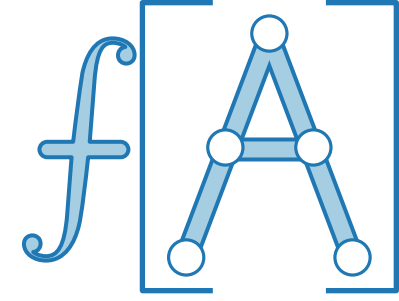
Ges. Ein Suffixarray A von T .

Idee.

- Wenn $n \in \mathcal{O}(1)$, dann benutze Brute-Force.
- Ansonsten zerlege T in Dreiergruppen.
- Interpretiere die Dreiergruppen als Buchstaben über einem Alphabet $\Sigma' \subseteq \Sigma^3$.
- Interpretiere T als String R über Σ' mit $|R| = \lceil n/3 \rceil$.
- Rekursion!

$T =$ y a b b a d a b b a

Konstruktion von Suffixarrays – Idee



Geg. Ein String T mit $n = |T|$ über Alphabet Σ .

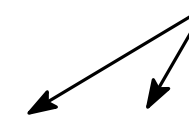
Ges. Ein Suffixarray A von T .

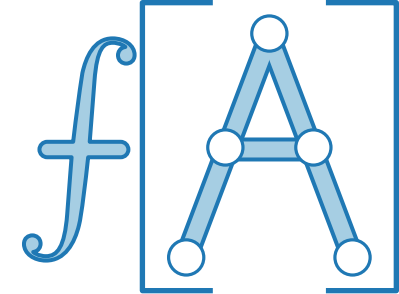
Idee.

- Wenn $n \in \mathcal{O}(1)$, dann benutze Brute-Force.
- Ansonsten zerlege T in Dreiergruppen.
- Interpretiere die Dreiergruppen als Buchstaben über einem Alphabet $\Sigma' \subseteq \Sigma^3$.
- Interpretiere T als String R über Σ' mit $|R| = \lceil n/3 \rceil$.
- Rekursion!

$R = [y \ a \ b] [b \ a \ d] [a \ b \ b] [a \ \$ \ \$]$

Auffüllung





Konstruktion von Suffixarrays – Idee

Geg. Ein String T mit $n = |T|$ über Alphabet Σ .

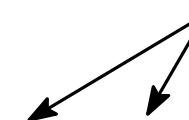
Ges. Ein Suffixarray A von T .

Idee.

- Wenn $n \in \mathcal{O}(1)$, dann benutze Brute-Force.
- Ansonsten zerlege T in Dreiergruppen.
- Interpretiere die Dreiergruppen als Buchstaben über einem Alphabet $\Sigma' \subseteq \Sigma^3$.
- Interpretiere T als String R über Σ' mit $|R| = \lceil n/3 \rceil$.
- Rekursion!

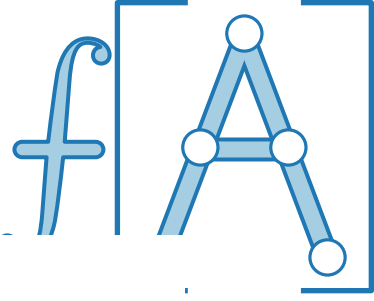
$R = [y \ a \ b] [b \ a \ d] [a \ b \ b] [a \ \$ \ \$]$

Auffüllung



Problem. Aber wie kann ein Suffixarray für R benutzt werden, um ein Suffixarray für T zu erstellen?

Konstruktion von Suffixarrays – Überblick



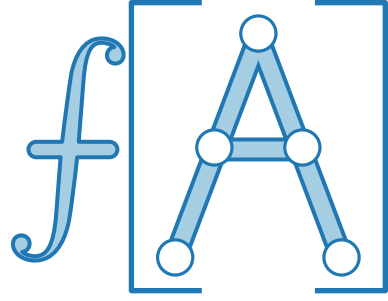
Kurze Notation: $T = t_0 t_1 \dots t_{n-1}$

	0	1	2	3	4	5	6	7	8	9	10	11
$T =$	y	a	b	b	a	d	a	b	b	a	d	o

$\mathcal{S}(T)$ = Suffixe von T =

S_0	y a b b a d a b b a d o
S_1	a b b a d a b b a d o
S_2	b b a d a b b a d o
S_3	b a d a b b a d o
S_4	a d a b b a d o
S_5	d a b b a d o
S_6	a b b a d o
S_7	b b a d o
S_8	b a d o
S_9	a d o
S_{10}	d o
S_{11}	o

Konstruktion von Suffixarrays – Überblick



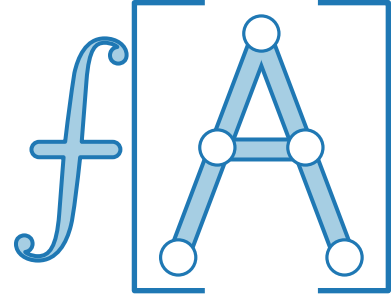
Kurze Notation: $T = t_0 t_1 \dots t_{n-1}$ und $x \equiv_3 y$ ist kurz für $x \bmod 3 = y$.

	0	1	2	3	4	5	6	7	8	9	10	11
$T =$	y	a	b	b	a	d	a	b	b	a	d	o

$\mathcal{S}(T) =$ Suffixe von $T =$

S_0	y a b b a d a b b a d o
S_1	a b b a d a b b a d o
S_2	b b a d a b b a d o
S_3	b a d a b b a d o
S_4	a d a b b a d o
S_5	d a b b a d o
S_6	a b b a d o
S_7	b b a d o
S_8	b a d o
S_9	a d o
S_{10}	d o
S_{11}	o

Konstruktion von Suffixarrays – Überblick



Kurze Notation: $T = t_0 t_1 \dots t_{n-1}$ und $x \equiv_3 y$ ist kurz für $x \bmod 3 = y$.

	0	1	2	3	4	5	6	7	8	9	10	11
$T =$	y	a	b	b	a	d	a	b	b	a	d	o

$\mathcal{S}_0 =$ Suffixe mit Index $i \equiv_3 0$

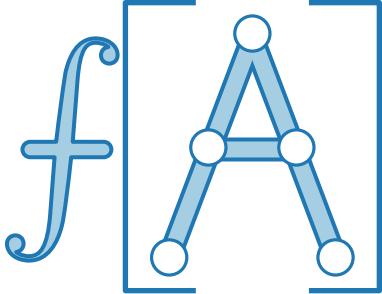
$\mathcal{S}_1 =$ Suffixe mit Index $i \equiv_3 1$

$\mathcal{S}_2 =$ Suffixe mit Index $i \equiv_3 2$

$\mathcal{S}(T) =$ Suffixe von $T =$

S_0	y a b b a d a b b a d o
S_1	a b b a d a b b a d o
S_2	b b a d a b b a d o
S_3	b a d a b b a d o
S_4	a d a b b a d o
S_5	d a b b a d o
S_6	a b b a d o
S_7	b b a d o
S_8	b a d o
S_9	a d o
S_{10}	d o
S_{11}	o

Konstruktion von Suffixarrays – Überblick



Kurze Notation: $T = t_0t_1 \dots t_{n-1}$ und $x \equiv_3 y$ ist kurz für $x \bmod 3 = y$.

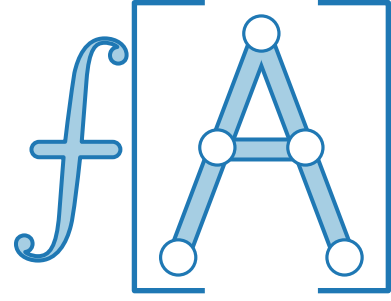
	0	1	2	3	4	5	6	7	8	9	10	11
$T =$	y	a	b	b	a	d	a	b	b	a	d	o

- $\mathcal{S}_0 =$ Suffixe mit Index $i \equiv_3 0$
- $\mathcal{S}_1 =$ Suffixe mit Index $i \equiv_3 1$
- $\mathcal{S}_2 =$ Suffixe mit Index $i \equiv_3 2$

$\mathcal{S}(T) =$ Suffixe von $T =$

S_0	y a b b a d a b b a d o
S_1	a b b a d a b b a d o
S_2	b b a d a b b a d o
S_3	b a d a b b a d o
S_4	a d a b b a d o
S_5	d a b b a d o
S_6	a b b a d o
S_7	b b a d o
S_8	b a d o
S_9	a d o
S_{10}	d o
S_{11}	o

Konstruktion von Suffixarrays – Überblick



Kurze Notation: $T = t_0 t_1 \dots t_{n-1}$ und $x \equiv_3 y$ ist kurz für $x \bmod 3 = y$.

	0	1	2	3	4	5	6	7	8	9	10	11
$T =$	y	a	b	b	a	d	a	b	b	a	d	o

KONSTRUIERE SUFFIXARRAY(T)

$\mathcal{S}_0 =$ Suffixe mit Index $i \equiv_3 0$

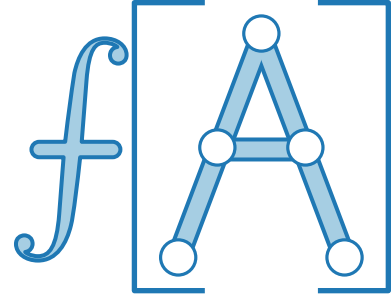
$\mathcal{S}_1 =$ Suffixe mit Index $i \equiv_3 1$

$\mathcal{S}_2 =$ Suffixe mit Index $i \equiv_3 2$

$\mathcal{S}(T) =$ Suffixe von $T =$

S_0	y a b b a d a b b a d o
S_1	a b b a d a b b a d o
S_2	b b a d a b b a d o
S_3	b a d a b b a d o
S_4	a d a b b a d o
S_5	d a b b a d o
S_6	a b b a d o
S_7	b b a d o
S_8	b a d o
S_9	a d o
S_{10}	d o
S_{11}	o

Konstruktion von Suffixarrays – Überblick



Kurze Notation: $T = t_0 t_1 \dots t_{n-1}$ und $x \equiv_3 y$ ist kurz für $x \bmod 3 = y$.

	0	1	2	3	4	5	6	7	8	9	10	11
$T =$	y	a	b	b	a	d	a	b	b	a	d	o

KONSTRUIERE SUFFIXARRAY(T)

if $n = \mathcal{O}(1)$ **then**
 | erstelle A in $\mathcal{O}(1)$ Zeit.

$\mathcal{S}_0 =$ Suffixe mit Index $i \equiv_3 0$

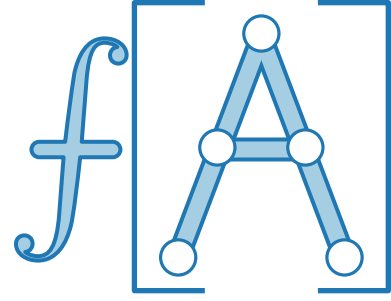
$\mathcal{S}_1 =$ Suffixe mit Index $i \equiv_3 1$

$\mathcal{S}_2 =$ Suffixe mit Index $i \equiv_3 2$

$\mathcal{S}(T) =$ Suffixe von $T =$

S_0	y a b b a d a b b a d o
S_1	a b b a d a b b a d o
S_2	b b a d a b b a d o
S_3	b a d a b b a d o
S_4	a d a b b a d o
S_5	d a b b a d o
S_6	a b b a d o
S_7	b b a d o
S_8	b a d o
S_9	a d o
S_{10}	d o
S_{11}	o

Konstruktion von Suffixarrays – Überblick



Kurze Notation: $T = t_0 t_1 \dots t_{n-1}$ und $x \equiv_3 y$ ist kurz für $x \bmod 3 = y$.

	0	1	2	3	4	5	6	7	8	9	10	11
$T =$	y	a	b	b	a	d	a	b	b	a	d	o

KONSTRUIERE SUFFIXARRAY(T)

```

if  $n = \mathcal{O}(1)$  then
  | erstelle  $A$  in  $\mathcal{O}(1)$  Zeit.
else
  |

```

$\mathcal{S}_0 =$ Suffixe mit Index $i \equiv_3 0$

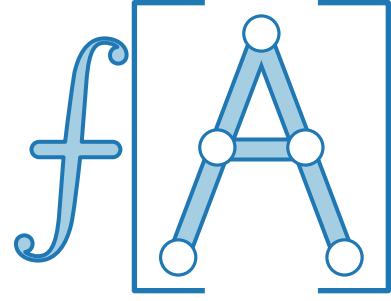
$\mathcal{S}_1 =$ Suffixe mit Index $i \equiv_3 1$

$\mathcal{S}_2 =$ Suffixe mit Index $i \equiv_3 2$

$\mathcal{S}(T) =$ Suffixe von $T =$

S_0	y a b b a d a b b a d o
S_1	a b b a d a b b a d o
S_2	b b a d a b b a d o
S_3	b a d a b b a d o
S_4	a d a b b a d o
S_5	d a b b a d o
S_6	a b b a d o
S_7	b b a d o
S_8	b a d o
S_9	a d o
S_{10}	d o
S_{11}	o

Konstruktion von Suffixarrays – Überblick



Kurze Notation: $T = t_0 t_1 \dots t_{n-1}$ und $x \equiv_3 y$ ist kurz für $x \bmod 3 = y$.

	0	1	2	3	4	5	6	7	8	9	10	11
$T =$	y	a	b	b	a	d	a	b	b	a	d	o

KONSTRUIERE SUFFIXARRAY(T)

if $n = \mathcal{O}(1)$ **then**

└ erstelle A in $\mathcal{O}(1)$ Zeit.

else

└ sortiere $\mathcal{S}_1 \cup \mathcal{S}_2$ in ein Array A_{12}

$\mathcal{S}_0 =$ Suffixe mit Index $i \equiv_3 0$

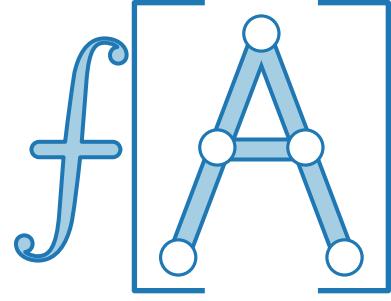
$\mathcal{S}_1 =$ Suffixe mit Index $i \equiv_3 1$

$\mathcal{S}_2 =$ Suffixe mit Index $i \equiv_3 2$

$\mathcal{S}(T) =$ Suffixe von $T =$

S_0	y a b b a d a b b a d o
S_1	a b b a d a b b a d o
S_2	b b a d a b b a d o
S_3	b a d a b b a d o
S_4	a d a b b a d o
S_5	d a b b a d o
S_6	a b b a d o
S_7	b b a d o
S_8	b a d o
S_9	a d o
S_{10}	d o
S_{11}	o

Konstruktion von Suffixarrays – Überblick



Kurze Notation: $T = t_0 t_1 \dots t_{n-1}$ und $x \equiv_3 y$ ist kurz für $x \bmod 3 = y$.

	0	1	2	3	4	5	6	7	8	9	10	11
$T =$	y	a	b	b	a	d	a	b	b	a	d	o

$\mathcal{S}_0 =$ Suffixe mit Index $i \equiv_3 0$

$\mathcal{S}_1 =$ Suffixe mit Index $i \equiv_3 1$

$\mathcal{S}_2 =$ Suffixe mit Index $i \equiv_3 2$

KONSTRUIERE SUFFIXARRAY(T)

if $n = \mathcal{O}(1)$ **then**

└ erstelle A in $\mathcal{O}(1)$ Zeit.

else

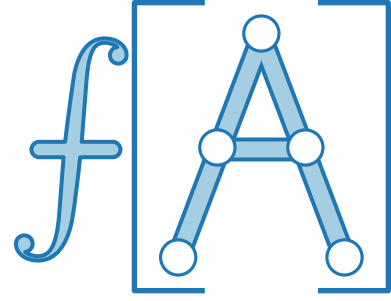
└ sortiere $\mathcal{S}_1 \cup \mathcal{S}_2$ in ein Array A_{12}

└ benutze A_{12} , um $\mathcal{S}_0$ in ein Array A_0 zu sortieren

$\mathcal{S}(T) =$ Suffixe von $T =$

S_0	y a b b a d a b b a d o
S_1	a b b a d a b b a d o
S_2	b b a d a b b a d o
S_3	b a d a b b a d o
S_4	a d a b b a d o
S_5	d a b b a d o
S_6	a b b a d o
S_7	b b a d o
S_8	b a d o
S_9	a d o
S_{10}	d o
S_{11}	o

Konstruktion von Suffixarrays – Überblick



Kurze Notation: $T = t_0 t_1 \dots t_{n-1}$ und $x \equiv_3 y$ ist kurz für $x \bmod 3 = y$.

	0	1	2	3	4	5	6	7	8	9	10	11
$T =$	y	a	b	b	a	d	a	b	b	a	d	o

KONSTRUIERE SUFFIXARRAY(T)

if $n = \mathcal{O}(1)$ **then**

 | erstelle A in $\mathcal{O}(1)$ Zeit.

else

 | sortiere $\mathcal{S}_1 \cup \mathcal{S}_2$ in ein Array A_{12}

 | benutze A_{12} , um $\mathcal{S}_0$ in ein Array A_0 zu sortieren

 | merge A_{12} mit A_0

$\mathcal{S}_0 =$ Suffixe mit Index $i \equiv_3 0$

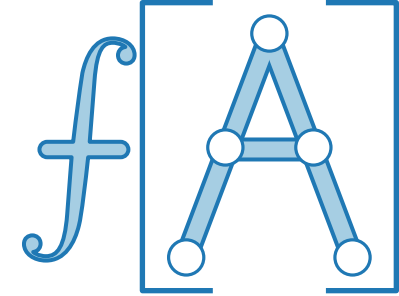
$\mathcal{S}_1 =$ Suffixe mit Index $i \equiv_3 1$

$\mathcal{S}_2 =$ Suffixe mit Index $i \equiv_3 2$

$\mathcal{S}(T) =$ Suffixe von $T =$

$\mathcal{S}_0$	y a b b a d a b b a d o
$\mathcal{S}_1$	a b b a d a b b a d o
$\mathcal{S}_2$	b b a d a b b a d o
$\mathcal{S}_3$	b a d a b b a d o
$\mathcal{S}_4$	a d a b b a d o
$\mathcal{S}_5$	d a b b a d o
$\mathcal{S}_6$	a b b a d o
$\mathcal{S}_7$	b b a d o
$\mathcal{S}_8$	b a d o
$\mathcal{S}_9$	a d o
$\mathcal{S}_{10}$	d o
$\mathcal{S}_{11}$	o

Konstruktion von Suffixarrays – Überblick



Kurze Notation: $T = t_0 t_1 \dots t_{n-1}$ und $x \equiv_3 y$ ist kurz für $x \bmod 3 = y$.

	0	1	2	3	4	5	6	7	8	9	10	11
$T =$	y	a	b	b	a	d	a	b	b	a	d	o

KONSTRUIERE SUFFIXARRAY(T)

if $n = \mathcal{O}(1)$ **then**

 | erstelle A in $\mathcal{O}(1)$ Zeit.

else

 | sortiere $\mathcal{S}_1 \cup \mathcal{S}_2$ in ein Array A_{12}

 | benutze A_{12} , um $\mathcal{S}_0$ in ein Array A_0 zu sortieren

 | merge A_{12} mit A_0

$\mathcal{S}_0 =$ Suffixe mit Index $i \equiv_3 0$

$\mathcal{S}_1 =$ Suffixe mit Index $i \equiv_3 1$

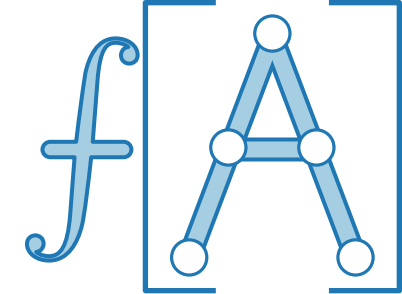
$\mathcal{S}_2 =$ Suffixe mit Index $i \equiv_3 2$

$\mathcal{S}(T) =$ Suffixe von $T =$

$\mathcal{S}_0$	y a b b a d a b b a d o
$\mathcal{S}_1$	a b b a d a b b a d o
$\mathcal{S}_2$	b b a d a b b a d o
$\mathcal{S}_3$	b a d a b b a d o
$\mathcal{S}_4$	a d a b b a d o
$\mathcal{S}_5$	d a b b a d o
$\mathcal{S}_6$	a b b a d o
$\mathcal{S}_7$	b b a d o
$\mathcal{S}_8$	b a d o
$\mathcal{S}_9$	a d o
$\mathcal{S}_{10}$	d o
$\mathcal{S}_{11}$	o

Der Einfachheit halber nehmen wir $n \equiv_3 0$ an.

Konstruktion von Suffixarrays – Überblick



Kurze Notation: $T = t_0 t_1 \dots t_{n-1}$ und $x \equiv_3 y$ ist kurz für $x \bmod 3 = y$.

	0	1	2	3	4	5	6	7	8	9	10	11
$T =$	y	a	b	b	a	d	a	b	b	a	d	o

$\mathcal{S}_0 =$ Suffixe mit Index $i \equiv_3 0$

$\mathcal{S}_1 =$ Suffixe mit Index $i \equiv_3 1$

$\mathcal{S}_2 =$ Suffixe mit Index $i \equiv_3 2$

KONSTRUIERE SUFFIXARRAY(T)

if $n = \mathcal{O}(1)$ **then**

└ erstelle A in $\mathcal{O}(1)$ Zeit.

else

└ sortiere $\mathcal{S}_1 \cup \mathcal{S}_2$ in ein Array A_{12}

└ benutze A_{12} , um $\mathcal{S}_0$ in ein Array A_0 zu sortieren

└ merge A_{12} mit A_0

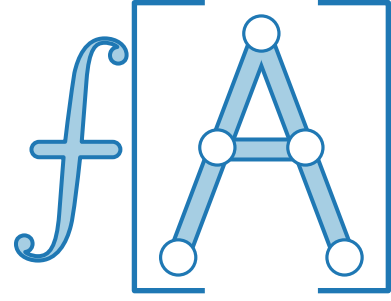
mit der Idee von der
vorigen Folie

$\mathcal{S}(T) =$ Suffixe von $T =$

$\mathcal{S}_0$	y a b b a d a b b a d o
$\mathcal{S}_1$	a b b a d a b b a d o
$\mathcal{S}_2$	b b a d a b b a d o
$\mathcal{S}_3$	b a d a b b a d o
$\mathcal{S}_4$	a d a b b a d o
$\mathcal{S}_5$	d a b b a d o
$\mathcal{S}_6$	a b b a d o
$\mathcal{S}_7$	b b a d o
$\mathcal{S}_8$	b a d o
$\mathcal{S}_9$	a d o
$\mathcal{S}_{10}$	d o
$\mathcal{S}_{11}$	o

Der Einfachheit halber nehmen wir $n \equiv_3 0$ an.

Konstruktion von Suffixarrays – Überblick



Kurze Notation: $T = t_0 t_1 \dots t_{n-1}$ und $x \equiv_3 y$ ist kurz für $x \bmod 3 = y$.

	0	1	2	3	4	5	6	7	8	9	10	11
$T =$	y	a	b	b	a	d	a	b	b	a	d	o

$\mathcal{S}_0 =$ Suffixe mit Index $i \equiv_3 0$

$\mathcal{S}_1 =$ Suffixe mit Index $i \equiv_3 1$

$\mathcal{S}_2 =$ Suffixe mit Index $i \equiv_3 2$

KONSTRUIERE SUFFIXARRAY(T)

if $n = \mathcal{O}(1)$ **then**

└ erstelle A in $\mathcal{O}(1)$ Zeit.

else

└ sortiere $\mathcal{S}_1 \cup \mathcal{S}_2$ in ein Array A_{12}

└ benutze A_{12} , um $\mathcal{S}_0$ in ein Array A_0 zu sortieren

└ merge A_{12} mit A_0

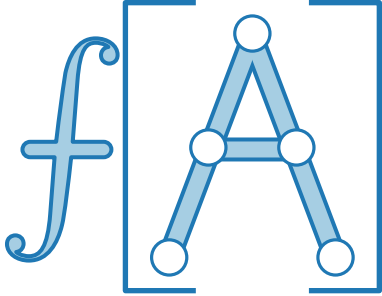
mit der Idee von der
vorigen Folie

$\mathcal{S}(T) =$ Suffixe von $T =$

$\mathcal{S}_0$	y a b b a d a b b a d o
$\mathcal{S}_1$	a b b a d a b b a d o
$\mathcal{S}_2$	b b a d a b b a d o
$\mathcal{S}_3$	b a d a b b a d o
$\mathcal{S}_4$	a d a b b a d o
$\mathcal{S}_5$	d a b b a d o
$\mathcal{S}_6$	a b b a d o
$\mathcal{S}_7$	b b a d o
$\mathcal{S}_8$	b a d o
$\mathcal{S}_9$	a d o
$\mathcal{S}_{10}$	d o
$\mathcal{S}_{11}$	o

Der Einfachheit halber nehmen wir $n \equiv_3 0$ an.

Schritt 1: sortiere $\mathcal{S}_1 \cup \mathcal{S}_2$



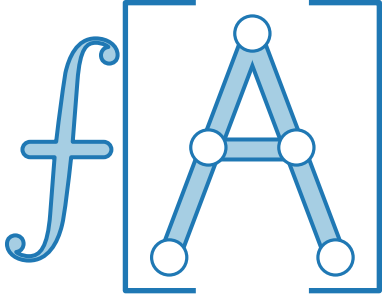
Kurze Notation: $T = t_0t_1 \dots t_{n-1}$ und $x \equiv_3 y$ ist kurz für $x \bmod 3 = y$.

- $\mathcal{S}_0$ = Suffixe mit Index $i \equiv_3 0$
- $\mathcal{S}_1$ = Suffixe mit Index $i \equiv_3 1$
- $\mathcal{S}_2$ = Suffixe mit Index $i \equiv_3 2$

$\mathcal{S}(T)$ = Suffixe von T =

S_0	y a b b a d a b b a d o
S_1	a b b a d a b b a d o
S_2	b b a d a b b a d o
S_3	b a d a b b a d o
S_4	a d a b b a d o
S_5	d a b b a d o
S_6	a b b a d o
S_7	b b a d o
S_8	b a d o
S_9	a d o
S_{10}	d o
S_{11}	o

Schritt 1: sortiere $\mathcal{S}_1 \cup \mathcal{S}_2$



Kurze Notation: $T = t_0t_1 \dots t_{n-1}$ und $x \equiv_3 y$ ist kurz für $x \bmod 3 = y$.

Zerlege S_1 und S_2 in Dreiergruppen und verknüpfe sie:

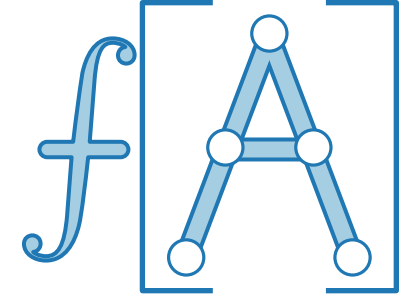
$\mathcal{S}_0$ = Suffixe mit Index $i \equiv_3 0$

$\mathcal{S}_1$ = Suffixe mit Index $i \equiv_3 1$

$\mathcal{S}_2$ = Suffixe mit Index $i \equiv_3 2$

$\mathcal{S}(T)$ = Suffixe von T =

S_0	y a b b a d a b b a d o
S_1	a b b a d a b b a d o
S_2	b b a d a b b a d o
S_3	b a d a b b a d o
S_4	a d a b b a d o
S_5	d a b b a d o
S_6	a b b a d o
S_7	b b a d o
S_8	b a d o
S_9	a d o
S_{10}	d o
S_{11}	o



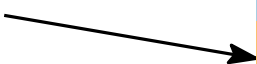
Schritt 1: sortiere $\mathcal{S}_1 \cup \mathcal{S}_2$

Kurze Notation: $T = t_0t_1 \dots t_{n-1}$ und $x \equiv_3 y$ ist kurz für $x \bmod 3 = y$.

Zerlege S_1 und S_2 in Dreiergruppen und verknüpfe sie:

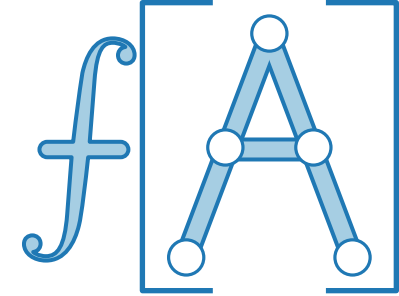
- $\mathcal{S}_0$ = Suffixe mit Index $i \equiv_3 0$
- $\mathcal{S}_1$ = Suffixe mit Index $i \equiv_3 1$
- $\mathcal{S}_2$ = Suffixe mit Index $i \equiv_3 2$

$$R_1 = [t_1t_2t_3][t_4t_5t_6] \dots = [abb][ada][bba][do\$]$$



$\mathcal{S}(T)$ = Suffixe von T =

S_0	y a b b a d a b b a d o
S_1	a b b a d a b b a d o
S_2	b b a d a b b a d o
S_3	b a d a b b a d o
S_4	a d a b b a d o
S_5	d a b b a d o
S_6	a b b a d o
S_7	b b a d o
S_8	b a d o
S_9	a d o
S_{10}	d o
S_{11}	o



Schritt 1: sortiere $\mathcal{S}_1 \cup \mathcal{S}_2$

Kurze Notation: $T = t_0t_1 \dots t_{n-1}$ und $x \equiv_3 y$ ist kurz für $x \bmod 3 = y$.

Zerlege S_1 und S_2 in Dreiergruppen und verknüpfe sie:

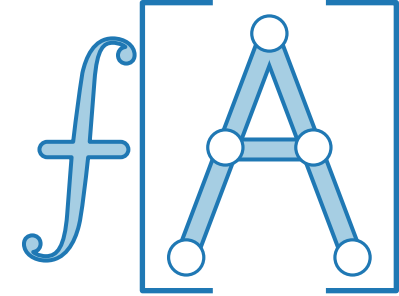
- $\mathcal{S}_0$ = Suffixe mit Index $i \equiv_3 0$
- $\mathcal{S}_1$ = Suffixe mit Index $i \equiv_3 1$
- $\mathcal{S}_2$ = Suffixe mit Index $i \equiv_3 2$

$$R_1 = [t_1t_2t_3][t_4t_5t_6] \dots = [abb][ada][bba][do\$]$$

$$R_2 = [t_2t_3t_4][t_5t_6t_7] \dots = [bba][dab][bad][o\$\$]$$

$\mathcal{S}(T)$ = Suffixe von T =

S_0	y a b b a d a b b a d o
S_1	a b b a d a b b a d o
S_2	b b a d a b b a d o
S_3	b a d a b b a d o
S_4	a d a b b a d o
S_5	d a b b a d o
S_6	a b b a d o
S_7	b b a d o
S_8	b a d o
S_9	a d o
S_{10}	d o
S_{11}	o



Schritt 1: sortiere $\mathcal{S}_1 \cup \mathcal{S}_2$

Kurze Notation: $T = t_0t_1 \dots t_{n-1}$ und $x \equiv_3 y$ ist kurz für $x \bmod 3 = y$.

Zerlege S_1 und S_2 in Dreiergruppen und verknüpfe sie:

$$R = [abb][ada][bba][do\$][bba][dab][bad][o\$\$]$$

$\mathcal{S}_0$ = Suffixe mit Index $i \equiv_3 0$

$\mathcal{S}_1$ = Suffixe mit Index $i \equiv_3 1$

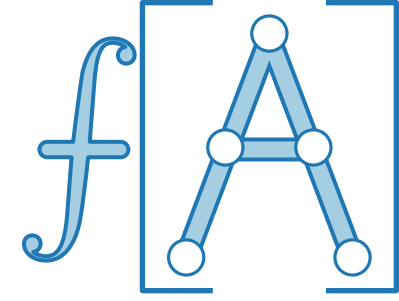
$\mathcal{S}_2$ = Suffixe mit Index $i \equiv_3 2$

$$R_1 = [t_1t_2t_3][t_4t_5t_6] \dots = [abb][ada][bba][do\$]$$

$$R_2 = [t_2t_3t_4][t_5t_6t_7] \dots = [bba][dab][bad][o\$\$]$$

$\mathcal{S}(T)$ = Suffixe von T =

S_0	y a b b a d a b b a d o
S_1	a b b a d a b b a d o
S_2	b b a d a b b a d o
S_3	b a d a b b a d o
S_4	a d a b b a d o
S_5	d a b b a d o
S_6	a b b a d o
S_7	b b a d o
S_8	b a d o
S_9	a d o
S_{10}	d o
S_{11}	o



Schritt 1: sortiere $\mathcal{S}_1 \cup \mathcal{S}_2$

Kurze Notation: $T = t_0t_1 \dots t_{n-1}$ und $x \equiv_3 y$ ist kurz für $x \bmod 3 = y$.

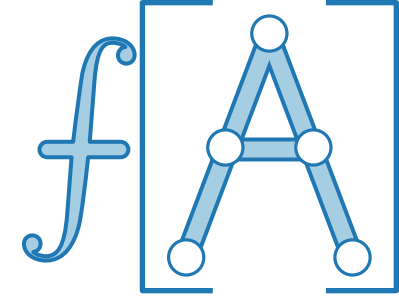
Zerlege S_1 und S_2 in Dreiergruppen und verknüpfe sie:

$R = [abb][ada][bba][do\$][bba][dab][bad][o\$\$]$

- $\mathcal{S}_0 =$ Suffixe mit Index $i \equiv_3 0$
- $\mathcal{S}_1 =$ Suffixe mit Index $i \equiv_3 1$
- $\mathcal{S}_2 =$ Suffixe mit Index $i \equiv_3 2$

$\mathcal{S}(T) =$ Suffixe von $T =$

S_0	y a b b a d a b b a d o
S_1	a b b a d a b b a d o
S_2	b b a d a b b a d o
S_3	b a d a b b a d o
S_4	a d a b b a d o
S_5	d a b b a d o
S_6	a b b a d o
S_7	b b a d o
S_8	b a d o
S_9	a d o
S_{10}	d o
S_{11}	o



Schritt 1: sortiere $\mathcal{S}_1 \cup \mathcal{S}_2$

Kurze Notation: $T = t_0t_1 \dots t_{n-1}$ und $x \equiv_3 y$ ist kurz für $x \bmod 3 = y$.

Zerlege S_1 und S_2 in Dreiergruppen und verknüpfe sie:

$R =$ [abb][ada][bba][do\$][bba][dab][bad][o\$]\$]

$\mathcal{S}(R) =$

$S_1(R)$	[abb][ada][bba][do\$][bba][dab][bad][o\$]\$]
$S_2(R)$	[ada][bba][do\$][bba][dab][bad][o\$]\$]
$S_3(R)$	[bba][do\$][bba][dab][bad][o\$]\$]
$S_4(R)$	[do\$][bba][dab][bad][o\$]\$]
$S_5(R)$	[bba][dab][bad][o\$]\$]
$S_6(R)$	[dab][bad][o\$]\$]
$S_7(R)$	[bad][o\$]\$]
$S_8(R)$	[o\$]\$]

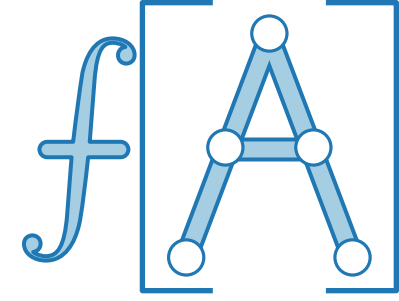
$\mathcal{S}_0 =$ Suffixe mit Index $i \equiv_3 0$

$\mathcal{S}_1 =$ Suffixe mit Index $i \equiv_3 1$

$\mathcal{S}_2 =$ Suffixe mit Index $i \equiv_3 2$

$\mathcal{S}(T) =$ Suffixe von $T =$

S_0	y a b b a d a b b a d o
S_1	a b b a d a b b a d o
S_2	b b a d a b b a d o
S_3	b a d a b b a d o
S_4	a d a b b a d o
S_5	d a b b a d o
S_6	a b b a d o
S_7	b b a d o
S_8	b a d o
S_9	a d o
S_{10}	d o
S_{11}	o



Schritt 1: sortiere $\mathcal{S}_1 \cup \mathcal{S}_2$

Kurze Notation: $T = t_0t_1 \dots t_{n-1}$ und $x \equiv_3 y$ ist kurz für $x \bmod 3 = y$.

Zerlege S_1 und S_2 in Dreiergruppen und verknüpfe sie:

$$R = [\text{abb}][\text{ada}][\text{bba}][\text{do\$}][\text{bba}][\text{dab}][\text{bad}][\text{o\$\$}]$$

$$\mathcal{S}(R) =$$

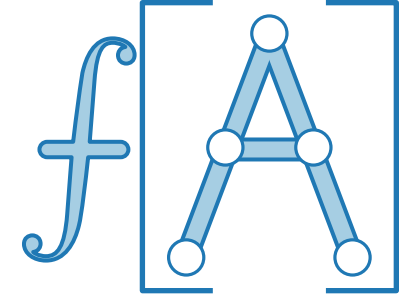
$S_1(R)$	$[\text{abb}][\text{ada}][\text{bba}][\text{do\$}][\text{bba}][\text{dab}][\text{bad}][\text{o\$\$}]$
$S_2(R)$	$[\text{ada}][\text{bba}][\text{do\$}][\text{bba}][\text{dab}][\text{bad}][\text{o\$\$}]$
$S_3(R)$	$[\text{bba}][\text{do\$}][\text{bba}][\text{dab}][\text{bad}][\text{o\$\$}]$
$S_4(R)$	$[\text{do\$}][\text{bba}][\text{dab}][\text{bad}][\text{o\$\$}]$
$S_5(R)$	$[\text{bba}][\text{dab}][\text{bad}][\text{o\$\$}]$
$S_6(R)$	$[\text{dab}][\text{bad}][\text{o\$\$}]$
$S_7(R)$	$[\text{bad}][\text{o\$\$}]$
$S_8(R)$	$[\text{o\$\$}]$

Beobachtung. $\mathcal{S}(R)$ entspricht bijektiv $\mathcal{S}_1 \cup \mathcal{S}_2$

- $\mathcal{S}_0$ = Suffixe mit Index $i \equiv_3 0$
- $\mathcal{S}_1$ = Suffixe mit Index $i \equiv_3 1$
- $\mathcal{S}_2$ = Suffixe mit Index $i \equiv_3 2$

$\mathcal{S}(T)$ = Suffixe von T =

S_0	y a b b a d a b b a d o
S_1	a b b a d a b b a d o
S_2	b b a d a b b a d o
S_3	b a d a b b a d o
S_4	a d a b b a d o
S_5	d a b b a d o
S_6	a b b a d o
S_7	b b a d o
S_8	b a d o
S_9	a d o
S_{10}	d o
S_{11}	o



Schritt 1: sortiere $\mathcal{S}_1 \cup \mathcal{S}_2$

Kurze Notation: $T = t_0t_1 \dots t_{n-1}$ und $x \equiv_3 y$ ist kurz für $x \bmod 3 = y$.

Zerlege S_1 und S_2 in Dreiergruppen und verknüpfe sie:

$$R = [\text{abb}][\text{ada}][\text{bba}][\text{do\$}][\text{bba}][\text{dab}][\text{bad}][\text{o\$\$}]$$

$\mathcal{S}(R) =$

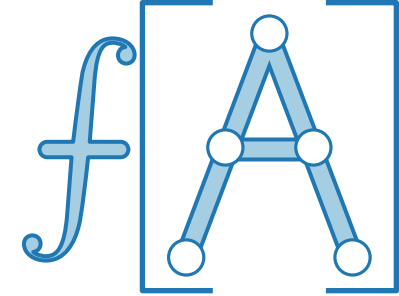
$S_1(R)$	$[\text{abb}][\text{ada}][\text{bba}][\text{do\$}][\text{bba}][\text{dab}][\text{bad}][\text{o\$\$}]$
$S_2(R)$	$[\text{ada}][\text{bba}][\text{do\$}][\text{bba}][\text{dab}][\text{bad}][\text{o\$\$}]$
$S_3(R)$	$[\text{bba}][\text{do\$}][\text{bba}][\text{dab}][\text{bad}][\text{o\$\$}]$
$S_4(R)$	$[\text{do\$}][\text{bba}][\text{dab}][\text{bad}][\text{o\$\$}]$
$S_5(R)$	$[\text{bba}][\text{dab}][\text{bad}][\text{o\$\$}]$
$S_6(R)$	$[\text{dab}][\text{bad}][\text{o\$\$}]$
$S_7(R)$	$[\text{bad}][\text{o\$\$}]$
$S_8(R)$	$[\text{o\$\$}]$

- $\mathcal{S}_0 =$ Suffixe mit Index $i \equiv_3 0$
- $\mathcal{S}_1 =$ Suffixe mit Index $i \equiv_3 1$
- $\mathcal{S}_2 =$ Suffixe mit Index $i \equiv_3 2$

$\mathcal{S}(T) =$ Suffixe von $T =$

S_0	y a b b a d a b b a d o
S_1	a b b a d a b b a d o
S_2	b b a d a b b a d o
S_3	b a d a b b a d o
S_4	a d a b b a d o
S_5	d a b b a d o
S_6	a b b a d o
S_7	b b a d o
S_8	b a d o
S_9	a d o
S_{10}	d o
S_{11}	o

Beobachtung. $\mathcal{S}(R)$ entspricht bijektiv $\mathcal{S}_1 \cup \mathcal{S}_2$



Schritt 1: sortiere $\mathcal{S}_1 \cup \mathcal{S}_2$

Kurze Notation: $T = t_0t_1 \dots t_{n-1}$ und $x \equiv_3 y$ ist kurz für $x \bmod 3 = y$.

Zerlege S_1 und S_2 in Dreiergruppen und verknüpfe sie:

$$R = [\text{abb}][\text{ada}][\text{bba}][\text{do\$}][\text{bba}][\text{dab}][\text{bad}][\text{o\$\$}]$$

$\mathcal{S}(R) =$

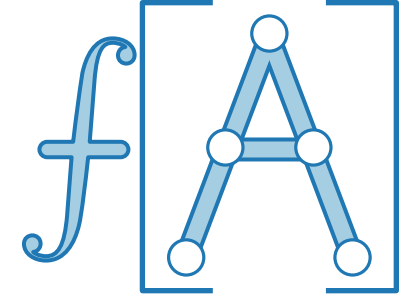
$S_1(R)$	$[\text{abb}][\text{ada}][\text{bba}][\text{do\$}][\text{bba}][\text{dab}][\text{bad}][\text{o\$\$}]$
$S_2(R)$	$[\text{ada}][\text{bba}][\text{do\$}][\text{bba}][\text{dab}][\text{bad}][\text{o\$\$}]$
$S_3(R)$	$[\text{bba}][\text{do\$}][\text{bba}][\text{dab}][\text{bad}][\text{o\$\$}]$
$S_4(R)$	$[\text{do\$}][\text{bba}][\text{dab}][\text{bad}][\text{o\$\$}]$
$S_5(R)$	$[\text{bba}][\text{dab}][\text{bad}][\text{o\$\$}]$
$S_6(R)$	$[\text{dab}][\text{bad}][\text{o\$\$}]$
$S_7(R)$	$[\text{bad}][\text{o\$\$}]$
$S_8(R)$	$[\text{o\$\$}]$

- $\mathcal{S}_0 =$ Suffixe mit Index $i \equiv_3 0$
- $\mathcal{S}_1 =$ Suffixe mit Index $i \equiv_3 1$
- $\mathcal{S}_2 =$ Suffixe mit Index $i \equiv_3 2$

$\mathcal{S}(T) =$ Suffixe von $T =$

S_0	y a b b a d a b b a d o
S_1	a b b a d a b b a d o
S_2	b b a d a b b a d o
S_3	b a d a b b a d o
S_4	a d a b b a d o
S_5	d a b b a d o
S_6	a b b a d o
S_7	b b a d o
S_8	b a d o
S_9	a d o
S_{10}	d o
S_{11}	o

Beobachtung. $\mathcal{S}(R)$ entspricht bijektiv $\mathcal{S}_1 \cup \mathcal{S}_2$



Schritt 1: sortiere $\mathcal{S}_1 \cup \mathcal{S}_2$

Kurze Notation: $T = t_0 t_1 \dots t_{n-1}$ und $x \equiv_3 y$ ist kurz für $x \bmod 3 = y$.

Zerlege S_1 und S_2 in Dreiergruppen und verknüpfe sie:

$R = [\text{abb}][\text{ada}][\text{bba}][\text{do\$}][\text{bba}][\text{dab}][\text{bad}][\text{o\$\$}]$

$\mathcal{S}(R) =$

$S_1(R)$	$[\text{abb}][\text{ada}][\text{bba}][\text{do\$}][\text{bba}][\text{dab}][\text{bad}][\text{o\$\$}]$
$S_2(R)$	$[\text{ada}][\text{bba}][\text{do\$}][\text{bba}][\text{dab}][\text{bad}][\text{o\$\$}]$
$S_3(R)$	$[\text{bba}][\text{do\$}][\text{bba}][\text{dab}][\text{bad}][\text{o\$\$}]$
$S_4(R)$	$[\text{do\$}][\text{bba}][\text{dab}][\text{bad}][\text{o\$\$}]$
$S_5(R)$	$[\text{bba}][\text{dab}][\text{bad}][\text{o\$\$}]$
$S_6(R)$	$[\text{dab}][\text{bad}][\text{o\$\$}]$
$S_7(R)$	$[\text{bad}][\text{o\$\$}]$
$S_8(R)$	$[\text{o\$\$}]$

$\mathcal{S}_0 =$ Suffixe mit Index $i \equiv_3 0$

$\mathcal{S}_1 =$ Suffixe mit Index $i \equiv_3 1$

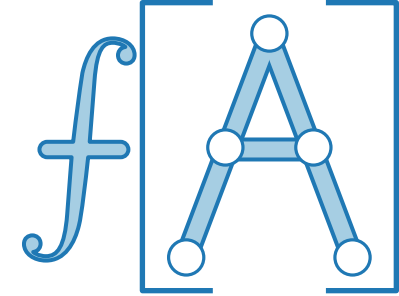
$\mathcal{S}_2 =$ Suffixe mit Index $i \equiv_3 2$

$\mathcal{S}(T) =$ Suffixe von $T =$

S_0	y a b b a d a b b a d o
S_1	a b b a d a b b a d o
S_2	b b a d a b b a d o
S_3	b a d a b b a d o
S_4	a d a b b a d o
S_5	d a b b a d o
S_6	a b b a d o
S_7	b b a d o
S_8	b a d o
S_9	a d o
S_{10}	d o
S_{11}	o

Beobachtung. $\mathcal{S}(R)$ entspricht bijektiv $\mathcal{S}_1 \cup \mathcal{S}_2$

$$S_i \leftrightarrow [t_i t_{i+1} t_{i+2}][t_{i+3} t_{i+4} t_{i+5}] \dots$$



Schritt 1: sortiere $\mathcal{S}_1 \cup \mathcal{S}_2$

Kurze Notation: $T = t_0 t_1 \dots t_{n-1}$ und $x \equiv_3 y$ ist kurz für $x \bmod 3 = y$.

Zerlege $\mathcal{S}_1$ und $\mathcal{S}_2$ in Dreiergruppen und verknüpfe sie:

$R = [\text{abb}][\text{ada}][\text{bba}][\text{do\$}][\text{bba}][\text{dab}][\text{bad}][\text{o\$\$}]$

$\mathcal{S}(R) =$

$S_1(R)$	$[\text{abb}][\text{ada}][\text{bba}][\text{do\$}][\text{bba}][\text{dab}][\text{bad}][\text{o\$\$}]$
$S_2(R)$	$[\text{ada}][\text{bba}][\text{do\$}][\text{bba}][\text{dab}][\text{bad}][\text{o\$\$}]$
$S_3(R)$	$[\text{bba}][\text{do\$}][\text{bba}][\text{dab}][\text{bad}][\text{o\$\$}]$
$S_4(R)$	$[\text{do\$}][\text{bba}][\text{dab}][\text{bad}][\text{o\$\$}]$
$S_5(R)$	$[\text{bba}][\text{dab}][\text{bad}][\text{o\$\$}]$
$S_6(R)$	$[\text{dab}][\text{bad}][\text{o\$\$}]$
$S_7(R)$	$[\text{bad}][\text{o\$\$}]$
$S_8(R)$	$[\text{o\$\$}]$

$\mathcal{S}_0 =$ Suffixe mit Index $i \equiv_3 0$

$\mathcal{S}_1 =$ Suffixe mit Index $i \equiv_3 1$

$\mathcal{S}_2 =$ Suffixe mit Index $i \equiv_3 2$

$\mathcal{S}(T) =$ Suffixe von $T =$

S_0	y a b b a d a b b a d o
S_1	a b b a d a b b a d o
S_2	b b a d a b b a d o
S_3	b a d a b b a d o
S_4	a d a b b a d o
S_5	d a b b a d o
S_6	a b b a d o
S_7	b b a d o
S_8	b a d o
S_9	a d o
S_{10}	d o
S_{11}	o

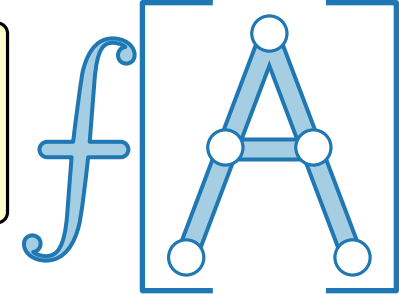
Beobachtung. $\mathcal{S}(R)$ entspricht bijektiv $\mathcal{S}_1 \cup \mathcal{S}_2$

$$S_i \leftrightarrow [t_i t_{i+1} t_{i+2}][t_{i+3} t_{i+4} t_{i+5}] \dots$$

und eine Sortierung von $\mathcal{S}(R)$ entspricht einer Sortierung von $\mathcal{S}_1 \cup \mathcal{S}_2$.

Schritt 1: sortiere $\mathcal{S}_1 \cup \mathcal{S}_2$

$S_i < S_j \Leftrightarrow S_i\$ < S_j\$ \Leftrightarrow S_i\$ \dots < S_j\$ \dots$
weil die Positionen der ersten \$ Symbole in den Strings $S_k(R)$ paarweise disjunkt sind.



Kurze Notation: $T = t_0t_1 \dots t_{n-1}$ und $x \equiv_3 y$ ist kurz für $x \bmod 3 = y$.

Zerlege S_1 und S_2 in Dreiergruppen und verknüpfe sie:

$R = [abb][ada][bba][do\$][bba][dab][bad][o\$\$]$

$\mathcal{S}(R) =$

$S_1(R)$	$[abb][ada][bba][do\$][bba][dab][bad][o\$\$]$
$S_2(R)$	$[ada][bba][do\$][bba][dab][bad][o\$\$]$
$S_3(R)$	$[bba][do\$][bba][dab][bad][o\$\$]$
$S_4(R)$	$[do\$][bba][dab][bad][o\$\$]$
$S_5(R)$	$[bba][dab][bad][o\$\$]$
$S_6(R)$	$[dab][bad][o\$\$]$
$S_7(R)$	$[bad][o\$\$]$
$S_8(R)$	$[o\$\$]$

$\mathcal{S}_0 =$ Suffixe mit Index $i \equiv_3 0$

$\mathcal{S}_1 =$ Suffixe mit Index $i \equiv_3 1$

$\mathcal{S}_2 =$ Suffixe mit Index $i \equiv_3 2$

$\mathcal{S}(T) =$ Suffixe von $T =$

S_0	y a b b a d a b b a d o
S_1	a b b a d a b b a d o
S_2	b b a d a b b a d o
S_3	b a d a b b a d o
S_4	a d a b b a d o
S_5	d a b b a d o
S_6	a b b a d o
S_7	b b a d o
S_8	b a d o
S_9	a d o
S_{10}	d o
S_{11}	o

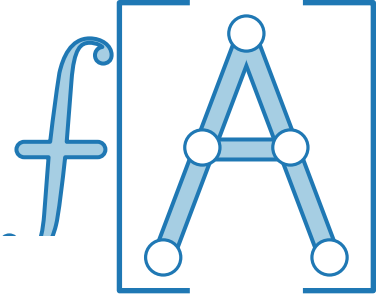
Beobachtung. $\mathcal{S}(R)$ entspricht bijektiv $\mathcal{S}_1 \cup \mathcal{S}_2$

$$S_i \leftrightarrow [t_i t_{i+1} t_{i+2}][t_{i+3} t_{i+4} t_{i+5}] \dots$$

und eine Sortierung von $\mathcal{S}(R)$ entspricht einer Sortierung von $\mathcal{S}_1 \cup \mathcal{S}_2$.

Sortiere $\mathcal{S}(R)$

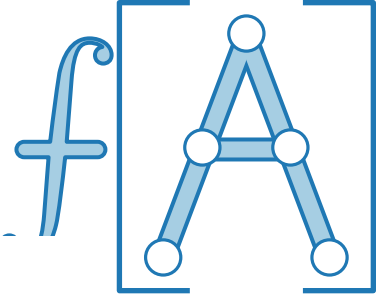
Sortiere die „Buchstaben“ (= Dreiergruppen) von R via RADIXSORT.



$R =$ [abb][ada][bba][do\$][bba][dab][bad][o\$]

$\mathcal{S}(R) =$	$S_1(R)$	[abb][ada][bba][do\$][bba][dab][bad][o\$]
	$S_2(R)$	[ada][bba][do\$][bba][dab][bad][o\$]
	$S_3(R)$	[bba][do\$][bba][dab][bad][o\$]
	$S_4(R)$	[do\$][bba][dab][bad][o\$]
	$S_5(R)$	[bba][dab][bad][o\$]
	$S_6(R)$	[dab][bad][o\$]
	$S_7(R)$	[bad][o\$]
	$S_8(R)$	[o\$]

Sortiere $\mathcal{S}(R)$



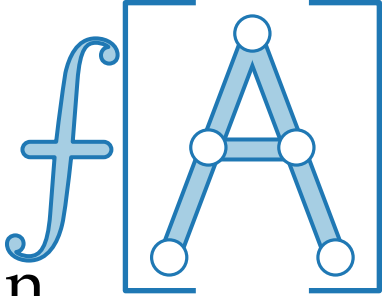
Sortiere die „Buchstaben“ (= Dreiergruppen) von R via RADIXSORT.

$R =$ [abb][ada][bba][do\$][bba][dab][bad][o\$]

Rang	3-Gr.
1	[abb]
2	[ada]
3	[bad]
4	[bba]
5	[dab]
6	[do\$]
7	[o\$]

$\mathcal{S}(R) =$	$S_1(R)$	[abb][ada][bba][do\$][bba][dab][bad][o\$]
	$S_2(R)$	[ada][bba][do\$][bba][dab][bad][o\$]
	$S_3(R)$	[bba][do\$][bba][dab][bad][o\$]
	$S_4(R)$	[do\$][bba][dab][bad][o\$]
	$S_5(R)$	[bba][dab][bad][o\$]
	$S_6(R)$	[dab][bad][o\$]
	$S_7(R)$	[bad][o\$]
	$S_8(R)$	[o\$]

Sortiere $\mathcal{S}(R)$



Sortiere die „Buchstaben“ (= Dreiergruppen) von R via RADIXSORT. Das geht in

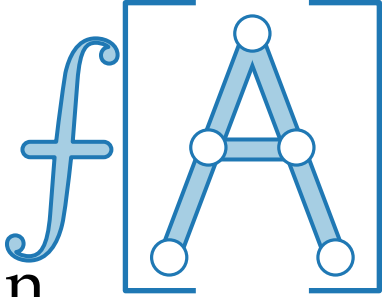
$$\mathcal{O}\left(3\left(\frac{2}{3}n + |\Sigma|\right)\right) \subseteq \mathcal{O}(n) \text{ Zeit.}$$

$R =$ [abb][ada][bba][do\$][bba][dab][bad][o\$]

Rang	3-Gr.
1	[abb]
2	[ada]
3	[bad]
4	[bba]
5	[dab]
6	[do\$]
7	[o\$]

$\mathcal{S}(R) =$	$S_1(R)$	[abb][ada][bba][do\$][bba][dab][bad][o\$]
	$S_2(R)$	[ada][bba][do\$][bba][dab][bad][o\$]
	$S_3(R)$	[bba][do\$][bba][dab][bad][o\$]
	$S_4(R)$	[do\$][bba][dab][bad][o\$]
	$S_5(R)$	[bba][dab][bad][o\$]
	$S_6(R)$	[dab][bad][o\$]
	$S_7(R)$	[bad][o\$]
	$S_8(R)$	[o\$]

Sortiere $\mathcal{S}(R)$



Sortiere die „Buchstaben“ (= Dreiergruppen) von R via RADIXSORT. Das geht in

$$\mathcal{O}\left(3\left(\frac{2}{3}n + |\Sigma|\right)\right) \subseteq \mathcal{O}(n) \text{ Zeit.}$$

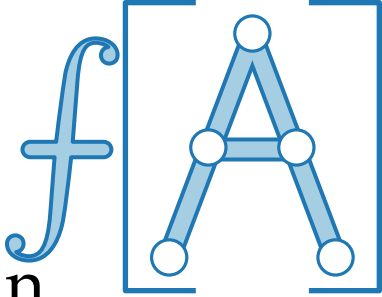
#Stellen

$R =$ [abb][ada][bba][do\$][bba][dab][bad][o\$]

Rang	3-Gr.
1	[abb]
2	[ada]
3	[bad]
4	[bba]
5	[dab]
6	[do\$]
7	[o\$]

$\mathcal{S}(R) =$	$S_1(R)$	[abb][ada][bba][do\$][bba][dab][bad][o\$]
	$S_2(R)$	[ada][bba][do\$][bba][dab][bad][o\$]
	$S_3(R)$	[bba][do\$][bba][dab][bad][o\$]
	$S_4(R)$	[do\$][bba][dab][bad][o\$]
	$S_5(R)$	[bba][dab][bad][o\$]
	$S_6(R)$	[dab][bad][o\$]
	$S_7(R)$	[bad][o\$]
	$S_8(R)$	[o\$]

Sortiere $\mathcal{S}(R)$



Sortiere die „Buchstaben“ (= Dreiergruppen) von R via RADIXSORT. Das geht in

$\mathcal{O}\left(3\left(\frac{2}{3}n + |\Sigma|\right)\right) \subseteq \mathcal{O}(n)$ Zeit.

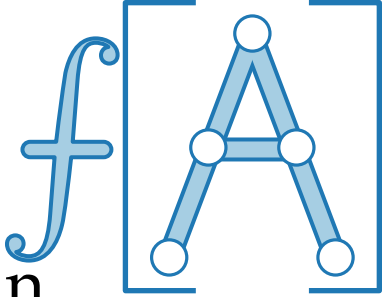
#Stellen #Objekte

$R =$ [abb][ada][bba][do\$][bba][dab][bad][o\$]

Rang	3-Gr.
1	[abb]
2	[ada]
3	[bad]
4	[bba]
5	[dab]
6	[do\$]
7	[o\$]

$\mathcal{S}(R) =$	$S_1(R)$	[abb][ada][bba][do\$][bba][dab][bad][o\$]
	$S_2(R)$	[ada][bba][do\$][bba][dab][bad][o\$]
	$S_3(R)$	[bba][do\$][bba][dab][bad][o\$]
	$S_4(R)$	[do\$][bba][dab][bad][o\$]
	$S_5(R)$	[bba][dab][bad][o\$]
	$S_6(R)$	[dab][bad][o\$]
	$S_7(R)$	[bad][o\$]
	$S_8(R)$	[o\$]

Sortiere $\mathcal{S}(R)$



Sortiere die „Buchstaben“ (= Dreiergruppen) von R via RADIXSORT. Das geht in

$\mathcal{O}\left(3\left(\frac{2}{3}n + |\Sigma|\right)\right) \subseteq \mathcal{O}(n)$ Zeit.

#Stellen #Objekte Größe des Alphabets

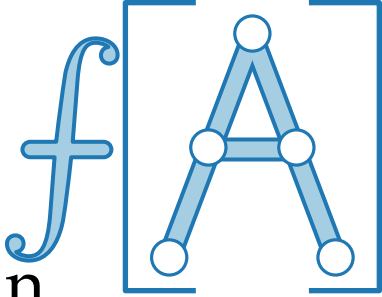
$R =$ [abb][ada][bba][do\$][bba][dab][bad][o\$]

Rang	3-Gr.
1	[abb]
2	[ada]
3	[bad]
4	[bba]
5	[dab]
6	[do\$]
7	[o\$]

$\mathcal{S}(R) =$

$S_1(R)$	[abb][ada][bba][do\$][bba][dab][bad][o\$]
$S_2(R)$	[ada][bba][do\$][bba][dab][bad][o\$]
$S_3(R)$	[bba][do\$][bba][dab][bad][o\$]
$S_4(R)$	[do\$][bba][dab][bad][o\$]
$S_5(R)$	[bba][dab][bad][o\$]
$S_6(R)$	[dab][bad][o\$]
$S_7(R)$	[bad][o\$]
$S_8(R)$	[o\$]

Sortiere $\mathcal{S}(R)$



Sortiere die „Buchstaben“ (= Dreiergruppen) von R via RADIXSORT. Das geht in

$\mathcal{O}\left(3\left(\frac{2}{3}n + |\Sigma|\right)\right) \subseteq \mathcal{O}(n)$ Zeit.

#Stellen #Objekte Größe des Alphabets

Ersetze jede Dreiergruppe von R mit seinem Rang

➔ String R' mit Alphabet der Größe $\leq \frac{2}{3}n \leq n$.

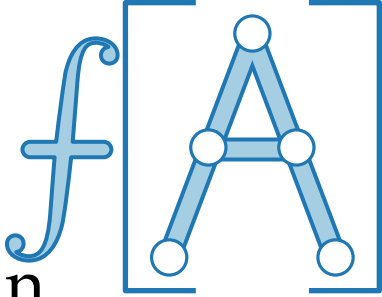
$R =$ [abb][ada][bba][do\$][bba][dab][bad][o\$]

Rang	3-Gr.
1	[abb]
2	[ada]
3	[bad]
4	[bba]
5	[dab]
6	[do\$]
7	[o\$]

$\mathcal{S}(R) =$

$S_1(R)$	[abb][ada][bba][do\$][bba][dab][bad][o\$]
$S_2(R)$	[ada][bba][do\$][bba][dab][bad][o\$]
$S_3(R)$	[bba][do\$][bba][dab][bad][o\$]
$S_4(R)$	[do\$][bba][dab][bad][o\$]
$S_5(R)$	[bba][dab][bad][o\$]
$S_6(R)$	[dab][bad][o\$]
$S_7(R)$	[bad][o\$]
$S_8(R)$	[o\$]

Sortiere $\mathcal{S}(R)$



Sortiere die „Buchstaben“ (= Dreiergruppen) von R via RADIXSORT. Das geht in

$\mathcal{O}\left(3\left(\frac{2}{3}n + |\Sigma|\right)\right) \subseteq \mathcal{O}(n)$ Zeit.

#Stellen #Objekte Größe des Alphabets

Ersetze jede Dreiergruppe von R mit seinem Rang

➔ String R' mit Alphabet der Größe $\leq \frac{2}{3}n \leq n$.

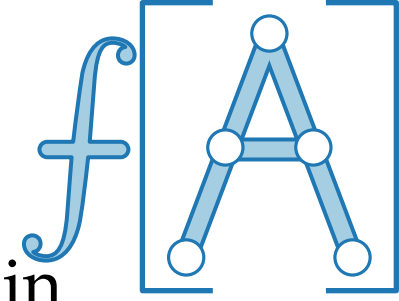
$R =$ [abb][ada][bba][do\$][bba][dab][bad][o\$]

$R' =$ 1 2 4 6 4 5 3 7

Rang	3-Gr.
1	[abb]
2	[ada]
3	[bad]
4	[bba]
5	[dab]
6	[do\$]
7	[o\$]

$\mathcal{S}(R) =$

$S_1(R)$	[abb][ada][bba][do\$][bba][dab][bad][o\$]
$S_2(R)$	[ada][bba][do\$][bba][dab][bad][o\$]
$S_3(R)$	[bba][do\$][bba][dab][bad][o\$]
$S_4(R)$	[do\$][bba][dab][bad][o\$]
$S_5(R)$	[bba][dab][bad][o\$]
$S_6(R)$	[dab][bad][o\$]
$S_7(R)$	[bad][o\$]
$S_8(R)$	[o\$]



Sortiere $\mathcal{S}(R)$

Sortiere die „Buchstaben“ (= Dreiergruppen) von R via RADIXSORT. Das geht in

$\mathcal{O}\left(3\left(\frac{2}{3}n + |\Sigma|\right)\right) \subseteq \mathcal{O}(n)$ Zeit.

#Stellen #Objekte Größe des Alphabets

Ersetze jede Dreiergruppe von R mit seinem Rang

➔ String R' mit Alphabet der Größe $\leq \frac{2}{3}n \leq n$.

Eine Sortierung von $\mathcal{S}(R')$ entspricht einer Sortierung von $\mathcal{S}(R)$ und kann rekursiv erhalten werden.

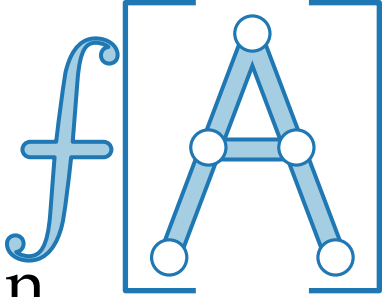
$R =$ [abb][ada][bba][do\$][bba][dab][bad][o\$]

$R' = 1\ 2\ 4\ 6\ 4\ 5\ 3\ 7$

Rang	3-Gr.
1	[abb]
2	[ada]
3	[bad]
4	[bba]
5	[dab]
6	[do\$]
7	[o\$]

$\mathcal{S}(R) =$	$S_1(R)$	[abb][ada][bba][do\$][bba][dab][bad][o\$]
	$S_2(R)$	[ada][bba][do\$][bba][dab][bad][o\$]
	$S_3(R)$	[bba][do\$][bba][dab][bad][o\$]
	$S_4(R)$	[do\$][bba][dab][bad][o\$]
	$S_5(R)$	[bba][dab][bad][o\$]
	$S_6(R)$	[dab][bad][o\$]
	$S_7(R)$	[bad][o\$]
	$S_8(R)$	[o\$]

Sortiere $\mathcal{S}(R)$



Sortiere die „Buchstaben“ (= Dreiergruppen) von R via RADIXSORT. Das geht in

$\mathcal{O}\left(3\left(\frac{2}{3}n + |\Sigma|\right)\right) \subseteq \mathcal{O}(n)$ Zeit.

#Stellen #Objekte Größe des Alphabets

Ersetze jede Dreiergruppe von R mit seinem Rang

➔ String R' mit Alphabet der Größe $\leq \frac{2}{3}n \leq n$.

Eine Sortierung von $\mathcal{S}(R')$ entspricht einer Sortierung von $\mathcal{S}(R)$ und kann rekursiv erhalten werden.

$R = [abb][ada][bba][do\$][bba][dab][bad][o\$\$]$ $R' = 1\ 2\ 4\ 6\ 4\ 5\ 3\ 7$

Rang	3-Gr.
1	[abb]
2	[ada]
3	[bad]
4	[bba]
5	[dab]
6	[do\$]
7	[o\$ \$]

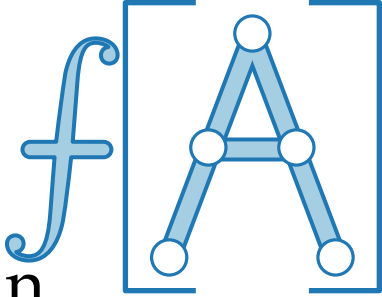
$\mathcal{S}(R) =$

$S_1(R)$	[abb][ada][bba][do\$][bba][dab][bad][o\$ \$]
$S_2(R)$	[ada][bba][do\$][bba][dab][bad][o\$ \$]
$S_3(R)$	[bba][do\$][bba][dab][bad][o\$ \$]
$S_4(R)$	[do\$][bba][dab][bad][o\$ \$]
$S_5(R)$	[bba][dab][bad][o\$ \$]
$S_6(R)$	[dab][bad][o\$ \$]
$S_7(R)$	[bad][o\$ \$]
$S_8(R)$	[o\$ \$]

$\mathcal{S}(R') =$

$S_1(R')$	1 2 4 6 4 5 3 7
$S_2(R')$	2 4 6 4 5 3 7
$S_3(R')$	4 6 4 5 3 7
$S_4(R')$	6 4 5 3 7
$S_5(R')$	4 5 3 7
$S_6(R')$	5 3 7
$S_7(R')$	3 7
$S_8(R')$	7

Sortiere $\mathcal{S}(R)$



Sortiere die „Buchstaben“ (= Dreiergruppen) von R via RADIXSORT. Das geht in

$$\mathcal{O}\left(3\left(\frac{2}{3}n + |\Sigma|\right)\right) \subseteq \mathcal{O}(n) \text{ Zeit.}$$

#Stellen
#Objekte
Größe des Alphabets

KONSTRUIERE SUFFIXARRAY(R')

Ersetze jede Dreiergruppe von R mit seinem Rang

➔ String R' mit Alphabet der Größe $\leq \frac{2}{3}n \leq n$.

Eine Sortierung von $\mathcal{S}(R')$ entspricht einer Sortierung von $\mathcal{S}(R)$ und kann rekursiv erhalten werden.

$R =$
[abb][ada][bba][do\$][bba][dab][bad][o\$]

$R' =$ 1 2 4 6 4 5 3 7

Rang	3-Gr.
1	[abb]
2	[ada]
3	[bad]
4	[bba]
5	[dab]
6	[do\$]
7	[o\$]

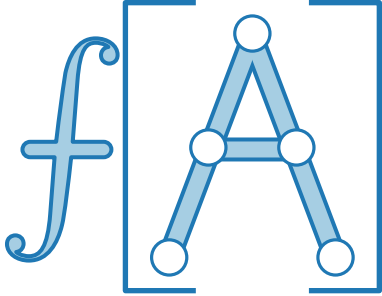
$\mathcal{S}(R) =$

$S_1(R)$	[abb][ada][bba][do\$][bba][dab][bad][o\$]
$S_2(R)$	[ada][bba][do\$][bba][dab][bad][o\$]
$S_3(R)$	[bba][do\$][bba][dab][bad][o\$]
$S_4(R)$	[do\$][bba][dab][bad][o\$]
$S_5(R)$	[bba][dab][bad][o\$]
$S_6(R)$	[dab][bad][o\$]
$S_7(R)$	[bad][o\$]
$S_8(R)$	[o\$]

$\mathcal{S}(R') =$

$S_1(R')$	1 2 4 6 4 5 3 7
$S_2(R')$	2 4 6 4 5 3 7
$S_3(R')$	4 6 4 5 3 7
$S_4(R')$	6 4 5 3 7
$S_5(R')$	4 5 3 7
$S_6(R')$	5 3 7
$S_7(R')$	3 7
$S_8(R')$	7

Zusammenfassung von Schritt 1

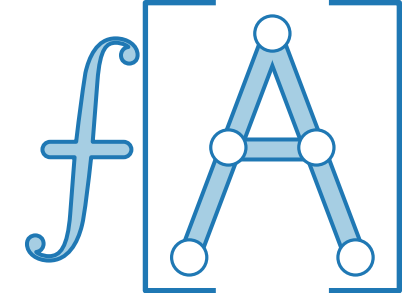


Vollständiges Beispiel.

$\mathcal{S}(T)=$

S_0	y a b b a d a b b a d o
S_1	a b b a d a b b a d o
S_2	b b a d a b b a d o
S_3	b a d a b b a d o
S_4	a d a b b a d o
S_5	d a b b a d o
S_6	a b b a d o
S_7	b b a d o
S_8	b a d o
S_9	a d o
S_{10}	d o
S_{11}	o

Zusammenfassung von Schritt 1



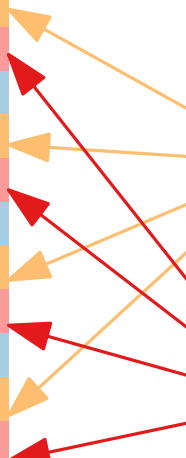
Vollständiges Beispiel.

$\mathcal{S}(T)=$

S_0	y a b b a d a b b a d o
S_1	a b b a d a b b a d o
S_2	b b a d a b b a d o
S_3	b a d a b b a d o
S_4	a d a b b a d o
S_5	d a b b a d o
S_6	a b b a d o
S_7	b b a d o
S_8	b a d o
S_9	a d o
S_{10}	d o
S_{11}	o

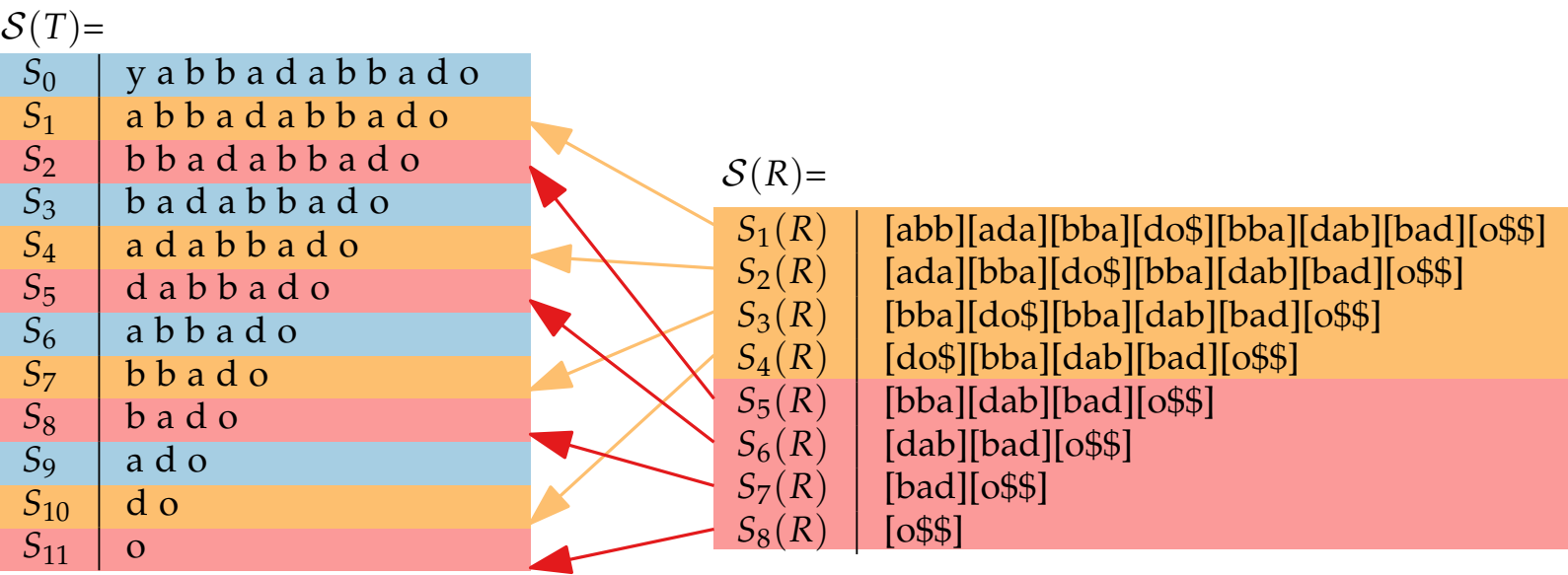
$\mathcal{S}(R)=$

$S_1(R)$	[abb][ada][bba][do\$][bba][dab][bad][o\$]\$]
$S_2(R)$	[ada][bba][do\$][bba][dab][bad][o\$]\$]
$S_3(R)$	[bba][do\$][bba][dab][bad][o\$]\$]
$S_4(R)$	[do\$][bba][dab][bad][o\$]\$]
$S_5(R)$	[bba][dab][bad][o\$]\$]
$S_6(R)$	[dab][bad][o\$]\$]
$S_7(R)$	[bad][o\$]\$]
$S_8(R)$	[o\$]\$]



Zusammenfassung von Schritt 1

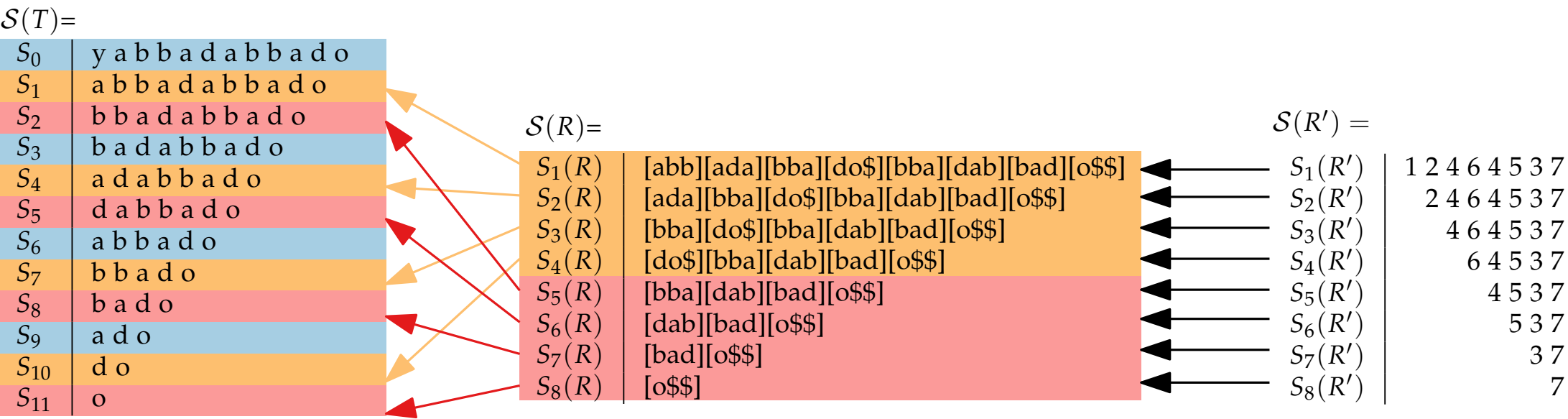
Vollständiges Beispiel.



Rang	3-Gr.
1	[abb]
2	[ada]
3	[bad]
4	[bba]
5	[dab]
6	[do\$]
7	[o\$]\$]

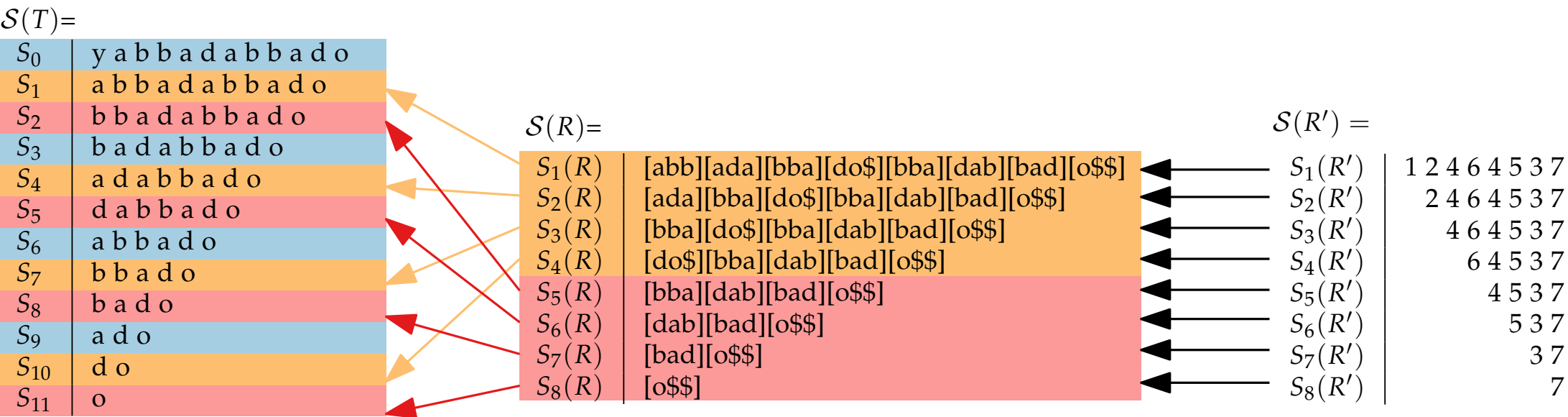
Zusammenfassung von Schritt 1

Vollständiges Beispiel.



Zusammenfassung von Schritt 1

Vollständiges Beispiel.

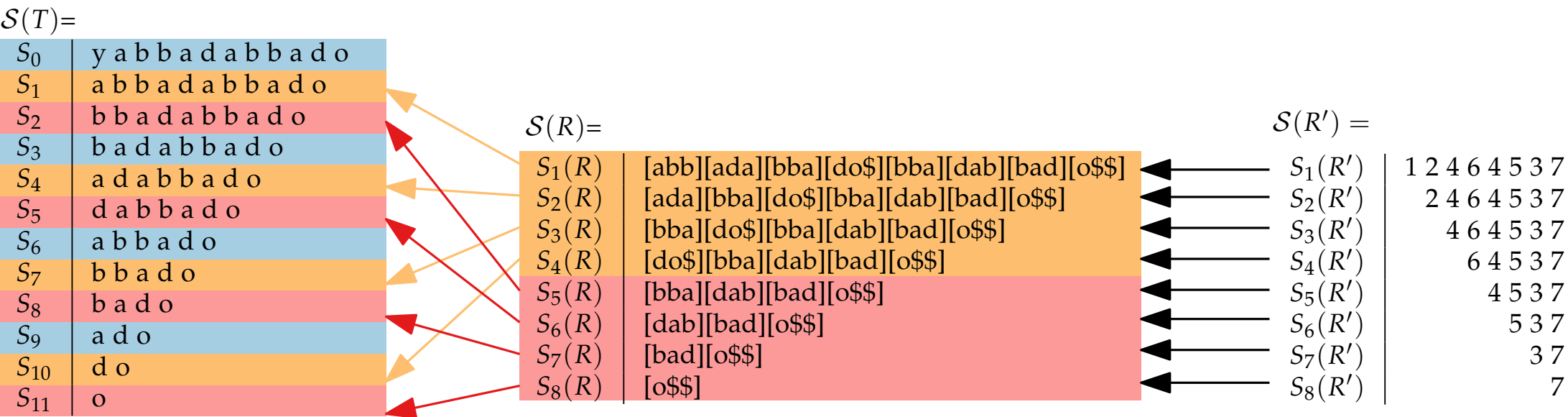


Rang	3-Gr.
1	[abb]
2	[ada]
3	[bad]
4	[bba]
5	[dab]
6	[do\$]
7	[o\$]\$

A_{12}			
1	S_1	a b b a d a b b a d o	$S_1(R')$ 1 2 4 6 4 5 3 7
2	S_4	a d a b b a d o	$S_2(R')$ 2 4 6 4 5 3 7
3	S_8	b a d o	$S_7(R')$ 3 7
4	S_2	b b a d a b b a d o	$S_5(R')$ 4 5 3 7
5	S_7	b b a d o	$S_3(R')$ 4 6 4 5 3 7
6	S_5	d a b b a d o	$S_6(R')$ 5 3 7
7	S_{10}	d o	$S_4(R')$ 6 4 5 3 7
8	S_{11}	o	$S_8(R')$ 7

Zusammenfassung von Schritt 1

Vollständiges Beispiel.



Rang	3-Gr.
1	[abb]
2	[ada]
3	[bad]
4	[bba]
5	[dab]
6	[do\$]
7	[o\$]\$

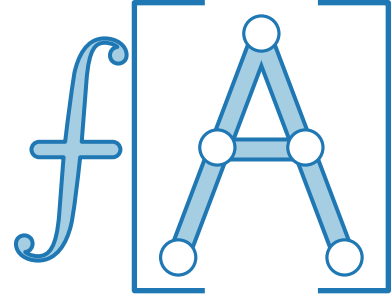
A_{12}			
1	S_1	a b b a d a b b a d o	$S_1(R')$ 1 2 4 6 4 5 3 7
2	S_4	a d a b b a d o	$S_2(R')$ 2 4 6 4 5 3 7
3	S_8	b a d o	$S_7(R')$ 3 7
4	S_2	b b a d a b b a d o	$S_5(R')$ 4 5 3 7
5	S_7	b b a d o	$S_3(R')$ 4 6 4 5 3 7
6	S_5	d a b b a d o	$S_6(R')$ 5 3 7
7	S_{10}	d o	$S_4(R')$ 6 4 5 3 7
8	S_{11}	o	$S_8(R')$ 7

Laufzeit.

$$T_1(n) = \mathcal{O}(n) + T(\frac{2}{3}n)$$

wobei $T(n)$ der Zeit entspricht,
KONSTRUIERESUFFIXARRAY auf
einem String der Länge n
auszuführen.

Konstruktion von Suffixarrays – Überblick



Kurze Notation: $T = t_0 t_1 \dots t_{n-1}$ und $x \equiv_3 y$ ist kurz für $x \bmod 3 = y$.

	0	1	2	3	4	5	6	7	8	9	10	11
$T =$	y	a	b	b	a	d	a	b	b	a	d	o

KONSTRUIERE SUFFIXARRAY(T)

if $n = \mathcal{O}(1)$ **then**

 | erstelle A in $\mathcal{O}(1)$ Zeit.

else

 | sortiere $\mathcal{S}_1 \cup \mathcal{S}_2$ in ein Array A_{12}

 | benutze A_{12} , um $\mathcal{S}_0$ in ein Array A_0 zu sortieren

 | merge A_{12} mit A_0

$\mathcal{S}_0 =$ Suffixe mit Index $i \equiv_3 0$

$\mathcal{S}_1 =$ Suffixe mit Index $i \equiv_3 1$

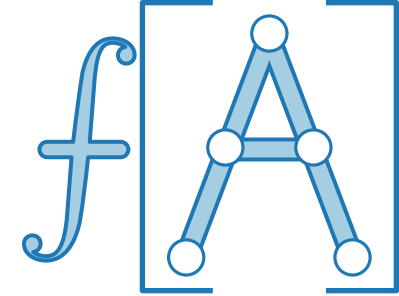
$\mathcal{S}_2 =$ Suffixe mit Index $i \equiv_3 2$

$\mathcal{S}(T) =$ Suffixe von $T =$

$\mathcal{S}_0$	y a b b a d a b b a d o
$\mathcal{S}_1$	a b b a d a b b a d o
$\mathcal{S}_2$	b b a d a b b a d o
$\mathcal{S}_3$	b a d a b b a d o
$\mathcal{S}_4$	a d a b b a d o
$\mathcal{S}_5$	d a b b a d o
$\mathcal{S}_6$	a b b a d o
$\mathcal{S}_7$	b b a d o
$\mathcal{S}_8$	b a d o
$\mathcal{S}_9$	a d o
$\mathcal{S}_{10}$	d o
$\mathcal{S}_{11}$	o

Der Einfachheit halber nehmen wir $n \equiv 0(3)$ an.

Konstruktion von Suffixarrays – Überblick



Kurze Notation: $T = t_0 t_1 \dots t_{n-1}$ und $x \equiv_3 y$ ist kurz für $x \bmod 3 = y$.

	0	1	2	3	4	5	6	7	8	9	10	11
$T =$	y	a	b	b	a	d	a	b	b	a	d	o

KONSTRUIERE SUFFIXARRAY(T)

if $n = \mathcal{O}(1)$ **then**

 | erstelle A in $\mathcal{O}(1)$ Zeit.

else

 | sortiere $\mathcal{S}_1 \cup \mathcal{S}_2$ in ein Array A_{12}

 | benutze A_{12} , um $\mathcal{S}_0$ in ein Array A_0 zu sortieren

 | merge A_{12} mit A_0

$\mathcal{S}_0 =$ Suffixe mit Index $i \equiv_3 0$

$\mathcal{S}_1 =$ Suffixe mit Index $i \equiv_3 1$

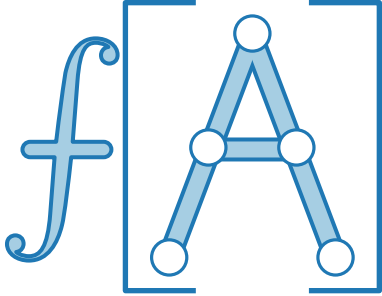
$\mathcal{S}_2 =$ Suffixe mit Index $i \equiv_3 2$

$\mathcal{S}(T) =$ Suffixe von $T =$

$\mathcal{S}_0$	y a b b a d a b b a d o
$\mathcal{S}_1$	a b b a d a b b a d o
$\mathcal{S}_2$	b b a d a b b a d o
$\mathcal{S}_3$	b a d a b b a d o
$\mathcal{S}_4$	a d a b b a d o
$\mathcal{S}_5$	d a b b a d o
$\mathcal{S}_6$	a b b a d o
$\mathcal{S}_7$	b b a d o
$\mathcal{S}_8$	b a d o
$\mathcal{S}_9$	a d o
$\mathcal{S}_{10}$	d o
$\mathcal{S}_{11}$	o

Der Einfachheit halber nehmen wir $n \equiv 0(3)$ an.

Schritt 2: sortiere $\mathcal{S}_0$



Kurze Notation: $T = t_0t_1 \dots t_{n-1}$ und $x \equiv_3 y$ ist kurz für $x \bmod 3 = y$.

	0	1	2	3	4	5	6	7	8	9	10	11
$T =$	y	a	b	b	a	d	a	b	b	a	d	o

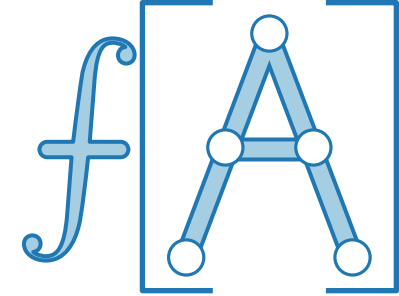
$\mathcal{S}_0 =$ Suffixe mit Index $i \equiv_3 0$

$\mathcal{S}_1 =$ Suffixe mit Index $i \equiv_3 1$

$\mathcal{S}_2 =$ Suffixe mit Index $i \equiv_3 2$

$\mathcal{S}(T) =$ Suffixe von $T =$

S_0	y a b b a d a b b a d o
S_1	a b b a d a b b a d o
S_2	b b a d a b b a d o
S_3	b a d a b b a d o
S_4	a d a b b a d o
S_5	d a b b a d o
S_6	a b b a d o
S_7	b b a d o
S_8	b a d o
S_9	a d o
S_{10}	d o
S_{11}	o



Schritt 2: sortiere $\mathcal{S}_0$

Kurze Notation: $T = t_0t_1 \dots t_{n-1}$ und $x \equiv_3 y$ ist kurz für $x \bmod 3 = y$.

	0	1	2	3	4	5	6	7	8	9	10	11
$T =$	y	a	b	b	a	d	a	b	b	a	d	o

Jedes $S_i \in \mathcal{S}_0$ kann als (t_i, S_{i+1}) geschrieben werden, wobei $S_{i+1} \in \mathcal{S}_1$.

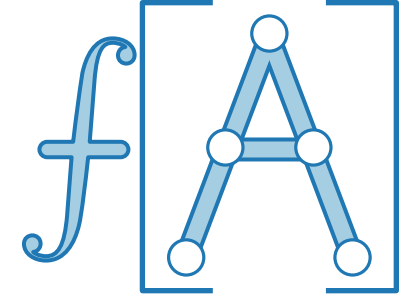
$\mathcal{S}_0$ = Suffixe mit Index $i \equiv_3 0$

$\mathcal{S}_1$ = Suffixe mit Index $i \equiv_3 1$

$\mathcal{S}_2$ = Suffixe mit Index $i \equiv_3 2$

$\mathcal{S}(T)$ = Suffixe von $T =$

S_0	y a b b a d a b b a d o
S_1	a b b a d a b b a d o
S_2	b b a d a b b a d o
S_3	b a d a b b a d o
S_4	a d a b b a d o
S_5	d a b b a d o
S_6	a b b a d o
S_7	b b a d o
S_8	b a d o
S_9	a d o
S_{10}	d o
S_{11}	o



Schritt 2: sortiere $\mathcal{S}_0$

Kurze Notation: $T = t_0t_1 \dots t_{n-1}$ und $x \equiv_3 y$ ist kurz für $x \bmod 3 = y$.

	0	1	2	3	4	5	6	7	8	9	10	11
$T =$	y	a	b	b	a	d	a	b	b	a	d	o

Jedes $S_i \in \mathcal{S}_0$ kann als (t_i, S_{i+1}) geschrieben werden, wobei $S_{i+1} \in \mathcal{S}_1$.

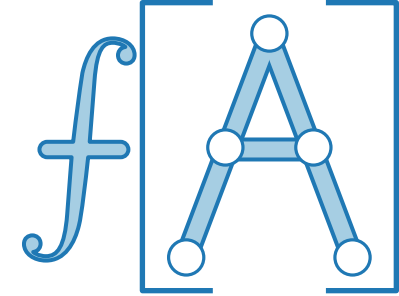
Beobachtung. Seien $S_i, S_j \in \mathcal{S}_0$. Dann ist $S_i < S_j$ genau dann wenn

- $t_i < t_j$; oder
- $t_i = t_j$ und $S_{i+1} < S_{j+1}$.

- $\mathcal{S}_0$ = Suffixe mit Index $i \equiv_3 0$
- $\mathcal{S}_1$ = Suffixe mit Index $i \equiv_3 1$
- $\mathcal{S}_2$ = Suffixe mit Index $i \equiv_3 2$

$\mathcal{S}(T)$ = Suffixe von $T =$

S_0	y a b b a d a b b a d o
S_1	a b b a d a b b a d o
S_2	b b a d a b b a d o
S_3	b a d a b b a d o
S_4	a d a b b a d o
S_5	d a b b a d o
S_6	a b b a d o
S_7	b b a d o
S_8	b a d o
S_9	a d o
S_{10}	d o
S_{11}	o



Schritt 2: sortiere $\mathcal{S}_0$

Kurze Notation: $T = t_0t_1 \dots t_{n-1}$ und $x \equiv_3 y$ ist kurz für $x \bmod 3 = y$.

	0	1	2	3	4	5	6	7	8	9	10	11
$T =$	y	a	b	b	a	d	a	b	b	a	d	o

Jedes $S_i \in \mathcal{S}_0$ kann als (t_i, S_{i+1}) geschrieben werden, wobei $S_{i+1} \in \mathcal{S}_1$.

Beobachtung. Seien $S_i, S_j \in \mathcal{S}_0$. Dann ist $S_i < S_j$ genau dann wenn

- $t_i < t_j$; oder
- $t_i = t_j$ und $S_{i+1} < S_{j+1}$.

$\Rightarrow \mathcal{S}_0$ kann sortiert werden, indem man alle Tupel (t_i, S_{i+1}) mit $i \equiv_3 0$ sortiert.

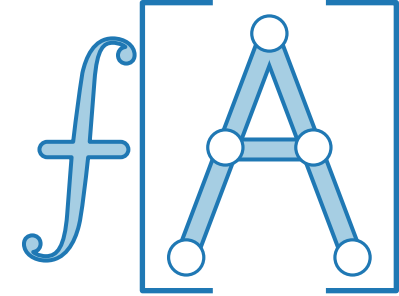
$\mathcal{S}_0$ = Suffixe mit Index $i \equiv_3 0$

$\mathcal{S}_1$ = Suffixe mit Index $i \equiv_3 1$

$\mathcal{S}_2$ = Suffixe mit Index $i \equiv_3 2$

$\mathcal{S}(T)$ = Suffixe von $T =$

S_0	y a b b a d a b b a d o
S_1	a b b a d a b b a d o
S_2	b b a d a b b a d o
S_3	b a d a b b a d o
S_4	a d a b b a d o
S_5	d a b b a d o
S_6	a b b a d o
S_7	b b a d o
S_8	b a d o
S_9	a d o
S_{10}	d o
S_{11}	o



Schritt 2: sortiere $\mathcal{S}_0$

Kurze Notation: $T = t_0 t_1 \dots t_{n-1}$ und $x \equiv_3 y$ ist kurz für $x \bmod 3 = y$.

	0	1	2	3	4	5	6	7	8	9	10	11
$T =$	y	a	b	b	a	d	a	b	b	a	d	o

Jedes $S_i \in \mathcal{S}_0$ kann als (t_i, S_{i+1}) geschrieben werden, wobei $S_{i+1} \in \mathcal{S}_1$.

Beobachtung. Seien $S_i, S_j \in \mathcal{S}_0$. Dann ist $S_i < S_j$ genau dann wenn

- $t_i < t_j$; oder
- $t_i = t_j$ und $S_{i+1} < S_{j+1}$.

$\Rightarrow \mathcal{S}_0$ kann sortiert werden, indem man alle Tupel (t_i, S_{i+1}) mit $i \equiv_3 0$ sortiert. Dies kann durch RADIXSORT in $\mathcal{O}(n)$ Zeit getan werden, weil die Reihenfolge der Einträge in $\mathcal{S}_1$ implizit in A_{12} ist.

$\mathcal{S}_0 =$ Suffixe mit Index $i \equiv_3 0$

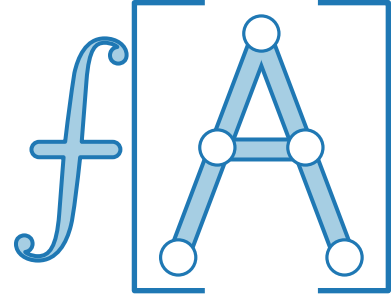
$\mathcal{S}_1 =$ Suffixe mit Index $i \equiv_3 1$

$\mathcal{S}_2 =$ Suffixe mit Index $i \equiv_3 2$

$\mathcal{S}(T) =$ Suffixe von $T =$

S_0	y a b b a d a b b a d o
S_1	a b b a d a b b a d o
S_2	b b a d a b b a d o
S_3	b a d a b b a d o
S_4	a d a b b a d o
S_5	d a b b a d o
S_6	a b b a d o
S_7	b b a d o
S_8	b a d o
S_9	a d o
S_{10}	d o
S_{11}	o

Konstruktion von Suffixarrays – Überblick



Kurze Notation: $T = t_0 t_1 \dots t_{n-1}$ und $x \equiv_3 y$ ist kurz für $x \bmod 3 = y$.

	0	1	2	3	4	5	6	7	8	9	10	11
$T =$	y	a	b	b	a	d	a	b	b	a	d	o

KONSTRUIERE SUFFIXARRAY(T)

if $n = \mathcal{O}(1)$ **then**

 | erstelle A in $\mathcal{O}(1)$ Zeit.

else

 | sortiere $\mathcal{S}_1 \cup \mathcal{S}_2$ in ein Array A_{12}

 | benutze A_{12} , um $\mathcal{S}_0$ in ein Array A_0 zu sortieren

 | merge A_{12} mit A_0

$\mathcal{S}_0 =$ Suffixe mit Index $i \equiv_3 0$

$\mathcal{S}_1 =$ Suffixe mit Index $i \equiv_3 1$

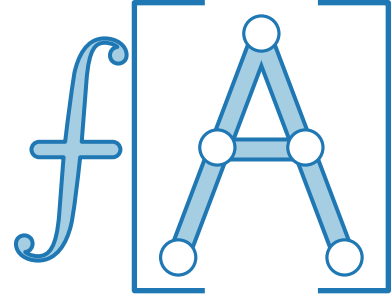
$\mathcal{S}_2 =$ Suffixe mit Index $i \equiv_3 2$

$\mathcal{S}(T) =$ Suffixe von $T =$

$\mathcal{S}_0$	y a b b a d a b b a d o
$\mathcal{S}_1$	a b b a d a b b a d o
$\mathcal{S}_2$	b b a d a b b a d o
$\mathcal{S}_3$	b a d a b b a d o
$\mathcal{S}_4$	a d a b b a d o
$\mathcal{S}_5$	d a b b a d o
$\mathcal{S}_6$	a b b a d o
$\mathcal{S}_7$	b b a d o
$\mathcal{S}_8$	b a d o
$\mathcal{S}_9$	a d o
$\mathcal{S}_{10}$	d o
$\mathcal{S}_{11}$	o

Der Einfachheit halber nehmen wir $n \equiv_3 0$ an.

Konstruktion von Suffixarrays – Überblick



Kurze Notation: $T = t_0 t_1 \dots t_{n-1}$ und $x \equiv_3 y$ ist kurz für $x \bmod 3 = y$.

	0	1	2	3	4	5	6	7	8	9	10	11
$T =$	y	a	b	b	a	d	a	b	b	a	d	o

KONSTRUIERE SUFFIXARRAY(T)

if $n = \mathcal{O}(1)$ **then**

 | erstelle A in $\mathcal{O}(1)$ Zeit.

else

 | sortiere $\mathcal{S}_1 \cup \mathcal{S}_2$ in ein Array A_{12}

 | benutze A_{12} , um $\mathcal{S}_0$ in ein Array A_0 zu sortieren

 | merge A_{12} mit A_0

$\mathcal{S}_0 =$ Suffixe mit Index $i \equiv_3 0$

$\mathcal{S}_1 =$ Suffixe mit Index $i \equiv_3 1$

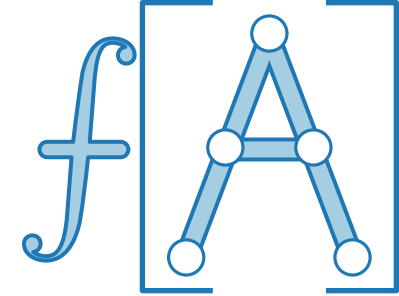
$\mathcal{S}_2 =$ Suffixe mit Index $i \equiv_3 2$

$\mathcal{S}(T) =$ Suffixe von $T =$

$\mathcal{S}_0$	y a b b a d a b b a d o
$\mathcal{S}_1$	a b b a d a b b a d o
$\mathcal{S}_2$	b b a d a b b a d o
$\mathcal{S}_3$	b a d a b b a d o
$\mathcal{S}_4$	a d a b b a d o
$\mathcal{S}_5$	d a b b a d o
$\mathcal{S}_6$	a b b a d o
$\mathcal{S}_7$	b b a d o
$\mathcal{S}_8$	b a d o
$\mathcal{S}_9$	a d o
$\mathcal{S}_{10}$	d o
$\mathcal{S}_{11}$	o

Der Einfachheit halber nehmen wir $n \equiv_3 0$ an.

Schritt 3: merge A_{12} und A_0



Kurze Notation: $T = t_0t_1 \dots t_{n-1}$ und $x \equiv_3 y$ ist kurz für $x \bmod 3 = y$.

	0	1	2	3	4	5	6	7	8	9	10	11
$T =$	y	a	b	b	a	d	a	b	b	a	d	o

$\mathcal{S}_0 =$ Suffixe mit Index $i \equiv_3 0$

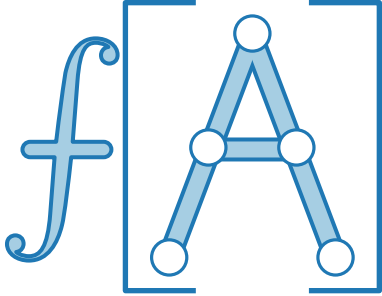
$\mathcal{S}_1 =$ Suffixe mit Index $i \equiv_3 1$

$\mathcal{S}_2 =$ Suffixe mit Index $i \equiv_3 2$

$\mathcal{S}(T) =$ Suffixe von $T =$

S_0	y a b b a d a b b a d o
S_1	a b b a d a b b a d o
S_2	b b a d a b b a d o
S_3	b a d a b b a d o
S_4	a d a b b a d o
S_5	d a b b a d o
S_6	a b b a d o
S_7	b b a d o
S_8	b a d o
S_9	a d o
S_{10}	d o
S_{11}	o

Schritt 3: merge A_{12} und A_0



Kurze Notation: $T = t_0t_1 \dots t_{n-1}$ und $x \equiv_3 y$ ist kurz für $x \bmod 3 = y$.

	0	1	2	3	4	5	6	7	8	9	10	11
$T =$	y	a	b	b	a	d	a	b	b	a	d	o

Jedes $S_i \in \mathcal{S}_0$ kann als (t_i, S_{i+1}) mit $S_{i+1} \in \mathcal{S}_1$ und als (t_i, t_{i+1}, S_{i+2}) mit $S_{i+2} \in \mathcal{S}_2$ geschrieben werden.

$\mathcal{S}_0 =$ Suffixe mit Index $i \equiv_3 0$

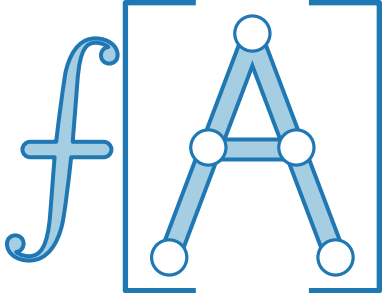
$\mathcal{S}_1 =$ Suffixe mit Index $i \equiv_3 1$

$\mathcal{S}_2 =$ Suffixe mit Index $i \equiv_3 2$

$\mathcal{S}(T) =$ Suffixe von $T =$

S_0	y a b b a d a b b a d o
S_1	a b b a d a b b a d o
S_2	b b a d a b b a d o
S_3	b a d a b b a d o
S_4	a d a b b a d o
S_5	d a b b a d o
S_6	a b b a d o
S_7	b b a d o
S_8	b a d o
S_9	a d o
S_{10}	d o
S_{11}	o

Schritt 3: merge A_{12} und A_0



Kurze Notation: $T = t_0t_1 \dots t_{n-1}$ und $x \equiv_3 y$ ist kurz für $x \bmod 3 = y$.

	0	1	2	3	4	5	6	7	8	9	10	11
$T =$	y	a	b	b	a	d	a	b	b	a	d	o

Jedes $S_i \in \mathcal{S}_0$ kann als (t_i, S_{i+1}) mit $S_{i+1} \in \mathcal{S}_1$ und als (t_i, t_{i+1}, S_{i+2}) mit $S_{i+2} \in \mathcal{S}_2$ geschrieben werden.

Beobachtung. Sei $S_i \in \mathcal{S}_0$.

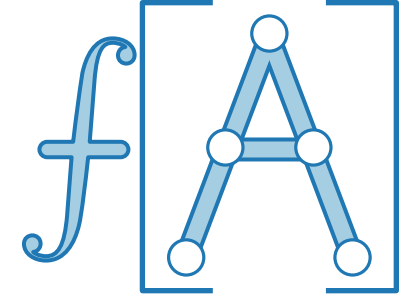
$\mathcal{S}_0 =$ Suffixe mit Index $i \equiv_3 0$

$\mathcal{S}_1 =$ Suffixe mit Index $i \equiv_3 1$

$\mathcal{S}_2 =$ Suffixe mit Index $i \equiv_3 2$

$\mathcal{S}(T) =$ Suffixe von $T =$

S_0	y a b b a d a b b a d o
S_1	a b b a d a b b a d o
S_2	b b a d a b b a d o
S_3	b a d a b b a d o
S_4	a d a b b a d o
S_5	d a b b a d o
S_6	a b b a d o
S_7	b b a d o
S_8	b a d o
S_9	a d o
S_{10}	d o
S_{11}	o



Schritt 3: merge A_{12} und A_0

Kurze Notation: $T = t_0t_1 \dots t_{n-1}$ und $x \equiv_3 y$ ist kurz für $x \bmod 3 = y$.

	0	1	2	3	4	5	6	7	8	9	10	11
$T =$	y	a	b	b	a	d	a	b	b	a	d	o

Jedes $S_i \in \mathcal{S}_0$ kann als (t_i, S_{i+1}) mit $S_{i+1} \in \mathcal{S}_1$ und als (t_i, t_{i+1}, S_{i+2}) mit $S_{i+2} \in \mathcal{S}_2$ geschrieben werden.

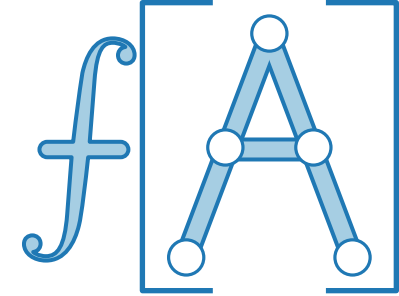
Beobachtung. Sei $S_i \in \mathcal{S}_0$.

- Sei $S_j \in \mathcal{S}_1$. Dann ist $S_i < S_j$ genau dann wenn
 - $t_i < t_j$; oder
 - $t_i = t_j$ und $S_{i+1} < S_{j+1}$ mit $S_{j+1} \in \mathcal{S}_2$.

- $\mathcal{S}_0 =$ Suffixe mit Index $i \equiv_3 0$
- $\mathcal{S}_1 =$ Suffixe mit Index $i \equiv_3 1$
- $\mathcal{S}_2 =$ Suffixe mit Index $i \equiv_3 2$

$\mathcal{S}(T) =$ Suffixe von $T =$

S_0	y a b b a d a b b a d o
S_1	a b b a d a b b a d o
S_2	b b a d a b b a d o
S_3	b a d a b b a d o
S_4	a d a b b a d o
S_5	d a b b a d o
S_6	a b b a d o
S_7	b b a d o
S_8	b a d o
S_9	a d o
S_{10}	d o
S_{11}	o



Schritt 3: merge A_{12} und A_0

Kurze Notation: $T = t_0t_1 \dots t_{n-1}$ und $x \equiv_3 y$ ist kurz für $x \bmod 3 = y$.

	0	1	2	3	4	5	6	7	8	9	10	11
$T =$	y	a	b	b	a	d	a	b	b	a	d	o

Jedes $S_i \in \mathcal{S}_0$ kann als (t_i, S_{i+1}) mit $S_{i+1} \in \mathcal{S}_1$ und als (t_i, t_{i+1}, S_{i+2}) mit $S_{i+2} \in \mathcal{S}_2$ geschrieben werden.

Beobachtung. Sei $S_i \in \mathcal{S}_0$.

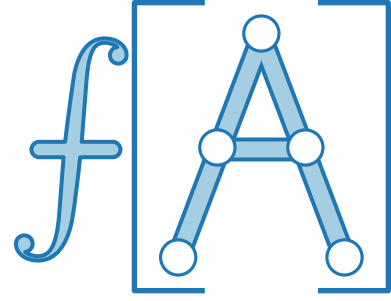
- Sei $S_j \in \mathcal{S}_1$. Dann ist $S_i < S_j$ genau dann wenn
 - $t_i < t_j$; oder
 - $t_i = t_j$ und $S_{i+1} < S_{j+1}$ mit $S_{j+1} \in \mathcal{S}_2$.
- Sei $S_j \in \mathcal{S}_2$. Dann ist $S_i < S_j$ genau dann wenn
 - $t_i < t_j$; oder
 - $t_i = t_j$ und $t_{i+1} < t_{j+1}$; oder
 - $t_it_{i+1} = t_jt_{j+1}$ und $S_{i+2} < S_{j+2}$ mit $S_{j+2} \in \mathcal{S}_1$.

- $\mathcal{S}_0 =$ Suffixe mit Index $i \equiv_3 0$
- $\mathcal{S}_1 =$ Suffixe mit Index $i \equiv_3 1$
- $\mathcal{S}_2 =$ Suffixe mit Index $i \equiv_3 2$

$\mathcal{S}(T) =$ Suffixe von $T =$

S_0	y a b b a d a b b a d o
S_1	a b b a d a b b a d o
S_2	b b a d a b b a d o
S_3	b a d a b b a d o
S_4	a d a b b a d o
S_5	d a b b a d o
S_6	a b b a d o
S_7	b b a d o
S_8	b a d o
S_9	a d o
S_{10}	d o
S_{11}	o

Schritt 3: merge A_{12} und A_0



Kurze Notation: $T = t_0 t_1 \dots t_{n-1}$ und $x \equiv_3 y$ ist kurz für $x \bmod 3 = y$.

	0	1	2	3	4	5	6	7	8	9	10	11
$T =$	y	a	b	b	a	d	a	b	b	a	d	o

$\mathcal{S}_0 =$ Suffixe mit Index $i \equiv_3 0$

$\mathcal{S}_1 =$ Suffixe mit Index $i \equiv_3 1$

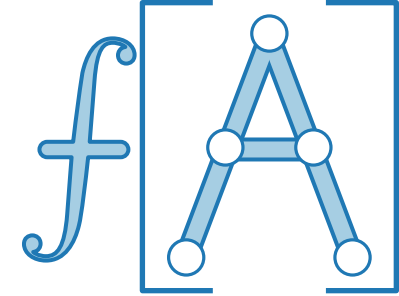
$\mathcal{S}_2 =$ Suffixe mit Index $i \equiv_3 2$

Jedes $S_i \in \mathcal{S}_0$ kann als (t_i, S_{i+1}) mit $S_{i+1} \in \mathcal{S}_1$ und als (t_i, t_{i+1}, S_{i+2}) mit $S_{i+2} \in \mathcal{S}_2$ geschrieben werden.

Beobachtung. Sei $S_i \in \mathcal{S}_0$.

- Sei $S_j \in \mathcal{S}_1$. Dann ist $S_i < S_j$ genau dann wenn
 - $t_i < t_j$; oder
 - $t_i = t_j$ und $S_{i+1} < S_{j+1}$ mit $S_{j+1} \in \mathcal{S}_2$.
- Sei $S_j \in \mathcal{S}_2$. Dann ist $S_i < S_j$ genau dann wenn
 - $t_i < t_j$; oder
 - $t_i = t_j$ und $t_{i+1} < t_{j+1}$; oder
 - $t_i t_{i+1} = t_j t_{j+1}$ und $S_{i+2} < S_{j+2}$ mit $S_{j+2} \in \mathcal{S}_1$.

Weil die Reihenfolge von $\mathcal{S}_1 \cup \mathcal{S}_2$ schon implizit in A_{12} ist, können wir diese Vergleiche in $\mathcal{O}(1)$ Zeit durchführen.



Schritt 3: merge A_{12} und A_0

Kurze Notation: $T = t_0 t_1 \dots t_{n-1}$ und $x \equiv_3 y$ ist kurz für $x \bmod 3 = y$.

	0	1	2	3	4	5	6	7	8	9	10	11
$T =$	y	a	b	b	a	d	a	b	b	a	d	o

$\mathcal{S}_0$ = Suffixe mit Index $i \equiv_3 0$

$\mathcal{S}_1$ = Suffixe mit Index $i \equiv_3 1$

$\mathcal{S}_2$ = Suffixe mit Index $i \equiv_3 2$

Jedes $S_i \in \mathcal{S}_0$ kann als (t_i, S_{i+1}) mit $S_{i+1} \in \mathcal{S}_1$ und als (t_i, t_{i+1}, S_{i+2}) mit $S_{i+2} \in \mathcal{S}_2$ geschrieben werden.

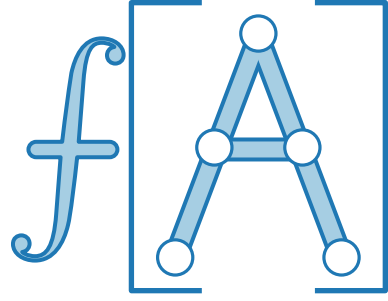
Beobachtung. Sei $S_i \in \mathcal{S}_0$.

- Sei $S_j \in \mathcal{S}_1$. Dann ist $S_i < S_j$ genau dann wenn
 - $t_i < t_j$; oder
 - $t_i = t_j$ und $S_{i+1} < S_{j+1}$ mit $S_{j+1} \in \mathcal{S}_2$.
- Sei $S_j \in \mathcal{S}_2$. Dann ist $S_i < S_j$ genau dann wenn
 - $t_i < t_j$; oder
 - $t_i = t_j$ und $t_{i+1} < t_{j+1}$; oder
 - $t_i t_{i+1} = t_j t_{j+1}$ und $S_{i+2} < S_{j+2}$ mit $S_{j+2} \in \mathcal{S}_1$.

Weil die Reihenfolge von $\mathcal{S}_1 \cup \mathcal{S}_2$ schon implizit in A_{12} ist, können wir diese Vergleiche in $\mathcal{O}(1)$ Zeit durchführen.

➔ A_{12} und A_0 können wie in MERGESORT zu A gemerged werden.

Konstruktion von Suffixarrays – Überblick



KONSTRUIERESUFFIXARRAY(T)

if $n = \mathcal{O}(1)$ **then**

 | erstelle A in $\mathcal{O}(1)$ Zeit.

else

 | sortiere $\mathcal{S}_1 \cup \mathcal{S}_2$ in ein Array A_{12}

 | benutze A_{12} , um $\mathcal{S}_0$ in ein Array A_0 zu sortieren

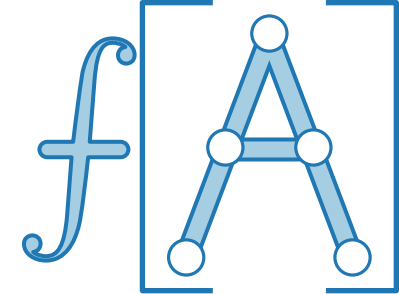
 | merge A_{12} mit A_0

$$\mathcal{O}(n) + T\left(\frac{2}{3}n\right)$$

$$\mathcal{O}(n)$$

$$\mathcal{O}(n)$$

Konstruktion von Suffixarrays – Überblick



KONSTRUIERESUFFIXARRAY(T)

if $n = \mathcal{O}(1)$ **then**

 | erstelle A in $\mathcal{O}(1)$ Zeit.

else

 | sortiere $\mathcal{S}_1 \cup \mathcal{S}_2$ in ein Array A_{12}

 | benutze A_{12} , um $\mathcal{S}_0$ in ein Array A_0 zu sortieren

 | merge A_{12} mit A_0

$$\mathcal{O}(n) + T\left(\frac{2}{3}n\right)$$

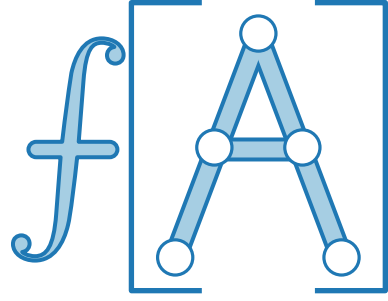
$$\mathcal{O}(n)$$

$$\mathcal{O}(n)$$

Gesamtlaufzeit:

$$T(n) = \begin{cases} \mathcal{O}(1), & \text{falls } n = \mathcal{O}(1) \\ \mathcal{O}(n) + T\left(\frac{2}{3}n\right), & \text{sonst} \end{cases}$$

Konstruktion von Suffixarrays – Überblick



KONSTRUIERE SUFFIXARRAY(T)

if $n = \mathcal{O}(1)$ **then**

└ erstelle A in $\mathcal{O}(1)$ Zeit.

else

└ sortiere $\mathcal{S}_1 \cup \mathcal{S}_2$ in ein Array A_{12}

└ benutze A_{12} , um $\mathcal{S}_0$ in ein Array A_0 zu sortieren

└ merge A_{12} mit A_0

Gesamtlaufzeit:

$$T(n) = \begin{cases} \mathcal{O}(1), & \text{falls } n = \mathcal{O}(1) \\ \mathcal{O}(n) + T\left(\frac{2}{3}n\right), & \text{sonst} \end{cases}$$



$$T(n) = c n + T\left(\frac{2}{3}n\right)$$



NATURAL LANGUAGE



MATH INPUT



Input

$$T(n) = c n + T\left(\frac{2}{3}n\right)$$

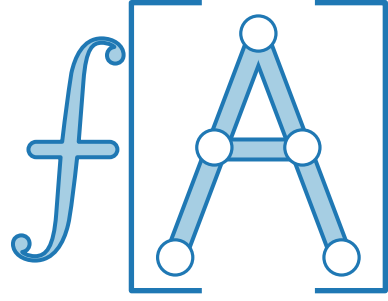
Recurrence equation solution

$$T(n) = 3 c (n - 1) + c_1 \quad (\text{where } c_1 \text{ is an arbitrary parameter})$$

Asymptotic bound

$$T(n) \in \Theta(n)_{n \rightarrow \infty}$$

Konstruktion von Suffixarrays – Überblick



KONSTRUIERE SUFFIXARRAY(T)

if $n = \mathcal{O}(1)$ **then**

└ erstelle A in $\mathcal{O}(1)$ Zeit.

else

└ sortiere $\mathcal{S}_1 \cup \mathcal{S}_2$ in ein Array A_{12}

└ benutze A_{12} , um $\mathcal{S}_0$ in ein Array A_0 zu sortieren

└ merge A_{12} mit A_0

Gesamtlaufzeit:

$$T(n) = \begin{cases} \mathcal{O}(1), & \text{falls } n = \mathcal{O}(1) \\ \mathcal{O}(n) + T\left(\frac{2}{3}n\right), & \text{sonst} \end{cases}$$

$$\Rightarrow T(n) \in \mathcal{O}(n)$$



$$T(n) = c n + T\left(\frac{2}{3}n\right)$$



NATURAL LANGUAGE



MATH INPUT



Input

$$T(n) = c n + T\left(\frac{2}{3}n\right)$$

Recurrence equation solution

$$T(n) = 3 c (n - 1) + c_1 \quad (\text{where } c_1 \text{ is an arbitrary parameter})$$

Asymptotic bound

$$T(n) \in \Theta(n)_{n \rightarrow \infty}$$

Zusammenfassung und Diskussion



Sei T über Alphabet Σ mit $n = |T|$.

Satz. Ein Suffixarray für T kann in $\mathcal{O}(n)$ Zeit und Platz berechnet werden. Damit kann man STRINGMATCHING Queries der Länge m in $\mathcal{O}(m \log n + k)$ Zeit beantworten.

Zusammenfassung und Diskussion



Sei T über Alphabet Σ mit $n = |T|$.

Satz. Ein Suffixarray für T kann in $\mathcal{O}(n)$ Zeit und Platz berechnet werden. Damit kann man STRINGMATCHING Queries der Länge m in $\mathcal{O}(m \log n + k)$ Zeit beantworten.

Lemma. Ein Suffixarray für T kann benutzt werden, um einen Suffixbaum von T in $\mathcal{O}(n)$ Zeit zu berechnen. (ohne Beweis)

Zusammenfassung und Diskussion



Sei T über Alphabet Σ mit $n = |T|$.

Satz. Ein Suffixarray für T kann in $\mathcal{O}(n)$ Zeit und Platz berechnet werden. Damit kann man STRINGMATCHING Queries der Länge m in $\mathcal{O}(m \log n + k)$ Zeit beantworten.

Lemma. Ein Suffixarray für T kann benutzt werden, um einen Suffixbaum von T in $\mathcal{O}(n)$ Zeit zu berechnen. (ohne Beweis)

Satz. Ein Suffixbaum für T kann in $\mathcal{O}(n)$ Zeit und Platz berechnet werden. Man kann damit STRINGMATCHING Queries der Länge m in $\mathcal{O}(m \log |\Sigma| + k)$ Zeit beantworten.

Zusammenfassung und Diskussion



Sei T über Alphabet Σ mit $n = |T|$.

Satz. Ein Suffixarray für T kann in $\mathcal{O}(n)$ Zeit und Platz berechnet werden. Damit kann man STRINGMATCHING Queries der Länge m in $\mathcal{O}(m \log n + k)$ Zeit beantworten.

Lemma. Ein Suffixarray für T kann benutzt werden, um einen Suffixbaum von T in $\mathcal{O}(n)$ Zeit zu berechnen. (ohne Beweis)

Satz. Ein Suffixbaum für T kann in $\mathcal{O}(n)$ Zeit und Platz berechnet werden. Man kann damit STRINGMATCHING Queries der Länge m in $\mathcal{O}(m \log |\Sigma| + k)$ Zeit beantworten.

Anmerkung. Das Suffixarray ist eine kompaktere Alternative zum Suffixbaum.

Zusammenfassung und Diskussion



Sei T über Alphabet Σ mit $n = |T|$.

Satz. Ein Suffixarray für T kann in $\mathcal{O}(n)$ Zeit und Platz berechnet werden. Damit kann man STRINGMATCHING Queries der Länge m in $\mathcal{O}(m \log n + k)$ Zeit beantworten.

Lemma. Ein Suffixarray für T kann benutzt werden, um einen Suffixbaum von T in $\mathcal{O}(n)$ Zeit zu berechnen. (ohne Beweis)

Satz. Ein Suffixbaum für T kann in $\mathcal{O}(n)$ Zeit und Platz berechnet werden. Man kann damit STRINGMATCHING Queries der Länge m in $\mathcal{O}(m \log |\Sigma| + k)$ Zeit beantworten.

Anmerkung. Das Suffixarray ist eine kompaktere Alternative zum Suffixbaum.

Der Suffixbaum (und das Suffixarray) hat mehrere zusätzliche Anwendungen:

Zusammenfassung und Diskussion



Sei T über Alphabet Σ mit $n = |T|$.

Satz. Ein Suffixarray für T kann in $\mathcal{O}(n)$ Zeit und Platz berechnet werden. Damit kann man STRINGMATCHING Queries der Länge m in $\mathcal{O}(m \log n + k)$ Zeit beantworten.

Lemma. Ein Suffixarray für T kann benutzt werden, um einen Suffixbaum von T in $\mathcal{O}(n)$ Zeit zu berechnen. (ohne Beweis)

Satz. Ein Suffixbaum für T kann in $\mathcal{O}(n)$ Zeit und Platz berechnet werden. Man kann damit STRINGMATCHING Queries der Länge m in $\mathcal{O}(m \log |\Sigma| + k)$ Zeit beantworten.

Anmerkung. Das Suffixarray ist eine kompaktere Alternative zum Suffixbaum.

Der Suffixbaum (und das Suffixarray) hat mehrere zusätzliche Anwendungen:

- Finde den längsten sich wiederholenden Teilstring

Zusammenfassung und Diskussion



Sei T über Alphabet Σ mit $n = |T|$.

Satz. Ein Suffixarray für T kann in $\mathcal{O}(n)$ Zeit und Platz berechnet werden. Damit kann man STRINGMATCHING Queries der Länge m in $\mathcal{O}(m \log n + k)$ Zeit beantworten.

Lemma. Ein Suffixarray für T kann benutzt werden, um einen Suffixbaum von T in $\mathcal{O}(n)$ Zeit zu berechnen. (ohne Beweis)

Satz. Ein Suffixbaum für T kann in $\mathcal{O}(n)$ Zeit und Platz berechnet werden. Man kann damit STRINGMATCHING Queries der Länge m in $\mathcal{O}(m \log |\Sigma| + k)$ Zeit beantworten.

Anmerkung. Das Suffixarray ist eine kompaktere Alternative zum Suffixbaum.

Der Suffixbaum (und das Suffixarray) hat mehrere zusätzliche Anwendungen:

- Finde den längsten sich wiederholenden Teilstring
- Finde den längsten gemeinsamen Teilstring zwischen zwei Strings

Zusammenfassung und Diskussion



Sei T über Alphabet Σ mit $n = |T|$.

Satz. Ein Suffixarray für T kann in $\mathcal{O}(n)$ Zeit und Platz berechnet werden. Damit kann man STRINGMATCHING Queries der Länge m in $\mathcal{O}(m \log n + k)$ Zeit beantworten.

Lemma. Ein Suffixarray für T kann benutzt werden, um einen Suffixbaum von T in $\mathcal{O}(n)$ Zeit zu berechnen. (ohne Beweis)

Satz. Ein Suffixbaum für T kann in $\mathcal{O}(n)$ Zeit und Platz berechnet werden. Man kann damit STRINGMATCHING Queries der Länge m in $\mathcal{O}(m \log |\Sigma| + k)$ Zeit beantworten.

Anmerkung. Das Suffixarray ist eine kompaktere Alternative zum Suffixbaum.

Der Suffixbaum (und das Suffixarray) hat mehrere zusätzliche Anwendungen:

- Finde den längsten sich wiederholenden Teilstring
- Finde den längsten gemeinsamen Teilstring zwischen zwei Strings
- ...